Table of Contents

Chapter 1: Glossary: Core Terms Every Frontend Engineer Should Know....2

Chapter 2: Browser & DOM Internals: How the Browser Actually Renders Your App....5

Chapter 3: Communication & Storage Protocols: WebSockets, REST vs GraphQL & Browser Storage Explained 12

Chapter 4: Requirements Gathering & Scoping for Frontend Systems....20

Chapter 5: Rendering Architecture: CSR, SSR, SSG, ISR & Hydration Trade-offs....37

Chapter 6: Performance & Scalability Architecture....58

Chapter 7: Frontend Data Layer, Caching & Real-Time Architecture....80

Chapter 8: State Management Architecture....100

Chapter 9: Frontend Component Design (LLD)....120

Chapter 10: Design Systems & Component Library Architecture....148

Chapter 11: Micro-Frontends & Scaling Frontend Architecture....167

Chapter 12: Accessibility Architecture....182

Chapter 13: Frontend Security Architecture....200

Chapter 14: Testing & Release Strategy....217

Chapter 15: Frontend Production, Monitoring & Deployment....238

Chapter 16: Putting It Together: Frontend System Design Case Studies....249

Chapter 17: When Your Tech Lead Says This... (Rapid-Fire Judgment Calls)....264

Chapter 18: Top 15 Most Asked Frontend System Design Questions....278

Chapter 19: Top 10 Most Asked Frontend Component Design (LLD) Questions....294

Chapter 1: Glossary: Core Terms Every Frontend Engineer Should Know

Every other chapter in this book assumes you already know the vocabulary of frontend architecture. If any of these terms feel unfamiliar, start here, five minutes now will make every later chapter click instead of feeling like a wall of jargon.

Performance & Core Web Vitals

- **LCP (Largest Contentful Paint):** how long it takes for the largest visible element (usually a hero image or heading) to render on screen. It's the primary Core Web Vital for perceived loading speed, if LCP is slow, the page feels slow even if other things finished quickly.
- **CLS (Cumulative Layout Shift):** a measure of how much visible content unexpectedly shifts around while a page loads (an image popping in and pushing text down, for example). High CLS is what makes a page feel janky and untrustworthy, even if it's technically fast.
- **INP (Interaction to Next Paint) / FID (First Input Delay):** INP measures how responsive a page feels across all user interactions during its lifetime (click, tap, key press, to the next visual update). FID was an earlier, narrower metric that only measured the delay before the first interaction. INP replaced FID as the official Core Web Vital.
- **TTFB (Time to First Byte):** how long it takes from the request being sent until the first byte of the response arrives, capturing server processing time, network latency, and any redirects, before the browser has anything to render at all.
- **TTI (Time to Interactive):** the point at which a page looks ready and reliably responds to user input, as opposed to looking ready but silently ignoring clicks because JavaScript is still loading or executing in the background.
- **Critical rendering path:** the sequence of steps the browser must complete (downloading HTML/CSS/JS, building the DOM and CSSOM, layout, paint) before it can render pixels on screen. Optimizing it means cutting or reordering that sequence so something meaningful appears sooner.
- **Reflow / repaint:** a repaint happens when the browser redraws pixels without changing layout (a color change). A reflow (or layout) happens when the browser has to recalculate the position and size of elements, which is far more expensive and can cascade to the rest of the page.

Rendering Strategies

- **CSR (Client-Side Rendering):** the server sends a mostly empty HTML shell, and the browser downloads JavaScript that then builds the actual page. Fast subsequent navigations, but a slower first paint and a blank page if JavaScript fails or is slow to load.
- **SSR (Server-Side Rendering):** the server renders the full HTML for a page on each request and sends it already populated, so the browser has something meaningful to show immediately, with JavaScript then "taking over" the page afterward.
- **SSG (Static Site Generation):** HTML pages are generated once, ahead of time at build time, and served as plain static files from a CDN, giving the fastest possible delivery for content that doesn't change per request.
- **ISR (Incremental Static Regeneration):** a hybrid where statically generated pages are automatically regenerated in the background after a set interval or on demand, combining the speed of static files with content that doesn't go permanently stale.
- **Hydration:** the process of attaching JavaScript event handlers and interactive behavior to server-rendered HTML that's already visible on the page, turning static markup into a fully interactive app without re-rendering everything from scratch.

- **Islands architecture / partial hydration:** an architecture where most of a page stays static HTML, and only specific interactive components ("islands") are hydrated with JavaScript individually, instead of hydrating the entire page as one unit.

Loading & Bundling

- **Code splitting:** breaking a JavaScript bundle into smaller chunks that are loaded only when they're needed (a route, a modal, a rarely used feature), instead of forcing every user to download the entire application's code upfront.
- **Tree shaking:** a build-time process that removes code which is never actually used or imported, so it doesn't get included in the final bundle shipped to the browser.
- **Lazy loading:** deferring the loading of a resource, whether an image, a component, or a route, until it's actually about to be needed (for example, when it scrolls into view), rather than loading everything the moment the page starts.
- **Bundle size / bundle analyzer:** bundle size is the total amount of JavaScript (and other assets) a page has to download and parse. A bundle analyzer visualizes what's inside a bundle, which libraries and modules are taking up the most space, so teams can find what to cut or split.
- **Virtualization / windowing:** rendering only the small subset of items in a long list that are currently visible on screen, and swapping DOM nodes in and out as the user scrolls, instead of rendering every single item in a list of thousands at once.

Networking & Caching

- **CDN (Content Delivery Network):** a network of servers spread across many physical locations that cache and serve static assets from a location close to the user, instead of every request traveling back to a single origin server.
- **Cache invalidation:** explicitly telling browsers or CDNs that a previously cached asset is no longer valid and should be re-fetched, typically triggered by deploying a new, differently named build of that asset (a "cache-busted" filename).
- **QPS / rate limiting / throttling:** QPS (queries per second) is how many requests an API receives per second. Rate limiting caps how many requests a given client can make in a time window; throttling slows a client down rather than rejecting it outright. Frontend engineers hit all three constantly when calling backend APIs, especially in search-as-you-type or polling features.
- **Debounce vs. throttle (event handling):** debouncing delays running a function until a burst of events has stopped firing for a set period (useful for search-as-you-type). Throttling runs a function at most once every fixed interval no matter how many events fire (useful for scroll or resize handlers).
- **WebSocket / SSE (Server-Sent Events) / polling:** a WebSocket is a persistent, two-way connection that lets server and client send messages to each other at any time. SSE is a one-way stream where the server can push updates to the client over a single long-lived HTTP connection. Polling is the client periodically asking the server "anything new?" on a timer. Each is a different trade-off between real-time freshness, server load, and implementation complexity.
- **Optimistic UI update:** updating the interface immediately as if an action (like a like or a comment) already succeeded, before the server has actually confirmed it, then quietly rolling back the UI if the server call ends up failing.
- **Race condition (stale response):** when multiple asynchronous requests are in flight and a response to an older request arrives and gets applied after a response to a newer request, showing outdated data on screen even though the correct data already arrived first.
- **BFF (Backend for Frontend):** a backend service built specifically to serve one frontend's needs, aggregating and reshaping calls to other backend services into exactly the shape that frontend wants, instead of the frontend having to orchestrate many calls itself.

Architecture & Organization

- **Design system:** a shared, versioned library of reusable UI components, design tokens (colors, spacing, typography), and usage guidelines that multiple teams or products build on top of, so the same button doesn't get reinvented five different ways.
- **Micro-frontend:** an architecture where a single web application is composed of multiple independently built and deployed frontend applications, often owned by different teams, that are stitched together at runtime or build time.
- **Monorepo vs. polyrepo:** a monorepo keeps the code for multiple projects or packages in a single repository with shared tooling and versioning. A polyrepo splits them across separate repositories, each with its own independent release cycle.
- **Feature flag:** a runtime switch that turns a feature on or off (or on for only a subset of users) without requiring a new deployment, used for gradual rollouts, A/B tests, and quick kill switches.
- **Canary / rolling / blue-green deployment:** a canary deployment sends a new version to a small slice of traffic first to catch problems early. A rolling deployment gradually replaces old instances with new ones. A blue-green deployment keeps two full environments and switches traffic from one to the other all at once, with the old one kept ready as an instant rollback.

Accessibility & Security

- **a11y:** shorthand for "accessibility" (the 11 stands for the number of letters between the "a" and the "y"), the practice of making a product usable by people with disabilities, including those using screen readers, keyboard-only navigation, or assistive devices.
- **ARIA (Accessible Rich Internet Applications):** a set of HTML attributes that add extra information for assistive technologies, describing roles, states, and properties of custom UI elements (a div acting as a button, for example) that native HTML alone can't communicate.
- **WCAG (Web Content Accessibility Guidelines):** the internationally recognized standard defining specific, testable criteria (organized into levels A, AA, AAA) for what makes web content accessible, most legal and organizational accessibility requirements point back to this standard.
- **XSS (Cross-Site Scripting):** a vulnerability where an attacker manages to inject and run their own JavaScript in a victim's browser session, typically by getting unsanitized user input rendered as executable code on the page.
- **CSRF (Cross-Site Request Forgery):** an attack that tricks a logged-in user's browser into unknowingly sending a request (like a fund transfer) to a site they're authenticated on, exploiting the fact that the browser automatically attaches that site's cookies to the request.
- **CSP (Content Security Policy):** an HTTP header that tells the browser exactly which sources of scripts, styles, images, and other resources are allowed to load on a page, so even if an attacker manages to inject a malicious script tag, the browser refuses to execute it.

Chapter 2: Browser & DOM Internals: How the Browser Actually Renders Your App

Every other chapter in this book uses terms like Virtual DOM, reflow, SPA, and tree shaking as if you already know exactly what's happening underneath them. This chapter is where that assumption gets earned. Each topic starts from zero, builds up with a plain analogy, and ends with the one-line version you'd actually say out loud in an interview.

Q1. [Foundations] What's the actual difference between the Real DOM and the Virtual DOM, and why does React use one?

Answer:

The Real DOM is the browser's actual live tree of objects representing the page. Every node is a full object with methods, and mutating it, adding, removing, or changing an element, can trigger the browser's layout and paint pipeline, which is comparatively expensive to do repeatedly.

The Virtual DOM is a lightweight, in-memory JavaScript object tree that mirrors what the UI should look like. It isn't connected to the browser at all, it's just plain data that's cheap to create and compare.

How React actually uses it

On a state change, React builds a new virtual DOM tree, diffs it against the previous one (a process called reconciliation), and computes the minimal set of real changes needed. Only that minimal batch of changes gets applied to the real DOM.

Why this is faster in practice

Touching the real DOM repeatedly, one small change at a time, is expensive because each touch can trigger layout recalculation. Diffing cheap, plain JS objects first and applying one batched, minimal update to the real DOM avoids redundant layout work.

An important nuance

The Virtual DOM isn't universally "faster than the DOM", hand-optimized direct DOM manipulation can still beat it. Its real value is developer ergonomics: you write declarative "what should be on screen" code, and the diffing algorithm figures out the efficient path to get there.

The Virtual DOM is a cheap, in-memory copy of the UI that React diffs against the previous version so it can apply only the minimal necessary changes to the real, expensive-to-touch DOM.

Interview Answer:

"React keeps a lightweight virtual copy of the UI tree in memory. On every state change, it builds a new virtual tree, diffs it against the last one, and only applies the minimal set of real changes to the actual DOM. That batching is what avoids the cost of touching the real DOM repeatedly for every small change."

Q2. [Foundations] What's a Shadow DOM, and how is it different from the Virtual DOM?

Answer:

The Shadow DOM is a real, native browser feature, not a React concept, for encapsulating a chunk of DOM and its styles inside a "shadow root" attached to an element, so its internal structure and CSS don't leak out, and outside CSS doesn't leak in.

The analogy

It's like sealing a component inside its own private bubble: its internal markup is invisible to a normal document.querySelector by default, and a global button { color: red; } rule from the rest of the page won't reach inside it.

Where it actually shows up

Native Web Components (custom elements) use Shadow DOM for real style and markup isolation. The browser also uses it internally, a `<video>` element's built-in play/pause/volume controls are literally implemented using a Shadow DOM the page can't see into or accidentally restyle.

The key difference from the Virtual DOM

Shadow DOM is a real DOM encapsulation boundary enforced by the browser itself. Virtual DOM is an in-memory diffing technique used by libraries like React. They solve completely different problems, style and markup isolation versus efficient re-rendering, and aren't alternatives to each other.

Shadow DOM is a native browser mechanism for encapsulating a component's markup and CSS so it can't leak in or out; the Virtual DOM is an unrelated, library-level diffing technique for efficient re-rendering.

Interview Answer:

"Shadow DOM is a real browser feature that isolates a component's DOM and styles from the rest of the page, used by native Web Components and by browser built-ins like the video element's controls. It has nothing to do with the Virtual DOM, which is just React's in-memory diffing technique, they solve completely different problems."

Q3. [Foundations] What's the difference between the DOM and the BOM?

Answer:

The DOM (Document Object Model) is the tree representation of the actual HTML document, elements, attributes, text nodes, that you manipulate with things like document.getElementById.

The BOM (Browser Object Model) is everything about the browser window itself that isn't part of the page's content: window, navigator (browser and device info), location (the URL), history (back/forward navigation), and screen (display info).

A concrete distinction

document.title is DOM, it's part of the page. window.location.href or window.history.back() is BOM, it's about the browser chrome and environment, not document content.

Why this matters practically

Server-side rendering environments (like Node.js during SSR) have no BOM at all, since there's no actual browser window. That's exactly why code that touches window or document directly can crash during SSR unless it's guarded to only run once the app is actually running in a browser.

The DOM is the document's content tree; the BOM is everything about the browser environment surrounding it (window, location, history, navigator), and SSR environments have no BOM at all.

Interview Answer:

"The DOM is the tree of the actual page content. The BOM is the browser environment around it, window, location, history, navigator. The practical reason this matters is that server-side rendering has no BOM, so any code touching window or navigator directly needs a guard to only run in the browser."

Q4. [Foundations] What actually happens between typing a URL and pixels appearing on screen?

Answer:

- DNS resolution, then a TCP connection and TLS handshake are established with the server.
- The browser receives HTML and starts parsing it incrementally, building the DOM tree as bytes arrive, it doesn't wait for the whole document.
- CSS is parsed into the CSSOM (CSS Object Model), a tree of style rules.
- The DOM and CSSOM are combined into the Render Tree, containing only visible nodes (a display:none element is excluded entirely).
- Layout (also called reflow): the browser computes the exact size and position of every element in the render tree.
- Paint: the browser fills in actual pixels, text, colors, borders, shadows, for every element.
- Composite: separate layers are combined together in the correct stacking order onto the screen, using the GPU where possible.

Why script and stylesheet placement matters

A `<script>` tag without defer or async blocks HTML parsing entirely until it downloads and executes, because the script could use document.write to alter the document. CSS is render-blocking by default because the browser can't safely paint content whose final styles it doesn't fully know yet, it might have to redo the work.

The browser streams and parses HTML into a DOM, parses CSS into a CSSOM, merges them into a render tree, computes layout, paints pixels, and composites layers onto the screen, and both render-blocking CSS and blocking scripts exist to avoid doing that work with incomplete information.

Interview Answer:
"It's parse HTML into the DOM, parse CSS into the CSSOM, combine them into a render tree of only visible nodes, compute layout, paint pixels, then composite layers onto the screen. CSS blocks rendering because painting with incomplete styles risks having to redo the work, and a plain script tag blocks HTML parsing because it could rewrite the document with document.write."

Q5. [Foundations] What's the difference between a reflow and a repaint, and why does reflow cost more?

Answer:

A repaint happens when the browser redraws pixels because something visual changed, but the layout, positions and sizes, stayed the same, changing a background-color or a text color, for instance.

A reflow (also called layout) happens when the browser has to recompute the geometry of elements because something changed that affects layout: width, adding or removing an element, changing font size. A reflow almost always forces a repaint afterward too, but not the other way around.

Why reflow is more expensive

A layout change to one element can ripple outward and force recalculating the position of many other elements around it, sometimes the entire page, while a repaint only has to redraw pixels within an already-known layout.

The practical implication for animation

Properties like transform and opacity are popular for animations specifically because they can often be handled purely by the compositor: no layout, no repaint of surrounding elements, just moving an already-painted layer. That's why they're so much smoother than animating width, top, or margin, which force a reflow on every single frame of the animation.

A repaint just redraws pixels with layout unchanged; a reflow recomputes positions and sizes and can ripple across the whole page, which is why animating transform/opacity (compositor-only) is far smoother than animating width or top (forces reflow every frame).

Interview Answer:

"A repaint redraws pixels without changing layout. A reflow recomputes element positions and sizes, and because that can ripple out to affect surrounding elements, it's far more expensive. That's exactly why you animate with transform and opacity instead of width or top, those two can be handled by the compositor alone without triggering layout at all."

Q6. [Foundations] How does the browser parse HTML and CSS differently, and why is CSS render-blocking?

Answer:

HTML parsing is incremental and forgiving: the browser starts building the DOM as bytes stream in, and it tolerates malformed markup, auto-correcting mismatched tags rather than halting on an error.

CSS parsing builds the CSSOM, a tree of style rules, but the browser needs the complete picture of all applicable styles before it can safely compute final layout and paint anything, since a rule later in the stylesheet could override one seen earlier.

Why this makes CSS render-blocking

Painting content before all CSS is parsed risks painting it with the wrong styles and then having to redo that work the moment the rest of the stylesheet arrives. Browsers hold off on rendering until CSSOM construction is complete, which is exactly why CSS is loaded as early as possible in the head, to get that blocking cost out of the way sooner.

A related note on scripts

HTML parsing itself blocks on a synchronous script tag because that script could use document.write to alter the document being parsed. That's the actual mechanism behind why script tags without defer or async pause DOM construction until they finish executing.

HTML parsing is incremental because the browser can safely build partial DOM as bytes arrive; CSS parsing blocks rendering because a later rule could override an earlier one, so the browser needs the complete CSSOM before it can safely paint.

Interview Answer:

"HTML parses incrementally because partial DOM is still safe to build. CSS blocks rendering because the browser can't know a rule is final until it's seen the whole stylesheet, a later rule could override it. That's why CSS goes early in the head, and why an unmarked script tag also blocks parsing, since it could rewrite the document via document.write."

Q7. [Foundations] What's the actual difference between an SPA and an MPA?

Answer:

An MPA (Multi-Page Application) treats every navigation as a full server round trip: the browser requests a whole new HTML document, throws away the current page, and rebuilds everything from scratch, DOM, CSS, and the JS execution context all reset.

An SPA (Single-Page Application) loads one HTML shell once, and JavaScript takes over subsequent "navigation," swapping content in place via client-side routing without a full page reload, so shared UI like headers and app state don't need to be rebuilt on every navigation.

The trade-offs

SPAs feel faster after the initial load, no full-page reloads, but usually ship more JavaScript upfront and need extra engineering effort for things browsers give MPAs for free, like a working back and forward button and per-page SEO metadata. MPAs are simpler and get SEO and initial-load correctness for free, but every navigation pays a full round trip.

The middle ground

Modern meta-frameworks like Next.js and Remix blend both: server-rendering the initial HTML like an MPA for a fast first paint and good SEO, then hydrating into SPA-like client-side navigation for everything after.

An MPA reloads a whole new document from the server on every navigation; an SPA loads one shell once and handles subsequent navigation client-side, trading a heavier upfront bundle and extra engineering for faster post-load navigation.

Interview Answer:

"An MPA does a full server round trip on every navigation, a fresh document each time. An SPA loads a shell once and handles routing client-side after that, which feels faster after the initial load but has to reimplement things like back-button behavior and SEO metadata that MPAs get for free. Most modern frameworks land in the middle: SSR for the first load, SPA-style navigation after."

Q8. [Foundations] What's the difference between minification and compression, and why do you need both?

Answer:

Minification happens at build time, before the file ever reaches the network. It rewrites your actual JS, CSS, or HTML source to be smaller by removing whitespace and comments and shortening variable names, producing a smaller but still plain-text file with the exact same behavior.

Compression happens at transfer time: the server encodes the already minified file using an algorithm like gzip or Brotli before sending it over the network, and the browser decompresses it automatically on arrival. It's about shrinking the bytes actually sent over the wire, not the source code itself.

Why you need both

Minification reduces the raw size of the code, which also matters for parse and execution time in the browser. Compression further shrinks whatever you're sending over the network on top of that. The two stack well together: a minified file compresses even better than an unminified one, since minification already removed a lot of repetitive, easily-compressible text like long variable names and comments.

Minification shrinks source code itself at build time (smaller file, same behavior, faster to parse); compression shrinks the bytes sent over the network at transfer time, and applying both together compounds the savings.

Interview Answer:
"Minification happens at build time and shrinks the actual source, removing whitespace, comments, and shortening names. Compression happens at transfer time, gzip or Brotli encoding the file before it goes over the network. They stack: a minified file compresses even better since a lot of the easily-compressible repetition is already stripped out."

Q9. [Foundations] What's the actual difference between tree shaking and dead code elimination?

Answer:

Dead code elimination is a general compiler optimization: removing code that's provably never reached or never used, regardless of language or bundler, an `if (false) { ... }` block, for example.

Tree shaking is a more specific, ES-module-aware technique bundlers like webpack and Rollup use. Because ES module imports and exports are static, you can tell exactly what's imported from where at build time, unlike CommonJS's dynamic require, the bundler can statically trace exactly which exported functions from a library are actually used, and exclude every unused export from the final bundle, even if the library itself has no idea it's being tree-shaken.

Why this matters practically

This is exactly why importing `{ debounce } from 'lodash-es'` can produce a much smaller bundle than importing the whole lodash package. The ES-module version is tree-shakeable and lets the bundler drop everything except debounce, while the older CommonJS version generally isn't, since the bundler can't statically prove what's used from a dynamically-required object.

Dead code elimination removes provably unreachable code in general; tree shaking is the specific, static-analysis technique bundlers use on ES modules to drop unused exports from a library, which is why ES-module imports produce smaller bundles than CommonJS ones.

Interview Answer:
"Dead code elimination is the general compiler concept of stripping unreachable code. Tree shaking is the specific version of that bundlers do with ES modules, since static imports let the bundler know exactly which exports are used and drop the rest. That's why importing from an ES-module build of a library gives a smaller bundle than the CommonJS version."

Q10. [Foundations] How does CSS positioning actually work under the hood?

Answer:

- static: the default. An element just follows normal document flow, top/left/etc. have no effect.
- relative: still takes up its original space in the flow, but top/left/right/bottom now shift it visually from where it would otherwise sit. Critically, it establishes a positioning context for any absolutely positioned children.
- absolute: removed from normal flow entirely (other elements act like it isn't there), and positioned relative to its nearest ancestor with a position other than static, its "containing block", or relative to the page if no such ancestor exists.

- fixed: removed from flow and positioned relative to the browser viewport itself, so it stays in place as the page scrolls, unless an ancestor has a transform, filter, or will-change property, which can create a new containing block and "trap" a fixed element inside it instead.
- sticky: behaves like relative until the element would scroll past a specified threshold (like top: 0), at which point it "sticks" and behaves like fixed relative to its nearest scrolling ancestor, until that ancestor's own boundary is reached.

The core mental model

Positioning is really about which containing block an element's offsets are measured against, and whether the element still occupies space in the normal flow or has been pulled out of it entirely.

Each position value differs in two dimensions: whether the element still occupies space in normal flow, and which containing block its offsets are measured against, with sticky being a hybrid that switches between relative and fixed behavior at a scroll threshold.

Interview Answer:

"The way I think about it is two questions: does the element still take up space in normal flow, and what containing block are its offsets measured against. Static and relative stay in flow, absolute and fixed are pulled out of it, and sticky is a relative element that switches to fixed-like behavior once it crosses a scroll threshold."

Chapter 3: Communication & Storage Protocols: WebSockets, REST vs GraphQL & Browser Storage Explained

WebSockets, polling, REST, GraphQL, localStorage, cookies, these show up constantly in the scenario questions throughout this book, but they're never explained from first principles anywhere else. This chapter closes that gap: how each of these protocols and storage mechanisms actually works, and exactly when you'd reach for one over another.

Q1. [Foundations] How does short polling actually work, and why is it usually the wrong choice?

Answer:

The client sends a request on a fixed interval, say, every five seconds, asking "anything new?" regardless of whether anything has actually changed, and the server responds immediately either way.

What it gets you

It's simple to implement and works with plain HTTP, no special infrastructure needed on either side.

The problem

Most requests return "nothing changed," wasting server resources and client battery and bandwidth. There's an inherent trade-off between freshness and overhead: a short interval means more waste, a longer interval means laggier updates, and there's no way to get both good freshness and low overhead at the same time with this approach.

When it's actually fine

Low-frequency updates where a few seconds of staleness genuinely doesn't matter, and a small enough number of clients that the wasted requests aren't a real cost.

Short polling checks on a fixed interval regardless of whether anything changed, which forces a direct trade-off between wasted requests and staleness, with no way to get both good freshness and low overhead.

Interview Answer:

"Short polling just asks 'anything new?' on a timer, whether or not anything's actually changed. It's simple, but it forces a trade-off, shorter intervals waste requests, longer intervals mean stale data, and you can't get both good freshness and low overhead at once. It's fine for low-frequency updates with small user counts, and the wrong choice for anything real-time."

Q2. [Foundations] How does long polling work, and how is it actually different from short polling under the hood?

Answer:

The client sends a request, but the server doesn't respond immediately, it holds the connection open and only responds once new data actually becomes available, or a timeout is hit, after which the client immediately reconnects and asks again.

What this flips

Instead of many requests mostly returning "nothing," you get fewer requests that mostly return real data, since the server only replies when there's something to say.

The cost

Holding many open connections waiting for data ties up server resources, a thread or connection slot per waiting client, which is part of why it doesn't scale as gracefully as a true persistent connection for very high concurrency.

It's still fundamentally request-response

It's still built on top of regular HTTP requests, just with the response deliberately delayed. Unlike WebSockets, there's no single connection carrying many messages back and forth, each update still requires a fresh request and response cycle.

Long polling delays the server's response until there's real data to send, flipping short polling's "mostly wasted requests" problem into "fewer, more meaningful requests," at the cost of tying up a server connection per waiting client.

Interview Answer:

"Long polling holds the request open on the server side until there's actually new data, then the client immediately reopens it. That means far fewer wasted round trips than short polling, but it ties up a server connection per waiting client, and it's still request-response under the hood, not a single persistent channel like a WebSocket."

Q3. [Foundations] How do WebSockets actually establish a persistent, full-duplex connection?

Answer:

It starts as a normal HTTP request with an Upgrade: websocket header. The server, if it supports it, responds with a 101 Switching Protocols status, and from that point on, the exact same underlying TCP connection stops speaking HTTP and starts speaking the WebSocket protocol instead.

What full-duplex actually means

Both sides can send messages at any time, independently, over that same open connection, unlike HTTP's strict request-then-response pattern. Either the client or the server can push a message the moment it has something to say, without waiting to be asked.

Why this is efficient

Because the connection stays open, there's no per-message overhead of establishing a new connection or repeating headers, which is what makes WebSockets efficient for frequent, bidirectional traffic like live chat, multiplayer cursors, or live price ticks.

The trade-off

The server now has to actively manage a large number of long-lived open connections, and handle reconnects, missed messages during a drop, and horizontal scaling of "which server holds which open connection," which is meaningfully more operational complexity than stateless HTTP requests.

A WebSocket starts as an HTTP request that upgrades to a persistent TCP connection speaking a different protocol, letting either side push messages at any time without per-message connection overhead, at the cost of real server-side connection management complexity.

Interview Answer:

"A WebSocket handshake starts as a normal HTTP request with an Upgrade header, and once the server accepts with a 101 response, that same TCP connection switches to the WebSocket protocol and stays open. From then on either side can push messages any time, full-duplex, without the overhead of a new connection per message. The trade-off is the server now has to manage a lot of long-lived open connections."

Q4. [Foundations] How do Server-Sent Events (SSE) actually work, and how are they different from WebSockets?

Answer:

SSE is a one-way channel: the client opens a regular HTTP connection to an endpoint using the EventSource API, and the server keeps that response stream open indefinitely, writing new "event" chunks into it over time as plain text in a simple data: ... line-based format.

A feature WebSockets don't give you for free

Because it's plain HTTP under the hood, SSE gets automatic reconnection built into the browser's EventSource API: if the connection drops, the browser retries automatically and can even resume from the last received event ID.

The key limitation compared to WebSockets

SSE is server-to-client only. The client cannot send messages back over that same connection, if the client needs to send data, it does so with a regular, separate HTTP request.

When to reach for SSE instead of WebSockets

Whenever data only needs to flow in one direction, server pushing updates to the client: live notification feeds, live score updates, streaming AI-generated text. It's simpler to run and gets free reconnection, with no need to pay for full bidirectional complexity you won't use.

SSE is a one-directional, server-to-client stream over plain HTTP that gets automatic reconnection for free, making it the simpler choice whenever the client never needs to push data back over the same channel.

Interview Answer:

"SSE is a one-way stream, the server keeps an HTTP response open and pushes text-formatted events into it over time, and the browser's EventSource API automatically reconnects if it drops. It can't send data back to the server over that connection, which is exactly why I'd reach for it over a WebSocket whenever the use case is purely server-to-client, like live notifications."

Q5. [Foundations] What's a webhook, and how is it structurally different from polling, WebSockets, and SSE?

Answer:

All of the previous mechanisms are about a client staying connected to, or repeatedly checking, a server it's already talking to. A webhook flips that entirely: instead of your app reaching out to ask for updates, an external service calls a URL on your server the moment something happens on their end, a payment provider hitting your /webhooks/payment-completed endpoint when a charge succeeds, for example.

What it actually is

It's really just a plain HTTP POST request. There's no special protocol, persistent connection, or client-side API involved, "webhook" is just the agreed-upon convention that the other service will proactively call your endpoint with an event payload.

Why frontend engineers rarely touch webhooks directly

Because it's server-to-server, it's not something a browser tab uses directly to get live updates for a user interface. A frontend engineer's usual involvement is indirect: the backend receives a webhook, updates a database or triggers a job, and then the frontend finds out about that change through one of the other mechanisms, polling that data, or a WebSocket or SSE push.

Webhooks are server-to-server event notifications sent as plain HTTP requests, structurally unrelated to WebSockets, SSE, or polling, which are all about how a browser client stays updated, not how two backend services notify each other.

Interview Answer:

"A webhook is just a plain HTTP POST that an external service sends to your server when something happens on their end, it's server-to-server, not something a browser uses directly. The connection to the frontend is indirect, a webhook updates the backend, and the frontend finds out through polling, a WebSocket, or SSE, whichever the app already uses for live updates."

Q6. [Foundations] What actually makes an API "RESTful," and how is that structurally different from GraphQL and gRPC?

Answer:

REST models an API around resources and standard HTTP verbs: a URL identifies a resource (/users/42), and the HTTP method, GET, POST, PUT, DELETE, describes the action on it. Each endpoint typically returns a fixed shape of data.

The main practical pain point

Over-fetching (an endpoint returns fields you don't need) and under-fetching (you need data from three different endpoints and have to make three round trips), especially on data-heavy UIs like dashboards or nested feeds.

Where it sits relative to the other two

Both REST and GraphQL usually run over plain HTTP and JSON, human-readable and easy to inspect in a browser's network tab. gRPC, covered next, is structurally different even at the transport level.

REST structures an API as fixed-shape resources accessed via HTTP verbs, simple and cacheable, but prone to over-fetching and under-fetching once a UI needs data shaped differently than the endpoints provide.

Interview Answer:

"REST is built around resources and HTTP verbs, a URL identifies a resource and the method describes the action, and each endpoint returns a fixed shape. The practical pain is over-fetching and under-fetching once your UI needs a different shape than what the endpoints give you, which is exactly the gap GraphQL is designed to close."

Q7. [Foundations] How is GraphQL actually different from REST under the hood, and what problem does it solve?

Answer:

Instead of many fixed-shape endpoints, GraphQL exposes a single endpoint with a strongly-typed schema, and the client sends a query describing exactly which fields it wants, potentially spanning multiple related resources in one request.

How it actually resolves a query

The server resolves that query field by field using "resolver" functions, walking the schema's graph of relationships, and returns exactly the shape the client asked for, nothing more, nothing less.

The problem this fixes

A mobile app screen that needs a post's title, its author's name, and the first three comments can get all of that in a single request with a single round trip, instead of hitting /posts/1, then /users/{authorId}, then /posts/1/comments.

The trade-off

That flexibility moves real complexity onto the server, schema design, resolver performance, avoiding the "N+1 query" problem where naive resolvers each independently hit the database. Caching is also harder than REST's, since GraphQL typically uses a single POST endpoint, which doesn't play as naturally with HTTP-level caching by URL the way distinct REST resource URLs do.

GraphQL lets the client specify exactly which fields it needs across related resources in one request, fixing REST's over/under-fetching, but shifts real complexity onto server-side resolver design and makes HTTP-level caching harder.

Interview Answer:

"GraphQL exposes one endpoint with a typed schema, and the client's query says exactly which fields it wants, even across related resources, resolved in a single round trip. That directly solves REST's over-fetching and under-fetching. The cost is that resolver design and avoiding N+1 queries becomes a real server-side problem, and caching is harder since it's not many distinct URLs anymore."

Q8. [Foundations] How does gRPC actually work, and why is it usually service-to-service rather than browser-to-server?

Answer:

gRPC sends messages as Protocol Buffers (protobufs), a compact binary format, not human-readable JSON, defined upfront in a .proto schema file that both client and server compile into strongly-typed code.

It runs over HTTP/2, which supports multiplexing many requests over a single connection and true bidirectional streaming, letting gRPC support not just request-response but also client-streaming, server-streaming, and full bidirectional streaming calls natively.

Why it's rare directly in browser traffic

Browsers don't give JavaScript low-level enough control over HTTP/2 frames to speak native gRPC directly, and binary protobuf isn't something you can casually inspect in a network tab the way you can JSON. A project called gRPC-Web exists as a workaround, but it requires a proxy in front of the actual gRPC server.

Where it actually shines

Internal service-to-service calls inside a backend, one microservice calling another, where both ends are known, controlled, and can share compiled protobuf schemas, and the performance win from a compact binary format and HTTP/2 multiplexing genuinely matters at that scale.

gRPC uses compact binary protobuf messages over HTTP/2 for efficient, strongly-typed, streamable communication, but browsers can't speak it natively, which is why it's mostly used for internal service-to-service traffic rather than direct frontend calls.

Interview Answer:

"gRPC uses binary Protocol Buffers over HTTP/2, giving it multiplexing and true bidirectional streaming that plain REST doesn't have. Browsers can't natively send gRPC though, there's no low-level HTTP/2 frame access in JavaScript, so it needs a proxy via gRPC-Web. That's why you see it mostly for internal service-to-service calls rather than talking directly to a frontend."

Q9. [Foundations] How is localStorage actually different from sessionStorage under the hood?

Answer:

Both store simple key-value string pairs directly in the browser, synchronously, with roughly 5 to 10MB of space depending on the browser, and neither is automatically sent to the server on requests, unlike cookies.

The entire difference: lifetime and scope

localStorage persists indefinitely, across tabs, browser restarts, and days or months later, until explicitly cleared by code or the user. sessionStorage is scoped to a single tab and is cleared the moment that tab is closed, and it isn't even shared between two tabs of the same site open at the same time, each tab gets its own separate sessionStorage.

A shared caveat

Because both are synchronous, reading or writing a large amount of data can briefly block the main thread, which is part of why they're used for small values, tokens, preferences, form drafts, rather than large datasets.

localStorage and sessionStorage share the same synchronous key-value API and per-origin scope; the only real difference is lifetime, localStorage persists indefinitely across tabs and restarts, sessionStorage dies with the tab.

Interview Answer:

"They share the same synchronous string key-value API, the only real difference is lifetime and scope. localStorage persists indefinitely across tabs and browser restarts. sessionStorage is scoped to one tab and disappears the moment that tab closes, it's not even shared between two tabs of the same site open at once."

Q10. [Foundations] How are cookies structurally different from localStorage, and why does auth commonly use cookies?

Answer:

A cookie is set either by the server, via a Set-Cookie response header, or by JavaScript, and critically, it's automatically attached to every subsequent HTTP request to that domain by the browser itself. No application code has to remember to send it.

localStorage is never sent to the server automatically. If you want a localStorage value to reach the backend, your JavaScript has to explicitly read it and attach it to a request, for example as an Authorization header.

Flags cookies support that plain storage doesn't

- HttpOnly: makes the cookie invisible to JavaScript entirely, so an XSS attack can't steal it via document.cookie.
- Secure: only sent over HTTPS.
- SameSite: controls whether the cookie is sent on cross-site requests, a major CSRF defense.

Why auth commonly uses cookies

Putting a session or auth token in an HttpOnly cookie removes it from JavaScript's reach entirely, closing off an entire class of "steal the token via XSS" attacks that a token sitting in localStorage remains exposed to.

Cookies are automatically attached to every request by the browser and support HttpOnly/Secure/SameSite flags that remove them from JavaScript's reach entirely; localStorage is never auto-sent and stays fully exposed to any script running on the page.

Interview Answer:
"The key structural difference is that cookies get attached to every request automatically by the browser, localStorage never does, your JS has to explicitly send it. Cookies also support HttpOnly, which hides them from JavaScript completely, that's why auth tokens commonly go in an HttpOnly cookie instead of localStorage, it closes off the XSS-can-steal-the-token attack path entirely."

Q11. [Foundations] How does IndexedDB actually work, and when would you reach for it over localStorage?

Answer:

IndexedDB is a full transactional, asynchronous, object-oriented database built into the browser, not a simple string key-value store. It stores structured JavaScript objects, not just strings, in "object stores," supports indexes for fast lookups by a field other than the primary key, and wraps operations in transactions.

Why asynchronous matters

Reading or writing doesn't block the main thread the way localStorage's synchronous API does, which matters once you're dealing with meaningfully large amounts of data.

Storage capacity

Far larger than localStorage's roughly 5-10MB, commonly hundreds of megabytes or more depending on available disk space, making it suitable for offline-first apps that need to cache real datasets locally: an email client's message store, a mapping app's offline map tiles, a note-taking app's full document history.

When to reach for it

Any time you need to store more than a trivial amount of structured data locally, need to query it efficiently, or need storage operations that don't block the UI thread, exactly the profile of most real offline-support and PWA use cases, versus localStorage's sweet spot of a handful of small, simple values.

IndexedDB is an asynchronous, transactional, structured-object database built into the browser with far more capacity than localStorage, making it the right tool for real offline datasets rather than a handful of small string values.

Interview Answer:
"IndexedDB is a real asynchronous database in the browser, it stores structured objects with indexes and transactions, not just strings, and it doesn't block the main thread the way localStorage does. I'd reach for it

whenever I need to store a meaningful amount of structured data locally, offline caches, downloaded content, versus localStorage's sweet spot of a few small values like a token or a preference."

Chapter 4: Requirements Gathering & Scoping for Frontend Systems

The single biggest predictor of whether a frontend system design interview goes well is what you do in the first five minutes, before a single component is drawn. Hearing "design the frontend for a photo-sharing app" and immediately sketching boxes is a weak instinct. A stronger one treats the prompt as underspecified on purpose and spends the opening minutes pinning down devices, network conditions, scale, accessibility obligations, and what "done" even means, the frontend equivalent of a backend candidate asking about QPS and read/write ratios before reaching for a whiteboard. Use the questions in this chapter to practice turning a vague ask into a scoped, measurable, time-boxed plan, since that skill matters more in real jobs than any specific component pattern you might know.

> **War Story:** *A team once spent two sprints building an elaborate offline-sync layer for a feature that, it turned out, only needed to work for users already on office WiFi, because nobody asked what the actual usage context was before designing for the hardest possible case.*

> **Avoid:** jumping straight into rendering strategy or state management before asking who the users are • never asking about devices or network conditions before proposing an architecture
>
> **Aim For:** asking about device, network, and audience constraints before proposing any architecture • explicitly narrowing an open-ended prompt before drawing anything

Q1. [Junior] You're asked to design the frontend for a photo-sharing app like Instagram, and your first instinct is to immediately start sketching a component tree for the feed. What's missing from that approach, and what should you do differently in the first two minutes instead?

Answer:

The gap is skipping requirements gathering entirely and jumping to a solution before the problem is defined. **Spend the first few minutes clarifying scope, constraints, and success criteria before drawing any boxes, because an app for 50 million globally distributed users and an internal prototype for a 20-person pilot are completely different systems even though the prompt sounds identical.**

What a strong opening looks like

- Restate the prompt and immediately narrow it: "When you say design the frontend for Instagram, do you want the whole app or should we focus on one flow, like the feed or the upload flow?"
- Ask about scale and audience: is this a global consumer app, a regional launch, or an internal demo?
- Ask about device and network floor: are we assuming modern phones on WiFi, or does this need to work on a mid-range Android device on a spotty 3G connection?
- Ask what "success" looks like for this interview: is the interviewer grading on component architecture, state management, performance, or all three?

Why this matters

- Skipping this step usually means designing the wrong thing well, for example a fully client-rendered SPA (single-page application, a site that renders in the browser rather than the server) when the real requirement was fast first paint for anonymous visitors.
- It also wastes interview time on a build you'll have to backtrack from once the interviewer clarifies constraints halfway through.

Interview Answer:

"Before I draw anything, I want to pin down what we're actually building, because 'design Instagram' could mean a global consumer feed or a small internal tool, and those lead to completely different architectures. I'd ask about scale, target devices and network conditions, and which flow we're focusing on, since a 45-minute interview can't cover the whole app. Then I'd restate what I heard back to confirm before touching the whiteboard. That upfront alignment is what lets me spend the rest of the time on a design that actually answers the question being asked."

Q2. [Junior] A product manager tells your team "make the checkout page feel fast." How do you turn that into something your team can actually build against?

Answer:

"Feels fast" is not an engineering requirement, it is a product feeling that has to be translated into numbers a team can build and test against. **The job of requirements gathering here is converting a vague adjective into specific, measurable targets, typically Core Web Vitals thresholds plus interaction-specific latency budgets, so engineering has an unambiguous definition of done.**

Turning the ask into numbers

- Largest Contentful Paint (LCP, the time until the biggest visible element renders): target under 2.5 seconds on a mid-tier mobile device on a throttled 4G connection.
- Interaction to Next Paint (INP, the delay between a user action like a tap and the next visual update): target under 200 milliseconds for the "Place Order" button.
- Cumulative Layout Shift (CLS, how much visible content jumps around while loading): target under 0.1, so the total price or shipping estimate never jumps after the user has started reading it.
- A specific interaction budget for the riskiest step, for example the payment field validation should give feedback within 100 milliseconds of the user finishing typing a card number.

Why this matters

- Without concrete numbers, "fast" gets re-litigated in every design review and every QA (quality assurance) bug report, since nobody agreed what fast means.
- Concrete targets also tell you where to invest engineering effort, for example lazy-loading the order-summary sidebar images matters much less than making sure the "Place Order" click responds instantly.

Interview Answer:

"'Fast' isn't testable, so my first move is to turn it into numbers everyone agrees on. I'd propose Core Web Vitals targets, something like LCP under 2.5 seconds and INP under 200 milliseconds, measured on a mid-range device on throttled 4G rather than my own laptop on office WiFi. Then I'd add a specific budget for the highest-stakes interaction on that page, like the Place Order button responding within 100 milliseconds of the tap. Once those numbers exist, they go into our performance budget and CI checks, so 'fast' stops being a subjective argument in a design review."

Q3. [Junior] Before you start designing, would you ask the same clarifying questions for a public marketing landing page as you would for an internal analytics dashboard used only by your company's ops team? Walk me through the difference.

Answer:

No, and treating them the same is a common junior mistake that leads to over-engineering one and under-engineering the other. **The audience, traffic pattern, and business risk of a public marketing page versus an internal tool are different enough that the clarifying questions, and the resulting architecture, should diverge from the very first conversation.**

Questions specific to a public marketing page

- What is the expected traffic pattern, and is there a launch spike (a product announcement, a Super Bowl ad) that needs to be handled?
- Does this page need to rank in search results, which pushes toward server-side rendering (SSR, rendering HTML on the server before sending it to the browser) or static generation?
- What is the acceptable bundle size, since anonymous visitors on a cellular connection will bounce if the page is slow to appear?

Questions specific to an internal ops dashboard

- Who exactly uses this, and can we assume a modern browser and a corporate laptop on reliable WiFi?
- Is SEO (search engine optimization) even relevant, given the page sits behind a login and is never indexed?
- Is development speed more valuable than bundle size here, since the user base is 50 people, not 5 million?

Why this matters

- Applying marketing-site rigor (SSR, strict bundle budgets, SEO metadata) to an internal tool wastes engineering time on constraints that do not exist for that audience.
- Applying internal-tool looseness (heavy client-side rendering, no SEO consideration, large bundles) to a marketing page directly costs the company conversions and search ranking.

Interview Answer:

"I would not ask the same questions, because the two have almost opposite constraints. For a marketing page, I care a lot about anonymous traffic, SEO, and first-paint speed on a phone with a weak connection, which usually pushes me toward server rendering or static generation. For an internal dashboard, I can usually assume a known set of users on modern laptops and reliable WiFi, so I'd rather spend my time budget on developer velocity and rich interactivity than on shaving kilobytes off the bundle. Asking 'who is the audience and what do they connect from' upfront is what tells me which of those two worlds I'm in."

Q4. [Junior] You're asked to design the frontend for a government benefits application form that needs to work across India, including in rural areas. What clarifying questions do you ask about network and device conditions, and how do those answers change your design?

Answer:

This prompt is deliberately testing whether you default to "assume a modern iPhone on WiFi" or actually ask about the real-world constraints of the target population. **In markets like rural India, a large share of users are on low-end Android devices with limited RAM and on 3G or unreliable 4G connections with metered, expensive data, and that reality should drive fundamental architecture choices, not just be a footnote about performance.**

Questions to ask

- What is the device floor, for example a $50-100 Android phone with 1-2GB of RAM, versus assuming a flagship device?
- What is the realistic network condition, for example 3G with high latency and packet loss rather than consistent broadband?
- Is data cost a concern for the target user, which would push toward minimizing total bytes transferred, not just initial load time?
- Does the form need to work with intermittent connectivity, where a user might lose signal mid-submission?

How the answers change the design

- A low RAM/CPU floor pushes toward a minimal JavaScript footprint, favoring server-rendered HTML with small islands of interactivity over a heavy client-side framework bundle.
- High latency and packet loss push toward aggressive caching, resumable form submission, and saving form progress locally (for example in `localStorage` or IndexedDB) so a dropped connection does not lose the user's work.
- Data cost sensitivity pushes toward avoiding auto-loading images or non-essential assets, and compressing every payload aggressively.

Interview Answer:
"I wouldn't assume a modern smartphone on WiFi just because that's what's on my desk. I'd ask what the actual device and network floor looks like for this population, and for a rural India rollout, I'd expect low-RAM Android devices on unreliable 3G with real data cost sensitivity. That changes my architecture, I'd lean toward server-rendered HTML with minimal JavaScript instead of a heavy SPA bundle, and I'd persist form progress locally so a dropped connection mid-submission doesn't wipe out the user's work. Getting that constraint wrong doesn't just mean a slower app, it means the app doesn't work for the people it was built for."

Q5. [Junior] You're asked to design a checkout flow, and accessibility never comes up in your answer. What should you cover if it's raised, and what does a strong answer on accessibility for a checkout flow actually include?

Answer:

Accessibility is a non-functional requirement (a quality attribute of the system rather than a specific feature) that is easy to skip entirely if nobody asks, so it is worth raising directly. **Treat accessibility compliance level as something to clarify upfront, the same way you'd clarify browser support, because it determines real engineering work like focus management, color contrast, and screen-reader labeling, not something to bolt on at the end.**

Questions to ask or raise

- Is there a specific compliance target, for example WCAG 2.1 AA (Web Content Accessibility Guidelines, level AA, a widely adopted accessibility standard), and is it a legal requirement in this market (for example ADA in the US or the EN 301 549 standard in the EU)?
- Does this checkout flow need to support keyboard-only navigation and screen readers, given that payment flows are high-stakes and commonly audited?
- Are there known user segments that require specific support, for example low-vision users needing sufficient color contrast on the "Place Order" button?

What it changes in the design

- Every custom interactive element (a custom dropdown for card expiration month, a custom modal for order confirmation) needs a documented plan for keyboard focus and ARIA (Accessible Rich Internet Applications) labeling, not just a visual design.
- Error messages need to be programmatically associated with their fields so a screen reader announces "card number is invalid" rather than leaving a sighted-only red border as the only signal.

Interview Answer:

"If I design a checkout flow and realize I never brought up accessibility, I'd stop and ask directly what compliance level we're targeting, because for something like checkout that's often a legal requirement, not a nice-to-have. A strong answer names something concrete, like WCAG 2.1 AA, and immediately connects it to real design decisions, like ensuring every custom form control is keyboard-operable and that error states are announced to screen readers, not just shown as a red border. That shows I see accessibility as a requirement to design for from the start, not a checklist item to retrofit right before launch."

Q6. [Junior] You're designing a settings page for a product that's about to launch in Japan, Germany, and Saudi Arabia. What internationalization questions do you ask before you start, and how do the answers affect your component design?

Answer:

Internationalization (i18n, designing software to support multiple languages and regions) is another requirement that is invisible until you ask about it directly, and it has real layout and architecture consequences. **The specific locales targeted determine concrete constraints like right-to-left text direction, text expansion, and date/currency formatting, and those constraints need to shape the component architecture from day one rather than being patched in after the English version ships.**

Questions to ask

- Which locales are launching, and does that include a right-to-left (RTL) language like Arabic, which is relevant here because of Saudi Arabia?
- How much can translated text expand compared to English, since German strings are commonly 30-40% longer and can break fixed-width buttons or labels?
- Do dates, currency, and number formats need locale-specific formatting, for example DD.MM.YYYY in Germany versus MM/DD/YYYY defaults?
- Is translation handled at build time or fetched at runtime, which affects whether you need a loading state for translated strings?

How it changes the design

- Supporting Arabic means the layout system needs to support mirroring (using logical CSS properties like `margin-inline-start` instead of `margin-left`), which is far easier to build in from the start than retrofit.

- Buttons and labels need flexible widths rather than fixed pixel widths, since a "Save Changes" button in German can be significantly longer than in English.

Interview Answer:

"Before I touch layout, I'd ask which locales are actually launching, because Saudi Arabia means I need right-to-left support, which is a layout decision, not just a translation file swap. I'd also ask about text expansion, since German strings run noticeably longer than English and can break a button that was sized for the English word. Practically, that pushes me toward using logical CSS properties instead of left/right values and avoiding fixed-width text containers from the start. Retrofitting RTL support after the fact touches nearly every component, so it's much cheaper to design for it upfront."

Q7. [Junior] In a 45-minute interview, you're asked to "design a notifications feature," and your instinct is to cover push notifications, in-app notifications, email digests, and a notification settings page all at once. How should you scope this down, and what should you cut?

Answer:

This is a scoping failure, trying to design everything means designing nothing in enough depth to demonstrate real skill. **In a time-boxed interview, explicitly narrow an open-ended prompt to one flow you can design in real depth, state what you're deliberately excluding, and only expand if time allows.**

How to scope it down

- Ask the interviewer directly: "Should I focus on in-app real-time notifications, or the settings and preferences side, or the delivery pipeline across channels?"
- Pick one, for example in-app real-time notifications (a bell icon with a live-updating badge count), and say out loud what is out of scope: "I'm going to assume push and email are handled by other teams and focus on the in-app experience."
- Within that narrowed scope, still budget time: roughly 5 minutes on requirements, 15-20 on component and data architecture, 10-15 on a specific hard part (for example ordering and deduplication of real-time events), and remaining time for follow-ups.

Why this matters

- Interviewers grade depth and judgment, not breadth. Designing one flow well, including edge cases like reconnection after a dropped WebSocket connection, shows more than sketching four flows shallowly.
- Explicitly stating what's out of scope also signals maturity, since it shows you know real systems have boundaries and that you're not just improvising.

Interview Answer:

"Notifications is a prompt that can expand infinitely, so the first thing I'd do is narrow it out loud and get buy-in. I'd say something like, 'there's push, email, in-app, and preferences here, I want to focus on the in-app real-time piece and treat the others as out of scope unless you want me to go there.' Then I'd budget my remaining time, a few minutes on requirements, the bulk on the component and data flow, and time reserved for a hard edge case like handling a dropped connection. That way I go deep enough on one thing to actually demonstrate judgment, instead of shallowly touching four things and running out of time."

Q8. [Mid] A product manager says "we want this web app to feel native." That's not an engineering spec. How do you push back and turn it into concrete, buildable requirements?

Answer:

"Feels native" is shorthand for a bundle of specific, checkable behaviors that users associate with native apps, and your job is to unpack which of those actually matter for this product. **Turning "feels native" into requirements means naming specific targets, such as 60 frames-per-second animations, sub-100-millisecond response to touch input, native-style gesture handling, and an app shell that appears instantly even before data loads, rather than accepting the phrase at face value.**

Questions to ask the PM

- Which specific native behaviors matter most: smooth scrolling and transitions, offline availability, home-screen installability, gesture support like swipe-to-dismiss, or all of the above?
- Is this a PWA (Progressive Web App, a web app that can be installed and behave like a native app) requirement, or purely a look-and-feel request?

Turning it into requirements

- Animation and transitions: target a consistent 60fps, meaning each frame has about 16.7 milliseconds of work, and avoid animating properties that trigger layout recalculation.
- Input responsiveness: touch and tap interactions should visually respond within 100 milliseconds, matching the threshold where input feels instantaneous to a human.
- App shell: the static layout (navigation, headers) should render immediately from cache, with data populating after, rather than a blank white screen during every load.
- Gestures: if swipe navigation is expected, that is a specific engineering task, not a CSS tweak, and should be scoped and estimated as such.

Interview Answer:
"'Feels native' isn't something I can build against directly, so I'd ask the PM which specific native behaviors they mean, things like buttery scrolling, instant touch feedback, offline support, or installability. Then I'd translate whichever of those matter into numbers: 60fps for animations, sub-100-millisecond feedback on taps, and an app shell that renders instantly from cache instead of showing a blank screen on every load. If gestures like swipe-to-dismiss are in scope, I'd call that out as its own scoped task, since that's meaningfully more engineering work than a visual tweak. That conversation usually surfaces that the PM cares about two or three specific things, not literally every native behavior."

Q9. [Mid] Your team is scoping a new marketing site and, separately, a new internal SPA dashboard. Would you set the same JavaScript bundle size budget for both? How would you decide the numbers?

Answer:

No, and the reasoning is a direct extension of who the audience is and what device and network conditions they're on. **Bundle size budgets should be set based on the specific audience's device and network profile and the cost of a slow first load to the business, not a single company-wide number applied everywhere.**

Best Choice

Set a strict, low budget (for example under 150-200KB of JavaScript, gzipped, for the initial route) for the public marketing site, and a looser budget (for example under 500KB-1MB for the initial route, with route-based code splitting) for the internal dashboard.

Why?

- The marketing site is the first and possibly only interaction an anonymous visitor has with the company, often on a mobile connection, and every extra second of load time measurably increases bounce rate, so the cost of a heavy bundle is direct lost revenue.
- The internal dashboard has a known, captive audience on reliable corporate networks who will use the tool daily regardless of a slightly slower initial load, so the cost-benefit favors shipping richer functionality and developer velocity over shaving kilobytes.
- In both cases, the number should come from a real target device and network profile, for example "loads in under 3 seconds on a mid-tier Android on throttled 4G" for the marketing site, translated backward into a KB budget, rather than picking a round number arbitrarily.

Interview Answer:

"I wouldn't use the same budget, because the two have very different costs when they're slow. For the marketing site, I'd set a tight budget, something like under 200KB of JavaScript for the initial route, because that's often an anonymous mobile visitor's first impression and a slow load directly costs conversions. For the internal dashboard, I'd allow a much larger budget with route-based code splitting, since the users are known, on reliable networks, and value richer functionality more than shaving off a few hundred milliseconds. I'd derive the actual number by picking a target device and network condition and working backward, rather than picking a number that sounds good."

Q10. [Mid] You're designing the frontend for a live-commenting feature during a major sporting event. How do you estimate concurrent users and device capability, and how do those estimates change your architecture?

Answer:

This question is testing whether you treat "real-time" as a single feature or actually reason about scale and device floor before choosing an architecture. **Estimating realistic concurrent load and the lowest-capability device you must support should directly decide the transport mechanism (WebSockets versus polling versus server-sent events) and how aggressively the client throttles and batches incoming updates.**

Questions to ask, and how to estimate

- What's the expected peak concurrency, for example "500,000 concurrent viewers during a World Cup final" versus "2,000 concurrent viewers for a regional match"? Ask the interviewer or make a reasonable order-of-magnitude assumption out loud.
- What's the expected message rate per user, since a comment feed with thousands of viewers might produce dozens of new comments per second, which is very different from a single live score update every few minutes.
- What's the device floor, since a mobile-heavy sports audience skews toward mid-range Android devices, not developer laptops.

How the estimate changes the design

- At high message rates, rendering every incoming comment immediately can overwhelm both the DOM and the CPU on a low-end device, so the client should batch and throttle updates, for example rendering at most every 250-500 milliseconds instead of on every message.

- At very high concurrency, a full duplex connection per client (WebSockets) has real server fan-out cost, so it is worth clarifying whether the backend can support that at this scale, versus a lighter approach like periodic polling or server-sent events for read-heavy, low-interactivity data like a live score.
- A low-end device floor pushes toward capping the number of DOM nodes kept in memory, for example virtualizing the comment list instead of appending every comment forever.

Interview Answer:

"Before picking WebSockets versus polling, I want a rough sense of scale, so I'd ask or estimate peak concurrency, something like hundreds of thousands of viewers for a major final, and a realistic message rate, since a comment feed under load could produce many messages per second. That estimate directly shapes the client design, I'd throttle and batch rendering so a low-end phone isn't trying to reflow the DOM on every single incoming comment, and I'd virtualize the list so memory doesn't grow unbounded over a two-hour match. It also shapes the transport conversation with the backend team, since full-duplex connections at that scale have real fan-out costs worth flagging early instead of assuming they're free."

Q11. [Mid] Your team needs to decide on a rendering strategy, server-side rendering versus a fully client-rendered single-page app, for two different projects: a new marketing site and a new internal admin tool. How does gathering requirements around SEO change that decision for each?

Answer:

SEO is one of the clearest examples of a requirement that should be resolved before architecture, not after, because switching rendering strategies later is expensive. **Whether search engines need to index the content is one of the first questions to ask, since it directly decides between server-side rendering or static generation for public content and justifies a simpler client-rendered SPA for anything that will never be crawled.**

Best Choice

Server-side rendering (SSR) or static site generation (SSG, pre-building HTML at deploy time) for the marketing site; a standard client-rendered SPA for the internal admin tool.

Why?

- The marketing site's entire value depends on being discoverable through search, so content needs to exist as real HTML that search crawlers and social media link previews can read without executing JavaScript first.
- The admin tool sits behind authentication and is never indexed, so there is no SEO requirement to satisfy, meaning the extra infrastructure complexity of SSR (server rendering pipelines, hydration) buys nothing and a simpler CSR (client-side rendering) SPA is the better trade.
- Confirming this with stakeholders early, "does this page need to be found on Google" and "is this ever publicly shared as a link," avoids a costly mid-project rendering-strategy migration.

Interview Answer:

"The deciding question is simply whether the content needs to be found by search engines or shared as a link with a preview, and the answer is completely different for these two projects. For the marketing site, yes, so I'd push for server-side rendering or static generation so crawlers and social previews see real content immediately. For the internal admin tool, it's behind a login and never indexed, so there's no SEO requirement at all, and I'd rather take the simpler, faster-to-build client-rendered SPA and not pay for SSR

infrastructure we don't need. Getting that question answered before architecture is set saves a very expensive rendering migration later."

Q12. [Mid] A requirements doc says "support the last two versions of all major browsers." How do you translate that single sentence into actual engineering decisions your team can act on?

Answer:

That sentence sounds precise but is actually still ambiguous, and your job is to press on it until it becomes a testable matrix and a concrete build configuration. **A browser support statement needs to be turned into a specific version matrix, a decision about polyfills and build targets, and a testing strategy, otherwise "support" ends up meaning different things to different engineers on the same team.**

Questions to ask

- Does "major browsers" include Safari on iOS specifically, since Safari has historically lagged on certain web platform features and iOS Safari is the only browser engine allowed on iPhones?
- Does this include older corporate environments that might still be on an outdated browser for policy reasons, which is common for enterprise/internal tools?
- What does "support" mean exactly: pixel-perfect parity, or "functional but visually degraded is acceptable"?

Turning it into engineering decisions

- Pin the exact matrix, for example "Chrome and Edge last 2 versions, Firefox last 2 versions, Safari last 2 versions including iOS Safari," and put it in the project's `browserslist` config so build tooling (autoprefixer, Babel/TypeScript compilation targets) enforces it automatically.
- Decide the polyfill strategy: ship polyfills conditionally based on feature detection rather than to every user, to avoid penalizing modern-browser users with unnecessary bytes.
- Decide the testing strategy: which browsers get automated visual regression tests versus manual smoke testing before each release.

Interview Answer:
"That sentence needs to become a concrete matrix before it's actionable, so I'd pin down exact browsers and versions, including whether iOS Safari is explicitly included, since that's often the trickiest one. I'd put that matrix directly into the build config, like a browserslist setting, so compilation targets and autoprefixing are enforced automatically instead of relying on developers remembering it. I'd also decide upfront whether unsupported features get polyfilled for everyone or only loaded conditionally for browsers that need them, since shipping polyfills to every user penalizes the majority who don't need them. And I'd agree on what 'support' means, full parity or just functional, since that changes how much QA effort this actually costs."

Q13. [Mid] You're designing the order-tracking screen for a food delivery app. The product requirement says the app "must work offline" and also "must always show the driver's current location." Those two requirements are in direct conflict. How do you resolve it?

Answer:

This is a classic case of two reasonable-sounding non-functional requirements that cannot both be fully true at the same time, and you need to recognize the conflict and propose a resolution instead of silently picking

one. **When requirements conflict, like offline availability versus always-current data, the right move is to scope the requirement per piece of data rather than treating the whole screen as one monolithic requirement, since order history can be fully offline-capable while live driver location fundamentally cannot be.**

Best Choice

Split the screen's data into tiers: static/recent data (order details, restaurant info, past status updates) is cached and available offline; live data (driver's current GPS position) is clearly marked as requiring connectivity, with an explicit stale-state UI when the connection drops.

Why?

- Order details, items, and price do not change once placed, so caching them locally (for example via a service worker and IndexedDB) genuinely satisfies "works offline" for that data with zero downside.
- A driver's live location is, by definition, a real-time value that cannot be meaningfully shown "offline," so promising both literally is not achievable; the honest resolution is showing the last known location with a clear timestamp and a "reconnecting" indicator rather than either hiding the feature or silently showing stale data as if it were live.
- Surfacing this conflict back to the product manager with a proposed resolution, rather than quietly deciding it alone, is itself a signal of good judgment, since it's a product trade-off as much as a technical one.

Interview Answer:

"I'd flag that these two requirements can't both be fully true for the same piece of data, and then split the problem instead of picking one side. Order details and status history are static once they exist, so those can genuinely be cached and work offline. The driver's live GPS position is inherently real-time, so instead of pretending it works offline, I'd show the last known location with a clear timestamp and a visible 'reconnecting' state when we lose the connection. I'd also bring that distinction back to the PM explicitly, since silently choosing one interpretation of a conflicting requirement is worse than surfacing the trade-off and agreeing on it together."

Q14. [Mid] You have 40 minutes to design the frontend for "a food delivery app like DoorDash." What do you explicitly cut from scope, and how do you decide what to keep?

Answer:

A prompt like this covers browsing, search, cart, checkout, order tracking, ratings, and a driver-side app, far more than any single interview can meaningfully cover. **Pick the flow with the most interesting design tension, typically live order tracking or the browse-and-cart experience, state everything else as explicitly out of scope, and justify the choice based on what best demonstrates your frontend design judgment.**

What to cut and why

- Cut anything that's mostly backend or business logic with a thin frontend, for example the restaurant-side order management app, unless the interviewer redirects there.
- Cut authentication, payment processing details, and account settings, calling them out as "standard, assume they exist" rather than designing them from scratch.
- Cut multi-market/i18n concerns unless the interviewer specifically raises them, since they add breadth without adding architectural depth for this particular prompt.

What to keep and why

- Keep the live order-tracking screen or the browse-to-cart-to-checkout flow, since both have real frontend design tension: real-time updates, offline/stale-data handling, optimistic UI on cart actions, and state synchronization across screens.
- State the chosen scope and cuts out loud in one sentence before starting, for example: "I'll focus on the customer-facing order tracking experience after checkout, and treat auth, payments, and the restaurant dashboard as out of scope."

Interview Answer:

"Given 40 minutes, I can't cover browsing, checkout, tracking, and the driver app in any real depth, so I'd say upfront which one I'm focusing on and why. I'd pick order tracking, since it has the most interesting frontend problems, real-time location updates, offline and stale-data handling, and state that auth, payment processing, and the restaurant-side dashboard are out of scope for this session. That single sentence upfront means the interviewer and I are aligned on what 'done' looks like, and I spend the full 40 minutes going deep on one flow instead of shallowly sketching four."

Q15. [Senior] You're given the prompt "design the frontend for Twitter" with no other context, at a senior level, and your instinct is to start designing immediately without asking a single clarifying question. Is that a real problem specifically because you're senior? What should you do instead?

Answer:

At the senior level, yes, this is a meaningful gap, since driving the requirements conversation is exactly the skill that differentiates senior engineers from more junior ones in ambiguous situations. **You're expected to actively drive the requirements-gathering conversation the way you would with a real product manager, proposing specific scale numbers and constraints and asking the interviewer to confirm or correct them, rather than waiting passively to be told everything.**

What to do differently at the senior level

- Propose concrete assumptions rather than open-ended questions: "I'm going to assume this is a global-scale timeline feature, on the order of hundreds of millions of daily active users, mixed mobile and desktop, and design for that unless you tell me otherwise," rather than just asking "what's the scale?" and waiting.
- Identify the two or three requirements that most change the architecture and prioritize clarifying those first, for example real-time delivery of new tweets versus periodic refresh, rather than asking a long unprioritized checklist of questions.
- Notice and name second-order constraints unprompted, for example "since this needs to support very high fan-out when a popular account posts, that affects how aggressively we can do real-time push versus pull," showing they understand why the requirement matters, not just that they should ask about it.

Why this matters

- A senior engineer in a real job usually does not get a fully specified spec, they have to extract it from an underspecified ask, and this exercise is a direct proxy for that skill.
- Diving straight into a component tree without confirming scale or scope usually produces a technically fine but contextually wrong design, and worse, does it confidently.

Interview Answer:

"At the senior level, I'd actually want to catch myself if I skipped straight to design, because handling ambiguity like this is a core part of the job in real product work. What matters is proactively proposing assumptions, something like 'I'll assume global scale, hundreds of millions of daily users, and design around a real-time-ish timeline, correct me if that's wrong,' rather than passively asking open questions and waiting to be spoon-fed constraints. That proactive framing also usually surfaces the two or three decisions that most affect the architecture, like how real-time the feed actually needs to be, early enough to shape the whole design instead of becoming a surprise halfway through."

Q16. [Senior] You're asked to design a comment thread component, similar to Reddit or YouTube comments. What should you explicitly decide is in scope versus out of scope before writing any code or drawing the component tree?

Answer:

A comment thread sounds like a single component but actually bundles several independent, non-trivial design decisions, nested replies, real-time updates, moderation, and sorting, any one of which could fill the whole interview on its own. **Explicitly state which of nesting depth, real-time updates, moderation/reporting, and sort order you're designing for, since silently assuming the simplest version of each without saying so hides the fact that harder versions exist and were never considered.**

Reasonable in-scope defaults for a component design question

- A fixed or capped nesting depth (for example, replies nest up to 3 levels, then flatten), since unbounded nesting has real rendering and data-modeling implications worth naming even if not fully designed.
- Optimistic UI for posting a new comment, since that's a common, testable frontend pattern (show the comment immediately, reconcile or roll back based on the server response).
- One sort mode fully designed (for example "Top" using a score), with other modes mentioned as an extension.

Reasonable to explicitly push out of scope

- Real-time updates from other users' new comments arriving live, unless the interviewer wants to go there, since that adds a whole second design problem (WebSocket or polling-based sync, merge/ordering logic).
- Moderation tooling and abuse reporting, which is a legitimate related system but a different one.
- Rich text or media embedding inside comments, which is a content-model question more than a component-architecture one.

Interview Answer:

"Before I design anything, I'd name the specific decisions bundled into 'comment thread' and say which ones I'm actually solving. I'd design for a capped nesting depth, say three levels before flattening, optimistic posting so a new comment appears instantly and reconciles with the server response, and one sort mode designed in full, like 'Top,' with others mentioned as extensions. I'd explicitly put live real-time updates from other users and moderation tooling out of scope unless asked, since each of those is really its own design problem. Saying that out loud upfront means the interviewer knows exactly what I did and didn't consider, instead of assuming I either ignored those cases or ran out of time."

Q17. [Senior] Compare designing the frontend for a typical CRUD-style admin form against designing the frontend for a Figma-like collaborative design canvas. How

should the requirements-gathering conversation differ, and how early should that difference affect your architecture?

Answer:

These two are different enough in their fundamental interaction model that requirements gathering should surface the difference immediately, and it should shape the architecture from the very first decision, not emerge later as a refactor. **A highly interactive, canvas-style application has fundamentally different requirements around rendering performance, input latency, and state modeling than a typical CRUD form, and recognizing that early should push you toward a completely different rendering and state architecture from day one rather than starting with a standard component-per-DOM-element approach and hitting a wall later.**

Best Choice

For the CRUD admin form: standard component-per-field architecture using the framework's normal reconciliation (React's virtual DOM diffing, for example), backed by a form library and a request-driven data layer.

For the Figma-like canvas: a custom rendering layer (Canvas API or WebGL) driven by a dedicated, often mutable, high-frequency state store outside the framework's normal render cycle, with the framework used only for surrounding UI chrome like panels and toolbars.

Why?

- A CRUD form has low-frequency updates (a user types in one field, submits once), so the standard virtual DOM diffing approach is more than fast enough and gives you accessibility and framework conveniences for free.
- A design canvas has extremely high-frequency updates (every pixel of mouse movement while dragging a shape, potentially 60+ times per second, times however many collaborators are active), and running that through a full component re-render and diffing cycle causes visible input lag; direct canvas or WebGL rendering with a purpose-built state layer avoids that overhead entirely.
- The requirements questions that surface this early are about interaction frequency and latency tolerance: "how often does the visual state change, and what's the acceptable input-to-pixel latency," and the answer for a canvas tool ("every frame, must feel instantaneous") is different enough from a form ("on submit, a few hundred milliseconds is fine") that it should be asked before any architecture is chosen.

Interview Answer:

"The key requirement question here is interaction frequency and latency tolerance, and the answers are worlds apart for these two apps. A CRUD form updates rarely, on submit or on blur, so the framework's normal rendering and diffing is more than fast enough, and I'd just use standard components and a form library. A Figma-like canvas has to redraw on every pixel of drag movement, potentially dozens of times a second across multiple collaborators, and pushing that through a full component re-render cycle causes visible lag, so I'd reach for direct canvas or WebGL rendering with its own dedicated state layer, keeping the framework only for surrounding panels and toolbars. That's a decision I want to make in the first few minutes of requirements gathering, not discover after I've already built the wrong foundation."

Q18. [Senior] Legal and compliance says every user action on a financial dashboard needs to be logged for audit purposes, but your performance budget for that

dashboard is already tight. How do you prioritize when a compliance requirement conflicts with a performance requirement?

Answer:

This is a case where one requirement, legal compliance, is typically non-negotiable, while the other, performance, has more flexibility in how it's achieved, and the resolution is to find an implementation that satisfies both rather than trading one off against the other. **When a compliance requirement is truly mandatory, the right move is not to weigh it against performance as if they're equally negotiable, but to change how the requirement is implemented, for example moving audit logging off the critical rendering and interaction path, so it stops competing with the performance budget at all.**

Best Choice

Keep full audit logging, but implement it asynchronously and off the main thread: batch user action events and send them via a non-blocking mechanism (for example `navigator.sendBeacon`, or a Web Worker that batches and flushes events), rather than synchronously logging on every interaction in a way that blocks rendering.

Why?

- Audit/compliance requirements are frequently a legal obligation, not a product preference, so they're rarely truly optional in the way "make it feel a bit snappier" might be, meaning the right response is "how do we hit both," not "which one wins."
- The performance cost of audit logging usually comes specifically from doing it synchronously on the interaction path, for example logging inline inside a click handler in a way that delays the next paint. Moving that same logging to a background mechanism removes the performance cost without removing a single logged event.
- This is also a case worth surfacing explicitly to both stakeholders: "we can satisfy full audit logging and our performance budget, but it requires implementing logging as a background, non-blocking process, here's the plan," which shows the trade-off was actively managed rather than silently resolved.

Interview Answer:

"I wouldn't treat this as a straight trade-off, because compliance requirements like audit logging are usually non-negotiable, so the real question is how to satisfy both, not which one to compromise. In practice, the performance cost almost always comes from doing the logging synchronously and blocking the interaction, so I'd move it off the critical path, batching events and sending them with something like sendBeacon or through a Web Worker so logging never delays a paint or a click response. That gets us full compliance without touching the performance budget, and I'd bring that plan back to both legal and the performance-conscious stakeholders explicitly, so everyone sees the conflict was solved rather than quietly traded away."

Q19. [Senior] You're scoping the frontend for a Google-Docs-style real-time collaborative editor that needs to work for teams spread across offices in the US, India, and Brazil. Walk me through how you'd estimate concurrent editors per document, device capability floor, and network variability, and how those estimates shape your architecture.

Answer:

This question is testing whether you can produce reasonable, justified estimates for scale and constraints rather than either skipping the estimation step or asking for exact numbers you'll never get in a real job. **Realistic estimates for concurrent editors per document, the lowest-capability device in the target regions, and the network variability across those regions should directly decide the conflict-resolution strategy, the sync transport, and how conservatively the client buffers and retries, so producing those estimates is itself part of the design work, not a preamble to it.**

How to reason through the estimates

- Concurrent editors per document: for a typical collaborative document, most documents have very few simultaneous editors, likely 2-5, but the design still needs to handle an outlier case like an all-hands document with 50+ simultaneous editors, since that tail case is what breaks a naive design.
- Device floor: offices across the US, India, and Brazil suggests a wide range of hardware, so design for a mid-range laptop or Chromebook-class device as the floor, not the newest MacBook.
- Network variability: assume meaningfully different baseline latency and reliability across regions, for example higher latency and more frequent brief disconnects for the India and Brazil offices relative to a US office near the primary data center, which matters a lot for a real-time system.

How the estimates shape the architecture

- Low-single-digit typical concurrency with an occasional high-concurrency outlier pushes toward a conflict-resolution approach that scales reasonably at both ends, for example CRDTs (Conflict-free Replicated Data Types, a data structure designed so concurrent edits merge automatically without conflicts) or operational transformation, rather than a naive last-write-wins approach that falls apart under real concurrent editing.
- A mid-range device floor pushes toward keeping expensive operations, like recalculating document state from a long history of operations, off the main thread, for example in a Web Worker.
- Regional network variability pushes toward an offline-tolerant client that buffers local edits and reconciles on reconnect, rather than an architecture that assumes a stable, low-latency connection at all times.

Interview Answer:

"I'd reason through rough numbers out loud rather than skip the estimate. Most documents probably have a handful of simultaneous editors, but I'd design assuming an outlier case, something like fifty people in a live all-hands document, since that's what actually breaks a naive design. Given offices across the US, India, and Brazil, I'd assume a mid-range laptop as my device floor and meaningfully different network latency and reliability by region rather than uniform low-latency WiFi. Those estimates directly push me toward a CRDT-based conflict resolution approach that holds up at both low and high concurrency, keeping expensive recomputation off the main thread for weaker devices, and building the client to buffer edits locally and reconcile on reconnect, since assuming a stable connection everywhere would be wrong for at least two of those three regions."

Q20. [Senior] Before diving into architecture on a system design interview, you're asked, at a senior level, to talk through your requirements-gathering checklist and how you'd prioritize if there's only time to nail down half of it. What should that checklist and prioritization look like?

Answer:

You should have an internalized, repeatable checklist for frontend non-functional requirements, and just as importantly, a clear method for prioritizing under real time pressure rather than treating every item as

equally urgent. **A strong checklist covers audience and scale, device and network floor, performance targets, accessibility and compliance level, browser support matrix, SEO needs, and internationalization, and under time pressure you should prioritize whichever items most change the architecture, not whichever are easiest to ask about.**

The checklist

- Audience and scale: who uses this, and roughly how many, since that changes almost everything downstream.
- Device and network floor: what's the weakest realistic device and connection this must work on.
- Performance targets: specific Core Web Vitals and interaction latency numbers, not "make it fast."
- Accessibility compliance level: is there a specific standard (like WCAG 2.1 AA) or legal obligation.
- Browser support matrix: which browsers and versions, translated into a concrete build/testing target.
- SEO needs: does this content need to be indexed or shared, which decides the rendering strategy.
- Internationalization: which locales, including any that need right-to-left support or heavy text expansion.
- Offline/connectivity expectations: does any part of this need to work without a network connection, and how does that interact with data freshness needs.

How to prioritize under time pressure

- Rank items by how much they change the architecture if the answer flips, not by how quick they are to ask. Audience/scale, device/network floor, and SEO needs (which decides rendering strategy) usually rank highest, since getting those wrong means redesigning the foundation later.
- Items like exact browser version numbers or specific WCAG success criteria can often be reasonably assumed ("I'll assume WCAG 2.1 AA and the standard modern-browser matrix unless told otherwise") and revisited later, since being wrong about them costs a targeted fix, not a full redesign.

Interview Answer:

"My checklist covers audience and scale, device and network floor, concrete performance targets, accessibility compliance level, browser support, SEO needs, i18n, and offline expectations, essentially the frontend equivalent of QPS and read/write ratios on the backend side. If I only have time for half of it, I prioritize whichever answers would most change the architecture if they flipped, so audience and scale, device and network floor, and whether this needs to be indexed by search engines usually come first, since getting those wrong means rebuilding the foundation later. Things like the exact browser version matrix or the specific accessibility success criteria I'm often comfortable stating as a reasonable default assumption and moving on, since being wrong there costs a targeted fix rather than a redesign. That prioritization itself, knowing which unknowns are load-bearing and which aren't, is what I'd expect from someone operating at a senior level."

If you remember nothing else from this chapter: *the best frontend architecture for the wrong audience is still the wrong architecture.*

Chapter 5: Rendering Architecture: CSR, SSR, SSG, ISR & Hydration Trade-offs

Almost every frontend architecture decision starts with "where does this page get rendered, and when." Being able to recite that "SSR is server-side rendering" won't get you far; what actually matters is whether you can look at a real product (a dashboard, a checkout flow, a blog) and reason about SEO needs, time-to-interactive, server cost, caching, and personalization together, then land on a defensible choice. Use these questions to practice treating rendering strategy as an engineering trade-off tied to a specific product's constraints, rather than a single "best practice" you apply everywhere regardless of context. The strongest answers also volunteer the failure mode of their own recommendation before being asked.

War Story: *A marketing site rebuilt as a pure client-rendered app saw organic search traffic drop by half within a month, because the migration shipped before anyone checked whether the new app was actually crawlable.*

Avoid: picking one rendering strategy for the whole app without considering that different routes have different needs

Aim For: reasoning about rendering strategy per route type, not per app

Q1. [Junior] We're building a marketing landing page for a new product launch, mostly static copy, a hero image, a signup form, and it needs to rank well on Google and load fast on mobile. Would you build this with client-side rendering (CSR, where the browser downloads a mostly-empty HTML shell and a JS bundle, then builds the page in JavaScript), server-side rendering (SSR, where a server builds full HTML per request), or static site generation (SSG, where HTML is pre-built once at build time and served as plain files from a CDN, a content delivery network of geographically distributed caching servers)?

Answer:

This page has no per-user data and doesn't change between visitors, so **SSG is the right default here because it gives every visitor pre-built HTML from the nearest CDN edge, which means the fastest possible load and full content for search crawlers with zero server compute per request.**

Best Choice

- SSG. Build the page once at deploy time (or when a marketer edits copy in the CMS) and serve static files.

Why?

- SEO: crawlers see complete HTML immediately, no JavaScript execution required, which matters because Google's indexing budget and rendering queue can delay or degrade CSR-only pages.
- Speed: a static file served from a CDN edge node near the visitor beats a server round-trip or a client-side render every time, especially on a throttled mobile connection.

- Cost: no per-request server rendering means the page can absorb a traffic spike (a launch going viral) without scaling a server fleet.
- CSR would hurt both SEO and first paint here for no benefit, since there's no interactivity complex enough to justify shipping the page empty and filling it in with JS. SSR would work but adds a server render on every request for content that never changes per visitor, which is pure waste.

Interview Answer:

"For a static marketing page with no personalization, I'd reach for SSG every time. There's nothing here that needs to be computed per request, so pre-building the HTML once and serving it from a CDN gives me the best Lighthouse scores and the best SEO with basically zero infrastructure to run. SSR would technically work but it's paying a server-rendering cost for content that's identical for every visitor, and CSR would tank both my SEO and my time-to-first-paint on mobile. I'd only reconsider if the page needed live, per-visitor content like a countdown tied to their timezone or region-specific pricing."

Q2. [Junior] You're building the blog for a company's engineering team. Posts are written in a CMS, published a few times a week, and never change once published. Would you serve this with SSG or SSR, and how would new posts actually get published without a full site rebuild each time?

Answer:

SSG is the right fit since blog content is identical for every reader and rarely changes, but you don't want every single post publish to trigger a full site rebuild once you have hundreds of posts, so most frameworks let you pair SSG with on-demand or incremental regeneration triggered by a CMS webhook.

Best Choice

- SSG, with incremental regeneration: build the shell once, then regenerate only the specific new or edited post's page when the CMS fires a "published" webhook.

Why?

- Content-heavy blogs are the textbook SSG case: no personalization, low change frequency, huge SEO value in having fully-formed HTML for every article.
- A naive "rebuild everything on every publish" pipeline degrades badly at scale (hundreds or thousands of posts), turning a 10-second publish into a 10-minute build; regenerating just the affected page (or a small batch) avoids that.
- SSR would work correctness-wise but adds a server render on every page view for content that's the same for every visitor and changes maybe three times a week, which is unnecessary compute and unnecessary latency versus a CDN-cached static file.

Interview Answer:

"I'd go with SSG for a blog like this. It's the classic case: content is public, identical for everyone, and changes infrequently. The one wrinkle I'd flag is publishing at scale. If every new post triggers a full site rebuild, that gets slow once you have a few hundred posts, so I'd want the CMS to fire a webhook on publish that regenerates just that one page, or a small batch, instead of the whole site. That gets me the SEO and performance benefits of static HTML without a build pipeline that falls over as content grows."

Q3. [Junior] Your team owns an internal analytics dashboard used only by employees behind a login. It's data-dense, highly interactive (filters, drag-to-resize

charts, live updates), and never needs to show up in search results. Would you push for SSR here, or is CSR the better fit?

Answer:

CSR is the right call for an internal, authenticated, highly interactive tool because there's no SEO requirement to satisfy and the page's value comes entirely from client-side interactivity, so paying for server rendering on every request buys you nothing.

Best Choice

- CSR: ship a lean HTML shell, let the browser fetch data and render the dashboard with JavaScript.

Why?

- SEO is irrelevant behind a login wall, which removes the single biggest reason to reach for SSR or SSG in the first place.
- The page is dominated by client-side state (filter selections, chart interactions, live data polling or websockets) that has to run in the browser regardless of how the first render happened, so SSR's main benefit (faster first paint of meaningful content) is diluted by the fact that the "meaningful content" here is inherently interactive and data-heavy.
- Building and operating SSR infrastructure (server rendering, hydration, cache invalidation) for a page whose users are a few hundred internal employees is added operational complexity with no measurable payoff.
- You should still flag the exception: if this dashboard is loaded dozens of times a day by people on slow VPNs or the initial data fetch is huge and slow, a thin SSR pass that pre-renders the shell and first data batch can meaningfully cut perceived load time, so it's a real trade-off, not an absolute rule.

Interview Answer:
"For an internal, logged-in dashboard, I'd default to CSR. There's no SEO to worry about, and the whole point of the page is heavy client-side interactivity, filtering, live updates, charts you drag around, which has to run in JavaScript no matter what rendered the first HTML. Standing up SSR just adds server cost and hydration complexity for a login-gated audience of employees. The one time I'd reconsider is if the initial data payload is huge and users are complaining about a blank screen on load. In that case I'd look at server-prefetching the first screen of data rather than going full SSR."

Q4. [Junior] You're building the product detail page for an e-commerce site. It needs to rank on Google for the product name, and price and stock level need to reflect current inventory, which can change several times an hour during a sale. Would you use SSG, SSR, or ISR (incremental static regeneration, where a statically generated page can be automatically regenerated after deploy on a schedule or on demand, without rebuilding the whole site)?

Answer:

SSR is the safer default here because price and stock are volatile enough that stale static HTML could show a sold-out item as available, though ISR with a short revalidation window is a strong alternative if you can tolerate content being a minute or two out of date.

Best Choice

- SSR for the page shell and price/stock, or ISR with a short revalidation interval (for example, revalidate every 60 seconds) if a small staleness window is acceptable for the business.

Why?

- SEO still needs full HTML at request time, which rules out CSR-only.
- Pure SSG is dangerous here: a statically built page could show "in stock, $49.99" hours after the item sold out or the price changed, which is a business risk, not just a UX nitpick.
- SSR guarantees every request reflects current inventory, at the cost of a server render (and a database or inventory-service call) on every page view, which is real infrastructure cost at e-commerce traffic volumes.
- ISR splits the difference: you get cached, CDN-served HTML most of the time (cheap, fast) and it self-refreshes on an interval, so you accept "up to N seconds stale" in exchange for far lower server load; for a flash sale where stock can flip in seconds, that staleness window might be unacceptable, which is exactly the kind of constraint that should push you toward SSR or a client-side "top-up" fetch for just the price/stock widget.

Interview Answer:

"This one's a real trade-off. I need SEO, so CSR-only is out, and I need current price and stock, so pure static generation is risky since it could show stale availability. My default would be SSR for the whole page so every request is guaranteed accurate, but if traffic is high enough that per-request server rendering gets expensive, I'd move to ISR with a short revalidation window, maybe sixty seconds, and additionally have the price and stock badge re-fetch live on the client so the number in front of the shopper's eyes is always current even if the surrounding HTML was cached a minute ago. During an actual flash sale with inventory changing every few seconds, I'd lean SSR or even skip caching entirely on that one data point."

Q5. [Junior] A team ships their entire public marketing site, including the homepage, pricing page, and about page, as a CSR single-page app: nearly-empty HTML plus a large JS bundle. Three months later, organic search traffic is far below what they expected. Walk me through what's likely going wrong and what you'd tell them.

Answer:

A CSR-only public site forces search engines to execute JavaScript before they see any real content, which is slower and less reliable than reading server-delivered HTML, so pages can get indexed late, indexed incompletely, or ranked lower purely because of how the content is delivered, independent of how good the content itself is.

Best Choice

- Move public, SEO-sensitive pages (homepage, pricing, about, any landing pages) to SSG or SSR; CSR can stay for anything behind a login where SEO doesn't apply.

Why?

- Search crawlers can execute JavaScript, but it's a slower, resource-limited second pass; pages that depend entirely on JS to render content risk delayed indexing or partial indexing if the crawler's renderer times out or hits an error.
- Time-to-interactive and time-to-first-paint also suffer with CSR since the visitor stares at a blank or skeleton screen until the JS bundle downloads, parses, and executes, which increases bounce rate, an indirect SEO signal.

- The fix isn't "rewrite the whole app," it's recognizing that marketing pages and the interactive app can have different rendering strategies within the same codebase: static or server-rendered for anything crawlers and first-time visitors need to see fast, CSR fine for the authenticated product behind it.

Interview Answer:

"If organic traffic is underperforming and the whole site is CSR, my first suspect is exactly that: rendering strategy. Even though modern crawlers can run JavaScript, it's a slower, less reliable path than just handing them HTML, so pages can get indexed late or incompletely. My recommendation wouldn't be to rewrite the whole app, though. I'd separate the concerns: marketing and content pages that need to rank should move to SSG or SSR, while the actual logged-in product experience can stay CSR since crawlers never see it anyway. That's usually a much smaller, lower-risk change than it sounds like at first."

Q6. [Junior] A five-person startup is deciding between SSR and a static/CDN approach for their product's public pages, and they expect a huge, unpredictable traffic spike on launch day (a Product Hunt feature). What would you recommend, and what's the actual cost trade-off you're weighing?

Answer:

Static generation served from a CDN can absorb almost unlimited concurrent traffic at near-zero marginal cost because the CDN edge is just serving cached files, while SSR means every single request runs server compute, so an unpredictable spike directly translates into a server capacity and cost problem.

Best Choice

- SSG (or SSR with aggressive edge caching in front of it) for the public pages, specifically because of the traffic-spike risk.

Why?

- SSR at scale means provisioning (or auto-scaling) enough server capacity to render every request; a viral spike can either bankrupt a small startup's cloud bill or, worse, overwhelm the servers and cause an outage right when the most people are looking.
- Static files behind a CDN scale horizontally almost for free since the CDN's job is exactly this: serve the same bytes to millions of requesters from cache.
- If some pages genuinely need per-request rendering, a pragmatic middle ground is SSR with a caching layer (a reverse proxy or CDN caching the rendered HTML for a short window) so repeat requests hit cache instead of re-rendering, capturing most of SSR's freshness with most of SSG's cost profile.
- The team's real constraint here isn't "which is more correct," it's operational risk tolerance on a specific, high-stakes day, and that should drive the decision as much as any general best practice.

Interview Answer:

"Given they're expecting a traffic spike they can't predict the size of, I'd steer them away from raw SSR for the public pages. SSR means every request costs server compute, and an unpredictable spike turns into an unpredictable bill or an outage, which is the worst possible time for either. Static generation behind a CDN can absorb that same spike for close to nothing since it's just cache hits. If there's content that truly needs to be dynamic, I'd still put a cache in front of the SSR responses rather than serving every request freshly rendered. The launch-day risk profile is what tips this decision, not a general rule about SSR being bad."

Q7. [Junior] A news site's homepage shows a "Recommended for you" rail that's different for every logged-in reader, but the rest of the homepage (top headlines, layout) is the same for everyone. Does that personalized rail rule out using SSG for this homepage?

Answer:

It doesn't rule out SSG entirely, but it does mean you can't statically generate the whole page as one artifact; the practical answer is to statically generate the shared shell and layer the personalized rail in separately, either via a client-side fetch after load or an edge/SSR pass that injects just that slot.

Best Choice

- Statically generate the shared homepage shell (headlines, layout, nav) and treat the personalized rail as its own independently rendered slot, fetched client-side or injected at the edge.

Why?

- Per-user content, by definition, can't live in a build-time artifact that's the same file for every visitor, so pure SSG for the whole page is off the table the moment personalization enters the picture.
- Throwing out SSG entirely and moving the whole homepage to SSR "to be safe" sacrifices the SEO and cost benefits of static generation for content that's still 90% identical for every visitor.
- Splitting the page (static shell plus a personalized slot) keeps the SEO-critical, crawler-visible content static and fast, while isolating the per-user complexity to a small, independently cacheable or independently rendered piece.

Interview Answer:
"That personalized rail doesn't mean I throw out static generation for the whole page, it just means the page isn't one monolithic static artifact anymore. I'd statically generate the shared shell, the headlines and layout that every visitor sees, and treat the recommendation rail as a separate slot that gets filled in after load with a client-side fetch, or injected at the edge if I want it to appear without a visible pop-in. That way I keep the SEO and speed benefits of static generation for the ninety percent of the page that's identical for everyone, and only pay the personalization cost where it's actually needed."

Q8. [Mid] A retailer's product catalog page updates a few times a day (new products, price changes) and a separate real-time inventory counter shows "3 left in stock" that can change minute to minute. Would you use the same rendering strategy for both, or split them, and why?

Answer:

These have very different freshness requirements, so treating them identically is a mistake; ISR fits the catalog content well since a few-times-a-day change rate tolerates a revalidation window, while the live inventory counter needs to bypass the static cache entirely and be fetched fresh or streamed.

Best Choice

- ISR for the catalog page's core content (title, description, images, category listing), with a short revalidation interval (minutes, not seconds) triggered on a timer or on-demand when a product is updated in the CMS (content management system) or PIM (product information management system, the internal tool merchandisers use to manage catalog data). The live inventory counter

should be its own client-side fetch or a small server-rendered fragment excluded from the page cache.

Why?

- ISR is built exactly for content that changes occasionally but not so often that a full rebuild-per-change makes sense; regenerating a catalog page every few minutes, or on-demand when a product changes, keeps pages cheap to serve (CDN cache hits) while staying acceptably fresh.
- Real-time inventory is the opposite profile: it needs to be correct within seconds, so baking it into a page that's cached for minutes would show wrong numbers, which for stock counts can mean overselling.
- Splitting the page into a statically-revalidated shell plus a live data island is the standard pattern for this exact conflict: cache what's cacheable, isolate what isn't, rather than picking one rendering mode for the whole page and compromising on one side.

Interview Answer:
"I wouldn't use one strategy for the whole page here because the two pieces of content have completely different freshness needs. The catalog details, description, images, price, change a handful of times a day, so ISR is a great fit: I can regenerate that page every few minutes, or on-demand when the product record changes, and serve cached HTML the rest of the time. The 'three left in stock' counter is a different animal; it needs to be accurate within seconds, so I'd pull that out of the cached page entirely and have it fetched live on the client, or rendered as a small dynamic fragment that skips the page cache. Baking a real-time number into a page that's cached for minutes risks overselling stock, which is a business problem, not just a staleness annoyance."

Q9. [Mid] Your team just added SSR to a page, but users report that right after the page appears, buttons don't respond to clicks for a noticeable moment. Why does this happen even though the page rendered fast, and what's the actual cost you're paying here?

Answer:

SSR delivers HTML fast, but that HTML is inert until hydration runs, the process where the framework re-attaches its component tree and event listeners to the existing DOM, so there's a window where the page looks fully loaded but isn't actually interactive yet, often called the "uncanny valley" of rendering.

Best Choice

- Treat hydration as a cost to be budgeted and minimized, not an automatic side effect of SSR you get for free.

Why?

- SSR solves first paint and SEO, it does not solve time-to-interactive by itself; the browser still has to download the JS bundle, parse it, execute it, and rebuild the component tree over the existing DOM before any click handler actually fires.
- The gap between "looks ready" (HTML visible) and "is ready" (hydration complete) is exactly the uncanny valley: a user sees a fully formed page and instinctively clicks, but nothing happens, which reads as broken or laggy even though the system is working as designed.
- The bigger and heavier the page's JS bundle and component tree, the longer this window lasts; a page with dozens of components all needing to hydrate before any of them respond is strictly worse

here than a lean page, which is why bundle size and hydration scope are architectural decisions, not just performance nitpicks.

- Common mitigations include only hydrating the interactive parts of the page (partial hydration or islands architecture), streaming HTML and hydration progressively instead of all-at-once, and prioritizing hydration of above-the-fold interactive elements first.

Interview Answer:

"SSR gets you fast first paint and good SEO, but it doesn't automatically give you a fast time-to-interactive, because the HTML the server sends is inert. Nothing responds to a click until hydration finishes, meaning the framework has downloaded its JS, rebuilt its component tree, and reattached event listeners on top of that existing HTML. In that gap, the page looks completely ready to the user, so they click something and nothing happens, which is worse for perceived quality than a visible loading spinner would be. The fix isn't to abandon SSR, it's to shrink that window: hydrate only what's actually interactive instead of the whole tree, stream in the page and its hydration progressively, and prioritize the above-the-fold interactive bits first."

Q10. [Mid] You're building a recipe website page: mostly static content (ingredients, instructions, photos) with a couple of small interactive widgets, a "adjust servings" calculator and a "save to favorites" button. How would you architect the rendering so you're not paying full-page hydration cost for two small widgets?

Answer:

This is exactly the case for partial hydration or islands architecture, a pattern where the page is delivered as static HTML by default and only the specific interactive components are individually hydrated, rather than hydrating the entire page's component tree to make two small widgets work.

Best Choice

- Render the page as static HTML (SSG fits well since recipes rarely change), and mark only the servings calculator and the favorite button as hydration "islands," each shipping its own small bundle of JS that hydrates independently.

Why?

- The static content, ingredients, steps, photos, never needs JavaScript at all; hydrating it along with the rest of the page just to make two small widgets interactive wastes bytes and CPU on the client for zero benefit.
- Islands architecture treats each interactive widget as its own small, independently hydrated unit, so the JS shipped to the browser scales with how much interactivity the page actually needs, not with the total size of the page.
- This also improves resilience to slow networks: if the JS for the favorites button is still loading, the rest of the page (which needed no JS at all) is already fully functional to read, rather than the whole page being blocked on one shared hydration pass.
- The trade-off is added architectural complexity: you need a framework or bundler that supports shipping and hydrating components independently (rather than one monolithic client bundle), and you have to be deliberate about which components are "islands" versus which stay pure server-rendered markup.

Interview Answer:

"For a page like this, I wouldn't hydrate the whole thing just because two small widgets need interactivity. I'd render the page as static HTML, since recipe content barely changes, and treat the servings calculator and the favorites button as isolated islands, each shipping its own small JS bundle that hydrates independently of the rest of the page. That way the ninety percent of the page that's just text and images never pays any JavaScript cost at all, and the interactive pieces become usable as soon as their own tiny bundle loads, instead of waiting on one big hydration pass for the entire page. The cost is some added build tooling complexity, since not every framework makes it easy to hydrate components independently, but for a content-heavy page like this it's worth it."

Q11. [Mid] You have a dashboard page where most of the data loads fast, but one widget depends on a slow backend aggregation query that can take several seconds. How would streaming SSR (server-rendering and sending HTML to the browser in chunks as it becomes ready, rather than waiting for the entire page to be ready before sending anything) help here, and what would the boundary around that slow widget look like?

Answer:

Streaming SSR lets the server send the fast parts of the page immediately and "fill in" the slow widget later once its data is ready, instead of blocking the entire response on the slowest piece of data, so the user sees and can interact with most of the page right away instead of staring at a blank screen for as long as the slowest query takes.

Best Choice

- Wrap the slow widget in a suspense/streaming boundary so its slow data fetch doesn't block the rest of the page's server response.

Why?

- Without streaming, a single slow backend call anywhere on the page forces the entire SSR response to wait for it, since the server has to finish building the full HTML document before sending anything; one slow widget then determines the load time of the whole page.
- With a streaming boundary, the server sends the shell and all the fast content first, then streams in the slow widget's HTML (and triggers its hydration) once the data resolves, so the page becomes useful well before the slowest piece of data arrives.
- This is a meaningful architectural decision, not just a nice-to-have: it changes which query determines your page's perceived load time, from "the slowest thing on the page" to "the slowest thing above the fold that isn't wrapped in a boundary."

```
// Conceptual shape of a streaming boundary around a slow widget.
// The rest of the dashboard renders and streams immediately;
// this widget's HTML and hydration arrive later, in-place.
function DashboardPage() {
  return (
    <Dashboard>
      <FastSummaryCards />
      <FastRecentActivity />
      <Suspense fallback={<WidgetSkeleton />}>
        <SlowAggregationWidget />
      </Suspense>
    </Dashboard>
  );
}
```

Interview Answer:

"I'd wrap that slow widget in a streaming boundary rather than let it block the whole page's response. Without that, the server has to wait for the slowest query on the page, in this case that heavy aggregation, before it can send any HTML at all, so a three-second query becomes a three-second blank screen for the user. With streaming, the fast summary cards and recent activity can be sent and rendered immediately, and the slow widget shows a placeholder that gets swapped in once its data is ready. The architectural shift is that your page's perceived load time is no longer dictated by the single slowest data dependency, it's dictated by the slowest thing you didn't isolate behind a boundary."

Q12. [Mid] Design the rendering strategy for a SaaS product's pricing page: the plan tiers, features, and FAQ are the same for every visitor, but if the visitor is already logged in, the page should highlight "your current plan" and show a personalized upgrade recommendation. How do you architect this without either losing SSG's benefits or building a fully dynamic page from scratch?

Answer:

The right architecture separates the page into a statically generated shared shell and a small personalized slot that's resolved separately, since forcing the whole page into SSR or CSR just to support one personalized element would sacrifice the performance and SEO benefits that apply to the vast majority of the page's content.

Best Choice

- SSG (or ISR) for the shared shell: plan tiers, feature comparison, FAQ. A separate client-side fetch (using an auth token/cookie already present in the browser) resolves the "your current plan" and recommendation slot after the static shell loads, or, if you want it to appear without any visible pop-in, a lightweight edge function that checks for a session cookie and injects the personalized block before the response reaches the browser.

Why?

- The bulk of this page (pricing, features, FAQ) is public, SEO-relevant, and identical for every visitor, which is exactly SSG's sweet spot; nothing about supporting a small logged-in personalization should force that content off static generation.
- Client-side resolution of the personalized slot is the simplest option: ship the static shell, then fetch "your plan" data based on the session cookie already in the browser, and render it into a designated slot. The trade-off is a brief flash where the generic page is visible before the personalized bit appears.
- If that flash is unacceptable from a product standpoint, an edge function in front of the static asset can inspect the session cookie and stitch in the personalized fragment before the response leaves the CDN, avoiding client-side pop-in at the cost of a bit more infrastructure (an edge compute layer instead of a pure static file serve).
- The anti-pattern to call out is defaulting the entire page to SSR "just in case someone's logged in," which throws away CDN caching for content that's overwhelmingly the same for every visitor, to support a feature that only affects a fraction of visits.

Interview Answer:

"I'd keep the bulk of this page static since the plans, features, and FAQ are identical for everyone and there's real SEO value in that content being pre-rendered. The personalized part, 'you're on this plan, consider upgrading,' I'd treat as a separate slot rather than let it dictate the rendering strategy for the whole page.

The simplest version is a client-side fetch after the static shell loads, using the session cookie the browser already has, which does mean a brief flash before the personalized content appears. If that flash is a problem for the product team, I'd move that check to an edge function that can inject the personalized fragment before the response reaches the browser, without giving up static generation and CDN caching for the ninety percent of the page that's the same for every visitor."

Q13. [Mid] Your team is considering using edge functions (small serverless functions that execute at points of presence physically close to the user, rather than in one centralized region) to do things like geo-based redirects, A/B test bucketing, and lightweight personalization. When does this genuinely help, and when does it just add complexity you didn't need?

Answer:

Edge functions earn their keep when the decision they're making is small, needs to happen before the cached response is served, and benefits from proximity to the user, such as redirecting by country or assigning an experiment bucket; they add unnecessary complexity when the logic is heavier, needs a full database round-trip anyway, or could just as easily live in a regular server or in the client.

Best Choice

- Use edge functions for lightweight, latency-sensitive decisions that gate or lightly modify a cached response (geo-redirects, cookie-based experiment bucketing, simple auth checks). Keep heavier logic (anything needing a database call, complex business logic, or large computation) in a regular origin server or the client.

Why?

- The core value of "at the edge" is proximity and speed for small decisions; running a function within milliseconds of the user to redirect them to the correct locale, or to read/set an A/B test cookie, adds negligible latency and lets you keep serving a static, cached page instead of falling back to a full dynamic render.
- If the "lightweight" edge function ends up needing a database query, most edge runtimes either don't support that well or it defeats the "avoid the round-trip" advantage of being at the edge in the first place, since you're now doing a slow network call from the edge back to a centralized database anyway.
- Edge functions also come with real operational costs: a more constrained runtime (memory/CPU limits, restricted APIs), debugging that's harder because you're not looking at one centralized server's logs, and another deployment target and cold-start profile to reason about. Teams sometimes adopt edge functions because it's trendy, then discover their actual logic doesn't fit the constraints and they've added a second execution environment for no real gain.
- The right question to ask when evaluating an edge function use case is "does this decision need to happen before the cache serves a response, and is it cheap enough to run in a constrained runtime," and if the answer to either is no, it probably belongs somewhere else.

Interview Answer:

"Edge functions are great for small decisions that need to happen close to the user and before a cached response goes out, things like geo-redirects or bucketing someone into an A/B test based on a cookie. Where I'd push back is when a team wants to put heavier logic there, like anything that needs a database call, because then you're doing a network round-trip from the edge back to a centralized database anyway, which erases the latency benefit and you're stuck with a more constrained runtime and harder debugging

for no payoff. My rule of thumb is: if the decision is cheap and needs to gate what gets served before caching kicks in, edge is a great fit; if it needs real data or heavy computation, it belongs in a regular server or the client, and forcing it to the edge is just added complexity."

Q14. [Mid] Design the rendering strategy for a multi-tenant B2B SaaS dashboard where each tenant (customer company) has its own theming (logo, colors) and some tenants have different feature flags enabled. How do you avoid either baking tenant-specific HTML into a build step per tenant, or losing caching benefits entirely?

Answer:

Since tenant identity is known from the request (subdomain, path, or auth token) and theming/feature flags are relatively small, cacheable pieces of data, the right architecture is SSR (or edge rendering) that resolves tenant context per request and injects theming and flags into an otherwise shared template, with the tenant's theme/config itself cached separately so you're not re-fetching it from a database on every request.

Best Choice

- SSR or edge-rendered pages that read tenant identity from the request (subdomain like acme.yourapp.com, or a path segment), fetch that tenant's theme/config from a cache (not the primary database) on each request, and render the shared dashboard template with tenant-specific values injected.

Why?

- Per-tenant static builds (literally generating a separate static site for every customer) don't scale operationally past a small number of tenants: every theme change, feature flag flip, or app update means rebuilding and redeploying per tenant, which turns a five-minute change into an hours-long fan-out across hundreds of customers.
- A shared SSR template with per-request tenant resolution scales cleanly regardless of tenant count, since adding tenant #501 doesn't require any new build or deploy, just a new row in the tenant config store.
- The caching concern is real but solvable: tenant config (logo URL, color values, enabled flags) rarely changes, so it should sit in a fast cache (in-memory or a CDN-edge key-value store) keyed by tenant, not be queried from the primary database on every dashboard request; you're caching the tenant's configuration, not the whole personalized page.
- If most of the dashboard's actual data view is also heavily interactive and user-specific (typical for B2B dashboards), CSR for the app shell's data views combined with SSR only for the outer shell (theme, nav, auth-gated wrapper) is a reasonable further split, so you're not paying full SSR cost for data that's going to be client-fetched and manipulated anyway.

Interview Answer:

"I wouldn't build this as a separate static site per tenant, that falls apart operationally once you have more than a handful of customers, since every theme tweak or flag change turns into rebuilding and redeploying per tenant. Instead I'd resolve tenant identity from the request, usually the subdomain, and use SSR to render a shared dashboard template with that tenant's theme and flags injected in. The key to keeping this fast is not hitting the primary database for tenant config on every request, I'd cache each tenant's theme and flags in a fast key-value store so that lookup is cheap. And if the actual dashboard data views are heavily interactive, which they usually are in B2B tools, I'd keep those CSR and reserve SSR for just the outer shell,

theming, and auth-gated wrapper, so I'm not paying a full server render for data the client's going to fetch and manipulate anyway."

Q15. [Mid] A mid-size app has a public blog, a checkout flow, and a logged-in customer dashboard, all in the same codebase. Would you pick one rendering strategy for the whole app, or different strategies per route? Walk me through how you'd decide, and what complexity that adds.

Answer:

Different sections of the same app usually have genuinely different constraints, so a hybrid, per-route rendering strategy (SSG for the blog, SSR for checkout, CSR for the dashboard) generally serves users and the business far better than forcing one strategy across the entire app, at the cost of a build and deploy pipeline that has to support multiple rendering modes at once.

Best Choice

- SSG for the blog (public, low change frequency, SEO-critical), SSR for checkout (needs current pricing/inventory and SEO for product pages leading into it, plus security-sensitive server-side logic), CSR for the logged-in dashboard (no SEO need, heavy interactivity).

Why?

- Picking one strategy for the whole app means accepting its worst trade-off everywhere: all-CSR tanks the blog's SEO and checkout's crawlability, all-SSR adds server cost and complexity to the dashboard for zero benefit since it's behind a login and dominated by client interactivity, all-SSG can't handle checkout's need for live pricing/inventory or the dashboard's per-user state at all.
- Modern meta-frameworks (frameworks that support mixed rendering modes per route in one project) make this hybrid approach practical without maintaining three separate codebases; the real cost is operational and cognitive, since your team now needs to understand which rendering mode applies where, keep shared components compatible across modes (a component used on both the SSR checkout page and the CSR dashboard needs to render correctly in both contexts), and reason about a slightly more complex build/deploy pipeline.
- The decision framework to apply per route: does this route need SEO? Does it show the same content to everyone or per-user content? Does it need to reflect fast-changing data? Answering those three questions per route, rather than for the app as a whole, is what should drive the choice.

Interview Answer:

"I'd pick per-route rendering here rather than one strategy for the whole app, because the blog, checkout, and dashboard have genuinely different needs. The blog is public and barely changes, so SSG. Checkout needs SEO on the product pages leading into it and has to reflect live pricing and inventory, so SSR fits. The dashboard is behind a login, has no SEO need, and is dominated by client interactivity, so CSR is the right call there and SSR would just add cost for nothing. The trade-off is real: you need a framework that supports mixing these modes cleanly in one project, and your team has to keep track of which mode applies where, especially for shared components that get used across sections. But forcing one rendering strategy across all three would mean accepting that strategy's worst trade-off in the sections where it doesn't fit, which is a worse outcome than the added operational complexity."

Q16. [Senior] Your team owns a CSR-only React app for a content-driven product (think a job board or a marketplace) with thousands of public listing pages. SEO is

bad, initial load is slow, and leadership wants this fixed, but a full rewrite to a server-rendering framework is considered too risky to do as one big cutover. Design a migration path.

Answer:

The lowest-risk path is a strangler-style migration (a pattern where a new system takes over pieces of an old one incrementally, route by route, until the old system can be retired, rather than replacing it all at once): introduce server rendering at the routing layer for a small, well-chosen slice of pages first, validate it in production, and expand route by route, rather than attempting to convert the entire app's rendering model in one release.

Best Choice

- Route-by-route migration starting with the highest-value, lowest-risk pages (typically the public listing pages driving the SEO problem), using a reverse proxy or framework routing layer that can serve some routes via SSR and others via the existing CSR app during the transition.

Why?

- A big-bang rewrite of rendering for an entire app carries the risk of shipping a large number of regressions at once, makes rollback all-or-nothing, and delays any SEO improvement until the entire rewrite is done, which for a content-driven product can mean months of continued lost organic traffic.
- Instead, start by identifying the pages that actually drive the business pain, in this case the public listing pages that need to be indexed and are currently rendering as blank shells to crawlers, and migrate just those first. This can often be done by introducing a routing layer (a reverse proxy, or a meta-framework's ability to mix rendering modes per route) that serves specific paths via SSR while everything else continues to be served by the existing CSR build.

1. Instrument first: measure current SEO indexing rates, Core Web Vitals (Google's standard set of page-experience metrics, covering loading speed, responsiveness, and visual stability), and time-to-interactive on the pages you intend to migrate, so you have a before/after baseline instead of anecdotal impressions of improvement.
2. Pick the first slice: choose a small number of high-traffic, high-SEO-value listing pages, ideally ones that are relatively simple and mostly read-only, to minimize the surface area of the first migration.
3. Stand up SSR for just that slice, behind a routing layer that can direct only those specific paths to the new rendering path, while every other route continues to be served by the untouched CSR app.
4. Validate in production with real traffic (checking indexing improvements, Core Web Vitals, error rates) before expanding to the next slice of routes.
5. Repeat route by route, expanding to more listing pages and eventually other sections, keeping the CSR app as the fallback for anything not yet migrated.
   - This approach means the team ships incremental, measurable wins (the SEO-critical pages improve first, fastest), keeps a working rollback path at every step (only the migrated routes are at risk, not the whole app), and avoids the common failure mode of a rewrite that takes twice as long as planned and delivers zero value until it's entirely finished.
   - The main cost is running two rendering paths side by side for a while, and needing shared components/design system pieces to work correctly in both the new SSR pages and the legacy CSR app, which is real but bounded complexity compared to the risk of a single cutover.

Interview Answer:

"I'd avoid a big-bang rewrite here entirely, since converting the whole app's rendering model in one release means no SEO improvement ships until everything is done, and a rollback would be all-or-nothing if something breaks. Instead, I'd do a strangler-style migration: put a routing layer in front of the app that can direct specific paths to a new SSR-rendered path while everything else keeps being served by the existing CSR build. I'd start with a small number of high-traffic listing pages, the ones actually driving the SEO problem, get those into production, measure indexing and Core Web Vitals improvements against a baseline I captured beforehand, and then expand route by route. That way we get the highest-value SEO wins first, we can roll back a single slice of routes if something goes wrong instead of the whole app, and leadership sees measurable progress within weeks instead of waiting for a multi-month rewrite to finish before seeing any benefit."

Q17. [Senior] You're reviewing a PR where a team added SSR to their highest-traffic page to "fix performance." Post-launch metrics show first paint improved, but time-to-interactive barely moved, and there are hydration mismatch warnings in the browser console. What's your architectural read on what went wrong, and what would you tell them to fix?

Answer:

SSR only improves the part of the timeline before the browser has anything to show; if the page still ships and hydrates a large, mostly-unchanged component tree, time-to-interactive is gated by hydration, not by first paint, and hydration mismatches usually mean the server-rendered markup and the client's first render disagree, both of which point to the team treating SSR as a drop-in fix rather than redesigning what gets hydrated and how.

Best Choice

- Diagnose hydration mismatches first (they often cause the framework to discard and fully re-render the mismatched subtree, which is slower than a normal hydration and can explain why TTI, time-to-interactive, didn't improve), then reduce hydration scope with partial hydration or streaming rather than hydrating the full page tree eagerly.

Why?

- First paint improving but TTI not moving is a strong signal that the bottleneck moved from "server response and HTML parsing" to "JS bundle download, parse, and hydration of the full component tree," which SSR alone does nothing to address, since SSR's job ends at delivering HTML, not at making it interactive.
- Hydration mismatches (the server-rendered HTML doesn't match what the client would render on first pass, commonly caused by things like using browser-only values such as window size or the current time during the initial render, or by data that differs between the server's render and the client's) force the framework to reconcile or discard and re-render the mismatched parts, which is strictly more work than a clean hydration and directly hurts the TTI number they were trying to improve.
- Concretely, I'd tell the team to: find and fix the source of the mismatch (usually non-deterministic or environment-dependent values rendered during SSR), measure how much of the page's JS bundle is required just to hydrate versus what's needed for actual interactivity, and consider whether the whole page needs to hydrate eagerly at all versus lazily hydrating below-the-fold or non-critical sections.
- The broader lesson for the team: SSR is a first-paint and SEO tool, not a time-to-interactive tool by itself; TTI is a hydration and bundle-size problem, and needs to be solved with hydration strategy

(partial hydration, lazy hydration, streaming), not just by moving where the initial HTML comes from.

Interview Answer:

"The pattern here, first paint improving but TTI staying flat plus hydration warnings, tells me they treated SSR as a silver bullet instead of thinking about what happens after the HTML arrives. First paint got faster because the browser now has real markup to show immediately, but time-to-interactive is gated by hydration, and if they're still hydrating the same large component tree they had before, of course that number didn't move. The hydration mismatches are a separate but related problem, that usually means something in the server render doesn't match what the client renders first, often a browser-only value like window size sneaking into the initial render, and when that happens the framework often has to throw away and redo that part of the render, which makes hydration slower, not faster. I'd tell them to fix the mismatch source first, then actually look at reducing what needs to hydrate eagerly, through partial hydration or lazy-hydrating non-critical sections, because SSR was never going to solve a hydration-scope problem on its own."

Q18. [Senior] You're advising a team building an internal, Figma-style collaborative tool (real-time cursors, live document editing, dozens of interactive panels) that's entirely behind authentication. Someone on the team insists "we should SSR this for performance." Do you agree, and how do you frame the decision?

Answer:

For a page whose value is almost entirely in real-time, client-driven interactivity behind a login, SSR's main benefits (SEO and faster perceived first paint of static content) mostly don't apply, so I'd push back and frame the real performance problem as an initial-load and bundle-size problem to be solved with code-splitting and lazy loading, not a rendering-location problem.

Best Choice

- CSR, with performance work focused on bundle size, code-splitting, and progressive loading of panels/features, not on moving rendering to the server.

Why?

- SEO doesn't apply behind a login wall, which removes one of SSR's two headline benefits entirely.
- The other headline benefit, faster time-to-first-meaningful-paint, is weaker here because the "meaningful" content (live cursors, collaborative document state, real-time panels) is inherently dynamic and client-driven; SSR could render an initial static snapshot, but the second that snapshot needs to start receiving live updates (via websockets or similar), you're back to a fully client-driven experience, and the SSR'd snapshot risks being stale or mismatched the moment real-time data arrives.
- Introducing SSR here adds real cost: server compute for every page load, and now hydration has to reconcile server-rendered state with a real-time data source that may have already changed since the server rendered it, which is a genuinely harder correctness problem than hydrating a mostly-static page.
- The actual performance complaint (if there is one) is more likely a bundle-size or initial-load problem: are all the collaborative editor's panels and features being loaded upfront, or can heavy pieces (a rarely-used panel, a complex diagramming feature) be code-split and loaded on demand? That's the lever I'd pull instead of SSR.

Interview Answer:

"I'd push back on that, respectfully, because I don't think SSR is solving the right problem here. This tool is behind a login, so SEO isn't in play, and the actual content that matters, live cursors, real-time document state, is inherently client-driven and changes the instant a websocket message arrives, so even if we SSR'd an initial snapshot, it's stale the moment real-time data starts flowing and now we've added a reconciliation problem between the server-rendered state and the live state on top of it. If the real complaint is that initial load feels slow, I'd look at bundle size and code-splitting instead, are we loading every panel and feature upfront when most users only touch a handful of them in a given session. That's very fixable without introducing server rendering and its costs into a tool where it doesn't buy us the benefits it usually buys."

Q19. [Senior] Design the rendering and caching architecture for a large e-commerce catalog during a flash sale: normally inventory changes a few times a day and ISR handles it fine, but during the sale window, stock on featured items can sell out within seconds and prices can be dynamically adjusted. How do you architect for both states without running two completely separate systems?

Answer:

The architecture should keep ISR as the default caching behavior for the catalog year-round, but add an explicit, deliberate escape hatch, on-demand revalidation triggered by inventory events, and a client-side "top-up" fetch for the specific fields that must never be stale (price, stock count) on the small set of pages actually in a flash sale, rather than lowering the cache duration for the entire catalog or running a separate rendering system just for sale periods.

Best Choice

- Keep ISR with on-demand revalidation (the page regenerates immediately when triggered by an event, such as an inventory-service webhook firing on a stock change, rather than only on a fixed timer) as the baseline for all catalog pages, and layer a live client-side fetch for price/stock specifically on pages flagged as part of an active flash sale.

Why?

- Lowering the ISR revalidation interval globally (say, from ten minutes to five seconds) to handle a sale that affects a small subset of products punishes the entire catalog's caching efficiency for a problem that's localized to a handful of pages, driving up server/regeneration load across products that don't need it.
- On-demand revalidation (regenerate this specific page now, triggered by an event like "stock hit zero" or "price changed" from the inventory service) gives you near-real-time correctness for the pages that actually changed, without touching the cache behavior of the other thousands of pages that didn't.
- For the handful of products in an active flash sale, where even on-demand revalidation's regeneration latency (the time between the event firing and the new page being ready) might not be tight enough, layering a lightweight client-side fetch for just the price and stock count on top of the (possibly momentarily stale) cached HTML gives you a belt-and-suspenders guarantee on the one number that actually matters for not overselling.
- This design means the system doesn't need two parallel rendering architectures for "normal" and "sale" states, it's the same ISR-based system with an event-driven revalidation trigger and an optional live-data overlay that gets turned on only for the pages that need it, controlled by a flag on the product/sale record rather than a global mode switch.

Interview Answer:

"I wouldn't build two separate systems for normal days versus flash sale days, that's a lot of complexity to maintain for what's fundamentally the same catalog. I'd keep ISR as the baseline everywhere, but make sure it supports on-demand revalidation, meaning the inventory service can fire an event that says 'regenerate this specific product page now' the instant stock changes, rather than only regenerating on a fixed timer. That handles the vast majority of the catalog fine, including most sale items. For the specific products that are part of an active flash sale, where even that regeneration latency might not be tight enough given stock can hit zero in seconds, I'd layer a small client-side fetch just for price and stock count on top of the cached page, so that one critical number is always live even if the surrounding HTML was generated a few seconds ago. That's a flag on the product record, not a different rendering system, so we're not maintaining two architectures."

Q20. [Senior] A team wants to statically generate product pages, but the pages also need to vary by locale (12 languages), by a pricing experiment (3 variants), and by a logged-in/logged-out state. Someone proposes pre-building a static page for every combination. What's the problem with that, and how would you architect it instead?

Answer:

Pre-building a static variant for every combination of locale, experiment, and auth state causes combinatorial cache-key explosion (12 x 3 x 2 = 72 variants per product, and it multiplies further with every new dimension), which blows up build time and cache storage for dimensions that don't actually need to be baked into the HTML at build time, so the better architecture is to statically generate along only the dimensions that are truly build-time-safe and resolve the rest at request time or client-side.

Best Choice

- Statically generate per locale (locale is usually build-time-safe since it's stable for a given URL and genuinely changes the HTML content, i.e. translated text), but resolve pricing experiment and auth state at the edge or client-side rather than baking them into separate static builds.

Why?

- Combinatorial explosion isn't just an inconvenience, it's a scaling problem: 72 variants per product times a catalog of thousands of products means build times and storage balloon, and every new experiment or locale added multiplies the problem further rather than adding to it linearly.
- Locale is a reasonable dimension to bake into static generation because it's stable per URL (a French visitor to the French-locale URL always needs the French translation) and the difference is substantial (entire page content changes).
- The pricing experiment doesn't need to be a build-time dimension: it can be resolved by an edge function that reads an experiment-bucket cookie and serves the correct price into an otherwise-shared static page, or the base static page can render a placeholder price that's swapped client-side once the experiment bucket is known. Either avoids doubling or tripling the static build for a value that's a single number on the page.
- Auth state (logged-in vs logged-out) similarly doesn't need separate static builds; it's better handled the way personalization is handled generally, a shared static shell with a client-fetched or edge-injected personalized slot (as in the pricing-page personalization pattern), since "logged in or not" is knowable from a request-time cookie, not something that needs to exist as a distinct pre-built artifact.

- The architectural principle to name here: only bake a dimension into static generation if it's both stable for a given URL and represents a large enough portion of the page's content to be worth a separate build; for everything else, resolve it at request time (edge) or client-side against a shared static base.

Interview Answer:

"Pre-building every combination is a trap, twelve locales times three experiment variants times two auth states is seventy-two versions of every single product page, and that number only gets worse every time someone adds a new experiment or a new locale. I'd only bake locale into the static build, since it's stable for a given URL and it changes a large amount of the page's actual content, the translated text. The pricing experiment and the logged-in state I'd resolve at request time instead, either through an edge function reading a cookie to serve the right price into an otherwise shared static page, or by fetching that small piece of data client-side against a shared static base. The rule I'd apply going forward is: a dimension only earns a separate static build if it's stable per URL and it's substantial enough to justify multiplying your build matrix by it, everything else should be resolved dynamically against a shared artifact."

Q21. [Senior] Your app gets tens of millions of page views a day on public, mostly-static content. One architect wants full SSR everywhere for "always fresh, always correct" pages; another wants CSR plus a CDN for cost reasons. As the tech lead, how do you resolve this, and what would actually change your answer?

Answer:

At this scale, defaulting to full SSR for content that's overwhelmingly the same for every visitor means paying server compute (and its operational overhead, autoscaling, cold starts if using serverless/edge compute, incident risk) on every single one of tens of millions of daily requests for freshness that most of that content doesn't actually need, so I'd push for SSG/ISR as the default and reserve SSR for the specific slices of the app where per-request correctness genuinely can't be cached, rather than picking one extreme for the whole app.

Best Choice

- SSG or ISR as the default for the high-traffic, mostly-static content; SSR reserved for the specific pages or fragments that truly need per-request freshness or personalization; CSR reserved for logged-in, non-SEO sections. Not a single answer for the whole app, at this scale the cost difference between strategies is large enough to matter per-section.

Why?

- At tens of millions of daily views, the cost delta between "serve a cached static file from a CDN edge" and "run a server render on every request" is enormous, easily an order of magnitude or more in infrastructure spend, and that delta is being paid for freshness that content changing a few times a day doesn't need.
- Full SSR everywhere also introduces operational risk at this scale that a static/CDN approach doesn't have to the same degree: server fleets need to autoscale correctly under load, serverless/edge SSR functions can suffer cold-start latency under bursty traffic, and a server-side bug or outage can take down page rendering entirely, whereas a CDN serving static files is one of the most battle-tested, resilient parts of the stack.
- Full CSR plus CDN for everything has its own problem at this scale though: it throws away SEO for the fraction of pages that need to rank, and it accepts a worse time-to-first-paint for every single

one of tens of millions of visits in exchange for a cost saving that mostly applies to content that wasn't expensive to render server-side in the first place.

- What would change my answer: if a large fraction of the traffic is actually to pages needing true per-request personalization or real-time correctness (not the "mostly static" case as described), the cost/benefit shifts and more SSR becomes justified for that slice specifically; the decision should be made by measuring what fraction of pages/requests genuinely need per-request rendering, not by picking a philosophy and applying it universally.

Interview Answer:

"I wouldn't side fully with either architect here, I'd push for measuring first. At tens of millions of daily views, the cost and operational risk of full SSR everywhere is real, you're paying for server compute, autoscaling, and potential cold starts on every single request for freshness that content changing a few times a day doesn't need. But full CSR plus CDN for everything throws away SEO and first-paint speed for whatever fraction of pages actually needs to rank or needs fast perceived load, which at this scale is a lot of lost opportunity too. My actual recommendation would be SSG or ISR as the default, since most of this traffic is to mostly-static content, and SSR reserved specifically for the pages or fragments that have a genuine per-request freshness or personalization requirement. What would change my mind is if we measured and found a much larger share of traffic actually needs true per-request rendering than assumed, at which point the cost math shifts and more SSR for that slice becomes the right call. The decision should follow from what fraction of the traffic actually needs what, not from picking one extreme as a philosophy."

Q22. [Senior] You're the founding frontend architect for a new product: it will have marketing pages, a content blog, a logged-in customer dashboard, and a checkout flow, and leadership wants this decided before the team starts building so they don't end up retrofitting rendering strategy later. Walk me through the rendering architecture you'd propose for the whole app.

Answer:

Rather than picking one rendering mode for the entire product, I'd propose a strategy assigned per section based on each section's actual SEO, freshness, and personalization requirements, built on a meta-framework that supports mixing SSG, ISR, SSR, and CSR within one codebase, so the team never has to retrofit a different rendering model in later as requirements diverge.

Best Choice

- Marketing pages: SSG. Blog: SSG with on-demand ISR triggered by CMS publish events. Checkout: SSR (needs live pricing/inventory and SEO on the product pages funneling into it). Customer dashboard: CSR (behind login, no SEO need, interactivity-dominated).

Why?

- Marketing pages are public, identical for every visitor, and SEO-critical, the textbook SSG case; there's no reason to accept SSR's per-request cost or CSR's SEO and TTI penalty here.
- The blog is the same profile as marketing pages but with a publishing workflow, so SSG plus on-demand regeneration (webhook-triggered on publish, not full rebuilds) keeps publish-to-live latency low without rebuilding the whole site per post.
- Checkout needs both SEO (product pages need to be crawlable and rank) and per-request correctness (price, stock, and potentially per-session state like cart contents or a payment step), which makes it the one section where SSR's cost is actually justified by what it buys.

- The dashboard is behind authentication, has no SEO surface at all, and is dominated by interactive, per-user state (settings, data views, live updates), which is CSR's sweet spot; adding SSR here would mean paying server-rendering cost and hydration complexity for a section where neither of SSR's core benefits apply.
- Practically, I'd propose the team standardize on one meta-framework capable of mixing these modes per route from day one (rather than stitching together separate apps or frameworks per section), define the decision criteria (does this route need SEO? is content the same for every visitor or per-user? does it need live data?) as a checklist for any new route added later, and set up the CI/deploy pipeline to support incremental static regeneration and route-level rendering config from the start, since retrofitting a mixed-mode rendering pipeline onto a codebase built assuming one mode everywhere is exactly the costly rework leadership is trying to avoid.

Interview Answer:

"I'd design this as four different rendering strategies within one app, not one strategy for the whole product, because each section has genuinely different needs. Marketing pages are public and identical for everyone, so SSG. The blog is the same shape but needs a publishing workflow, so SSG plus on-demand regeneration triggered when a post goes live, rather than full rebuilds. Checkout is the one place I'd actually reach for SSR, because it needs both SEO on the product pages and per-request correctness on price, stock, and cart state, so the server-rendering cost is justified there. The dashboard is behind login with no SEO surface and is dominated by interactive per-user state, so CSR, and adding SSR there would just be cost with no upside. The important part of doing this upfront is picking a framework that supports mixing all four modes per route from day one, and writing down the decision checklist, does it need SEO, is the content shared or per-user, does it need live data, so that as the team adds new routes later, the rendering choice is a quick, consistent decision instead of a retrofit."

If you remember nothing else from this chapter: *rendering strategy is a per-page decision hiding inside what looks like a single, one-time architectural choice.*

Chapter 6: Performance & Scalability Architecture

Every frontend engineer can name the metrics (LCP, CLS, TTI) but far fewer can architect a system that keeps those metrics healthy once a codebase has fifty routes, a dozen teams shipping to it, and a product manager who wants three more third-party widgets on the homepage. Use the questions in this chapter to move past reciting performance vocabulary and practice designing the pipelines, budgets, and loading architectures that keep performance from decaying as a real product and team scale.

> **War Story:** *An app that felt instant in every internal demo turned out to take twelve seconds to load for real users, because every laptop in the office was on gigabit WiFi and nobody had ever tested it on a throttled connection.*

> **Avoid:** quoting performance numbers measured only on fast networks and modern devices
>
> **Aim For:** insisting on testing under throttled network and mid-range device conditions before trusting a performance number

Q1. [Junior] We're building a large multi-route web app, say twenty-plus pages behind a single-page app router. How would you decide the code-splitting strategy, route-based versus component-based, and where exactly do you draw the split boundaries?

Answer:

Code splitting (breaking one large JavaScript bundle into smaller chunks that load on demand instead of all at once) is the first and highest-leverage lever for keeping initial load fast as an app grows. **Start with route-based splitting because it lines up with a natural, user-triggered boundary, navigation, and only reach for component-based splitting inside a route once that route's own bundle is still too heavy.**

Route-Based Splitting First

- Each route (a dashboard page, a settings page, a checkout page) becomes its own chunk, loaded via dynamic import when the router matches that path.
- This is nearly free to reason about: a user visiting `/settings` should never pay for the JavaScript that powers `/checkout`.
- Most router libraries (React Router, Next.js's file-based routing) give you this boundary automatically if you use `lazy` or dynamic `import()` per route.

When to Add Component-Level Splitting

- Split inside a route when that single route still ships an outsized chunk, for example a checkout page that bundles a full rich-text editor, a charting library, or a PDF generator that only 5% of visitors on that page ever trigger.
- Good candidates for component-level splits: modals, tabs that aren't the default tab, anything behind a click or a feature flag, and heavy third-party SDKs.
- A bad candidate is anything visible on initial paint. Splitting something the user sees immediately just adds a network round trip and a loading spinner where there didn't need to be one.

Deciding the Boundary in Practice

1. Measure per-route bundle size with a bundle analyzer (a tool that visualizes what's inside your built JavaScript, like webpack-bundle-analyzer or Vite's rollup-plugin-visualizer).
2. Flag any route over your budget (see Q8 on performance budgets).
3. Inside a flagged route, find the single largest contributor that isn't needed for first paint and split that first, don't split everything at once.
4. Stop once the route is under budget. Over-splitting has a real cost too, every extra chunk is a network request, and on a flaky connection that's more round trips, not fewer.

```
// Route-level split: the whole /checkout page is its own chunk
const CheckoutPage = React.lazy(() => import('./pages/CheckoutPage'));

// Component-level split inside that page: only loaded when the user
// opens the "add promo code" flow, not on initial page load
const PromoCodeEditor = React.lazy(() => import('./components/PromoCodeEditor'));
```

Interview Answer:

"I'd default to route-based splitting first since it's a natural boundary that maps to how users actually move through the app, and it's usually a one-line change per route with the router's lazy-loading support. I wouldn't go further than that until I had data. I'd run a bundle analyzer per route, and if a specific route is still bloated, I'd look for the single heaviest piece that isn't needed for first paint, like a rich text editor or a charting library, and split just that behind a dynamic import. I try not to over-split, because every chunk is a network request, and on a slow connection that adds up fast."

Q2. [Junior] You're building an image-heavy product page, think an e-commerce listing with a hero image and a scrolling gallery below it. How do you decide what loads eagerly versus lazily, and how does that decision change for a global user base?

Answer:

The core design decision here is defining "above the fold" (the content visible without scrolling) for your actual layouts, not guessing, and then wiring the loading strategy to that definition. **Treat the hero image as a critical resource that loads eagerly with high priority, and treat everything below the first viewport as a candidate for lazy loading, deferred until the user is about to scroll it into view.**

Defining Above the Fold

- "Above the fold" isn't one number, it depends on viewport height, which varies by device. A reasonable approach is to pick a conservative viewport height (for example, the 25th percentile mobile viewport from your real user analytics) and treat anything rendered inside that as eager-load.
- For a responsive layout, this usually means the hero image and maybe the first row or two of a grid, not the whole gallery.

Eager-Loaded Content

- The hero image should use `fetchpriority="high"` and skip the native `loading="lazy"` attribute, since browsers already deprioritize below-the-fold images by default but won't know your hero is critical without a hint.
- Avoid lazy-loading anything that's part of the Largest Contentful Paint (LCP, the render time of the biggest visible element) candidate, since deferring it directly hurts your LCP score.

Lazy-Loaded Content and Global Delivery

- Everything else in the gallery uses native `loading="lazy"` or an IntersectionObserver-based lazy loader if you need more control (for example, loading a placeholder blur-up image first).
- For a global user base, pair lazy loading with a CDN (Content Delivery Network, a network of edge servers that cache and serve content close to the user) that does automatic format negotiation, serving AVIF or WebP to browsers that support it and falling back to JPEG, and automatic resizing so a user on a small phone in a region with expensive mobile data isn't downloading a 4000px desktop image.
- Serve responsive images with `srcset` and `sizes` so the browser picks the right resolution for its viewport and pixel density instead of always fetching the largest version.

Interview Answer:
"I'd start by defining above the fold concretely for our actual layout, using real viewport data rather than guessing, and that boundary decides the loading strategy. The hero image is above the fold, so it loads eagerly with a high fetch priority since it's almost certainly our LCP element. Everything in the gallery below that loads lazily, either with native lazy loading or an IntersectionObserver if I need a blur-up placeholder. For a global audience I'd put an image CDN in front of everything so it can do format negotiation, AVIF or WebP where supported, and automatic resizing based on the requesting device, because a user on a budget phone in a high-latency region shouldn't be downloading the same file as someone on a 4K monitor."

Q3. [Junior] Your team wants to add a performance budget so pages don't get slower over time. For a fairly simple marketing site, what metrics would you actually pick to gate on, and why those?

Answer:

A performance budget is only useful if it tracks things that correlate with what users actually feel, so the first design decision is picking a small set of metrics instead of trying to gate on everything. **For a marketing site, gate on Largest Contentful Paint (LCP, how fast the main content appears), Cumulative Layout Shift (CLS, how much visible content jumps around while loading), and total JavaScript bundle size, since those three map directly to perceived speed and stability without requiring a full lab setup.**

Why These Three

- LCP is the closest single number to "does this page feel fast to load," and it's one of Google's Core Web Vitals, which also ties performance to SEO for a marketing site.
- CLS catches the "I was about to tap that button and an ad shifted the page" problem, which is disproportionately damaging on a marketing or landing page where a bad first impression costs a conversion.
- Bundle size is a leading indicator, it's cheap to measure in CI (continuous integration, the automated pipeline that runs on every pull request) and tends to predict LCP and Time to Interactive (TTI) regressions before they show up in real user data.

What to Leave Out at This Stage

- Skip highly granular metrics like Total Blocking Time or Interaction to Next Paint for a simple marketing site early on, they matter more for interactive apps and add noise before the team has the basics under control.
- Avoid gating on synthetic scores like a single Lighthouse number in isolation, since it bundles multiple signals together and makes it hard to tell a team which specific thing regressed.

Interview Answer:

"For a marketing site I'd keep it to three things: LCP, CLS, and total JavaScript bundle size. LCP and CLS are Core Web Vitals, so they double as SEO signals and they map closely to what a visitor actually perceives as fast or janky. Bundle size is there because it's cheap to check on every PR and it's a leading indicator, it tends to predict LCP problems before they show up in real user monitoring. I'd hold off on things like Total Blocking Time early on, they matter more once the site gets more interactive, and starting with too many gates just trains the team to ignore the dashboard."

Q4. [Junior] Product wants a table that shows a user's full order history, and for power users that can be hundreds of thousands of rows. Would you paginate, render everything, or virtualize, and what's driving that decision?

Answer:

The decision comes down to how many DOM nodes the browser has to create and keep alive, not how many rows exist in the data. **For hundreds of thousands of rows, virtualize (render only the rows currently visible in the viewport, plus a small buffer, and recycle DOM nodes as the user scrolls), since rendering everything would crash or freeze the tab and pagination alone doesn't solve the "power user wants to scroll and scan quickly" use case.**

Why Not Render Everything

- Rendering hundreds of thousands of DOM nodes means hundreds of thousands of elements the browser has to layout, paint, and keep in memory, this will make the tab unresponsive or crash on lower-end devices well before you hit that row count.

Why Not Pure Pagination

- Pagination is a legitimate choice for some products, but it changes the interaction: the user has to click "next" repeatedly instead of scrolling, which is worse for a power user trying to scan or search visually through history.
- Pagination also complicates things like "scroll to a specific date" or keyboard-driven navigation across the full set.

Why Virtualization Fits Here

- A windowing library (react-window, TanStack Virtual) keeps a fixed, small number of real DOM rows mounted, recalculating which rows should be visible as the user scrolls, and reuses those DOM nodes instead of creating new ones per row.
- This gives you the smooth, scrollable feel of "all the data is right here" while keeping memory and render cost proportional to viewport size, not data size.
- It's still worth pairing with server-side pagination or infinite loading of the underlying data, virtualization solves the rendering cost, not the "how do I fetch 300,000 rows" problem, so you'd still fetch in pages behind the scenes even though the UI feels like one continuous scroll.

Interview Answer:

"I'd virtualize rather than render everything or paginate outright. Rendering every row directly is a non-starter at that scale, the browser would choke on the DOM node count alone. Pure pagination avoids that crash but changes the interaction in a way power users usually don't like, they want to scroll and scan, not click through pages. Virtualization gives you the best of both, it only mounts the rows currently in view plus a small buffer and recycles the DOM nodes as you scroll, so the render cost stays constant regardless of how many rows exist. I'd still pair it with paginated or infinite data fetching underneath so we're not pulling all

300,000 rows over the network at once, virtualization solves the rendering problem, not the data-fetching one."

Q5. [Junior] Marketing wants to add a third-party analytics script to every page. How do you make sure it doesn't block or slow down the actual product experience?

Answer:

The core design move is to treat any third-party script as untrusted and non-critical by default, and architect the loading so its failure or slowness can never block your own code. **Load the analytics script asynchronously, off the critical rendering path, using `async` or `defer` (or dynamically injecting the script tag after your app's own critical content is interactive), so a slow or hung third-party server can never delay your first paint.**

Loading Mechanics

- `async` downloads the script in parallel and executes it as soon as it's ready, which can still interrupt rendering at an unpredictable moment. `defer` downloads in parallel but waits to execute until after the HTML is parsed, which is usually the safer default for non-essential scripts.
- Never put a third-party `<script>` tag, blocking, in the `<head>` without one of these attributes, that's a direct hit to your Time to Interactive because the browser stops everything to fetch and run it.

Isolating Failure and Slowness

- Wrap the script load so a failure (blocked by an ad blocker, DNS timeout, 500 from their CDN) doesn't throw an unhandled error that breaks your own app, treat it as fire-and-forget.
- Consider loading it after a `requestIdleCallback` or after your own app's hydration/interactivity event, so it competes for resources only once the user-facing experience is already responsive.

Interview Answer:
"My default assumption with any third-party script is that it could be slow, broken, or blocked, and none of that should be able to touch the core experience. So I'd load it with `defer`, or inject the script tag dynamically after our own app is interactive, rather than putting a blocking script tag in the head. I'd also make sure a failure to load doesn't throw anything that affects our app, it should fail silently from the product's perspective. Basically, analytics should never be able to make checkout slower."

Q6. [Junior] A checkout button, when clicked, takes the user to a payment page that pulls in a decent amount of its own JavaScript. Would you prefetch that page's resources, preload them, or do nothing, and what would you base that decision on?

Answer:

This is about matching the resource hint to how confident you are that the resource will actually be needed. **For a checkout button specifically, prefetch (a low-priority hint that tells the browser "the user will probably navigate here next, fetch it when idle") on hover or on viewport entry is a good fit, because the intent signal is strong and the payoff, a faster perceived transition into payment, is high value.**

The Three Hints and When Each Fits

- `preload` says "I need this resource for the current page, fetch it now at high priority." Use it for something you already know the current page needs, like a critical font or hero image, not for a future page.
- `prefetch` says "I'll probably need this soon, fetch it at low priority when the browser is idle." This fits the checkout button case: hovering or focusing the button is a strong signal of intent.
- `preconnect` sets up the network connection (DNS, TLS handshake) to a domain ahead of time without fetching a specific resource, useful if the payment page loads from a different origin, like a payments provider's CDN.

Why Not "Do Nothing" Here

- Checkout is a high-value, low-ambiguity moment, unlike, say, prefetching every link on a content-heavy homepage where most users will only click one or two links. Wasted bandwidth on hover-to-click for a checkout button is a reasonable trade for a snappier transition into your highest-stakes flow.

Interview Answer:

"I'd prefetch the payment page's resources on hover or focus of the checkout button, not preload them, since preload is for things the current page needs right now. Prefetch tells the browser 'this is likely next, fetch it at low priority when idle,' which fits here because clicking checkout is a strong, deliberate signal, it's not like prefetching every link on a busy homepage where most won't be clicked. If the payment page is served from a different domain, like a third-party payments provider, I'd also add a preconnect so the DNS and TLS handshake are already done by the time the actual navigation happens."

Q7. [Junior] You're reviewing a pull request that adds a new date-formatting library as a dependency. It's a small, quality-of-life addition, but you've seen bundles balloon before. What do you actually check before approving it?

Answer:

The review has to look past "does this work" and evaluate what it costs every single user to download, since a dependency decision made once in a PR review is paid for by every page load afterward. **Check the library's actual shipped size (not its npm description), whether it tree-shakes so you only pay for the functions you use, and whether an existing dependency already covers the same need, before approving any new addition to the shared bundle.**

What to Check

- Look up the real gzip and minified size on a tool like Bundlephobia, a 2KB utility and a 90KB one both "just format a date," and the difference compounds across a team adding dependencies over a year.
- Check whether it supports tree-shaking (bundlers removing unused exports), some date libraries let you import just `format` and get a few KB, others force you to pull in the whole library even for one function.
- Check for overlap: if the codebase already has a date library, a chart library, or a utility belt like Lodash, ask whether this new one is duplicating functionality that already exists somewhere in the dependency tree.
- Check the import path used in the actual diff, it's common for a PR to accidentally import the full library (`import moment from 'moment'`) instead of a scoped, smaller import.

Interview Answer:

"I wouldn't just check that it works, I'd check what it actually costs. First I'd look up its real size on something like Bundlephobia, since a library's npm description doesn't tell you the gzip size. Then I'd check if it tree-shakes properly, so we're only paying for the pieces we import, and I'd check the actual import statement in the diff to make sure it's not accidentally pulling in the whole library. Last, I'd check if we already have something in the dependency tree that does this, because the real risk isn't one small library, it's five small libraries added over a year by five different people, each individually reasonable, that together bloat the bundle."

Q8. [Mid] You want to stop performance regressions from shipping at all, not just catch them after the fact. Design a CI-enforced performance budget: what metrics do you gate on, and what actually happens when a PR blows the budget?

Answer:

A performance budget only works as a system if it's automated, specific per surface, and has a clear, non-negotiable consequence, otherwise it degrades into a dashboard nobody looks at. **Gate CI on a small set of metrics per key route, LCP, CLS, Total Blocking Time, and JavaScript bundle size measured with something like Lighthouse CI or a bundle-size-check action, and fail the build (not just warn) when a PR crosses the threshold, forcing an explicit decision instead of a silent regression.**

What to Gate On, Per Route

- Bundle size per entry point (not one global number, since a marketing page and a data-heavy dashboard have very different reasonable budgets).
- LCP and CLS from a lab tool like Lighthouse CI running against a preview deployment of the PR, not just production data, since you want to catch the regression before merge.
- Total Blocking Time as a proxy for how much the new code will jank interactivity.

What Happens When a PR Blows the Budget

1. CI fails the check and posts the specific delta on the PR (for example, "bundle for `/dashboard` grew by 34KB, budget is +10KB").
2. The author either fixes it (code-split the addition, remove a dependency, defer non-critical work) or explicitly justifies and raises the budget with a reviewer's sign-off, budgets should be adjustable but never silently ignored.
3. Track budget overrides over time, if a team is routinely overriding, that's a signal the budget itself, or the team's architecture, needs a real conversation, not just more overrides.

Best Choice

Fail the build on regression, with an explicit, reviewed override path, rather than a warning-only check.

Why?

A warning that doesn't block merge gets ignored under deadline pressure almost every time. A hard fail forces the trade-off to be made consciously, by a human, at the moment it happens, which is the only point where it's cheap to fix.

```
// lighthouserc.js, CI budget gate example
module.exports = {
  ci: {
    assert: {
      assertions: {
        'largest-contentful-paint': ['error', { maxNumericValue: 2500 }],
        'cumulative-layout-shift': ['error', { maxNumericValue: 0.1 }],
```

```
        'total-blocking-time': ['error', { maxNumericValue: 300 }],
      },
    },
  },
};
```

Interview Answer:

"I'd set up automated budgets per route rather than one global number, since a marketing page and a data dashboard have very different reasonable costs. I'd gate on LCP, CLS, Total Blocking Time from something like Lighthouse CI against a preview build, plus a bundle size check per entry point. And critically, I'd make it a hard fail in CI, not a warning, because warnings get ignored the moment there's a deadline. If a PR blows the budget, the author either fixes it or explicitly justifies raising the budget with a reviewer's sign-off, so it's a conscious trade-off made in the room, not a silent regression that shows up in real user data three weeks later."

Q9. [Mid] Your app's main bundle has grown 40% over the last year even though no single feature seems huge. As the team scales past ten frontend engineers, how do you architect a system to stop this creeping, unbounded growth?

Answer:

Unbounded bundle growth at scale is rarely one big mistake, it's the accumulation of many individually-reasonable decisions made without shared visibility, so the fix has to be a process plus tooling, not a one-time cleanup. **Put a bundle-size check and a dependency review gate in CI on every PR, and pair that with a lightweight dependency budget review for anything that adds a new third-party package, so growth becomes visible and requires a decision instead of happening by accident.**

Automated Visibility in CI

- Run bundle analysis on every PR (a tool like `bundlesize`, `size-limit`, or webpack-bundle-analyzer's stats output) and post the delta directly on the pull request, so the person adding the code sees the cost immediately, not the person who discovers it in a retro six months later.
- Track bundle size over time on the main branch on a dashboard, so the team can see the trend line, not just individual PR deltas.

Dependency Governance

- Require a short justification for any new dependency above a size threshold, similar in spirit to Q7 but as a formal, lightweight process once you have ten-plus engineers who can't all rely on tribal knowledge of what's already in the tree.
- Maintain an internal "approved" list of common utilities (one date library, one HTTP client, one state management approach) so five teams don't independently reach for five different tools that do the same job.

Structural Levers Beyond Process

- Push shared, stable dependencies (a design system, a framework) into a separate vendor chunk that's cached long-term and rarely invalidated, so app code changes don't force users to re-download unchanged framework code.
- For a genuinely large org with independently deployed teams, consider module federation (a webpack/Vite capability letting separately built and deployed apps share code and load each other's modules at runtime) as a structural alternative, so teams ship independently without every team's code landing in one ever-growing bundle. It's a heavier architectural commitment, so it's worth it once team-count, not just bundle size, is the real driver of the growth.

Interview Answer:

"At that scale, the growth is almost never one big feature, it's death by a thousand reasonable-looking PRs. So I'd fix it with visibility plus governance rather than a one-time cleanup. Every PR gets an automated bundle-size check that posts the delta directly on the diff, and any new dependency above a size threshold needs a short justification, similar to a lightweight architecture review. I'd also push shared framework and design-system code into a long-lived vendor chunk so it's not being re-downloaded on every deploy. And if the real driver is that we now have ten independent teams shipping to one bundle, I'd seriously evaluate module federation, so teams can ship independently without everyone's code funneling into a single, ever-growing artifact."

Q10. [Mid] You're building an analytics dashboard: one critical chart the user came for, plus a dozen secondary widgets (recent activity, team leaderboard, tips panel, and so on) below and beside it. How do you architect what loads first versus what can be deferred?

Answer:

The design principle is to identify the one thing the user actually came for and protect its loading path from everything else on the page, treating every other widget as strictly lower priority regardless of how it's laid out visually. **Load the critical chart's data and code on the critical path with high priority, render the rest of the page as lightweight skeletons immediately, and code-split plus lazy-load each secondary widget independently so a slow leaderboard API can never delay the chart the user is actually here for.**

Critical Path: The Chart

- Fetch the chart's data as early as possible, ideally kicked off before or in parallel with the JavaScript bundle finishes loading (for example, via a server-rendered initial payload or a request fired from a `<link rel="preload">`-triggered script).
- Don't bundle the chart's code with all twelve widgets' code, if the chart is the reason the user opened this page, it shouldn't wait behind unrelated widget code in the same chunk.

Deferred: The Secondary Widgets

- Render the page shell and chart area immediately, with skeleton placeholders for every widget, so the layout doesn't shift once real content arrives (protecting CLS).
- Code-split each widget as its own lazy-loaded chunk, and fetch each widget's data independently, a slow "team leaderboard" endpoint should never block the "recent activity" widget next to it, let alone the chart.
- Use an IntersectionObserver to defer widgets that are below the initial viewport (see Q2's above-the-fold logic) until they're about to scroll into view, rather than firing off a dozen network requests immediately on page load.

Why Not Load Everything in Parallel Immediately

- Firing all thirteen data requests and all thirteen chunks at once contends for the same limited number of browser connections and the same main thread, competition that specifically delays the one thing (the chart) that matters most to this page's purpose.

Interview Answer:

"The first question I'd ask is, what did the user actually open this page for, and the answer here is the chart, not the leaderboard widget. So I'd treat the chart as the critical path: its data fetch starts immediately, ideally even before the full JS bundle is done loading, and its code isn't bundled together with the twelve

secondary widgets. Everything else renders as a skeleton right away and is code-split and lazy-loaded independently, so a slow API for the tips panel can't hold up the leaderboard or, more importantly, the chart. And for widgets below the fold, I'd defer their fetch until they're about to scroll into view with an IntersectionObserver, instead of firing thirteen requests the instant the page loads and having them all compete for the same connections."

Q11. [Mid] You're delivering images and video to users across North America, a low-bandwidth region in Southeast Asia, and everywhere in between. How do you architect the delivery pipeline so both ends of that spectrum get a good experience without maintaining two separate systems?

Answer:

The right architecture pushes the format, resolution, and quality decisions to the edge, close to the user, rather than baking one-size-fits-all assets into your build or your origin server. **Put an image/media CDN in front of a single set of high-quality source assets, and let it perform per-request transformation, resizing, format negotiation (serving AVIF or WebP where supported, JPEG otherwise), and quality adjustment based on the requesting device and, where available, network conditions, so one pipeline serves both a 4K monitor in Toronto and a budget Android phone on a 3G connection.**

Single Source, Edge Transformation

- Store one high-resolution master per image, don't pre-generate every possible size and format at upload time, that doesn't scale as you add breakpoints or formats. Let the CDN (Cloudinary, Imgix, or a self-hosted equivalent like an image proxy in front of your CDN) generate variants on request and cache them at the edge.
- Use `srcset` and `sizes` so the browser requests the right resolution for its viewport and pixel density, and let the CDN handle format negotiation via the `Accept` header rather than hardcoding format per device.

Adapting to Network Conditions, Not Just Device

- Where the Network Information API (`navigator.connection.effectiveType`) is available, use it as a signal, on a detected slow connection (2G/slow-3G), request a lower-quality tier or defer non-essential media entirely, rather than assuming device screen size alone tells you about bandwidth.
- For video, use adaptive bitrate streaming (HLS/DASH) so the player itself adjusts quality in real time based on measured throughput, instead of committing to one fixed bitrate for every viewer.

Regional Delivery

- Rely on the CDN's edge network to physically shorten the round trip for users far from your origin, a user in Manila hitting a Singapore edge node instead of a US origin server is a bigger latency win than almost any client-side optimization.

Interview Answer:

"I wouldn't build two systems for two regions, I'd build one pipeline that adapts per request. Store a single high-quality master asset, put an image CDN in front of it that transforms on the fly, resizing and picking AVIF, WebP, or JPEG based on the requesting browser's Accept header, and cache those variants at edge nodes close to the user. On top of that, I'd use srcset and sizes so the browser only requests the resolution it actually needs, and where the Network Information API is available, I'd use effective connection type as a signal to drop to a lower quality tier on a detected slow connection. For video I'd lean on adaptive bitrate

streaming so the player itself reacts to real throughput. The point is the decision about what to serve happens at request time, at the edge, not baked into the build."

Q12. [Mid] You need to virtualize a table where rows have variable heights (some rows wrap to two lines, some to four), there's a sticky header, and product wants a "jump to row 50,000" search feature. What does your windowing implementation need to support, and how do you architect it?

Answer:

Variable row heights break the simplest form of virtualization, which assumes every row is the same fixed height and can therefore compute scroll position with simple multiplication, so the architecture has to track and reconcile actual measured heights instead. **Maintain a position cache (an array or map of measured offsets and heights per row, updated as rows are measured after render) so the virtualizer can compute which rows are visible, keep the sticky header outside that scrolling coordinate system entirely, and implement scroll-to-index as "look up the cached offset, or estimate and correct" rather than a fixed-height calculation.**

Handling Variable Row Heights

- On first render, estimate each row's height (an average or a size based on content type) so you can render something immediately, then measure the real rendered height (via `ResizeObserver` or a ref callback) and update the position cache.
- Maintain a running cache of cumulative offsets, so "what's the scroll position of row N" is a lookup or a binary search into the cache rather than a recalculation of every prior row on each scroll event.
- Expect some scroll-jump correction: when a row above the viewport turns out to be taller or shorter than estimated, the virtualizer needs to adjust scroll offset to prevent the visible content from jumping.

Sticky Header

- Keep the header outside the virtualized, transformed scroll container, position it with `position: sticky` (or a separate fixed element synced to horizontal scroll only) so it's never treated as a "row" subject to recycling or offset calculations.

Scroll-to-Index

1. If the target index's offset is already in the position cache (because it's been rendered before), jump directly using that cached offset.
2. If it hasn't been rendered yet, jump to an estimated offset using the average row height, then re-measure and correct once the real rows around that estimate render, similar to how the initial estimate-then-measure cycle works for normal scrolling.
3. For very large jumps (row 50,000 in a 500,000-row table), this estimate-and-correct approach is unavoidable, you cannot know the exact real height of 50,000 unrendered rows without having rendered them at least once.

```
// Simplified shape of what the windowing engine needs to track
interface RowMeasurement {
  index: number;
  offset: number;   // cumulative pixels from top
  height: number;   // measured, not estimated, once available
}

interface VirtualizerState {
```

```
  estimatedRowHeight: number;
  measurements: RowMeasurement[]; // sparse until rows have been rendered
  scrollToIndex: (index: number) => void;
}
```

Interview Answer:

"The fixed-height trick, where you just multiply row index by row height, doesn't work here, so the core of the design is a position cache: I estimate each row's height up front, render it, measure the real height with something like ResizeObserver, and store that in a cache of cumulative offsets. Scroll position calculations become lookups or a binary search into that cache instead of a fresh calculation every scroll event. The sticky header stays completely outside that virtualized coordinate system, it's positioned with CSS sticky and never treated as a recyclable row. For scroll-to-index, if we've rendered that row before we jump straight to its cached offset, and if we haven't, like jumping to row 50,000 in a half-million row table, we jump to an estimate based on average row height and then correct once the real rows around it render and get measured, because there's no way to know the exact height of unrendered rows in advance."

Q13. [Mid] Support wants to add a third-party live chat widget to every page. In the past, a similar widget from a different vendor caused noticeable input lag on a form-heavy page. How do you architect this so it can't do that again?

Answer:

The design goal is isolation, making sure the chat widget's JavaScript can't compete with your own app for main-thread time or block user interactions, even if the vendor's code is poorly written. **Load the chat widget lazily, only after your own page is interactive, and isolate its execution as much as the vendor's integration allows, via an iframe for DOM/style isolation and, if the SDK supports it, running its script off the main thread using a tool like Partytown (which runs certain third-party scripts inside a web worker via a proxy).**

Defer the Load

- Don't load the widget on initial page load at all for most users, load it after your own critical content is interactive, or only once the user shows intent, like scrolling past a certain point or being on the page for more than a few seconds. Many users never open chat, and it's wasteful to make everyone pay the loading cost of a widget most of them won't use.

Isolate Execution

- If the vendor supports an iframe-based embed, use it, an iframe gets its own separate DOM, style, and (mostly) script execution context, so a heavy or buggy widget can't directly cause layout thrashing or long tasks in your app's own document.
- If the vendor only ships a same-document script and iframe isn't available, evaluate running it through Partytown, a tool that proxies certain third-party scripts into a web worker so their execution doesn't block the main thread where your form's input handling lives.

Cap the Blast Radius

- Set a timeout and error boundary around the widget's initialization so a hung or failing load doesn't leave the page in a broken state, and monitor its real-world impact on your own Core Web Vitals with Real User Monitoring (RUM) specifically segmented by "widget loaded" versus not, so you catch a regression from a vendor update quickly instead of discovering it anecdotally like last time.

Interview Answer:

"Given we've been burned by this before, I'd design for isolation from the start rather than trust the vendor's code to behave. First, I wouldn't load it on initial page load at all, I'd defer it until our own app is interactive, or until there's a real signal of intent like the user scrolling past a certain point. Second, wherever possible I'd use the vendor's iframe embed instead of a same-document script, since an iframe gives real isolation from our own DOM and main thread. If iframe isn't an option and it's a plain script, I'd look at running it through something like Partytown, which proxies certain third-party scripts into a web worker. And I'd specifically monitor our Core Web Vitals segmented by whether the widget loaded, so if a vendor pushes a bad update, we catch it in a dashboard instead of finding out from a support ticket about laggy forms again."

Q14. [Mid] You want to prefetch resources for a user's likely next navigation on a content site, articles with a lot of internal links, but you don't want to waste bandwidth prefetching pages most visitors will never click into. How aggressive would you be, and what would you base the decision on?

Answer:

The right level of aggressiveness is a function of how strong your intent signal is and how expensive being wrong is, so the design should use graduated signals rather than one blanket "prefetch everything" or "prefetch nothing" rule. **Prefetch only on a meaningful intent signal, hover with a short dwell time, touchstart, or a link scrolling into the viewport combined with dwell time on the current article, and respect the user's network conditions and data-saver preference before firing anything, rather than prefetching every link the moment the page loads.**

Graduated Signals, Not All-or-Nothing

- Viewport visibility alone is a weak signal, on an article page most links are visible at some point but most won't be clicked, so visibility alone would prefetch far too much.
- Hover with a short delay (for example, 60 to 100 milliseconds of sustained hover before firing) is a much stronger signal on desktop, most accidental mouse-overs won't sustain that long.
- On touch devices, `touchstart` fires close enough to an actual navigation that prefetching there wastes very little, since a touch that starts a tap is very likely to complete it.
- A library like `quicklink` implements a middle ground automatically: it prefetches links currently in the viewport but only when the browser is idle and only on good connections, deprioritizing the whole thing when resources are scarce.

Respecting the User's Constraints

- Check `navigator.connection.saveData` (the user's explicit data-saver preference) and `navigator.connection.effectiveType`, and skip prefetching entirely on slow connections or when data-saver is on, aggressive prefetching is a bandwidth cost the user didn't necessarily agree to.
- Cap concurrent prefetches (don't fire off ten at once) and only prefetch when the browser reports it's idle, so prefetching never competes with active, user-triggered work.

Interview Answer:

"I wouldn't go with a blanket rule, I'd tier it based on how strong the signal is. Just being in the viewport isn't enough, on an article page most links are visible but most won't get clicked, so that alone would waste a lot of bandwidth. Hover with a short dwell time, something like 60 to 100 milliseconds, is a much better signal on desktop, and on touch, touchstart is close enough to an actual tap that it's basically free to prefetch there. I'd also respect the user's own signals, checking the Network Information API for save-data mode or a slow effective connection type, and skip prefetching entirely in those cases, since it's their bandwidth

budget, not just ours to spend. A library like quicklink already implements a lot of this idle-time, connection-aware behavior, so I'd start there rather than hand-rolling it."

Q15. [Mid] Your organization has four separate frontend teams that used to ship four separate apps, and now leadership wants them stitched into one cohesive product experience. Would you reach for a single monorepo with route-based code splitting, or module federation, and why?

Answer:

The deciding factor isn't bundle size, it's organizational: whether the teams need to deploy independently on their own schedules. **If the four teams need independent deploys, choose module federation (letting each team build and deploy its own app independently while the host app loads their code at runtime as if it were one application), and if they can live with a shared build and release train, a monorepo with route-based code splitting is simpler and should be preferred.**

Best Choice

It depends on deploy independence: module federation if the teams must ship on their own cadence, monorepo with route splitting if a shared release process is acceptable.

Why?

- A monorepo with route-based splitting is architecturally simpler: one build pipeline, one set of shared dependencies deduplicated automatically by the bundler, one performance budget system to maintain. Its cost is coupling, every team's release now depends on the same CI pipeline and the same deploy window.
- Module federation decouples deploys, team A can ship a fix to their section of the product without waiting on or coordinating a release with teams B, C, and D, which matters a lot once four teams have four different velocities and on-call rotations. Its cost is real complexity: shared dependency versions have to be managed carefully (two teams on different React versions can cause runtime errors), and you lose some of the automatic deduplication and bundle analysis simplicity a single build gives you for free.
- A useful gut check: if the honest answer to "why can't this be one monorepo" is "our teams are organizationally separate and need to ship independently," that's a real reason to pay module federation's complexity cost. If the honest answer is just "our repos happen to be separate right now," a monorepo migration is usually the better long-term investment.

Interview Answer:

"I'd base the decision on organizational independence, not bundle mechanics. If these four teams genuinely need to deploy on their own schedules, maybe one owns a checkout flow with its own release cadence tied to payment provider certifications, module federation is worth its complexity, because it lets each team ship independently while still composing into one product at runtime. But if they can live with a shared release process, I'd push for a monorepo with route-based code splitting instead, it's simpler to reason about, you get automatic dependency deduplication from a single build, and one performance budget system instead of four. Module federation solves a real problem, but it's an organizational problem more than a technical one, so I'd want to be honest about whether that's actually the constraint before taking on the complexity."

Q16. [Senior] You've just joined a company where the frontend bundle has grown to 3.2MB, no team owns it, and every team says their own feature is small. Design

the end-to-end system that prevents this from recurring, including how you'd introduce it without stalling every team's roadmap.

Answer:

This requires both a technical system and a rollout strategy, because a perfect budget system that every team resents and routes around will fail just as surely as having no system at all. **Architect a layered system, automated per-route bundle budgets enforced in CI, a lightweight dependency review gate for new packages, a shared vendor chunk for common dependencies, and clear ownership per bundle surface, then roll it out with a grace period and a ratchet (tightening budgets gradually) rather than an immediate hard cutoff on a 3.2MB baseline.**

Diagnose Before Prescribing

- Run a bundle analysis to find where the 3.2MB actually lives: is it duplicate dependencies across teams (three date libraries, two HTTP clients), dead code that's never imported but still bundled, or genuinely necessary feature code. The fix differs a lot depending on which of these dominates.

The System, Layered

1. **Ownership**: assign each major bundle surface (each route, each shared chunk) an owning team, so "nobody owns it" stops being true. Ownership is what makes a budget alert actionable instead of ignorable.
2. **CI enforcement**: bundle-size checks per route on every PR (as in Q8/Q9), failing the build on regression past budget, with an explicit override path that requires a second engineer's sign-off.
3. **Dependency governance**: a lightweight approval step for new dependencies above a size threshold, plus a small "approved utilities" list to stop duplicate libraries from creeping back in.
4. **Structural fixes**: consolidate genuinely duplicate dependencies found in the audit, and split shared, stable code (framework, design system) into a long-cached vendor chunk separate from frequently-changing app code.

Rolling It Out Without Stalling Roadmaps

1. Start by publishing the current baseline per route publicly, visibility alone changes behavior before enforcement even starts.
2. Set the initial CI budget at "current size plus a small margin," not at some aspirational target, teams shouldn't be blocked by history they didn't create on day one.
3. Ratchet the budget down gradually on a schedule (for example, a 5% reduction target per quarter for the worst offenders), giving teams runway to plan the work rather than a fire drill.
4. Fix the worst structural offenders (duplicate dependencies, the biggest dead-code contributors) centrally as a platform-team effort, since that's not fairly a single feature team's backlog item, it's an inherited, collective mess.

Interview Answer:

"First I'd actually diagnose where the 3.2MB is coming from, duplicate dependencies, dead code, or genuinely necessary feature weight, because that changes the fix. Then I'd put a layered system in place: clear ownership per route or bundle surface so alerts have an actual owner, CI checks that fail on regression with a reviewed override path, and a lightweight approval step for new dependencies so we don't recreate this problem in a year. For rollout, I wouldn't set the CI budget at some ideal target on day one, since that would block every team immediately for a mess they inherited. I'd baseline at current size plus a small margin, publish the numbers so there's visibility, and then ratchet the budget down gradually on a quarterly

schedule while a platform effort tackles the worst structural offenders, like duplicate dependencies, centrally rather than asking each feature team to fix a collective problem on their own time."

Q17. [Senior] Design a virtualized table for a trading platform: 200,000+ rows, updates streaming in via WebSocket for visible rows in real time, variable row heights, a sticky header and a sticky first column, sorting and filtering, and a "jump to symbol" search. Walk me through the architecture.

Answer:

This combines everything from basic windowing with two harder problems: two-dimensional stickiness (row and column) and reconciling a live data stream against a virtualized viewport without causing layout thrash or stale renders. **Architect it as a windowing core (tracking measured row offsets, as in Q12) wrapped by a data layer that only subscribes to WebSocket updates for currently-rendered symbols, with sorting and filtering handled server-side or in a Web Worker to keep the main thread free for scroll and update rendering.**

Windowing Core

- Same position-cache approach as Q12: estimate-then-measure row heights, cumulative offset cache for scroll position lookups, sticky header kept outside the virtualized transform.
- Sticky first column (common in trading tables for the symbol/ticker name) needs its own transform axis, independent horizontal scroll for the data columns while the first column stays fixed, typically implemented as two synchronized scroll containers or a CSS grid with a pinned column.

Reconciling Live Data With a Virtualized Viewport

- Don't subscribe to WebSocket updates for all 200,000 symbols at once, that's both a backend cost and a client-side one. Subscribe only to the symbols currently in the rendered window (plus a small buffer), and re-subscribe as the visible range changes on scroll, unsubscribing from rows that scrolled out of view.
- Batch incoming price updates and apply them on animation frame boundaries (`requestAnimationFrame`), rather than re-rendering on every individual WebSocket message, a busy symbol can tick many times a second, and rendering every tick independently would flood the main thread.
- Visually flag updated cells (a brief highlight flash) without re-triggering height measurement for rows whose height didn't actually change, to avoid unnecessary layout recalculation on every price tick.

Sorting, Filtering, and Search at This Scale

- Sorting and filtering 200,000 rows on every keystroke on the main thread will jank the UI. Move that work to a Web Worker, or better, to the server/API layer if the dataset already lives there, so the main thread stays free to handle scrolling and live update rendering.
- "Jump to symbol" reuses the scroll-to-index estimate-and-correct approach from Q12, but the lookup itself (symbol to row index) should be indexed (a hash map) rather than a linear scan, since it needs to feel instant on every keystroke of a search box.

Interview Answer:

"I'd build this in layers. The base is the same windowing core as a variable-height virtualized list: measured row heights in a position cache, sticky header outside the scroll transform. On top of that, this table also needs a sticky first column, which I'd handle with a separate, synchronized horizontal scroll region so the symbol column stays pinned while the rest of the row scrolls. The harder part is the live data. I wouldn't

subscribe to WebSocket updates for all 200,000 symbols, only for whatever's currently rendered plus a small buffer, resubscribing as the visible range changes on scroll. Incoming ticks get batched and applied on animation frame boundaries instead of triggering a render per message, since a hot symbol can tick many times a second. For sort and filter, I'd push that work off the main thread, either to a Web Worker or the backend, so scrolling and live updates don't jank while someone re-sorts 200,000 rows. And jump-to-symbol uses a hash map lookup from symbol to row index, combined with the same estimate-and-correct scroll approach, so it feels instant even before the target row has ever been rendered."

Q18. [Senior] You're designing the frontend architecture for a new analytics dashboard product from scratch, similar scenario to Q10 but now you own the whole stack: rendering strategy, resource loading, and code splitting. What's your end-to-end architecture for making the critical chart appear fast while a dozen secondary widgets load around it?

Answer:

Owning the whole stack means the prioritization decision can start on the server instead of being purely a client-side loading trick, which is strictly more powerful. **Use streaming server-side rendering (SSR, sending HTML to the browser in chunks as it becomes ready, rather than waiting for the entire page to render before sending anything) so the chart's markup and data stream to the browser first, paired with resource hints (`preload` for the chart's critical CSS/JS/data, `fetchpriority="high"` on its assets) and independently code-split, client-lazy-loaded secondary widgets.**

Server-Side: Streaming and Prioritized Data

- Use a streaming SSR framework capability (React's `renderToPipeableStream`, or an equivalent in your framework) with Suspense boundaries around each secondary widget, so the shell and the critical chart flush to the client immediately while slower widgets stream in as their data resolves, without blocking the initial response.
- Kick off the chart's data fetch as the very first thing on the server request, before fetching any secondary widget data, so its result is ready to flush first.

Client-Side: Hints and Splitting

- Add `<link rel="preload">` for the chart's critical JS chunk and its CSS, and `fetchpriority="high"` on any chart-critical image or font, so the browser's own resource scheduler treats it as top priority alongside everything else competing for bandwidth.
- Code-split each secondary widget as its own chunk (same reasoning as Q10), hydrated independently, so a slow-to-hydrate widget doesn't delay the chart becoming interactive.
- Use React's Suspense/concurrent rendering (or the equivalent in your framework) so hydration of secondary widgets doesn't block the main thread from responding to interaction with the already-visible chart.

Tying It Together

1. Server starts the chart's data fetch immediately, streams its HTML first.
2. Browser receives and paints the chart shell essentially immediately, preload hints ensure its JS/CSS arrive with priority.
3. Secondary widgets stream in as their own Suspense boundaries resolve, each hydrating independently once its own chunk and data are ready.

4. Client-side lazy-loading and IntersectionObserver-based deferral (Q10's logic) still apply for widgets below the fold, even within this streaming architecture.

Interview Answer:

"Since I own the whole stack here, I'd start the prioritization decision on the server, not just in client-side loading tricks. I'd use streaming SSR with a Suspense boundary around each secondary widget, so the shell and the critical chart's markup and data flush to the browser first while the other twelve widgets stream in independently as their own data resolves, without blocking that initial response. On the client, I'd back that up with preload hints and high fetch priority on the chart's JS, CSS, and any critical image, and each secondary widget is still its own code-split chunk that hydrates independently, so a slow widget can't delay the chart from becoming interactive. Combined with deferring below-the-fold widgets via IntersectionObserver, this means the chart gets nearly the entire budget of server response time and network priority, and everything else is explicitly secondary."

Q19. [Senior] You've inherited a five-year-old product. Load times have crept up every year, nobody remembers why several of the twenty-plus third-party scripts on the homepage are there, the bundle has three different date libraries, and there's no performance monitoring in place. Diagnose the situation and give me a concrete, prioritized remediation plan, not just a list of ideas.

Answer:

The trap in a situation like this is trying to fix everything at once, which stalls for months and ships nothing. **Sequence the plan as measure first, cut obviously dead or duplicate weight second, then invest in structural, harder-to-regress fixes third, prioritized by highest user impact per unit of engineering effort, not by what's technically most interesting.**

Phase 1: Instrument Before Touching Anything (Week 1-2)

- Stand up Real User Monitoring (RUM, measuring actual users' performance, not just lab tests) and a basic CI performance budget check, even a rough one, before making any changes, you need a before/after baseline or you can't prove any of this worked, and you can't prioritize by real impact without real data on where users actually suffer (which pages, which devices, which regions).
- Run a bundle analysis and a third-party script audit in parallel, since both are quick to gather and inform every later decision.

Phase 2: Cut Obvious, Low-Risk Weight (Week 2-4)

- Third-party script audit: for each of the twenty-plus scripts, find its owner and its actual usage. Anything with a stakeholder who can't justify it, or with analytics showing near-zero engagement, gets removed first, this is usually the single highest-impact, lowest-effort win in a legacy product, because dead weight nobody remembers costs real load time for zero benefit.
- Consolidate the three date libraries down to one, this is mechanical, low-risk work (find/replace usages) with a direct bundle-size payoff, and it's a good early win to build momentum and prove the initiative's value to stakeholders.

Phase 3: Structural Fixes (Month 2-4)

- Introduce route-based code splitting if it isn't there already, this is usually the single biggest lever left after the audit-driven cuts.
- Put the CI performance budget from Q8 into hard-enforcement mode now that a clean baseline exists, so the next five years don't repeat the last five.

- Re-architect the highest-traffic, worst-performing page (identified from the RUM data in Phase 1) specifically, applying the prioritization and loading architecture patterns from Q10/Q18, rather than trying to fix every page at once.

Why This Order

- Measuring first isn't busywork, it's what turns "several third-party scripts" and "load times have crept up" into a ranked, defensible list, and it's what lets you prove impact to leadership, which funds Phase 3's larger structural investment.
- Cutting before rebuilding matters because a legacy codebase's biggest performance problem is very often just accumulated, unowned weight, not a fundamentally wrong architecture, and removing weight is far lower-risk than a rewrite.

Interview Answer:

"I wouldn't start by rewriting anything, I'd start by measuring, because right now every decision would be a guess. So week one is standing up real user monitoring and a rough CI bundle check, plus an audit of exactly what those twenty-plus third-party scripts are and who owns them, and a bundle analysis to see what's actually in there, like the three date libraries. Phase two is cutting obvious dead weight, any third-party script nobody can justify gets removed, and I consolidate the date libraries down to one, both low-risk, mechanical wins that build momentum and prove the effort is working. Only in phase three, once there's a clean baseline and real data on which pages hurt the most, do I invest in the bigger structural work, introducing proper code splitting if it's missing, turning on a hard-enforced performance budget so this doesn't happen again over the next five years, and re-architecting specifically the highest-traffic, worst-performing page rather than trying to fix everything simultaneously. The whole plan is ordered by impact per unit of effort, not by what's most technically interesting to work on."

Q20. [Senior] For a large e-commerce site, would you build a predictive, server-driven prefetching system (using aggregate analytics to decide what to prefetch for a given user segment) or stick with client-side heuristics like hover and viewport signals? Walk me through the trade-off at scale.

Answer:

The trade-off is prediction accuracy and personalization versus operational complexity and staleness risk, and the right answer usually isn't either extreme but a combination where server-driven prediction sets broad priors and client heuristics refine them in real time. **At meaningful e-commerce scale, layer a server-driven model, using aggregate navigation data to predict likely next pages for a given landing page or user segment, as a baseline priority list, and let real-time client-side signals like hover and viewport override or refine that baseline for the specific user in front of you right now.**

Best Choice

A layered approach: server-driven predictions as a default priority ranking, client-side heuristics as the real-time refinement layer on top.

Why?

- Pure client-side heuristics (hover, viewport) are cheap to build and always reflect real-time intent, but they're reactive, they only fire once the user is already showing intent, missing the chance to prefetch something with near-certain demand before any signal appears, like the most common next product from a specific landing page during a flash sale.

- A pure server-driven model (analytics-informed predictions computed offline) can prefetch with zero required user interaction, which is powerful for high-confidence patterns, "80% of users landing on this sale page click into this specific category next," but it's blind to the individual user in front of you, and it risks being stale if built from data that doesn't reflect a sudden shift, a flash sale changing normal traffic patterns overnight, for instance.
- Combining them gets the strengths of both: the server-driven model sets a sensible default prefetch priority for a page before the user does anything, and client-side signals adjust it in real time for what this specific visitor is actually doing, without needing a model retrain to react to an individual's behavior.

Operational Considerations at Scale

- A server-driven model needs a feedback loop and monitoring of its own, wasted-prefetch rate (predicted but never navigated to) and hit rate, since a model that's confidently wrong is worse than no model, it burns bandwidth for a false signal instead of a weak but honest one.
- Still gate everything, model-driven or heuristic, on the user's own network conditions and data-saver preference from Q14, prediction accuracy doesn't override the ethical floor of respecting a user's bandwidth constraints.

Interview Answer:
"I wouldn't pick one or the other exclusively, I'd layer them. A server-driven model built from aggregate navigation analytics can set a sensible default prefetch priority the moment a page loads, for example, knowing that 80% of visitors landing on a flash-sale page click into a specific category next, which a purely reactive client-side heuristic can't do since it only fires once the user shows intent. But I'd still run client-side signals like hover and viewport on top of that, because the server-side model is blind to what this specific visitor is doing right now, and it can go stale if traffic patterns shift suddenly, like during a flash sale that doesn't match historical data. So the model sets the baseline, real-time client signals refine it, and everything, model-driven or not, still respects the user's own network conditions and data-saver settings, since prediction accuracy doesn't give us a pass on wasting someone's limited data plan."

Q21. [Senior] You're the frontend architecture lead for an org with fifty teams sharing a common design system and component library. Design the governance model for performance budgets across the org, who sets them, who enforces them, and what happens when teams disagree.

Answer:

At fifty-team scale, a single central team dictating budgets to everyone becomes a bottleneck and breeds resentment, while fully decentralized, per-team budgets lose the benefit of a shared platform entirely, so the governance model has to split responsibility deliberately between what's centrally owned and what's team-owned. **Have a central platform team own the shared, cross-cutting budget, the design system and vendor chunk's size and its own CI checks, while each product team owns and sets its own route-level budget within org-wide guardrails, with a lightweight, documented escalation path (not a committee) for disputes.**

What's Centrally Owned

- The design system and shared component library's own bundle size budget, enforced in that library's own CI, since a regression there multiplies across all fifty teams' apps.
- Org-wide guardrails: a maximum reasonable budget ceiling per page type (for example, a marketing page shouldn't exceed X, a data-heavy internal tool has a higher, separately justified ceiling), and a

shared measurement toolchain (the same Lighthouse CI setup, the same bundle analyzer config) so every team's numbers are comparable.

What's Team-Owned

- Each team sets and enforces its own specific route-level budget within the org guardrail ceiling, since only that team has the context on their page's actual content needs, a checkout flow and an internal admin tool have very different reasonable budgets even under the same ceiling.
- Each team owns responding to their own CI failures and override decisions, day to day, this shouldn't route through a central team for every single PR, that doesn't scale to fifty teams.

Escalation When Teams Disagree

1. Default path: team lead and one platform engineer review the specific case together, most disagreements are resolved here, for example "is this new dependency worth the 15KB" is usually answerable with the concrete trade-off in front of both people.
2. If unresolved, escalate to a standing, lightweight architecture review (not a large committee, two to three senior engineers with rotating membership) that meets on a fixed cadence, not ad hoc, so it doesn't become a blocker teams route around.
3. Document every override and its reasoning in a shared, searchable log, so repeated disputes over the same category of decision (the same kind of dependency, the same kind of pattern) become visible and can turn into a standing guideline, reducing the same argument happening fifty separate times across fifty teams.

Interview Answer:

"At this scale I wouldn't centralize everything or decentralize everything, I'd split ownership deliberately. The platform team owns the shared design system's own budget, since a regression there hits all fifty teams at once, plus org-wide guardrail ceilings and a shared measurement toolchain so every team's numbers are apples to apples. Within those guardrails, each product team sets and owns their own route-level budget and handles their own day-to-day CI overrides, because routing every PR decision through a central team just doesn't scale to fifty teams. When there's a real disagreement, the first step is just the team lead and a platform engineer talking through the specific trade-off, which resolves almost everything. If it's still unresolved, it goes to a small, standing architecture review that meets on a fixed cadence, not an ad hoc committee people can route around, and every override gets logged with its reasoning, so if the same kind of dispute keeps coming up, that's a signal to turn it into a standing guideline instead of relitigating it fifty separate times."

Q22. [Senior] Your homepage has accumulated fifteen different third-party scripts over several years, analytics, ads, a chat widget, a survey tool, session replay, and more, each added by a different team with a different vendor relationship. Design a system to govern and load all of them without any one of them being able to degrade the core experience.

Answer:

With this many independently-added third parties, the problem is as much about organizational control as loading mechanics, an ungoverned free-for-all where any team can drop in a script tag is what caused the sprawl in the first place. **Architect a centralized script-loading gateway, a single internal system every third-party script must register with (declaring priority tier, consent requirements, and loading strategy) rather than being hand-added to the page by whichever team wants it, so loading order, priority, and**

failure isolation are enforced structurally instead of depending on every team remembering best practices.

A Central Loading Gateway, Not Ad Hoc Script Tags

- Require every third-party script to be registered in a config (owner, purpose, priority tier, whether it needs user consent under privacy regulation) rather than pasted directly into the page's HTML by whichever team owns that vendor relationship, this is what makes governance enforceable instead of aspirational.
- The gateway loads scripts by tier: critical-to-business (payment fraud detection, say) gets a small budget and prioritized loading, "nice to have" (a survey tool, session replay) loads well after the page is interactive, gated on idle time, and in some cases gated on explicit user consent, which also delays loading for users who decline, saving them the cost entirely.

Isolation and Failure Handling

- Every script's initialization is wrapped so a failure, hang, or thrown error can't propagate to the app or to other third-party scripts, one bad vendor script shouldn't be able to take down analytics too.
- Where the vendor supports it, prefer iframe-based embeds or worker-based execution (Partytown, as in Q13) for anything that isn't strictly required to run in the main document context.

Ongoing Governance, Not a One-Time Cleanup

- Require a lightweight architecture sign-off before any new third-party script is added to the gateway config, similar in spirit to the dependency review in Q9, so a sixteenth script doesn't get added the same ungoverned way the first fifteen were.
- Schedule a recurring (for example, quarterly) audit of every registered script against its stated owner and purpose, and remove anything an owner can't justify anymore, that's exactly the mechanism that would have prevented "nobody remembers why several of these are here" from the Q19 scenario.
- Monitor aggregate third-party impact on Core Web Vitals via RUM, segmented by which scripts were loaded for a given session, so a specific vendor's update regressing performance is traceable to that vendor, not just visible as a vague, org-wide slowdown.

Interview Answer:

"With fifteen independently-added scripts, the real problem is that there was no gate stopping any team from just adding a sixteenth the same way. So I'd build a central script-loading gateway that every third-party script has to register with, declaring its owner, its priority tier, and its consent requirements, rather than letting teams paste script tags directly into the page. The gateway loads by tier, genuinely business-critical scripts get priority, and things like a survey tool or session replay load only after the page is interactive and only once the user has consented where required, which also means users who decline never pay the loading cost at all. Every script's init is wrapped so one vendor's failure can't take down another vendor's script or our own app, and where possible I'd push non-essential scripts into an iframe or a worker via something like Partytown for real isolation. And critically, this can't be a one-time cleanup, I'd require sign-off before anything new gets added to the gateway and run a quarterly audit against each script's stated owner and purpose, which is exactly the mechanism that prevents ending up back here with fifteen more scripts nobody can explain in another few years."

If you remember nothing else from this chapter: *a performance number measured on the team's own hardware is a best case, not a real one.*

Chapter 7: Frontend Data Layer, Caching & Real-Time Architecture

Almost every production frontend eventually becomes a data synchronization problem: keeping server state, client cache, and the UI consistent while requests fail, connections drop, and multiple screens show the same underlying record. If you've only ever called fetch inside a useEffect, you'll tend to describe individual requests; if you've owned a real app's data layer, you'll talk about cache keys, invalidation, race conditions, and what happens when a mutation succeeds but the rollback path was never designed. Use the questions below to practice making these architectural calls under real constraints, not just relying on a library that made the decisions for you.

> **War Story:** *A stale-cache bug showed users their old account balance for up to ten minutes after a purchase, because a cache invalidation call was quietly failing and nobody had alerting on cache-write failures.*

Avoid: treating client-side caching as "set a TTL and forget it" with no invalidation-on-write story

Aim For: designing invalidation on write as the primary mechanism and TTL as the backstop, not the other way around

Q1. [Junior] Your team is starting a new internal dashboard on top of a REST API with maybe fifteen distinct resources. Would you write a thin fetch wrapper yourselves, or pull in a full data-fetching library like React Query or SWR? How do you decide?

Answer:

Both solve real problems, but they solve different layers of the problem, so the decision comes down to how much server-state behavior (caching, retries, sync) you actually need versus how much you just need a consistent place to make requests. **The real question isn't "which is better," it's whether your app needs a caching and synchronization engine or just a disciplined place to centralize request logic.**

What a thin wrapper buys you

- A single place to attach auth headers (tokens, tenant IDs), a base URL, and consistent error shaping so every call site doesn't reimplement `try/catch` and status-code parsing.
- Predictable, easy-to-read code: a junior engineer can trace a network call end to end without learning a library's caching model.
- Zero dependency risk and full control, which matters for a small app with few shared entities and little cross-screen data reuse.

What a full data-fetching library buys you

- Deduplication of identical in-flight requests (two components asking for the same user at the same time become one network call).
- Background revalidation, retry policies, and stale-while-revalidate semantics (serving cached data immediately while quietly refetching in the background) without hand-rolling a cache.

- Built-in loading/error/success state machines per query, which removes a large class of "flash of stale data" and race-condition bugs that show up once an app has more than a handful of screens sharing data.

Best Choice

For a 15-resource internal dashboard with light data sharing across screens, a thin wrapper is often enough on day one, but the moment two or more screens need the same entity kept in sync, or you need retry/backoff and background refresh, that's the signal to adopt React Query or SWR rather than rebuilding their feature set by hand.

Why?

Teams that resist adopting a data-fetching library "to keep things simple" often end up reinventing request deduplication, cache invalidation, and stale-data bugs piecemeal, which costs far more engineering time than adopting a well-tested library up front.

Interview Answer:

"I'd start by asking how much data actually gets shared across screens. If it's mostly independent, page-scoped fetches, a thin wrapper around fetch that centralizes auth headers and error handling is enough, and it keeps the codebase simple for a small team. But the moment I see the same entity, like a user or an order, needed by multiple components that need to stay in sync, I'd bring in React Query or SWR, because at that point I'm really building a caching and synchronization layer, and those libraries have already solved deduplication, background refetching, and stale-state handling better than we would in a sprint or two."

Q2. [Junior] You're setting up the API consumption layer for a large frontend app that dozens of engineers will touch. Before picking any specific library, how do you structure this layer so it doesn't turn into a mess of scattered fetch calls?

Answer:

The goal is to isolate three concerns that get tangled together in unstructured codebases: how requests are transported, what each business resource looks like, and how components consume that data. **A well-structured API layer separates the "how do we talk to the network" transport code from the "what does a Product or Order look like" domain code, so a backend contract change touches one file instead of forty components.**

Layering

1. **Transport layer**: one HTTP client (wrapping `fetch` or `axios`) that owns the base URL, auth token injection, retry/backoff, and consistent error normalization.
2. **Domain/service layer**: one module per resource (e.g. `catalogApi.js`, `ordersApi.js`) that knows the actual endpoints and shapes, and translates raw API responses into the shapes the UI expects.
3. **Consumption layer**: hooks or thin data-fetching wrappers that components actually call, so a component asks for "the current user's orders," not for `GET /v3/orders?userId=...`.

Why this matters at scale

- When the backend renames a field or splits an endpoint, the fix is contained to the domain layer instead of a search-and-replace across the app.
- New engineers have one obvious place to add a new resource, instead of guessing where the fortieth `fetch(...)` call should live.

- It gives you a seam to add cross-cutting concerns later, like request logging, feature-flagged endpoints, or swapping REST for GraphQL under the hood, without rewriting every component.

```
// domain/ordersApi.js
export const ordersApi = {
  getByUser: (userId) => http.get(`/users/${userId}/orders`).then(normalizeOrders),
  cancel: (orderId) => http.post(`/orders/${orderId}/cancel`),
};
```

Interview Answer:

"I'd split it into three layers before touching any specific library: a transport layer that owns the HTTP client, headers, and retry logic; a domain layer with one module per resource that knows the actual endpoints and normalizes responses; and a consumption layer of hooks that components call in business terms, like 'give me this user's orders.' The payoff is that when the API changes, which it always does, the blast radius is one file instead of every component that happened to call fetch directly. It also gives me a clean seam to later add things like request batching or swap transport without touching UI code."

Q3. [Junior] What is a Backend-for-Frontend, or BFF, and in what situation would you actually push to have your frontend team own one, instead of just calling the existing microservices directly?

Answer:

A BFF (Backend-for-Frontend) is a thin server layer, owned by or dedicated to a specific frontend, that sits between the client and the "real" backend services and does aggregation, reshaping, and orchestration tailored to that UI's needs. **You push for a BFF when your frontend's data needs don't match any single backend service's shape, forcing the client to orchestrate multiple calls itself; a BFF moves that orchestration to a place better suited to do it.**

When it's justified

- A mobile or dashboard screen needs data assembled from five microservices, and doing that fan-out from the client causes request waterfalls and a chatty, slow screen.
- Different frontends (web, iOS, a partner widget) need meaningfully different shapes or aggregation of the same underlying services, so a shared generic API can't satisfy all of them without becoming bloated.
- You want to hide backend complexity or versioning churn from the client so frontend releases aren't coupled to every backend service's deploy schedule.

When it's not worth it

- A small app with one or two backend services and simple, mostly 1:1 resource mapping doesn't need an extra hop. Adding a BFF here is added infrastructure and an extra team to staff, for no real aggregation benefit.

Interview Answer:

"A BFF is a server layer built specifically for one frontend's needs, usually to aggregate and reshape calls to several backend services into one response tailored for that UI. I'd push for one when I see a screen that has to fan out to four or five services from the client just to render, because that creates slow waterfalls and puts orchestration logic in the wrong layer. A concrete example is a mobile home screen that needs a blend of user profile, recent orders, and recommendations. Owning a BFF lets the frontend team control that aggregation and iterate on it without waiting on every backend team, but if the app only talks to one or two services with a clean 1:1 mapping, I wouldn't add the extra hop."

Q4. [Junior] You're building a social feed: posts, each with nested comments, each comment with an author and a like count, and the UI needs different subsets of that on the feed view versus the post detail view. Would you reach for REST or GraphQL here?

Answer:

This is exactly the shape of problem GraphQL was built for: deeply nested, view-dependent data where a fixed set of REST endpoints either over-fetches or forces many round trips. **When the same nested entity needs different shapes on different screens, GraphQL lets each view ask for exactly the fields and nesting it needs in one request, instead of over-fetching a fixed REST payload or making N follow-up calls per comment.**

Best Choice

GraphQL for this feature.

Why?

- A REST feed endpoint would either return every post with all nested comments and authors (heavy over-fetching for the feed view, which only needs the first two comments) or force the client to make a separate request per post to fetch its comments (an N+1 request pattern that gets worse as the feed grows).
- GraphQL lets the feed query ask for `posts { id, text, comments(first: 2) { author { name }, likeCount } }` while the detail page asks for the full comment tree, from the same schema, without the backend team creating a new bespoke endpoint for every screen variation.
- The trade-off is real: GraphQL adds server-side complexity (resolver design, avoiding the N+1 problem on the server instead, usually with a batching tool like DataLoader) and needs care around caching, since GraphQL responses don't map cleanly onto HTTP caching the way REST resource URLs do.

Interview Answer:

"I'd choose GraphQL here because the core problem is that a nested entity, posts with comments with authors, needs different shapes on different screens. With REST I'm stuck either over-fetching a fixed payload or firing off a request per comment thread, which is a classic N+1 pattern. GraphQL lets the feed view ask for a shallow slice, like the first two comments per post, while the detail view asks for the full tree, off the same schema and the same single request. I'd just go in knowing the backend needs batching, like DataLoader, to avoid pushing that N+1 problem down into its own resolvers, and that caching gets more deliberate than plain HTTP caching gives you for free."

Q5. [Junior] Now flip it: you're building an internal admin tool for support staff to view and edit user accounts and orders, mostly simple CRUD screens, one table, one form. REST or GraphQL?

Answer:

Simple, resource-shaped CRUD (create, read, update, delete) is the case REST was designed for, and introducing GraphQL here adds cost without solving a real problem. **When your screens map cleanly to "one resource, one form, one table," REST's simplicity and free HTTP-level tooling usually beat GraphQL's flexibility, which you're not using anyway.**

Best Choice

REST for this tool.

Why?

- Each screen maps almost 1:1 onto a resource (`GET /users/:id`, `PATCH /users/:id`, `GET /orders?userId=`), so there's no nested, view-dependent shape problem for GraphQL to solve.
- REST gives you HTTP-level caching, simple browser devtools inspection, and straightforward permissions per endpoint, all useful for an internal tool that a small team will maintain for years without much churn.
- Adding a GraphQL server, schema, and resolver layer here is extra infrastructure and onboarding cost for a tool where the flexibility GraphQL provides (arbitrary nested queries, field-level selection) is never actually exercised.

Interview Answer:

"For a straightforward admin CRUD tool, I'd stick with REST. Each screen is basically one resource and one form, so there's no over-fetching or nested-shape problem to solve, and REST gives me simple, cacheable, easy-to-debug endpoints without standing up a GraphQL server and schema for a tool that will never use most of what GraphQL is good at. I'd only reconsider if this admin tool started growing complex nested views, like a single dashboard blending users, their orders, and their support tickets in different shapes per screen."

Q6. [Junior] You're caching server responses on the client for a product listing page that supports search text, category filters, sorting, and pagination. How do you design the cache keys so you don't serve the wrong data for the wrong filters?

Answer:

A cache key has to encode every input that can change the response, or you'll serve stale or simply wrong data when one of those inputs changes but the key doesn't reflect it. **A cache key must be a deterministic serialization of every parameter that affects the response, resource name plus filters plus sort plus page, or two different queries will collide and show one filter's results under another's key.**

Designing the key

- Treat the key as `[resource, params]`, where `params` is a normalized object: `{ search: 'shoes', category: 'mens', sort: 'price_asc', page: 2 }`.
- Normalize before hashing: sort object keys alphabetically and trim/lowercase text inputs, so `{search:'Shoes'}` and `{search:'shoes'}` don't accidentally create two separate cache entries for what the backend treats as the same query.
- Keep pagination as part of the key, not a separate cache dimension, since page 2 of one filter set is not interchangeable with page 2 of another.

```
// query key used by a cache like React Query
const key = ['products', { search, category, sort, page }];
```

Common mistake to watch for

- Using only the resource name as the key (`'products'`) and mutating a single cached array in place as filters change; this causes flicker, wrong-data flashes, and makes back/forward navigation unreliable because there's no distinct entry per filter combination to restore.

Interview Answer:

"I'd build the cache key from the resource name plus every parameter that changes the response, search text, category, sort, and page, normalized consistently, so the same logical query always produces the same key. That's the model React Query uses with array query keys, for example `['products', { search, category, sort, page }]`. The mistake I've seen teams make is caching under just the resource name and mutating one shared array as filters change, which causes stale flashes and breaks back-button navigation because there's no separate cache entry per filter combination to go back to."

Q7. [Junior] You need to show an unread-notifications badge count in the header that other users' actions can change. Would you refresh it by polling on an interval, or by invalidating and refetching only when the current user causes a mutation?

Answer:

These solve different sources of change, so the right answer usually depends on whether the count can change from outside the current user's own actions. **Invalidate-on-mutation covers changes the current user causes themselves, but a badge that can change from someone else's action needs either a push channel or a polling fallback, because there's no local mutation to hook into.**

Best Choice

A short interval poll (or a push-based update if you already have a real-time channel for something else), not invalidate-on-mutation alone.

Why?

- Invalidate-on-mutation only refreshes data after an action the current tab knows about, like the user marking something as read; it does nothing when a teammate's action on another device is what changed the count.
- Polling every 30 to 60 seconds is a pragmatic, low-engineering-cost way to keep a low-stakes number reasonably fresh without building a WebSocket or SSE (Server-Sent Events) channel just for one badge.
- If the app already has a real-time connection open for another feature (say, live chat), piggyback the notification count on that channel instead of adding a second polling loop, since maintaining two independent freshness mechanisms for similar data is a common source of bugs and wasted requests.

Interview Answer:

"For a notification badge, invalidate-on-mutation alone isn't enough, because it only catches changes the current user makes themselves, not something a teammate did on another device. I'd poll on a modest interval, something like every 30 to 60 seconds, since it's a low-stakes number where a little staleness is fine, and that's much cheaper to build than a dedicated real-time channel. But if the app already has a WebSocket or SSE connection open for something else, like live chat, I'd just push the count update over that existing channel instead of adding a second polling loop."

Q8. [Mid] Design a staleness and revalidation strategy for a product listing page where prices and stock levels change throughout the day, but not every second. How do you decide what's "fresh enough," and how do you keep the UI from feeling laggy?

Answer:

This is a stale-while-revalidate problem: you want to show something instantly, usually cached data, while quietly checking in the background whether it's still accurate. **The design decision is choosing a staleTime, how long cached data is trusted without a refetch, separately from a cacheTime, how long unused data is kept around at all, and tuning both to the actual rate of change of the underlying data, not to a single global default.**

Designing the windows

- Give listing data a staleTime of a few minutes, since prices and stock don't change second to second; this avoids refetching on every navigation back to the page while still catching real changes within a session.
- Trigger revalidation on meaningful events instead of only a timer: refocServer on window focus (the user tabbed back after a while), on reconnect after being offline, and on navigation into the page from elsewhere in the app.
- Keep the previously cached page visible while revalidating in the background, only swapping in new data when it arrives, rather than showing a loading spinner over data you already have; that avoids a page that flickers empty every time the user revisits it.

```
useQuery(['products', filters], () => fetchProducts(filters), {
  staleTime: 2 * 60 * 1000,   // trust cached data for 2 minutes
  refetchOnWindowFocus: true, // catch changes when the tab regains focus
  keepPreviousData: true,     // avoid a flash of empty state while refetching
});
```

Where a longer window is wrong

- If this were a flash-sale page with inventory counting down in real time, a multi-minute staleTime would show sold-out items as available; that scenario calls for a much shorter staleTime or a push-based update instead, which is a sign that "staleness window" isn't a one-size setting across an app.

Interview Answer:

"I'd treat this as a stale-while-revalidate design problem: show cached data immediately, and refresh it in the background based on how fast the underlying data actually changes. For a normal product listing, I'd set a staleTime of a couple of minutes, since prices and stock don't move second to second, and trigger revalidation on window focus and reconnect rather than only a timer. I'd also keep previously loaded data visible while refetching so the page doesn't flash empty every time someone navigates back to it. But I'd flag that this window has to match the domain. a flash-sale countdown page needs a much shorter staleness window or a push update, because a few minutes of staleness there means showing sold-out stock as available."

Q9. [Mid] You're building a live comments feature under a blog post: readers see new comments appear without refreshing, but posting a comment is a normal, occasional action. Would you use WebSockets, Server-Sent Events, or polling?

Answer:

The deciding factor is directionality and frequency: this feature only needs the server pushing updates to many readers, not the client streaming continuous data back. **Since the client only occasionally sends data, posting a comment, but needs to receive updates continuously, Server-Sent Events (SSE), a one-way server-to-client push over plain HTTP, fits better than the added complexity of a bidirectional WebSocket.**

Best Choice

Server-Sent Events, with comment posting still going over a normal HTTP POST.

Why?

- WebSockets give you a full bidirectional channel, but this feature never needs the client to stream data continuously back to the server, so that bidirectionality is unused complexity: extra infrastructure (a WS-aware load balancer, connection state to manage) for a capability you're not using.
- SSE runs over plain HTTP, works through most existing infrastructure and proxies without special configuration, and has automatic reconnection built into the browser's `EventSource` API, which removes a whole category of reconnection logic you'd otherwise write yourself for WebSockets.
- Polling would work too and is the simplest to build, but on a busy post with thousands of concurrent readers, a poll every few seconds from every open tab creates far more load than one push per new comment, and it adds visible latency (comments appear a poll-interval late) that a live feature is specifically trying to avoid.

Interview Answer:

"This is server-to-client push with only occasional client-to-server writes, so I'd reach for Server-Sent Events rather than WebSockets. Posting a comment is still a normal POST request, but new comments stream to readers over an SSE connection, which runs over plain HTTP and reconnects automatically through the browser's EventSource API, so I don't have to hand-build reconnection logic. WebSockets would work but they're solving a bidirectional problem I don't have here, and that adds infrastructure cost, like WebSocket-aware load balancing, for capability I'd never use. I'd rule out plain polling mainly because of scale: thousands of readers polling every few seconds on a popular post generates a lot more load than one push per new comment."

Q10. [Mid] You're building an internal operations dashboard, used by about twenty people, that shows metrics refreshing every few seconds. Does this need a push-based real-time channel, or is polling actually the right call here?

Answer:

Real-time infrastructure has a cost, so the right question is whether this specific scenario's scale and freshness needs actually justify it. **With a small, known audience and a multi-second freshness tolerance, a short interval poll is often the pragmatically correct choice, since the operational cost of WebSockets or SSE only pays off once client count or freshness requirements get demanding enough to matter.**

Best Choice

Polling every 3 to 5 seconds, unless the freshness requirement tightens or the audience grows significantly.

Why?

- Twenty concurrent clients polling every few seconds is a trivial load for almost any backend; the operational complexity of maintaining persistent connections (scaling a WebSocket layer, handling reconnects, load-balancer stickiness) isn't justified by this scale.
- A poll-based design is also simpler to reason about for correctness. every request is a clean request/response with no persistent connection state to leak or get stuck, which matters for an internal tool a small team has to maintain.
- The calculus changes if this dashboard later gets embedded for hundreds of customers or the metrics need sub-second freshness (like a trading desk); at that point the polling load and latency both become real problems, and that's when SSE or WebSockets earn their infrastructure cost.

Interview Answer:

"For twenty internal users with a few seconds of acceptable staleness, I'd just poll every three to five seconds rather than standing up a WebSocket or SSE layer. At that scale the extra load from polling is trivial, and I'd rather keep the architecture simple, plain request and response, no persistent connection state to manage, for an internal tool a small team maintains. I'd revisit that decision if this dashboard's audience grew into the hundreds or the freshness requirement tightened to near real time, because that's the point where polling's overhead and latency actually start to hurt."

Q11. [Mid] You're adding collaborative cursors to a Figma-style whiteboard, so users see each other's cursor position update live while they work together. Which transport do you pick, and what does that decision hinge on?

Answer:

This is the case that actually needs a bidirectional, low-latency channel: every client is both sending its own cursor position and receiving everyone else's, at high frequency. **Collaborative cursors need every client to both send and receive high-frequency updates with minimal latency, which is exactly the bidirectional, persistent-connection use case WebSockets exist for; SSE only pushes one direction and polling can't hit the latency bar.**

Best Choice

WebSockets.

Why?

- Every participant is a sender and a receiver of frequent, small updates (cursor x/y, maybe every 50 to 100ms), so a one-way channel like SSE would only solve half the problem; you'd still need a separate channel for the client-to-server direction.
- Polling can't hit the latency bar this feature needs; cursors that update once a second or two feel broken for a "live collaboration" feature where users expect near-instant feedback.
- Beyond transport choice, the design has to include throttling or debouncing cursor events on the client before sending (to avoid flooding the socket with every pixel of movement) and a presence system to know when a user disconnects so their cursor is removed rather than frozen on screen.

Interview Answer:

"Collaborative cursors need bidirectional, low-latency updates, every user is sending their position and receiving everyone else's, so WebSockets are really the only fit. SSE only pushes server to client, so I'd need a second channel for the other direction anyway, and polling just can't get the latency down to something that feels live. Beyond picking WebSockets, I'd design in client-side throttling so we're not sending every pixel of mouse movement, and a presence mechanism so a dropped connection removes that user's cursor instead of leaving it frozen on screen."

Q12. [Mid] You've built a WebSocket-based live feature and now you need to handle what happens when a user's connection drops and reconnects, maybe from a laptop going to sleep or a flaky network. Walk me through how you'd architect reconnection and make sure messages don't arrive out of order or get lost.

Answer:

A dropped WebSocket connection means you've lost an unknown number of messages, so reconnecting the socket alone isn't enough; you need a way to detect what was missed and resync state, not just resume the

stream. **Treat reconnection as a state-resync problem, not just a socket-reconnect problem: reconnect with backoff, then reconcile via a sequence number or snapshot before trusting the live stream again, or you'll silently render on top of missed or out-of-order messages.**

Reconnection

1. Detect the drop (socket `close` or `error` event) and reconnect with exponential backoff plus jitter (increasing delay between attempts, with a bit of randomness) so a server restart doesn't get hammered by every client reconnecting at the exact same instant.
2. Show the user a subtle "reconnecting" state rather than silently freezing, so they know live updates have paused rather than assuming nothing changed.

Message ordering and gap detection

- Give every message a monotonically increasing sequence number (or a server timestamp plus ID) so the client can detect gaps and out-of-order delivery instead of assuming messages arrive in the order sent.
- On reconnect, send the last sequence number the client successfully processed; the server either replays the gap from a buffer or tells the client to fall back to a full resync.
- If the gap is too large to replay cheaply (the client was offline for ten minutes), fetch a fresh snapshot over a normal REST call and then resume the live stream from that point, rather than trying to replay a huge backlog of individual events.

```
// simplified client-side message handling
function handleMessage(msg) {
  if (msg.seq !== lastSeq + 1) {
    return requestResync(lastSeq); // gap detected, ask for replay or snapshot
  }
  lastSeq = msg.seq;
  applyUpdate(msg.payload);
}
```

Interview Answer:

"I treat a dropped connection as a state-resync problem, not just a reconnect-the-socket problem. On the transport side, I reconnect with exponential backoff and jitter so a server restart doesn't get hit by every client at once, and I show the user a reconnecting indicator rather than silently freezing. On the data side, every message carries a sequence number, so when the client reconnects it can tell the server the last sequence it processed. If the gap is small the server replays the missed messages from a buffer; if it's large, like the laptop was asleep for ten minutes, I just pull a fresh snapshot over REST and resume the live stream from there instead of replaying a huge backlog. Without sequence numbers, you can't even detect that you missed something, which is the bug I've seen bite teams that only handle the reconnect and not the resync."

Q13. [Mid] A user taps "like" on a post. Design the optimistic update and its rollback: what actually needs to happen in the UI the instant they click, and what happens if the request fails?

Answer:

Optimistic UI means updating the interface immediately, before the server confirms, on the assumption the request will usually succeed, but the design only really matters in how cleanly you undo it when that assumption is wrong. **The update, the in-flight request, and the rollback have to be one atomic unit of logic, so a failed request restores exactly the prior state rather than leaving the UI in a state the server never agreed to.**

Design

1. On click, immediately flip the like state and increment the count in local state or cache, so the UI feels instant.
2. Fire the mutation request in the background; keep a snapshot of the prior state (liked: false, count: N) captured before the optimistic change.
3. On success, reconcile with the server's actual response (in case count differs, e.g. someone else liked it in between) rather than just trusting the optimistic value forever.
4. On failure, roll back to the captured snapshot and surface a small, non-blocking signal (like the icon briefly shaking, or a toast) so the user understands their action didn't stick, rather than the like silently reverting with no explanation.

```
async function toggleLike(postId, current) {
  const snapshot = current;
  setLikeState(postId, { liked: !current.liked, count: current.count + (current.liked ?
-1 : 1) });
  try {
    const server = await api.toggleLike(postId);
    setLikeState(postId, server); // reconcile with source of truth
  } catch {
    setLikeState(postId, snapshot); // roll back
    showToast('Could not update like, please try again');
  }
}
```

Interview Answer:

"I'd flip the like state and count in local state the instant the user clicks, so the interaction feels instant, but before doing that I capture a snapshot of the prior state. The mutation fires in the background; if it succeeds, I reconcile with whatever the server actually returns, since the real count might have changed if someone else liked it in the meantime. If it fails, I restore the exact snapshot and show a small, visible signal, like a toast or the icon un-liking with a subtle animation, so the user understands their tap didn't stick rather than it just silently reverting with no explanation."

Q14. [Mid] Now scale that up: a Kanban board where dragging a card to a new column is optimistic, but the server can reject the move (say, a business rule like "can't move to Done without a reviewer assigned"). Other parts of the screen, like per-column card counts and an activity feed, also reflect card state. Design how a rollback propagates across the whole feature, not just the dragged card.

Answer:

The moment more than one piece of UI derives from the same piece of state, a rollback can't just be local to the component that triggered the optimistic update; every dependent view has to be driven from that same single source of truth so it rolls back automatically. **Design the board so column counts, the activity feed, and the card itself all read from one normalized store of card state, so rolling back one card's move updates every dependent view for free instead of requiring you to remember every place that needs to be told about the failure.**

Design

- Keep a single normalized store of cards (`{ [cardId]: { columnId, … } }`) rather than each widget (column counts, activity feed) keeping its own derived copy; column counts and the feed should be computed views over that one store, not independently updated.

- On drag, optimistically update the card's `columnId` in that shared store; every derived view (counts, board rendering) re-renders from the same source and is instantly consistent.
- On rejection, revert the card's `columnId` in the shared store to its snapshot; because the counts and other views are derived, not independently mutated, they correct themselves automatically rather than needing a manual "also fix the count" step.
- Still show an explicit signal at the point of interaction, the card animates back to its original column with a brief error message, so the user isn't left wondering why the board "snapped back."
- Log or surface the failed action distinctly in the activity feed if the product wants an audit trail (e.g. "move attempt failed: reviewer required"), rather than silently pretending the attempt never happened.

Why this is the harder version of Q13

- If each widget (counts, feed, board) held its own locally optimistic copy of card state, a rollback would require remembering to individually correct every one of them, and it's very easy to miss one, leaving the count wrong even after the card visibly snaps back. Centralizing state removes that whole class of bug.

Interview Answer:

"The key design decision here is not to let each widget, the board, the column counts, the activity feed, keep its own optimistic copy of card state. I'd centralize card state in one normalized store, keyed by card ID, and have counts and the feed be derived views over that store rather than independently updated. When the drag happens, I optimistically update the card's column in that one place; every dependent view updates for free because they all read from the same source. If the server rejects the move, I revert just that one field in the shared store, and everything downstream, counts included, corrects itself automatically instead of me having to remember every place that needs a manual fix. I'd still animate the card back and show a clear error at the point of interaction, so the user understands why the board changed instead of it just silently snapping back."

Q15. [Mid] A user's display name is editable from their profile page, but it also shows up in the header avatar menu, a team directory list, and the details panel of things they've created. After they save a new name, how do you make sure all of those places update, without over-fetching the entire app?

Answer:

This is a cache invalidation problem where the same entity is rendered in several independent places, so the fix has to target the entity itself, not each screen individually. **Model the user as one cache entity keyed by user ID that every view reads through, so a single mutation updates one cache entry and every screen referencing that entity re-renders consistently, instead of manually invalidating four separate queries and hoping you remembered all of them.**

Design options, in order of preference

1. **Normalized entity cache** (how Apollo's `InMemoryCache` or a normalized Redux/React Query setup works): the user record lives once in cache, keyed by ID; the header, directory list, and details panel all read a reference to that same entity rather than holding their own copies. Updating the one cache entry after the mutation updates every view automatically.
2. **Targeted invalidation by query key/tag**, if a fully normalized cache isn't in place: on successful mutation, invalidate every query tagged with that user ID (e.g. React Query's `invalidateQueries(['user', userId])` plus any list queries known to include that user), forcing them to refetch.

3. **Broad invalidation as a fallback**, only for small apps: invalidate a wide swath of queries after any user mutation; this is simple but wasteful and gets worse as the app grows, since it refetches data that didn't actually change.

Why option 1 is worth the setup cost

- Options 2 and 3 both rely on someone remembering every place a user record appears and keeping that invalidation list current as the app grows; a normalized cache removes that maintenance burden structurally, since there's only one copy of the entity to update.

Interview Answer:
"I'd treat the user as one entity in a normalized cache, keyed by user ID, that the header, the directory list, and the details panel all read through rather than each holding its own local copy. That way, when the mutation to update the display name succeeds, I update that single cache entry and every view referencing it re-renders automatically and consistently. That's the model Apollo's InMemoryCache uses, and you can get similar behavior with React Query if you're deliberate about deriving list and detail views from shared entity data. The alternative, manually invalidating four separate queries after every mutation, works at first but becomes a maintenance trap as the app grows, because it's very easy to add a fifth place the name is shown and forget to add it to the invalidation list."

Q16. [Senior] You're designing the data layer for a field-service app used by technicians who often work with no connectivity: they need to view job details and submit updates (photos, notes, status changes) that get synced once they're back online. Design the offline-first architecture: what does a "local-first write" mean here, and how does sync work?

Answer:

Offline-first means every write the user makes is committed to local storage immediately and treated as the source of truth for the UI, with syncing to the server happening asynchronously and separately, rather than the UI waiting on network success to consider the action "done." **Design writes to land in a local, durable store first and an outbox queue second, so the UI never blocks on network availability and a sync engine can retry the outbox independently of whatever the user is doing right now.**

Architecture

1. **Local store**: use IndexedDB (or a wrapper like Dexie) to hold both the technician's working data (jobs, notes, photos) and an outbox of pending mutations, since local storage needs to survive app restarts and outlast a single session.
2. **Write path**: a technician's action (mark job complete, attach a photo) writes to the local store immediately and enqueues a corresponding mutation record in the outbox; the UI reflects this local write right away, it doesn't wait on the network.
3. **Sync engine**: a background process, using the Background Sync API where supported or a simple retry loop on reconnect, drains the outbox in order, sending each queued mutation to the server and marking it synced (or failed) as responses come back.
4. **Idempotency**: tag each outbox mutation with a unique client-generated ID so that if a request succeeds on the server but the response is lost (e.g. connection drops right after), a retry doesn't create a duplicate job update; the server can dedupe by that ID.

5. **Conflict surfacing**: when the server rejects a queued mutation (stale data, permission changed while offline), surface that as a distinct sync-error state on that record, rather than silently dropping it or endlessly retrying a request that will never succeed.

Interview Answer:
"The core idea is that every write lands locally first, in something like IndexedDB, and the UI treats that local write as done immediately, it doesn't wait for the network. Alongside the actual data, I'd keep an outbox of pending mutations, each tagged with a client-generated idempotency ID so retries after a dropped connection don't create duplicates on the server. A background sync process drains that outbox whenever connectivity comes back, whether that's the Background Sync API or a simple retry loop, and updates each record's status as synced or failed. The part people forget is handling permanent failures distinctly, like the job was reassigned while the technician was offline, so those show up as a clear conflict for the user to resolve rather than retrying forever or silently vanishing."

Q17. [Senior] Continuing that field-service scenario: a technician edits a job's notes offline at 9am, and a dispatcher edits the same job's notes online at 9:15am, before the technician's device reconnects and syncs at 10am. Design the conflict resolution: what happens to that record?

Answer:

There's no universally "correct" resolution here, the right strategy depends on whether the data can be merged automatically, needs a human decision, or is safe to resolve with a simple rule, so the design has to pick a strategy per field or per entity, not one blanket policy for the whole app. **Decide conflict resolution at the field or operation level, not the whole-record level: some fields (like a status enum) can use a safe automatic rule, while free-text fields like notes usually need either a merge strategy or a surfaced choice, because silently picking one side always risks quietly discarding real information.**

Options, and when each fits

- **Last-write-wins by timestamp**: simplest to implement, fine for fields where losing one edit is low-stakes (e.g. a "priority" flag), but wrong for freeform notes, since it silently discards one person's genuine input with no record it ever existed.
- **Field-level merge**: track edits per field rather than per record; if the technician only changed the "notes" field and the dispatcher only changed the "assigned reviewer" field, both changes can be applied without conflict, since they don't actually overlap.
- **CRDML/CRDT-style merge (Conflict-free Replicated Data Type)**: for structured data like counters or append-only logs, use a data structure designed to merge concurrent edits deterministically without a central coordinator; this is often overkill for a full note field but works well for things like a checklist of completed steps.
- **Surfaced manual resolution**: when both parties genuinely edited the same field (both edited "notes"), the safest design is to keep both versions, timestamp them, and show the dispatcher (or whoever is online at sync time) a simple "your version / their version, pick one or merge" prompt, rather than guessing.

Design for this scenario

- Use field-level merge as the default rule, so non-overlapping edits from the technician and dispatcher both apply cleanly.

- For the specific case where both edited notes, don't silently overwrite; store both versions with author and timestamp, and surface a resolution prompt to the dispatcher (who has connectivity) rather than to the technician (who might not reconnect again for hours).

Interview Answer:
"I wouldn't pick one blanket rule for the whole record, I'd resolve at the field level. If the technician changed the notes and the dispatcher changed the assigned reviewer, those aren't actually in conflict, so both apply cleanly. The real conflict is when both edited the same field, notes in this case, and for that I would not do silent last-write-wins, because that quietly throws away one person's real input with no trace. Instead I'd keep both versions with their author and timestamp and surface a short resolution prompt to whoever's online at sync time, since guessing which edit matters more is exactly the kind of decision that should go to a human, not a heuristic. For lower-stakes fields, like a status flag, I'd allow simple last-write-wins by timestamp, since the cost of occasionally losing that edit is much lower than the cost of building conflict UI for every field."

Q18. [Senior] Design the pagination and data architecture for an infinite-scrolling feed, like a social timeline, that needs to support smooth scrolling through potentially thousands of items. What does the cache and fetching strategy look like?

Answer:

Infinite feeds fail in predictable ways when built naively: offset-based pagination breaks when items are inserted or removed mid-list, and holding every loaded item in memory eventually kills scroll performance, so the design has to address both from the start. **Use cursor-based pagination (each page's request carries a pointer to "after this item," not a page number) paired with a normalized cache of pages plus a virtualized list, so the feed stays correct under concurrent inserts/deletes and stays fast regardless of how many items the user has scrolled through.**

Pagination strategy

- Use cursor-based pagination, where each request says "give me items after cursor X," instead of offset/page-number pagination; offset pagination breaks when items are added or removed between requests, causing skipped or duplicated items as the list shifts underneath the user.
- Store the cache as an ordered list of page results plus a normalized entity map (`{ [postId]: post }`), so an individual post's data (like its like count) can be updated once and reflected wherever it appears, instead of duplicated across page arrays.

Rendering and memory

- Virtualize the list (render only the DOM nodes for items currently near the viewport, using a library like `react-window` or `react-virtual`) so memory and render cost stay roughly constant no matter how far the user has scrolled, rather than growing linearly with total items loaded.
- Keep a cap on how many pages are retained in memory for very long sessions, evicting far-offscreen pages' DOM (while keeping lightweight metadata like scroll position) if the feed is expected to be scrolled for a very long time.

Fetching behavior

- Prefetch the next page slightly before the user reaches the bottom (e.g. when they're within two screen-heights of the end) so loading doesn't visibly interrupt scrolling.

- Preserve scroll position and cached pages when the user navigates away and back (e.g. into a post and back to the feed), rather than refetching from page one, which is a common source of "the feed jumped back to the top" complaints.

Interview Answer:
"I'd use cursor-based pagination rather than page numbers, since offset pagination breaks the moment items are inserted or removed while the user is scrolling, causing duplicates or skipped posts. On the cache side, I'd keep an ordered list of pages plus a normalized map of post entities by ID, so something like a like count can be updated in one place and shown correctly everywhere that post appears. For rendering, I'd virtualize the list so we're only rendering DOM nodes near the viewport, which keeps performance flat no matter how far someone has scrolled. And I'd make sure navigating into a post and back preserves the cached pages and scroll position, since refetching from the top on every back-navigation is one of the most common complaints users have about infinite feeds."

Q19. [Senior] That same feed also needs to support live inserts: new posts from people the user follows should be able to arrive in real time while the user is scrolled down reading older content. How do you reconcile "infinite scroll going down" with "new items appearing at the top" without wrecking the user's scroll position or reading flow?

Answer:

The failure mode to avoid is inserting new items directly into the top of a list the user is actively scrolled through, since that shifts every item below it and yanks the user's scroll position or reading focus. **Buffer live inserts separately from the paginated list and surface them as an explicit "N new posts" affordance the user opts into, rather than splicing them into the live DOM automatically, so scroll position and reading flow are never disturbed by something the user didn't ask for.**

Design

1. Keep the live-update channel (WebSocket or SSE) separate from the paginated fetch/cache; incoming live posts land in a small pending buffer, not directly into the rendered list.
2. Show a non-disruptive affordance, a pill or button like "12 new posts" pinned near the top of the viewport, rather than auto-inserting; this is the same pattern most timeline products use, and it exists specifically to avoid the scroll-jump problem.
3. Only when the user taps that affordance (or scrolls back to the top themselves) do the buffered posts get merged into the head of the paginated list and rendered, at a point where the user has explicitly signaled they're ready for the view to change.
4. If the user is already at the very top and idle, it's reasonable to auto-insert, since there's no reading position to protect at that moment, but this should be a deliberate, narrow exception, not the default behavior while scrolled into the list.
5. Keep the normalized entity cache from Q18 as the single write target: whether a post arrives via initial page load or a live insert, it's written into the same entity map, so there's exactly one code path for "a post exists in this feed," not two divergent ones for paginated versus live data.

Interview Answer:
"The thing to avoid is splicing new posts directly into a list the user is actively scrolled through, because that shifts everything below and breaks their scroll position and reading flow. So I'd keep the live channel, WebSocket or SSE, feeding a small pending buffer that's separate from the rendered, paginated list, and surface it as a '12 new posts' affordance near the top instead of auto-inserting. Only when the user taps

that, or is already sitting at the top with nothing to disrupt, do those buffered posts actually get merged into the list. Underneath that, I'd make sure both the paginated fetch and the live channel write into the same normalized entity cache, so there's one source of truth for a post's data regardless of how it arrived, rather than two separate code paths that can drift out of sync."

Q20. [Senior] You review a PR for a trading-style dashboard with thirty independent widgets. Each widget was built by a different contributor and each one runs its own `setInterval` to poll its data every 500 milliseconds. What's the architectural problem, and how would you redesign it?

Answer:

The problem isn't polling itself, it's that each widget independently owns its own polling loop with no coordination, which multiplies request volume by the widget count and very likely re-fetches overlapping data multiple times per second. **Thirty independently polling widgets turn one dashboard load into sixty requests per second with no shared caching, so the fix is to centralize data fetching and real-time updates behind one shared layer that widgets subscribe to, instead of each widget owning its own fetch loop.**

What's wrong

- Request volume scales linearly with widget count regardless of whether widgets need overlapping data; if five widgets all show different views of the same underlying "positions" data, that's five redundant polling loops fetching the same thing every 500ms.
- No shared cache means no deduplication and no consistency guarantee. two widgets showing the same underlying number could display it slightly differently for a moment if their independent poll timers land at different offsets.
- Thirty independent `setInterval` timers is also a real client-side performance cost (thirty timer callbacks, thirty fetches, thirty re-renders competing for the main thread every half second), separate from the network cost.

Redesign

1. Replace per-widget polling with one shared real-time channel, most likely a single WebSocket connection (this data changes fast and needs low latency, similar reasoning to Q11), that the app owns centrally.
2. Introduce a pub-sub layer on top of that connection: incoming messages are tagged by entity/topic (e.g. `positions`, `orders`, `marketData`), and widgets subscribe only to the topics they actually need, rather than each polling the whole dataset.
3. Feed updates into a normalized shared cache/store (same pattern as Q15 and Q19), so if multiple widgets render the same entity, there's exactly one place holding its current value, and all subscribed widgets re-render consistently from that one update.
4. Widgets become dumb subscribers: `useEntity('position:AAPL')` style hooks that read from the shared store and re-render on change, with zero knowledge of transport or fetch timing.

Interview Answer:

"The real problem isn't that it polls, it's that thirty widgets each independently own their own poll loop with zero coordination, so you get up to sixty requests a second with no deduplication, plus thirty separate timers competing for the main thread. I'd redesign this around one shared connection, likely a single WebSocket given the sub-second freshness need, with a pub-sub layer on top so each widget subscribes only to the topics it actually cares about, like 'positions' or 'orders.' Updates would land in one normalized shared store, so if three widgets show the same underlying position, there's exactly one place holding that value and all

three update consistently from a single message. Widgets become simple subscribers to that store, with no knowledge of polling or transport at all, which also means a new widget can be added without adding another fetch loop to the page."

Q21. [Senior] Design the shared caching architecture for a dashboard product with dozens of independently-built widgets, each fetching its own data, but several widgets display the same underlying entities, for example a "Deal" record shown in a pipeline widget, a revenue widget, and a recent-activity widget. How do you avoid duplicate fetches and, more importantly, avoid one widget showing a stale copy of a Deal that another widget just updated?

Answer:

The failure mode to design against is each widget team treating its fetched data as private, which leads to the same entity being fetched N times and, worse, N independent copies that can silently drift out of sync the moment any one of them changes. **Every widget must read shared entities through one normalized cache keyed by entity ID, and any mutation must write back to that same cache, so all widgets referencing "Deal:482" are reading the same object and update together the instant it changes, instead of each holding its own copy that can go stale.**

Architecture

1. **Normalized entity store**: cache data by entity type and ID (`deals: { 482: {...} }`, `activities: { ... }`), not by "whatever shape widget X's query returned." Every widget's query result gets normalized into this store on arrival, and widgets render by reading references into it (this is the same core idea Apollo's `InMemoryCache` and Redux normalized state use, applied dashboard-wide).
2. **Request deduplication and batching**: if the pipeline widget and the revenue widget both request Deal 482 within the same tick, the fetching layer collapses that into a single request rather than firing two, and where possible, batches multiple different entity requests together (a DataLoader-style pattern, or a BFF endpoint that accepts a batch of IDs) rather than one HTTP request per widget.
3. **Single write path for mutations**: any action that changes a Deal (updating its stage, its owner) goes through one mutation function that updates the normalized store directly (or triggers a targeted invalidation of that entity), so every widget referencing that Deal re-renders from the updated value without needing to know a mutation happened elsewhere on the page.
4. **Widget contract**: widgets don't fetch raw endpoints directly; they declare the entities/queries they need (`useEntity('deal', 482)`, `useQuery('pipelineSummary')`) against the shared data layer, so the data layer, not each widget author, controls fetching, caching, and dedup policy. This also means a new widget added later automatically benefits from whatever entity is already cached, rather than a contributor having to know to "check if someone else already fetched this."
5. **Staleness and background refresh**: apply the same staleTime/background-revalidation thinking from Q8 per entity type, e.g. Deals refresh more eagerly than a slower-changing entity like account metadata, so widgets don't all inherit one global freshness policy that's wrong for most of them.

```
// widget-facing contract, transport/caching hidden behind it
function PipelineWidget() {
  const deals = useEntities('deal', dealIds);   // reads normalized shared cache
  ...
}

function RevenueWidget() {
  const deal = useEntity('deal', 482);          // same underlying cache entry
  ...
```

```
}
```

Interview Answer:

"The design principle is that no widget owns a private copy of shared data, everything reads through one normalized entity cache keyed by type and ID, so a Deal exists exactly once in memory no matter how many widgets reference it. On top of that I'd add request deduplication and batching, so if the pipeline widget and the revenue widget both ask for the same Deal, or for different Deals, in the same tick, that collapses into one request or one batched call instead of firing off separately per widget. Mutations go through a single path that updates that same normalized store, so when a Deal's stage changes, every widget showing that Deal re-renders automatically, nobody has to manually tell the revenue widget that the pipeline widget just changed something. And I'd expose this to widget authors as a contract, hooks like useEntity, so individual widget teams never touch raw fetch logic or have to coordinate with each other directly, the shared data layer is what actually enforces consistency."

Q22. [Senior] Zoom out: you're the engineer kicking off the frontend architecture for a brand-new analytics product. It'll have a multi-widget dashboard home screen, some data needs to update live, and customers have asked for basic offline viewing of recently loaded data on flaky connections. Multiple squads will build widgets independently over the next year. How do you decide which of the patterns we've discussed, BFF, GraphQL, normalized caching, WebSockets, offline sync, actually get built, and in what order?

Answer:

The trap here is building every pattern this chapter covers on day one because they're all individually reasonable; the senior move is sequencing investment against what will actually break first as the product and org scale. **Build the shared normalized cache and widget data contract first, since every later pattern (real-time updates, offline support, a BFF) plugs into that same store, and retrofitting a shared cache after dozens of widgets already fetch independently is far more expensive than building it up front.**

Sequencing and reasoning

1. **Normalized cache and widget contract, from day one.** With multiple squads building widgets independently over a year, this is the piece that prevents the Q20/Q21 problem (duplicate fetches, inconsistent entities) from ever accumulating; retrofitting it after thirty widgets exist means rewriting all of them, so it has to be the foundation, not an optimization.
2. **API consumption layer decision (REST vs GraphQL, and whether to stand up a BFF) next**, driven by the actual shape of the home screen. If the dashboard needs many overlapping entities assembled from several backend services, invest in a BFF (and consider GraphQL if screens have deeply nested, view-dependent needs) early, since this decision is expensive to reverse once dozens of widgets have coded against a particular API shape.
3. **Real-time layer third**, added as a subscription mechanism on top of the existing normalized cache (as in Q20), not as a parallel system. Only build this once you know which specific widgets actually need sub-minute freshness. don't wire every widget to a live channel by default when most dashboard data tolerates a short poll or staleTime window.
4. **Offline support last, and scoped narrowly.** "Basic offline viewing of recently loaded data" is a much smaller problem than full offline-first writes with conflict resolution (Q16/Q17): it likely just means persisting the normalized cache to IndexedDB and serving it when the network is unavailable, read-only, without building a sync/outbox engine the requirement didn't actually ask for.

Why this order

- Each later layer depends on the first: a real-time subscription system and an offline persistence layer both need something to write into, which is the shared cache. Building them before the cache exists means building throwaway plumbing that gets replaced once the cache is introduced.
- Sequencing this way also means the org gets the biggest risk-reduction (avoiding thirty independently-fetching widgets) first, while the more speculative pieces (exactly how much real-time is needed, how much offline is truly required) are decided closer to when squads actually build against them, with real requirements instead of guesses.

Interview Answer:

"I wouldn't build all of these on day one just because they're all reasonable patterns individually. I'd start with the normalized cache and a widget data contract, because that's the piece that prevents the classic multi-widget failure mode, duplicate fetches and inconsistent copies of the same entity, and it's far more expensive to retrofit once thirty widgets already exist than to establish up front. Next I'd settle the API shape, REST, GraphQL, or a BFF, based on how nested and overlapping the actual home-screen data is, since that's also expensive to change once squads have built against it. Real-time updates come after that, layered on top of the same cache as a subscription mechanism, and only for the specific widgets that actually need sub-minute freshness, not everything by default. And I'd scope offline support narrowly to what was actually asked for, read-only access to recently loaded data from a persisted cache, rather than building a full offline-write sync engine with conflict resolution that nobody requested yet. The theme across all of it is sequencing investment by what will actually break first, not building every pattern because it's available."

If you remember nothing else from this chapter: *a cache with no invalidation story isn't a performance feature, it's a bug waiting for someone to notice.*

Chapter 8: State Management Architecture

State management questions separate people who have memorized "Redux has a store, actions, and reducers" from people who have actually felt the pain of prop drilling, watched a Context provider re-render half a page, or shipped a stale cache bug to production. Use this chapter to practice treating state as a design problem, choosing a tool based on the shape and lifecycle of the data, rather than reaching for whatever library you used at your last job out of habit.

War Story: *A single global store that grew to hold every modal's open-or-closed flag eventually caused unrelated parts of a page to re-render on every click, and nobody could tell why the app felt sluggish until a render profiler pointed straight at it.*

Avoid: defaulting every piece of state to global without asking whether it's actually shared across the app

Aim For: explicitly reasoning about state scope, local versus shared, before reaching for a global store

Q1. [Junior] Say a component has a boolean flag for whether a dropdown menu is open. Would you put that in Redux, in Context, or just in local component state, and how do you decide?

Answer:

This is a warm-up to see whether you reflexively reach for a library or actually reason about where state needs to live. The dropdown's open/closed flag is a textbook case for local state (`useState` in that one component) because nothing outside that component tree cares about it.

What "Global State" Actually Means

- Global state is any data that more than one component needs to read or write, especially components that do not have a direct parent-child relationship. Examples include the logged-in user's profile, the current theme, or a shopping cart total shown in both a header badge and a checkout page.
- The label "global" is about scope of access, not about which tool stores it. You can have genuinely global state in a single `useState` at the top of the app just as easily as in Redux.

When Local State Is Enough

- If only one component (and its direct children via props) ever reads a piece of state, and no other part of the tree needs it, local state is simpler, faster, and easier to reason about than any shared store.
- **The default should always be local state; you only reach for something global when you can point to at least two unrelated parts of the UI that need the same data.**
- A dropdown's open state, a form field's current input value, and hover state are all classic local-only cases.

Interview Answer:

"I'd default to local state here. The dropdown's open/closed flag only matters to that one component, nobody else in the app needs to read or react to it, so putting it in Redux or Context would just add indirection with zero benefit. I only promote state to something global once I can point to two or more

components in different parts of the tree that genuinely need to share it. Reaching for a global store by default is a smell, it usually means someone copied a pattern without asking whether it was needed."

Q2. [Junior] I've got a `UserProfile` component eight levels deep in the tree that needs the current user's name, and the only way to get it there right now is passing `user` as a prop through every component in between. What's wrong with that, and what's this problem usually called?

Answer:

This is prop drilling, passing data through several layers of components that don't use the data themselves, just to hand it down to a descendant that does.

Why Prop Drilling Is a Problem

- Every intermediate component has to accept and forward a prop it has no actual use for, which couples components that shouldn't know about each other and makes the component signatures noisier.
- **Prop drilling itself isn't a bug, it's the motivating pain that pushes teams toward global state tools like Context or Redux once passing props through five or more layers becomes unmanageable.**
- Refactoring becomes risky: renaming or restructuring one of the intermediate components can silently break the chain of props being forwarded.

What It Costs in Practice

- Adding a new field to `user` means touching every intermediate component's prop types, even ones that never read `user` directly.
- It makes components harder to reuse in isolation, since a component in the middle of the chain can't be dropped into a different part of the tree without also wiring up the same pass-through props.

Interview Answer:
"That's prop drilling, passing a value down through components that don't need it themselves just to reach one that does. It's not dangerous by itself, but it couples every intermediate component to data it doesn't use, and it makes the codebase brittle because changing the shape of `user` means touching six files that never actually read it. That pain is exactly why tools like Context or Redux exist, they let the `UserProfile` component grab `user` directly without every component in between having to relay it."

Q3. [Junior] Walk me through how you'd use React's Context API to make the current user available to any component in the tree without prop drilling.

Answer:

Context lets a provider component expose a value at some point in the tree, and any descendant can subscribe to that value directly via a hook, skipping the intermediate layers entirely.

How It Works

- You create a context object with `createContext`, wrap the relevant part of the tree in a `<UserContext.Provider value={user}>`, and any descendant calls `useContext(UserContext)` to read `user` directly.

- **Context solves the wiring problem of prop drilling by letting a component subscribe directly to a value higher in the tree, without every component in between needing to know about it.**
- It's built into React, so there's no extra dependency for this kind of "pass this value down" use case.

A Realistic Setup

- A common pattern is wrapping the app in an `AuthProvider` that exposes `{ user, login, logout }`, so any component (a header avatar, a settings page, a permissions check) can call `useAuth()` and get what it needs.
- The provider itself usually still holds its own `useState` internally; Context is the transport mechanism, not the state storage mechanism.

Interview Answer:
"Context solves the wiring problem, not the storage problem. You create a context, wrap part of the tree in a provider that holds the actual value, usually backed by `useState` or `useReducer`, and any descendant component calls `useContext` to read it directly instead of having it passed down as props through every layer. For something like the current user, I'd build an `AuthProvider` that exposes `user`, `login`, and `logout`, and any component that needs auth info just calls a `useAuth` hook."

Q4. [Junior] Some teams use Context as their entire state management solution and just accept it. What's the real limitation that makes that a bad fit for state that changes often, like a live cursor position or a websocket message feed?

Answer:

Context is not built for high-frequency updates, its re-render behavior is coarse compared to a purpose-built state library.

The Re-render Problem

- Every component that calls `useContext` on a given context re-renders whenever that context's value changes, even if the component only cares about one small piece of a larger object.
- **Context wasn't designed as a performance-optimized state management tool, it was designed to avoid prop drilling, so any value that updates frequently will trigger broad re-renders across every consumer.**
- There's no built-in way to subscribe to just a slice of the context value the way Redux's `useSelector` lets a component subscribe to one specific piece of the store.

Where It Actually Breaks Down

- A live collaborative cursor position updating 30 times a second through Context would re-render every consumer of that context on every tick, even components that only display a static label.
- The usual workaround is splitting context into smaller, more targeted providers, or wrapping consumers in `React.memo` (a way to skip re-rendering a component if its props haven't changed), but at that point you're doing manual work that a library like Redux or Zustand gives you for free through selective subscriptions.

Interview Answer:
"Context re-renders every component that reads it whenever the value changes, there's no partial subscription like you get with Redux selectors. That's fine for something like a theme or the logged-in user that changes rarely, but it falls apart for anything high-frequency, like a live cursor position in a collaborative editor or a stream of websocket messages. Every consumer would re-render on every update whether it

cares about that specific piece of data or not. At that point I'd reach for something like Zustand or Redux that supports selecting just the slice of state a component actually needs."

Q5. [Junior] Explain Redux's core pieces to me, store, actions, reducers, dispatch, as if you were teaching a junior on your team who's never touched it.

Answer:

Redux keeps all application state in a single store, and the only way to change that state is to dispatch an action describing what happened, which a reducer function turns into a new state.

The Four Pieces

- The store holds the entire application's state as one JavaScript object, and it's the single source of truth that every connected component can read from.
- An action is a plain object describing something that happened, like `{ type: "cart/itemAdded", payload: { id: 42 } }`, it's a description of intent, not the state change itself.
- **A reducer is a function that takes the current state and an action and returns a brand-new state object, and it never mutates the state it's given.**
- Dispatch is the function you call to send an action to the store, which triggers the reducer and updates the state, which then notifies any subscribed components to re-render.

Interview Answer:
"Redux has one store holding all your app's state as a single object. When something happens, like a user adding an item to a cart, you dispatch an action, which is just a plain object describing what happened. That action goes into a reducer, a pure function that looks at the current state and the action and returns a new state object. The store then notifies any component that's subscribed to that piece of state so it can re-render. The key discipline is that components never touch state directly, everything flows through dispatching an action and letting the reducer decide what the new state looks like."

Q6. [Junior] Redux is often described as having a "unidirectional data flow." What does that actually mean, and why does it matter compared to letting any component just mutate shared state directly?

Answer:

Unidirectional data flow means state always moves in one direction through a fixed cycle: action dispatched, reducer computes new state, UI re-renders from that new state, and any UI interaction that changes state has to go back through dispatching another action.

Why the Cycle Matters

- **Because every state change has to go through the same dispatch-reducer cycle, you can trace any change in the UI back to a specific action, which makes debugging predictable instead of guessing which of ten components touched a shared object.**
- If components could mutate shared state directly from anywhere, two components changing the same object in the same render cycle could produce race conditions or updates that silently overwrite each other.
- Tools like Redux DevTools rely on this predictability, letting you scrub through a timeline of every action dispatched and see exactly what state looked like before and after, which wouldn't be possible if state changes happened from arbitrary places.

A Concrete Example

- In a checkout flow, clicking "Apply Coupon" dispatches `coupon/applied`, the reducer computes a new total, and the UI re-renders with that total. If instead five different components could reach into a shared `cart` object and edit `total` directly, tracking down why the total is wrong during a bug report becomes a search through the whole codebase instead of reading one action log.

Interview Answer:

"It means state changes always follow the same one-way cycle: something dispatches an action, a reducer computes a brand-new state from it, and the UI re-renders based on that new state. Nothing is allowed to just reach in and mutate the store directly. That matters because it makes changes traceable, if the cart total is wrong, I can open Redux DevTools and see the exact sequence of actions that led there instead of guessing which of several components touched the object. It trades a bit of ceremony for a debugging story that scales as the app grows."

Q7. [Junior] Redux reducers have to be pure functions, and updates have to be immutable, no mutating the existing state object. Why is that a hard rule instead of just a style preference?

Answer:

Purity and immutability aren't stylistic, they're what makes Redux's change detection, time-travel debugging, and predictable re-renders actually work.

Why Purity Matters

- A pure function's output depends only on its inputs, given the same state and action, a reducer must always return the same result, with no random values, no API calls, and no side effects like logging or mutating a variable outside its scope.
- **If a reducer isn't pure or mutates state in place, Redux's `connect` and `useSelector` can't reliably detect that anything changed, because they typically compare the old and new state by reference, and a mutated object still has the same reference.**
- Impure reducers also break Redux DevTools' time-travel debugging, which works by replaying past actions against past states, if a reducer has side effects, replaying it doesn't give you the same result twice.

A Concrete Bug

- If a reducer does `state.items.push(newItem)` instead of returning a new array, `state.items` still points to the same array reference. A component using `useSelector(state => state.items)` with a reference-equality check will think nothing changed and skip re-rendering, even though the array's contents did change.

Interview Answer:

"Redux detects changes by comparing references, not deep-comparing objects, so if a reducer mutates the existing state object instead of returning a new one, the reference stays the same and connected components never re-render even though the data changed underneath them. Purity matters for a similar reason: DevTools' time-travel debugging works by replaying actions against previous states, and that only produces trustworthy results if the reducer always returns the same output for the same input, with no random values or side effects baked in."

Q8. [Junior] Here's a reducer someone on your team wrote. What's wrong with it, and how would you fix it?

```
function cartReducer(state, action) {
  switch (action.type) {
    case "cart/itemAdded":
      state.items.push(action.payload);
      return state;
    default:
      return state;
  }
}
```

Answer:

This reducer mutates the existing `items` array in place with `push` and then returns the same `state` object reference, which breaks Redux's ability to detect the change.

What's Broken

- `state.items.push(...)` mutates the array that already exists in the store rather than creating a new one, and returning the same `state` object means nothing about the reference changed from Redux's point of view.
- **A correct Redux reducer must return a brand-new state object (and new nested arrays/objects for anything that changed) rather than editing the existing one in place.**
- Any component subscribed to `state.items` via reference-equality checks will silently fail to re-render when an item is added, since the array reference never changes.

The Fix

```
function cartReducer(state, action) {
  switch (action.type) {
    case "cart/itemAdded":
      return {
        ...state,
        items: [...state.items, action.payload],
      };
    default:
      return state;
  }
}
```

- The spread operator (`...`) copies the existing array or object into a brand-new one, so `items` becomes a fresh array reference containing the old items plus the new one, and `state` becomes a fresh object too.

Interview Answer:

"The bug is that it pushes directly onto `state.items`, which mutates the existing array in place, and then returns the same `state` reference. Redux and any `useSelector` hooks reading that slice will think nothing changed because the reference is identical, so the UI won't re-render even though data was added. The fix is to build a new array with the spread operator, `[...state.items, action.payload]`, and return a new state object with `{ ...state, items: newItems }`, so both the array and the top-level object get fresh references that Redux can detect as changed."

Q9. [Mid] You've got a component that fetches a list of orders from an API. Someone suggests just storing that response in Redux like any other piece of state.

What's the case for treating "server state" as fundamentally different from regular client state like a form input or a UI toggle?

Answer:

Client state is data your app owns and controls entirely, like whether a modal is open. Server state is a local cache of data that actually lives somewhere else and can go stale the moment it's fetched.

Why the Distinction Matters

- **Server state comes with problems client state never has, staleness (is this still accurate?), needing to be refetched, deduping concurrent requests for the same data, and background updates, none of which a plain reducer or `useState` handles for you out of the box.**
- A UI toggle is the single source of truth for itself; the orders list is not the source of truth, the server's database is, and your copy is just a snapshot that can drift out of sync the moment another user or another tab changes something.
- Treating server state like client state means you end up hand-rolling loading flags, error flags, retry logic, and cache invalidation inside every reducer that touches an API response, which is exactly the boilerplate that server-state libraries exist to remove.

A Concrete Example

- If two browser tabs both have the same order list open and one tab cancels an order, the other tab's Redux store still shows it as active until something explicitly refetches. A tool built for server state, like React Query, treats that staleness as a first-class concept rather than something you have to manually detect.

Interview Answer:
"Client state is stuff my app fully owns, like whether a sidebar is expanded, there's no 'staleness' concept because there's nothing external to go out of sync with. Server state is different, it's a cached snapshot of something that actually lives on a server and can change without my app knowing. If I store an orders list in Redux like any other slice, I have to hand-build the loading state, the error state, the refetch logic, and some way to know if it's stale, and I'll probably get some of that wrong or inconsistent across different parts of the app. That's the exact gap that libraries like React Query fill, they treat fetching, caching, and invalidation as the core problem instead of an afterthought."

Q10. [Mid] Redux doesn't come with a way to handle async logic like an API call out of the box, since reducers have to be pure and synchronous. How does middleware like redux-thunk fit into that gap?

Answer:

Middleware sits between dispatching an action and the action reaching the reducer, giving you a place to run side effects like API calls before a plain action object ever arrives at a reducer.

How redux-thunk Works

- Normally, `dispatch` only accepts a plain action object. Thunk middleware extends that so `dispatch` also accepts a function, and it calls that function with `dispatch` and `getState` as arguments instead of passing it straight to the reducer.

- **Thunk lets you write async logic, like fetching data and then dispatching a follow-up action with the result, as a function that gets dispatched, keeping the reducers themselves pure and synchronous.**
- A typical thunk dispatches a "loading" action immediately, awaits an API call, then dispatches either a "success" action with the data or a "failure" action with the error.

A Concrete Example

```
function fetchOrders() {
  return async function (dispatch) {
    dispatch({ type: "orders/loading" });
    try {
      const res = await fetch("/api/orders");
      const data = await res.json();
      dispatch({ type: "orders/succeeded", payload: data });
    } catch (err) {
      dispatch({ type: "orders/failed", payload: err.message });
    }
  };
}
```

Interview Answer:

"Reducers have to stay pure and synchronous, so Redux needs some other layer to handle things like API calls, and that's what middleware provides, it intercepts a dispatched value before it reaches the reducer. Thunk specifically lets you dispatch a function instead of a plain action object, and that function receives `dispatch` and `getState`, so it can do the async work itself, an API call for example, and then dispatch plain action objects with the loading state, the result, or the error at each step. The reducers only ever see simple, synchronous actions, all the async complexity lives in the thunk."

Q11. [Mid] Both redux-thunk and redux-saga solve the same problem, running async logic outside of reducers, but they're built very differently. Walk me through the trade-off and which one you'd pick for a checkout flow with several dependent API calls and the possibility of needing to cancel a request mid-flight.

Answer:

Thunk is a thin wrapper that lets you write async logic as plain functions with `async`/`await`. Saga uses generator functions to describe async flows as a series of yielded effects, which makes complex sequencing, cancellation, and testing easier at the cost of a steeper learning curve.

How They Differ

- Thunk code reads like normal JavaScript, `await` this, `dispatch` that, which is easy to pick up and fine for straightforward "fetch data, dispatch result" patterns.
- Saga uses generator functions (functions that can pause and resume execution, using `yield`) to describe effects declaratively, like "take this action, then call this API, then put this action," which the saga middleware then executes. That structure makes it much easier to express things like debouncing a search input, canceling an in-flight request when a newer one starts, or coordinating several dependent calls with retries.
- Saga's declarative effects are also easier to unit test in isolation, since you're asserting on the sequence of described effects rather than mocking `fetch` and `dispatch` calls directly.

Best Choice

- For a checkout flow with several dependent calls and a real cancellation requirement, redux-saga is the stronger fit.

Why?

- **Saga's built-in cancellation and sequencing primitives (`takeLatest`, `race`, `cancel`) handle exactly the "cancel this in-flight request when the user does something else" scenario that thunk would otherwise require hand-rolled `AbortController` plumbing to solve.**
- If the async logic were simpler, a single fetch-and-dispatch with no cancellation or complex sequencing, thunk would be the better choice since saga's generator syntax and extra concepts wouldn't be worth the added complexity for the team.

Interview Answer:

"They solve the same problem but at different levels of power. Thunk is just a function you dispatch that can do async work with plain `async`/`await`, it's simple and reads like normal JavaScript. Saga uses generators to describe a flow of effects declaratively, which makes it much better at things thunk struggles with, like canceling an in-flight request with `takeLatest`, or racing two calls against each other. For a checkout flow with several dependent calls and a real need to cancel requests mid-flight, I'd reach for saga, its cancellation primitives solve exactly that problem instead of me hand-rolling `AbortController` logic inside a thunk. If the flow were simpler, I'd just use thunk and skip the extra learning curve."

Q12. [Mid] You're starting a mid-sized internal dashboard app: a handful of shared filters, some data fetched from an API, and a few UI-only bits of state like which tab is active. Would you reach for Redux, Zustand, or Context plus useReducer, and why?

Answer:

For this specific app, the honest answer depends on splitting the state into what's actually server data (should go to a fetching library, not any of these three) versus what's genuinely shared client state (the filters and active tab).

What Each Option Actually Offers

- Redux gives you a mature ecosystem, DevTools, middleware, and strict conventions, but for a small set of shared filters it's meaningfully more boilerplate (actions, reducers, selectors, a store setup) than the problem needs.
- Zustand is a small state management library with a minimal API, no boilerplate actions, no providers wrapping your tree, just a hook-based store you call directly, and it supports selective subscriptions so components only re-render when the slice they read changes.
- Context plus `useReducer` is "free" (built into React, no dependency), but it inherits Context's broad re-render behavior discussed earlier, every consumer re-renders on any state change unless you do manual splitting.

Best Choice

- Zustand, paired with a server-state library like React Query for the actual API data.

Why?

- **Zustand gives selective, granular subscriptions without the setup overhead of Redux or the re-render coarseness of Context, which matches a handful of shared filters and UI flags better than either alternative.**

- Redux would be justified if the team already has strong DevTools/middleware conventions across many apps, or if the state graph were much larger and more interconnected; Context+useReducer would be reasonable if the team wanted zero dependencies and the update frequency were low enough that broad re-renders wouldn't matter.

Interview Answer:
"First I'd pull the actual API data out of this decision entirely, that belongs in something like React Query, not in any of these three, because it has caching and staleness concerns none of them solve well. For the remaining client state, the shared filters and active tab, I'd reach for Zustand. It gives me selective subscriptions like Redux without the action and reducer boilerplate, and it avoids the re-render-everything behavior of Context. I'd only reach for full Redux if this were a much bigger app with a large, deeply interconnected state graph where the team already benefits from DevTools and a stricter architecture, and I'd consider plain Context plus useReducer if I wanted zero extra dependencies and the update frequency were genuinely low."

Q13. [Mid] Before React Query and SWR became popular, most teams fetched data with `useEffect` and stored the result in `useState` or Redux. What specific pain did libraries like React Query actually solve that hand-rolled fetching didn't?

Answer:

Hand-rolled fetching handles the happy path fine but leaves a long list of edge cases as manual work: caching across components, deduping identical requests, refetching on window focus, retrying failed requests, and knowing when data is stale.

What Gets Solved Automatically

- **React Query and SWR treat a fetch as a cached, keyed query rather than a one-off side effect, so the same query key requested from two different components reuses the cached data and in-flight request instead of firing two network calls.**
- They provide built-in loading, error, and "is this stale" states per query, refetch-on-window-focus and refetch-on-reconnect behavior, and automatic retries with backoff, none of which a typical `useEffect` fetch gives you without a lot of manual code.
- Cache lifetime and garbage collection are handled for you, old queries that no components are subscribed to anymore get cleaned up automatically after a configurable time instead of living in a Redux store forever.

A Concrete Example

- If a `ProductList` and a `ProductCount` badge both need the same `/api/products` data, a hand-rolled `useEffect` in each component fires two separate network requests. React Query keys both requests by the same query key and serves the second one from cache (or dedupes the in-flight request), so the network only sees one call.

Interview Answer:
"With hand-rolled `useEffect` fetching, you're on your own for everything except the happy path, deduping the same request fired from two components, retrying on failure, refetching when the window regains focus, knowing whether the data you already have is stale. React Query treats a fetch as a cached query identified by a key, so if two components ask for the same key, it reuses the cached data or the in-flight request instead of firing duplicate network calls. It also handles retries, background refetching, and garbage collecting unused cache entries out of the box. That's a meaningful amount of code most teams were rewriting slightly differently in every app before these libraries existed."

Q14. [Mid] After a user edits a record, you want the UI to reflect the change everywhere it's shown. What are the two main cache invalidation strategies you'd use with something like React Query, and when would you reach for each?

Answer:

The two common approaches are explicitly invalidating and refetching a query after a mutation succeeds, and stale-while-revalidate, serving cached data immediately while a fresh fetch happens in the background.

Invalidate on Mutation

- After a mutation (like a `PATCH` request to update a record) succeeds, you tell the cache "anything tagged with this query key is now out of date," which triggers an automatic refetch of any currently-mounted query using that key.
- This is the right default for anything where correctness after a write matters more than saving a network round trip, editing a user's profile and wanting the profile page to show the new value immediately, for example.

Stale-While-Revalidate

- **Stale-while-revalidate means showing the cached (possibly outdated) data instantly while a background request fetches the latest version, then swapping it in once it arrives, which keeps the UI feeling instant instead of showing a loading spinner on every visit.**
- This fits data that doesn't need to be perfectly fresh on first paint but should self-correct shortly after, a dashboard of stats that update every few minutes, for example, where showing last-known numbers immediately and quietly updating them a second later is a better experience than a spinner.

Interview Answer:

"When a mutation happens, like a user editing their profile, I'd explicitly invalidate the related query key so React Query knows to refetch it, that guarantees the UI catches up to the write instead of showing stale data indefinitely. For data that's read far more often than it's written, like a stats dashboard, I'd lean on stale-while-revalidate, show the cached data immediately so the page feels instant, and let a background refetch quietly update it a moment later. The difference is whether you need the freshness guaranteed right after a specific action, versus just wanting things to gradually stay fresh in the background."

Q15. [Mid] Show me how you'd implement an optimistic update for a "like" button, updating the UI immediately before the server confirms, and rolling back if the request fails.

Answer:

An optimistic update means updating the local cache or state as if the mutation already succeeded, then rolling back to the previous value if the actual request comes back with an error.

The Pattern

- **The core idea is: snapshot the current value before mutating, apply the optimistic change immediately for a responsive feel, then either confirm it silently on success or restore the snapshot on failure.**

- Most data-fetching libraries give you hooks for exactly this: `onMutate` to apply the optimistic change and save a snapshot, `onError` to roll back using that snapshot, and `onSettled` to refetch and reconcile with the server's actual state.

A Concrete Example

```
function useLikeMutation(postId) {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: () => api.likePost(postId),
    onMutate: async () => {
      await queryClient.cancelQueries(["post", postId]);
      const previous = queryClient.getQueryData(["post", postId]);
      queryClient.setQueryData(["post", postId], (old) => ({
        ...old,
        liked: true,
        likeCount: old.likeCount + 1,
      }));
      return { previous };
    },
    onError: (err, vars, context) => {
      queryClient.setQueryData(["post", postId], context.previous);
    },
    onSettled: () => queryClient.invalidateQueries(["post", postId]),
  });
}
```

- `onMutate` cancels any in-flight fetch for that key (so it doesn't overwrite the optimistic value), snapshots the old data, and applies the new value right away. `onError` uses the snapshot from `onMutate`'s return value to restore the previous state if the server rejects the request.

Interview Answer:

"I'd apply the change to the local cache immediately so the like count and heart icon update the instant the user clicks, that's the whole point of optimistic updates, no waiting on the network for something low-risk. Before applying it, I snapshot the current value. If the mutation fails, I roll back to that snapshot in the `onError` handler. With React Query specifically, that's `onMutate` to snapshot and apply the optimistic change, `onError` to restore the snapshot, and `onSettled` to refetch and reconcile with whatever the server actually says, just in case the optimistic value and the real result ended up slightly different."

Q16. [Mid] You've got a store holding a list of blog posts, and each post has an `author` object embedded in it. A user updates their display name once, but it only changes on some posts in the UI, not others. What's going wrong, and how would normalizing the data fix it?

Answer:

The bug is duplicated copies of the same entity (the author) embedded separately inside each post object, so updating one copy doesn't touch the others, they silently drift out of sync.

The Problem With Nested Data

- If `posts` is `[{ id: 1, author: { id: 9, name: "Sam" } }, { id: 2, author: { id: 9, name: "Sam" } }]`, that's the same author appearing twice as separate, unlinked objects. Updating the name in one post's embedded copy does nothing to the other.
- **Normalizing means storing each entity type once, keyed by id, and having everything else reference it by id instead of embedding a full copy, so there's exactly one source of truth per entity.**

What Normalized State Looks Like

```
const state = {
  posts: {
    byId: {
      1: { id: 1, title: "Hello", authorId: 9 },
      2: { id: 2, title: "World", authorId: 9 },
    },
    allIds: [1, 2],
  },
  authors: {
    byId: {
      9: { id: 9, name: "Sam" },
    },
    allIds: [9],
  },
};
```

- Now updating `authors.byId[9].name` fixes the display name everywhere at once, because every post just holds `authorId: 9` and looks the author up from the single shared record instead of carrying its own copy.
- Libraries like Redux Toolkit's `createEntityAdapter` or normalizr exist specifically to manage this `byId`/`allIds` shape without writing it by hand every time.

Interview Answer:

"The bug is that each post has its own embedded copy of the author object instead of a reference to a shared one, so when the name changes, only the copy that gets explicitly updated changes, the rest are now stale duplicates of the same real-world entity. The fix is normalizing the state, storing authors once in an `authors.byId` map keyed by id, and having each post just hold an `authorId` reference instead of the full nested object. That way updating the author's name in one place, the single source of truth, is reflected everywhere a post looks it up, since there's no second copy left to go stale."

Q17. [Mid] A teammate stores `filteredOrders` in state alongside `orders` and a `statusFilter`, and updates `filteredOrders` in a `useEffect` whenever either changes. What's your take on that pattern?

Answer:

`filteredOrders` is derived state, it can always be recomputed from `orders` and `statusFilter`, so storing it separately just creates a second thing that has to be kept in sync and can drift out of sync if a code path updates one without the other.

Why This Is a Smell

- **If a value can be calculated directly from other state you already have, don't store it as its own piece of state, compute it during render (or memoize the computation) instead of syncing it with an effect.**
- The `useEffect` version adds a render where `filteredOrders` still reflects the old filter before the effect runs and updates it, which can cause a visible flash of stale data or an extra unnecessary render.
- It also adds a place for bugs: if someone updates `orders` through a new code path and forgets that a corresponding effect needs to also fire, `filteredOrders` silently goes stale.

The Fix

```
const filteredOrders = useMemo(
  () => orders.filter((o) => o.status === statusFilter),
  [orders, statusFilter]
```

```
);
```

- `useMemo` (a hook that caches a computed value and only recalculates it when its dependencies change) recomputes `filteredOrders` only when `orders` or `statusFilter` actually change, with no extra state, no effect, and no risk of the two getting out of sync.

Interview Answer:
"That's storing derived state, `filteredOrders` is just a function of `orders` and `statusFilter`, so keeping it as its own piece of state that gets synced through an effect means there are now two sources of truth that have to agree with each other. It also causes an extra render cycle, because the effect runs after the render where the inputs changed, so you can briefly see stale filtered results. I'd just compute it directly during render, wrapped in `useMemo` if the filtering is expensive, and drop the separate state and effect entirely. The rule of thumb is: if you can calculate it from state you already have, don't store it."

Q18. [Senior] You're building a collaborative kanban board, multiple users dragging cards between columns in real time, seeing each other's changes live. Walk me through how you'd split up the state: what's local, what's shared client state, and what's server state, and what tools you'd use for each.

Answer:

This app has three genuinely different kinds of state living side by side, and conflating them is the most common design mistake: per-user UI state, shared "room" state that multiple users affect in real time, and the durable board data that's persisted to a database.

Splitting the State

- Local-only state: which card is currently being dragged, whether a modal is open, which column is scrolled where. This lives in plain `useState` local to whatever component owns it, no other user or component needs to know about it.
- Real-time shared state: the live positions of cards as people drag them, other users' cursors or "currently editing" indicators. This is server state, but with a twist, it arrives via a websocket or a service like Pusher/Ably rather than a request/response fetch, and it needs to be merged into the same normalized store that holds the board data so a card only exists in one place.
- Durable board state: the columns, cards, and their assignments, fetched initially via a query (React Query or similar) and normalized (columns reference card ids, cards reference column ids) so that a card being edited from two different views stays in sync.

How the Pieces Fit Together

- **The key architectural move is normalizing all card and column data into a single entity store keyed by id, then treating both the initial fetch and incoming websocket events as different sources that write into that same normalized store, rather than keeping "the fetched board" and "the live updates" as two separate structures that have to be reconciled.**
- Drag-and-drop interactions should be optimistic: moving a card updates the local normalized store immediately for a responsive feel, then a mutation persists the move to the server, and incoming websocket events from other users apply the same kind of update to the same store.
- Conflict handling matters here in a way it doesn't for a typical CRUD app: if two users drag the same card at nearly the same time, you need either a last-write-wins policy with a version or timestamp field, or an operational-transform-style merge, decided deliberately rather than left to chance.

Interview Answer:

"I'd split this into three layers. Purely local UI state, like which card is mid-drag, stays in component state and never touches a shared store. The board's actual data, columns and cards, gets fetched once and normalized into an entity store keyed by id, so a card only exists in one place no matter how many components render it. Then live updates, other users dragging cards, come in over a websocket and get written into that same normalized store rather than kept as a separate 'live updates' structure, so there's one source of truth reconciling both the initial fetch and real-time events. Drags are optimistic locally for responsiveness, then persisted, and I'd explicitly decide on a conflict policy, most likely last-write-wins with a timestamp, for the case where two people move the same card at once, since that's a real scenario in a board like this, not an edge case you can ignore."

Q19. [Senior] You're building an analytics dashboard: five or six widgets, all of which need to react to a shared set of filters (date range, region, product line). Changing a filter should refetch and re-render every affected widget. How would you architect the state so widgets stay in sync without every widget re-fetching independently in an uncoordinated way?

Answer:

The filters themselves are shared client state, small, low-frequency, and read by many components, while each widget's data is independent server state that depends on the filters as query parameters.

Where the Filters Live

- The filter values (date range, region, product line) belong in a small shared store, Zustand or Context plus `useReducer` work well here since it's a handful of values changing infrequently (a user adjusting a date picker, not a stream of live data).
- **The critical design move is making the filters part of each widget's data-fetching query key, so changing a filter naturally and automatically invalidates and refetches exactly the widgets that depend on it, instead of manually telling each widget to refetch.**

How the Widgets React

```
function useRevenueWidget() {
  const { dateRange, region } = useFilters();
  return useQuery({
    queryKey: ["revenue", dateRange, region],
    queryFn: () => api.getRevenue({ dateRange, region }),
  });
}
```

- Because `dateRange` and `region` are part of the query key, React Query (or SWR) automatically treats a filter change as "this is now a different query," refetching only the widgets whose query key actually includes the filter that changed, and leaving unrelated widgets alone.
- Widgets that don't depend on a given filter (say, a widget that's region-agnostic) simply don't include `region` in their query key, so they're untouched when region changes, no manual coordination needed.

Interview Answer:

"I'd keep the filter values themselves in a small shared store, Zustand or Context with useReducer, since it's just a handful of values that change occasionally, not a firehose of updates. The part that actually keeps widgets in sync is making those filter values part of each widget's query key in whatever data-fetching library I'm using. That way, changing the date range naturally invalidates every widget whose query key includes date range, and React Query or SWR handles the refetching automatically, in parallel, deduped if two widgets happen to want the same query. Widgets that don't care about a particular filter just don't include it

in their key, so they're left alone. That avoids building a manual 'notify all widgets to refetch' system, the query key mechanism does that job for free."

Q20. [Senior] You tell your team "we split our Context providers into smaller pieces to fix a performance problem." Push on that yourself: what problem were you actually solving, and what's the trade-off of splitting too aggressively?

Answer:

Splitting a large context into smaller, more targeted ones is a direct response to Context's coarse re-render behavior, where every consumer of a context re-renders whenever any part of its value changes, even parts the consumer doesn't read.

What Splitting Actually Fixes

- If a single `AppContext` bundles `user`, `theme`, and `notifications` together, a component that only reads `theme` still re-renders every time `notifications` changes, since it's all one context value.
- **Splitting into `UserContext`, `ThemeContext`, and `NotificationsContext` means a component only re-renders when the specific context it actually subscribes to changes, which cuts unnecessary re-renders without introducing a new state library.**
- This is the same underlying fix as a Redux selector, narrow what a component subscribes to, just achieved by splitting providers instead of a built-in selector mechanism.

The Trade-off of Overdoing It

- Splitting too aggressively means wrapping the tree in many nested providers, which adds boilerplate and makes the provider tree harder to read, and every new context is one more thing a component might need to remember to consume correctly.
- At some point, if you're creating a dozen tiny contexts each with their own provider and update logic, you've essentially built a hand-rolled, worse version of a proper state library, and it's worth asking whether Zustand or Redux would just do the selective-subscription job more cleanly with less scaffolding.

Interview Answer:
"The actual problem is that Context re-renders every consumer whenever its value changes, with no way to subscribe to just part of it. If `user`, `theme`, and `notifications` are all bundled into one context, a component that only cares about theme still re-renders on every notification. Splitting into separate contexts per concern means each consumer only re-renders when the specific piece it reads actually changes, similar in spirit to a Redux selector. The trade-off is you can take this too far, if you end up with a dozen tiny contexts and providers nested five deep just to get selective updates, that's a sign you've reinvented a worse version of a real state library, and at that point Zustand's selector-based subscriptions would give you the same benefit with less scaffolding."

Q21. [Senior] Design the normalized state shape and the selectors for a project management app: projects contain tasks, tasks have an assignee, and the same task can appear in multiple filtered views (a "my tasks" view and a "by project" view). How do you avoid duplicating task data across those views?

Answer:

Each entity, projects, tasks, and users, gets stored exactly once in a normalized, id-keyed table, and every "view" (my tasks, by project) is a derived list of ids computed with a selector, not a separate copy of the task objects themselves.

The Normalized Shape

```
const state = {
  tasks: { byId: { t1: { id: "t1", projectId: "p1", assigneeId: "u1", title: "Fix
bug" } } },
  projects: { byId: { p1: { id: "p1", name: "Website" } } },
  users: { byId: { u1: { id: "u1", name: "Sam" } } },
};
```

- Every task is stored once in `tasks.byId`, referencing its project and assignee by id rather than embedding copies of those objects.

Views as Selectors, Not Copies

- **Views like "my tasks" or "tasks by project" are computed on demand by filtering `tasks.byId` by `assigneeId` or `projectId`, they are never a second stored copy of the task data, so there is exactly one place a task's title or status can be edited.**
- Wrapping those filters in a memoized selector (using something like `reselect` or `useMemo`) means the filtered list is only recomputed when the underlying `tasks` table or the relevant filter actually changes, not on every render.

```
const selectMyTasks = (state, userId) =>
  Object.values(state.tasks.byId).filter((t) => t.assigneeId === userId);
```

- Because both the "my tasks" view and the "by project" view read from the same `tasks.byId`, editing a task's title in either view updates the single stored copy, and both views reflect the change immediately since they're recomputed from that shared source, not from independently held copies.

Interview Answer:

"I'd store tasks, projects, and users each once, in a `byId` table keyed by id, with tasks referencing their project and assignee by id rather than embedding full objects. The 'my tasks' and 'by project' views aren't separate stored lists, they're selectors that filter the single `tasks.byId` table by `assigneeId` or `projectId` on demand, memoized with something like reselect so they don't recompute on every unrelated render. That guarantees there's exactly one copy of any given task's data, so editing it from either view updates the same underlying record, and both views immediately reflect that change because they're both just different filters over the same source of truth, not independent copies that could drift apart."

Q22. [Senior] Optimistic updates work well for a single mutation, but what happens when a user fires two optimistic mutations against the same record in quick succession, say, clicking "archive" and then immediately "unarchive" before the first request resolves? What can go wrong, and how do you guard against it?

Answer:

The failure mode is a race condition: if the two requests resolve out of order, or the rollback logic for the first mutation fires after the second one already applied its own optimistic change, the UI and the server can end up disagreeing about the final state.

What Can Go Wrong

- If "archive" is slow and "unarchive" resolves first, then "archive" finally resolves and its success handler blindly sets the local state to "archived," the UI now shows the opposite of what the user's last action actually intended, even though both requests individually succeeded.
- If "archive" fails and its `onError` handler rolls back to a snapshot taken before "archive" was ever clicked, that rollback can stomp on the optimistic "unarchive" change that was applied afterward, discarding a change the user made and that may have already succeeded.
- **The root cause is that each mutation's rollback snapshot is only valid relative to the state at the moment it was taken, so when mutations overlap, a later mutation's optimistic update or a rollback from an earlier mutation can clobber state that no longer matches what either snapshot assumed.**

How to Guard Against It

- Cancel or ignore in-flight requests for the same entity when a new mutation for that same entity starts (React Query's `cancelQueries` before applying a new optimistic update is meant exactly for this), so an old request's resolution can't overwrite a newer optimistic value.
- Use a version or timestamp on the entity and have the success/error handlers check "is this response still the most recent action for this entity" before applying it, discarding stale responses instead of blindly trusting whichever network call happens to resolve last.
- For genuinely high-stakes actions, disabling the second action while the first is in flight (rather than allowing both to race) is a simpler and often better UX trade-off than trying to perfectly reconcile overlapping optimistic updates.

Interview Answer:
"The risk is a race condition, if archive and unarchive are both in flight, whichever response resolves last can silently overwrite the user's actual last intent, or a rollback from the first mutation can stomp on the optimistic change the second mutation already applied. The guard I'd put in place is canceling any in-flight request for that same entity before applying a new optimistic update, which is what `cancelQueries` is for in React Query, so a stale response can't land after a newer one. I'd also tag responses with a version or timestamp and have the handlers check that the response they're processing is still the most recent action before applying it, discarding anything stale. For something high-stakes, I'd honestly just disable the button while the first request is in flight rather than trying to perfectly reconcile two racing optimistic updates, it's a simpler guarantee."

Q23. [Senior] Your team inherited an app where literally every piece of state, including form inputs and whether a tooltip is visible, lives in Redux. What signals would tell you it's time to pull some of that state back out, and how would you make that case to the team without it sounding like you're just chasing a trend?

Answer:

The signal isn't "Redux is bad," it's that a large fraction of that state has no business being global in the first place, and the cost shows up as unnecessary re-renders, bloated action/reducer boilerplate for things nobody outside one component cares about, and a mental tax on every new hire who has to learn the whole store just to change a tooltip.

Signals Worth Watching For

- If you can `git grep` for a piece of state in the store and find that only one component ever reads or dispatches actions related to it, that's a strong sign it should be local `useState` instead.

- If PRs regularly add a new action type, reducer case, and selector just to toggle a boolean that lives and dies within a single component's lifecycle (a modal's open state, for example), that's boilerplate paying for a flexibility nobody is using.
- **The concrete cost to point to, not an aesthetic preference, is unnecessary re-renders and cognitive overhead: every piece of state that doesn't need to be global still has to be traced through the store by anyone debugging an unrelated issue, and depending on selector granularity, can trigger re-renders in components that never asked for that update.**

Making the Case Without It Sounding Like Trend-Chasing

- Frame it around a specific, measurable pain the team already has, "these three components re-render on every keystroke in this form because the form state lives in the same slice as X" is concrete, "Redux is old-fashioned" is not.
- Propose the change incrementally, moving one component's genuinely local state (like a tooltip) to `useState` as a small, low-risk pull request, rather than pitching a rewrite, and letting the reduced boilerplate speak for itself.
- Keep Redux for what it's actually good at in that app, cross-cutting state with complex derived selectors and time-travel debugging needs, and be explicit that the proposal isn't "remove Redux," it's "stop defaulting every new piece of state into it."

Interview Answer:

"The tell isn't a vague sense that Redux is outdated, it's finding state in the store that only one component ever touches, a tooltip's visibility, a single form field, things that never needed to be global in the first place. The real cost is re-renders and cognitive load, every unrelated piece of global state is something a new engineer has to trace through when debugging, and depending on how selectors are written, can trigger re-renders nobody intended. To make that case without sounding like I'm chasing a trend, I'd point at a specific measurable symptom the team already has, like a form component that re-renders on every keystroke because its state shares a slice with something unrelated, and propose fixing that one case as a small PR rather than pitching a rewrite. I'd also keep Redux for what it's genuinely good at in that app, I'm not arguing to remove it, just to stop defaulting every new piece of state into it."

Q24. [Senior] Someone on your team wants to move all of your app's server data, the stuff currently fetched with React Query, into Redux "for consistency, so all state lives in one place." What's your response?

Answer:

Consistency for its own sake isn't a good enough reason to give up what React Query already solves for free, caching, deduping, background refetching, and staleness tracking, none of which Redux provides out of the box.

Where the "One Place" Instinct Goes Wrong

- Redux is excellent at coordinating client state, UI flags, form state, cross-cutting derived data, but it has no built-in concept of "this data is 10 minutes old and should be refetched in the background," "this component unmounted so cancel the in-flight request," or "these two components asked for the same data at the same time, dedupe the request." Moving server data into Redux means rebuilding all of that by hand, most likely worse than the version React Query already ships with, tested, for free.

- **Consistency of "where does state live" is worth less than correctness of "does the tool match the nature of the data," and server state and client state are different enough in their needs that forcing them into the same tool creates work without a corresponding benefit.**
- Redux Toolkit's own RTK Query exists precisely because the Redux team recognized this gap, and it deliberately mirrors React Query's caching model rather than treating server data like a normal reducer slice, which is itself evidence that "just put it in the normal store" isn't how the ecosystem actually solved this.

What I'd Propose Instead

- Keep client state (UI flags, filters, form state) in whatever the team's existing client-state tool is, and keep server state in React Query (or RTK Query if the team wants it under the Redux umbrella specifically for DevTools consistency).
- If the actual complaint is "we have two different mental models to reason about," that's a fair discussion, but the fix is picking the right two tools and being disciplined about which kind of state goes where, not collapsing a real distinction into one tool because it feels tidier.

Interview Answer:

"I'd push back gently. The appeal of 'one place for all state' sounds clean, but server data and client data have genuinely different needs, staleness, refetching, deduping, cache garbage collection, and Redux doesn't solve any of that out of the box. Moving it into Redux means we'd end up rebuilding a worse version of what React Query already gives us for free. I'd point out that even the Redux team built RTK Query specifically because server state doesn't fit the normal reducer model well, which tells you this isn't just a matter of taste. If the real concern is that the team finds two different state tools confusing to reason about, I'd take that seriously, but the fix is being disciplined about which kind of state goes where, not merging two things that are different for good reasons into one tool because it looks tidier on paper."

If you remember nothing else from this chapter: *the state management bugs that are hardest to debug are usually caused by state living one scope higher than it needed to.*

Chapter 9: Frontend Component Design (LLD)

This is the frontend equivalent of a backend low-level-design round: instead of designing a rate limiter class, you're asked to design a Modal or a virtualized list. Use these questions to practice thinking in terms of props, state, and accessibility from the start, rather than bolting accessibility and edge cases on as an afterthought once pushed, since that gap is exactly what separates someone who ships components other teams can safely reuse from someone who ships one-off UI that breaks the first time it's dropped into a slightly different context.

War Story: *A single shared button component grew fourteen boolean props over two years until nobody could safely change it without testing every screen in the app that used it.*

Avoid: adding another boolean prop to an already-overloaded component instead of asking whether it should be a different component

Aim For: recognizing when a component's prop surface has outgrown its original purpose and proposing splitting it

Q1. [Junior] Walk me through how you'd design a reusable Button component that supports a primary, secondary, and danger variant, plus a loading state. What props would it expose?

Answer:

A Button component earns its keep by being the one place every click target in the app goes through, so its prop surface should be small and predictable rather than a grab bag of one-off flags. **The core design decision is keeping Button a thin, mostly stateless wrapper around the native `<button>` element so you inherit built-in keyboard and focus behavior for free instead of reimplementing it.**

Props / API

- `variant` (`primary` | `secondary` | `danger` | `ghost`) controls the visual style, and `size` (`sm` | `md` | `lg`) controls padding and font size, both as string enums rather than a pile of booleans like `isPrimary`, `isSmall`.
- `disabled`, `loading`, `leftIcon`/`rightIcon`, and `onClick` cover the common behavioral needs, and `type` (`button` | `submit` | `reset`) matters because a Button silently defaulting to `submit` inside a `<form>` is a classic bug that fires an unwanted form submission.
- A polymorphic escape hatch, often called `as` or `asChild`, lets a consumer render the button's styling and behavior on top of an anchor tag (for a link styled like a button) without forking the component.

Internal State

- A well-designed Button usually has no internal state of its own; `loading` and `disabled` are passed in by the parent, which is what makes the component predictable and easy to test with a snapshot of props.
- Any internal state that does exist (like a brief "pressed" visual ripple) should be purely presentational and never something the parent needs to read back out.

Accessibility Considerations

- Use a real `<button>` element, not a styled `<div>` with an `onClick` handler, because the native element gives you keyboard activation (Space and Enter), focus handling, and the correct role for free.
- When `loading` is true, set `aria-busy="true"` and keep the button's accessible label stable (do not swap the text to just a spinner with no label), and for icon-only buttons always supply `aria-label` since there is no visible text for assistive technology to read.

Tricky Edge Case

- If a user double-clicks a submit button while an async request is in flight, you need to prevent a second submission without simply removing focus or making the button disappear, since a `disabled` button becomes unfocusable and unreachable by keyboard, which can strand a keyboard user mid-flow. A common fix is to keep the button focusable but ignore repeat clicks internally while `loading` is true, showing a spinner in place of the label but keeping the accessible name intact.

Interview Answer:
"I'd keep Button as a thin wrapper over a native `<button>` so I inherit keyboard and focus behavior instead of reimplementing it. The props would be `variant`, `size`, `disabled`, `loading`, optional icons, and an `onClick`, plus a polymorphic `as` prop so it can render as a link when needed. It shouldn't hold much internal state since predictability is the whole point. The trickiest part is the loading state: I want to block a second submit on double-click without disabling the button outright, because a disabled button drops out of the tab order and can strand a keyboard user, so I'd ignore repeat clicks internally while showing `aria-busy` instead."

Q2. [Junior] Design a text Input component with a label, helper text, and an error state. Would you make it controlled or uncontrolled, and why?

Answer:

An Input component needs to solve two problems at once: giving the parent form logic a clean way to read and write the value, and giving the user clear, accessible feedback when something is wrong. **The core design tension is controlled versus uncontrolled state, and getting that choice wrong is what causes the classic React warning about an input switching from uncontrolled to controlled mid-render.**

Props / API

- `value` and `onChange` for the controlled path, `defaultValue` for the uncontrolled path, plus `label`, `error`, `helperText`, `disabled`, `type`, and `name`.
- An `id` prop (or an internally generated one via `useId`) is required so the `<label>` can be associated with the input via `htmlFor`, which is what makes clicking the label focus the input.

Internal State

- If built as an uncontrolled component, the DOM input itself holds the value and you read it via a ref only when needed (e.g., on form submit), which avoids a re-render on every keystroke.
- If built as a controlled component, the value lives entirely in the parent's state, and the Input itself holds no value state, only transient UI state like whether the field currently has focus (for styling).

Accessibility Considerations

- Associate the error message with the input using `aria-describedby` pointing at the error message's `id`, and set `aria-invalid="true"` while an error is present, so a screen reader announces the problem when the user tabs into or interacts with the field.

- Helper text should also be wired into `aria-describedby` (both ids can be listed, space-separated) so a screen reader reads the hint even when there is no error.

Best Choice

For a component library, make Input controlled by default, but allow `defaultValue` for simple uncontrolled use in quick forms.

Why?

A controlled Input is what plays nicely with validation libraries like Formik or React Hook Form, which need to read and set the value from outside on every keystroke (e.g., a credit card field that auto-formats as you type); an uncontrolled input can't support that without extra plumbing, but forcing every simple search box to be controlled adds unnecessary re-renders for a component that doesn't need external control.

Tricky Edge Case

- Passing `defaultValue` and then later changing that prop does nothing, since `defaultValue` is only read on the initial render, this trips up developers who expect it to behave like `value`. Similarly, toggling a `value` prop between `undefined` and a real string across renders (rather than using `""` consistently) is exactly what triggers React's "a component is changing an uncontrolled input to be controlled" warning.

Interview Answer:

"I'd default Input to controlled, since that's what plays well with form libraries that need to read or transform every keystroke, like an auto-formatting credit card field. I'd still support `defaultValue` for simple, uncontrolled use cases where a full form library is overkill. For accessibility, the label needs an `htmlFor` match with the input's id, and the error and helper text both need to be wired in through `aria-describedby` so a screen reader actually announces them. The edge case I'd call out is that `defaultValue` only applies on first render, so if you're relying on it to react to prop changes later, it silently won't, and that's a common source of confused bug reports."

Q3. [Junior] Design a Checkbox and a Radio Group component. What's different about the keyboard behavior between the two, and how would that show up in your component's internal state?

Answer:

Checkboxes and radios look similar visually but represent very different data shapes and interaction models, and treating them as interchangeable usually means you haven't actually built one from scratch. **The key design point is that a checkbox group's state is a set of independently toggled booleans, while a radio group's state is a single selected value with arrow-key navigation baked into the native behavior.**

Props / API

- Checkbox: `checked`, `onChange`, `label`, `indeterminate` (for a "select all" checkbox representing a partially selected list), `disabled`.
- RadioGroup: `options` (array of value/label pairs), `value`, `onChange`, `name` (so the native radios are grouped correctly even if you're not using native radios directly).

Internal State

- A checkbox group's state at the parent level is typically an array or Set of selected values, since any subset can be checked independently.

- A radio group's state is a single selected value, and if you build custom-styled radios (divs instead of native `<input type="radio">`), you must manually track which item currently has `tabIndex={0}` (called roving tabindex, where only one item in the group is tab-stoppable at a time and arrow keys move that stop) because you lose the native grouping behavior the moment you stop using real radio inputs.

Accessibility Considerations

- Wrap a radio group in a `fieldset` with a `legend` so screen readers announce the group's purpose, and use `role="radiogroup"` if you're not using native inputs.
- Native radio buttons already move selection with the arrow keys automatically; if you rebuild radios with styled divs, you must replicate arrow-key movement and immediate selection-on-arrow yourself, which is easy to forget since Tab alone still "works" for basic navigation and the missing behavior only shows up under closer testing.

Tricky Edge Case

- A "select all" checkbox that controls a list of child checkboxes needs a third visual state, indeterminate, when some but not all children are checked; this state exists in the DOM (`element.indeterminate = true`) but has no matching HTML attribute, so it must be set imperatively via a ref rather than as a JSX prop, which trips up developers expecting a plain `indeterminate` prop to work like `checked`.

Interview Answer:

"The main difference is the shape of the state: a checkbox group is a set of independent booleans, so I'd model it as an array or Set of selected values, while a radio group is always exactly one selected value. The tricky part shows up if you build custom-styled radios instead of using native inputs, because you lose arrow-key navigation and selection for free and have to reimplement roving tabindex yourself. And for a 'select all' checkbox, the indeterminate state doesn't exist as an HTML attribute, so you have to set `element.indeterminate` imperatively through a ref rather than passing it as a normal prop."

Q4. [Junior] Design a Toggle/Switch component that flips a setting on and off. How is it different from a checkbox under the hood, and what happens if the toggle triggers an API call?

Answer:

A switch is visually distinct from a checkbox, but the more important difference is semantic: a checkbox represents "selected" while a switch represents "on right now," and that distinction matters for both ARIA roles and for how you handle asynchronous side effects. **The core design challenge is that toggling a switch often kicks off an async request, so the component needs a pending state that prevents a second toggle from racing the first, and a rollback path if the request fails.**

Props / API

- `checked`, `onChange`, `disabled`, `label`, and optionally `loading` if the parent wants to visually reflect a pending async operation.
- Keep the component fully controlled; a switch representing a real setting should never silently disagree with the source of truth (e.g., a server-persisted preference).

Internal State

- Ideally none beyond transient focus/hover styling, since `checked` should be driven by the parent so the UI can be rolled back if an API call fails.

- If the team wants optimistic UI (flip immediately, revert on failure), that optimistic flag is often local state layered on top of the controlled `checked` prop, distinct from the "confirmed" value.

Accessibility Considerations

- Use `role="switch"` with `aria-checked`, not just `role="checkbox"`, since a switch communicates a live on/off state rather than a selection, and screen readers announce it differently ("on"/"off" versus "checked"/"not checked").
- Support both Space and Enter for activation if built on a non-native element, and make sure the state change is announced (either through `aria-checked` updating or a live region for the associated setting name).

Tricky Edge Case

- If flipping the switch enables, say, two-factor authentication and calls an API, a user tapping it twice quickly before the first response returns can put the UI and the server out of sync. The fix is to disable further toggles (or queue them) while a request is in flight, show the optimistic state immediately for responsiveness, and revert to the server-confirmed value with a visible error message if the call fails, rather than leaving the switch showing "on" while the setting is actually still off on the backend.

Interview Answer:
"A switch is semantically different from a checkbox, it represents a live on/off state rather than a selection, so I'd use `role='switch'` and `aria-checked` rather than treating it as a styled checkbox. Functionally I'd keep it controlled, driven by the parent, so it always reflects the real, confirmed state. The interesting design problem is the async case: if toggling it calls an API, like turning on two-factor auth, I'd show the change optimistically for responsiveness but disable further toggles until the request resolves, and roll the UI back with an error if it fails, so the switch never lies about what's actually persisted on the server."

Q5. [Junior] Design a removable Chip or Tag component, the kind you'd see in a multi-select filter UI. What happens to focus when the user removes one?

Answer:

A Chip looks like a trivial component, a rounded label with an "x" button, but the removable interaction is where a lot of real accessibility bugs hide. **The core design point is that removing a chip changes the DOM structure the user was just focused on, so the component's focus management after removal matters as much as its visual design.**

Props / API

- `label`, `onRemove` (optional, omit it entirely for a non-removable chip rather than passing a no-op function), `variant`/`color`, and an optional `icon`.
- The parent typically owns the list of chips (e.g., selected filter values) and re-renders the list with one fewer item after a removal, rather than the Chip managing its own visibility.

Internal State

- None for the chip itself; it's a controlled, presentational piece, with the array of active chips living in the parent's state.

Accessibility Considerations

- The remove control needs its own accessible label, like `aria-label="Remove San Francisco filter"`, since a bare "x" glyph with no text is meaningless to a screen reader.

- If the whole chip is clickable for something other than removal (e.g., clicking the label edits it), separate the two interactive regions clearly rather than nesting a button inside another clickable element, which produces ambiguous or duplicated announcements.

Tricky Edge Case

- When a user removes a chip with the keyboard, focus was sitting on that chip's remove button, which is about to be unmounted; if you don't move focus somewhere explicit, focus falls back to `<body>`, and a keyboard or screen reader user effectively loses their place in the page. The fix is to programmatically move focus to the next chip in the list (or the previous one if the removed chip was last, or an input field if the list becomes empty) right after removal.

Interview Answer:

"The component itself is simple, a label plus a remove button, fully controlled by the parent that owns the list. The part I'd actually dig into in an interview is what happens to focus after removal: if you don't handle it, the removed element's DOM node disappears and focus resets to the body, which is disorienting for a keyboard or screen reader user. I'd move focus explicitly, to the next chip, or the previous one if you removed the last item, or to a fallback like the filter input if the list becomes empty, so the user never loses their place."

Q6. [Junior] Design an Avatar component that shows a user's photo, and falls back to their initials if the image fails to load. What state does that fallback logic need?

Answer:

An Avatar looks like a plain `<img>` wrapper, but any production app inevitably has broken image URLs (deleted photos, expired signed URLs, users who never uploaded one), so the fallback path is the actual design problem here. **The core design decision is tracking image load success or failure as explicit internal state, rather than assuming the `src` prop will always resolve.**

Props / API

- `src`, `alt`, `name` (used to derive initials for the fallback), `size`, and `shape` (`circle` | `square`).
- `alt` should default to the person's name if provided, and should be an empty string only when the avatar is purely decorative next to visible text that already names the person.

Internal State

- An internal `status` state (`loading` | `loaded` | `error`), updated by the `<img>` element's `onLoad` and `onError` handlers, decides whether to render the actual image or the initials fallback.

Accessibility Considerations

- If the avatar is the only visual identifier for a user (e.g., in a chat message list with no name shown), `alt` must contain the person's name so screen reader users get the same identifying information sighted users do.
- If a name is already displayed as text right next to the avatar, mark the avatar `alt=""` so it isn't announced twice.

Tricky Edge Case

- If the `src` prop changes (say, the user just uploaded a new photo, or you're scrolling through a virtualized list reusing the same Avatar instance for different users), a stale `error` state from a previous `src` can persist and incorrectly show initials for a perfectly valid new image, unless you

reset the internal status back to `loading` whenever `src` changes, typically by keying the effect or the component on `src` itself.

Interview Answer:
"The interesting part isn't rendering the image, it's the fallback path. I'd track an internal status, loading, loaded, or error, set from the image's onLoad and onError handlers, and render initials derived from the name whenever status is error. The edge case that actually bites people is reusing an Avatar instance across different users, like in a virtualized list, where a stale error state from the previous src can leak into the next render unless you explicitly reset that status whenever the src prop changes."

Q7. [Junior] Design an Accordion component where clicking a header expands or collapses its panel. Would you allow multiple panels open at once, and how would you animate the expand smoothly?

Answer:

An Accordion's design decisions mostly come down to two things: how many panels can be open simultaneously, and how you animate a height that you don't know in advance. **The core design point that's easy to miss is that you can't CSS-transition to `height: auto` directly, so the animation itself requires either a measured pixel height or a modern CSS trick.**

Props / API

- `items` (array of header/content pairs), `allowMultiple` (boolean controlling whether opening one panel closes the others), `defaultExpandedKeys`, and `onChange` for the parent to observe expand/collapse events.

Internal State

- A set (or single value, if `allowMultiple` is false) of currently expanded item keys, held internally if uncontrolled, or lifted to the parent via `expandedKeys`/`onChange` if the app needs to control it (e.g., a FAQ page that deep-links to a specific open question).

Accessibility Considerations

- Each header should be a real `<button>` wrapped in a heading element (like `<h3><button>...</button></h3>`), with `aria-expanded` reflecting open/closed state and `aria-controls` pointing at the panel's id, and the panel itself should have `role="region"` with a matching `aria-labelledby`.
- Collapsed panel content should be hidden from both sighted users and assistive technology (e.g., via `hidden` or `display: none`), not just visually clipped with `overflow: hidden` and zero height, since a screen reader can still read clipped-but-present text.

Tricky Edge Case

- Animating from `height: 0` to the panel's real height is hard because content height is often unknown until render (it can even change later, e.g., if the panel contains an image that loads after the animation starts). Two common approaches are measuring `scrollHeight` via a ref and animating to that pixel value, or using the newer CSS `grid-template-rows: 0fr` to `1fr` trick, which animates smoothly without ever needing to know the actual pixel height, though it requires slightly newer CSS Grid animation support in the browser.

Interview Answer:

"I'd hold a set of expanded keys internally, defaulting to a single-open pattern but exposing an `allowMultiple` prop for use cases like a settings page where several sections might legitimately be open at once. For accessibility, each header needs to be a real button with `aria-expanded` and `aria-controls` wired to the panel, and the panel needs `role='region'`. The genuinely tricky part is the animation: you can't transition to `height: auto` in CSS, so I'd either measure `scrollHeight` via a ref and animate to that pixel value, or use the `grid-template-rows: 0fr` to `1fr` trick, which sidesteps needing to know the height at all."

Q8. [Mid] Design a Tabs component with full keyboard support. Walk me through exactly which ARIA roles and keyboard interactions it needs.

Answer:

Tabs is one of the clearest components for testing whether you actually know the ARIA authoring practices versus just knowing tabs "should be accessible" in the abstract. **The core design point is roving tabindex: only the active tab is a normal tab stop, and arrow keys, not Tab, move focus between the other tabs in the list.**

Props / API

- `tabs` (array of id/label/content), `value`/`activeKey` and `onChange` for controlled usage, `defaultActiveKey` for uncontrolled, and `orientation` (`horizontal` | `vertical`) since vertical tabs use up/down arrows instead of left/right.

Internal State

- If uncontrolled, the currently active tab key; if controlled, that state lives in the parent and the component just renders based on `value`.
- A separate "focused tab index" is often needed distinct from "active tab," since arrow-key navigation can move focus across tabs before the user commits to activating one (depending on whether you choose automatic or manual activation, explained below).

Accessibility Considerations

- Use `role="tablist"` on the container, `role="tab"` with `aria-selected` on each tab button, and `role="tabpanel"` with `aria-labelledby` pointing back at its tab on the content region.
- Only the active tab gets `tabIndex={0}`; every other tab gets `tabIndex={-1}` (roving tabindex, a pattern where the group as a whole is one Tab stop, and arrow keys move an internal "focus pointer" among its children). Left/Right arrows (or Up/Down for vertical tabs) move focus, Home/End jump to the first/last tab.

Tricky Edge Case

- The WAI-ARIA spec actually describes two valid activation patterns: "automatic activation" (arrow keys immediately switch the visible panel as you move focus) and "manual activation" (arrow keys just move focus, and the user has to press Enter/Space to actually activate the tab). Automatic activation feels snappier but is a poor fit when switching tabs triggers an expensive operation, like a fresh data fetch, since arrowing quickly through five tabs would fire five requests; manual activation avoids that but requires clearly communicating to the user that they need to commit with Enter.

Interview Answer:

"I'd build this with roving tabindex: the tablist is one Tab stop for the page, and once focus is inside it, arrow keys move between tabs while Home and End jump to the first and last. Structurally that's `role='tablist'`, `role='tab'` with `aria-selected`, and `role='tabpanel'` with `aria-labelledby` linking back to its tab. The one design decision I'd flag explicitly is automatic versus manual activation, if switching tabs triggers something

expensive like a data fetch, I'd use manual activation so arrowing through tabs doesn't fire five requests, and only commit on Enter or Space."

Q9. [Mid] Design a Pagination controls component for a search results page. What happens if the result set shrinks (say, from an added filter) while the user is on page 5?

Answer:

Pagination looks like pure rendering logic, page numbers and prev/next buttons, but the interesting design problem is keeping the current page valid as the underlying data changes out from under it. **The core design point is that Pagination should never trust its own `currentPage` prop blindly; it needs to clamp against the actual page count derived from `totalItems` and `pageSize`.**

Props / API

- `totalItems`, `pageSize`, `currentPage`, `onPageChange`, and `siblingCount` (how many page numbers to show around the current page before collapsing the rest into an ellipsis, similar to Google's search results pagination).

Internal State

- Ideally none; Pagination should be a fully controlled, derived-from-props component, since the "current page" is really a property of the parent's data-fetching state, not something Pagination should own independently.

Accessibility Considerations

- Wrap the control in `<nav aria-label="Pagination">`, mark the current page button with `aria-current="page"`, and make sure prev/next buttons have clear labels ("Previous page," not just "<") since a lone arrow glyph is ambiguous out of context for a screen reader.
- Use real `<button>` elements unless each page genuinely maps to a distinct URL (in which case `<a>` tags are actually more correct, since it lets users open a page in a new tab or bookmark it).

Tricky Edge Case

- If the user applies a new filter that shrinks the result set from 12 pages down to 2 while they were sitting on page 5, `currentPage` is now out of range. The component (or the parent's data-fetching logic reacting to `totalItems` changing) needs to clamp the current page back into range, typically resetting to the last valid page or back to page 1, rather than rendering an empty results view with no items and no obvious explanation of why.

Interview Answer:

"I'd keep Pagination fully controlled and derived from props, no internal page state, since the current page really belongs to whatever is fetching the data. Accessibility-wise, it needs a `nav` landmark with a label, `aria-current='page'` on the active page, and real labels on the prev/next controls rather than bare arrow glyphs. The edge case I'd bring up unprompted is a filter shrinking the result set while the user is on, say, page 5 of what's now only 2 pages, if you don't clamp `currentPage` back into range when `totalItems` changes, the user lands on a blank page with no items and no explanation."

Q10. [Mid] Design a controlled Modal/Dialog component. Walk me through focus trapping, closing on Escape and outside click, and why you'd render it through a portal.

Answer:

A Modal is the single most common "design a component" prompt in frontend interviews because it touches almost every hard problem at once: focus management, event handling, and DOM placement. **The core design point is that a Modal has to actively trap keyboard focus inside itself while open and restore it to the triggering element on close, since the browser does nothing like that for you automatically.**

Props / API

- `isOpen`, `onClose`, `title`, `children`, `closeOnEscape`, `closeOnOutsideClick`, and `initialFocusRef` (letting the caller specify which element inside the modal should receive focus first, since it isn't always the first focusable element, e.g., a destructive confirm dialog should often focus "Cancel," not "Delete").

Internal State

- A ref storing the element that had focus right before the modal opened, so it can be restored on close.
- A mount/unmount timing state if the modal animates out, since you often need to keep it in the DOM briefly after `isOpen` goes false to let an exit transition finish before actually unmounting.

Accessibility Considerations

- The dialog container needs `role="dialog"`, `aria-modal="true"`, and `aria-labelledby` pointing at the title, and everything outside the modal should be excluded from the accessibility tree while it's open (either via the native `inert` attribute or `aria-hidden` on sibling content).
- Focus trapping means Tab and Shift+Tab cycle only through focusable elements inside the modal, wrapping from the last back to the first (and vice versa) instead of escaping into the page behind it.

Best Choice

Use a well-tested focus-trap primitive (like the one built into Radix UI or Reach UI, or the standalone `focus-trap-react` library) rather than hand-rolling the Tab-cycling logic.

Why?

Correctly enumerating "all currently tabbable elements" is deceptively hard, it has to account for `tabindex` ordering, elements hidden by CSS, disabled form controls, and even focusable elements inside nested iframes, and these libraries have already handled the years of edge cases that a first attempt at this logic in an interview setting almost always misses.

Tricky Edge Case

- Nested modals, for example, a confirmation dialog opened from inside a larger "delete item" modal, require the Escape handler and focus trap to be stack-aware. Pressing Escape should close only the topmost modal, not both, and focus needs to trap within whichever modal is currently on top of the stack, which is why a modal system usually needs a shared context tracking the open modal stack rather than each Modal instance reacting to `document`-level events independently.

Interview Answer:

"The main design challenge is focus management: on open, I'd save a ref to whatever had focus before, move focus into the modal, usually to a caller-specified element rather than just the first focusable one, and

trap Tab and Shift+Tab so they cycle only within the modal. I'd render it through a portal into `document.body` so it escapes any parent's `overflow: hidden` or z-index stacking context, while it still gets React context normally since a portal only changes where the DOM node lands, not the component tree. For the actual Tab-cycling logic, I'd reach for a library like Radix's focus trap rather than writing it by hand, since correctly enumerating tabbable elements, including disabled controls and nested iframes, has a lot of edge cases. The trickiest real scenario is nested modals, where Escape and the focus trap both need to be aware of which modal is currently on top of the stack."

Q11. [Mid] Design a Toast/notification system where multiple toasts can be queued, each with its own auto-dismiss timer. Where does the toast state actually live?

Answer:

A toast system is really two components in disguise: an imperative API (`toast.success("Saved!")`) that any part of the app can call from anywhere, and a rendering layer that turns a queue of toast objects into stacked, auto-dismissing UI. **The core design point is that toast state cannot live inside the component that triggers it, since that component might unmount before the toast even needs to disappear; it has to live in a global store or context that outlives any individual trigger.**

Props / API

- An imperative API surface: `toast.success(message, options)`, `toast.error(message, options)`, `toast.dismiss(id)`, with `options` covering `duration`, `position`, and an optional action button.
- A `<ToastProvider maxVisible={3} />` (or similar) rendered once near the root of the app, which owns the actual queue and rendering.

Internal State

- An array of active toast objects (`id`, `message`, `variant`, `createdAt`), each with its own dismiss timer (via `setTimeout`), plus a queue behind the scenes if more toasts are triggered than `maxVisible` allows at once.
- A pause flag per toast, set true on hover or keyboard focus, that stops its dismiss timer from counting down, and resumes it (with the remaining time, not a full reset) when the pointer leaves.

Accessibility Considerations

- Render toasts inside a container with `aria-live="polite"` (or `role="alert"`/`aria-live="assertive"` for errors that genuinely need immediate interruption) so screen readers announce new toasts without the user having to navigate to them.
- Auto-dismiss timers must pause on hover and on keyboard focus, since a toast that vanishes in 4 seconds gives almost no chance for a screen reader user, who navigates more slowly, to actually finish hearing it, let alone act on any button inside it.

Tricky Edge Case

- If a toast is triggered by an async operation, say, an API call that resolves after the triggering component has already unmounted (the user navigated to a different page while waiting), and the toast state lived inside that component, the toast would either never appear or throw a "setState on unmounted component" warning. Keeping the toast queue in a context or external store that lives at the app root, independent of any individual page's component tree, is what avoids this entirely.

Interview Answer:

"I'd split this into an imperative API, something like `toast.success(message)`, and a Provider mounted once near the app root that owns the actual queue of active toasts, each with its own id and dismiss timer. The reason the state has to live at that root level, not in whichever component triggered the toast, is that the triggering component can unmount before the toast's job is even done, for example an API call resolving after the user navigated away. For accessibility I'd put the toast container in an `aria-live='polite'` region, and I'd make sure hovering or focusing a toast pauses its auto-dismiss timer rather than letting it vanish while someone's still reading it."

Q12. [Mid] Design a Dropdown/Select component with keyboard navigation and closing on outside click. Where exactly do the event-timing bugs usually show up?

Answer:

A custom Dropdown has to reimplement everything the native `<select>` gives you for free: opening on click, closing on outside click or Escape, and moving a highlighted option with the arrow keys. **The core design point that causes real bugs is that "click outside to close" and "click an option to select it" are two listeners racing against the same click, and getting their event order wrong closes the dropdown before the selection registers.**

Props / API

- `options`, `value`, `onChange`, `placeholder`, and `multiple` (for a multi-select variant that keeps the dropdown open after each selection instead of closing).

Internal State

- `isOpen` (boolean), `highlightedIndex` (which option is currently keyboard-highlighted, distinct from the committed `value`), and, if the dropdown supports typeahead search, the current typed query string.

Accessibility Considerations

- Use `role="combobox"` on the trigger with `aria-expanded` and `aria-controls`, `role="listbox"` on the options container, and `aria-activedescendant` on the combobox pointing at the currently highlighted option's id, which is how a screen reader knows which option is highlighted without focus actually moving there.
- Arrow keys move `highlightedIndex`, Enter commits the highlighted option, and Escape closes the dropdown and returns focus to the trigger button rather than leaving focus stranded on a now-hidden option.

Tricky Edge Case

- A naive "outside click" listener attached on the `click` event can fire after the option's own `click` handler in some browsers but before it in others, depending on where the listener is attached and whether it's in the capture phase; a more common failure mode is attaching the outside-click listener on `mousedown` for snappier feel, which then fires and closes the dropdown before the option's `click` (which fires on `mouseup`) ever gets a chance to register the selection. The reliable fix is checking, inside the outside-click handler, whether the click target is contained within the dropdown's own ref before deciding to close, rather than assuming event order.

Interview Answer:

"Structurally this needs `role='combobox'` on the trigger and `role='listbox'` on the options, with `aria-activedescendant` tracking the highlighted option so a screen reader knows what's highlighted without focus actually leaving the input. State-wise I'd track `isOpen` and a separate `highlightedIndex` from the

committed value, since arrow keys move the highlight before the user commits with Enter. The bug I'd specifically watch for is event-order between the outside-click listener and the option's own click handler, if you close on `mousedown` for snappier feel, you can end up closing the dropdown before the option's `mouseup`-triggered click ever registers the selection, so I'd always check the click target against the dropdown's own ref rather than trusting event ordering."

Q13. [Mid] Design a debounced Search/Autocomplete input. Show me roughly how you'd wire up the debouncing and handle the case where responses come back out of order.

Answer:

An autocomplete box has to balance responsiveness against not hammering the backend on every keystroke, and the moment you add debouncing (delaying the actual search until typing pauses briefly) you also introduce the possibility that responses come back in a different order than the requests were sent. **The core design point is that debouncing solves the request-volume problem, but you still need request cancellation to solve the out-of-order-response problem, and they are not the same fix.**

Props / API

- `onSearch(query)` (or the component owns the fetch itself given a `searchFn`), `debounceMs` (default around 300), `minChars` before searching starts, and `renderSuggestion` for custom result rendering.

Internal State

- The current input text, the current suggestions list, a loading flag, `highlightedIndex` for keyboard navigation through results, and an `AbortController` reference (a browser API for cancelling an in-flight `fetch` request) for the most recent request.

Accessibility Considerations

- Use the ARIA combobox pattern: `aria-autocomplete="list"`, `aria-expanded`, `aria-controls` pointing at the suggestions listbox, and a live region announcing something like "5 results available" so screen reader users know results loaded without having to navigate into the list to find out.

Tricky Edge Case

- If a user types "ca," then quickly "cat," two requests fire close together; if the "ca" request is slower and resolves after the "cat" request, its results will overwrite the correct "cat" results unless you explicitly guard against it. Aborting the previous request (or tagging each request with an id and ignoring any response that isn't the latest) is what prevents a slow, stale response from clobbering a newer, correct one.

```
function useDebouncedSearch(searchFn, delayMs = 300) {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState([]);
  const controllerRef = useRef(null);

  useEffect(() => {
    if (query.length < 2) return setResults([]);
    const timer = setTimeout(async () => {
      controllerRef.current?.abort();
      const controller = new AbortController();
      controllerRef.current = controller;
      const data = await searchFn(query, controller.signal);
      setResults(data);
    }, delayMs);
```

```
    return () => clearTimeout(timer);
  }, [query]);

  return { query, setQuery, results };
}
```

Interview Answer:

"I'd debounce the input with a timer, roughly 300 milliseconds, so I'm not firing a request on every keystroke, but debouncing alone doesn't fully solve the problem. If a slower earlier request resolves after a faster later one, it can overwrite correct results with stale ones, so I'd also keep an AbortController for the most recent request and cancel the previous one every time a new request starts. For accessibility I'd follow the ARIA combobox pattern with `aria-autocomplete='list'` and a live region announcing the result count, since a sighted user sees the dropdown appear but a screen reader user needs that announced explicitly."

Q14. [Mid] Design a Tooltip/Popover component with positioning logic that avoids overflowing the viewport. What's the actual difference between a Tooltip and a Popover, and how do you compute the flip when there isn't room?

Answer:

Tooltip and Popover are often used interchangeably in casual conversation, but they solve different problems, and treating them identically is where accessibility bugs creep in. **The core design point is that a Tooltip must never contain interactive content (that's what actually makes it a Popover instead), and both need positioning logic that measures the trigger and viewport after render, then flips or shifts placement to avoid clipping off-screen.**

Props / API

- `content`, `placement` (`top` | `bottom` | `left` | `right`, often with `-start`/`-end` variants), `trigger` (`hover` | `focus` | `click`), and `offset` (gap between the trigger and the floating element).

Internal State

- `isOpen`, and a computed `{ top, left }` (or `x`/`y`) position, recalculated whenever the trigger scrolls, resizes, or the window resizes, since position has to be derived from the trigger's real, current bounding box, not a fixed offset decided at mount time.

Accessibility Considerations

- A Tooltip should be pure supplementary text describing the trigger, use `role="tooltip"` and connect it to the trigger with `aria-describedby`, and it must appear on keyboard focus, not only on mouse hover, or keyboard users never see it at all.
- A Popover, by contrast, can contain interactive content (buttons, links, a small form) and should use `role="dialog"` or similar, since ARIA tooltips are explicitly not allowed to contain focusable elements, that's the real distinguishing line between the two, not just visual size.

Tricky Edge Case

- You can't know a floating element's real rendered size until after the browser has laid it out, so placement logic like Floating UI's (a popular positioning library) has to render, measure, then flip or shift the position in a second pass if the preferred placement (say, "top") would clip past the viewport edge, in which case it falls back to "bottom," and further nudges horizontally ("shift") if even the fallback would clip a side edge. Doing this in a single synchronous layout pass without causing a visible flash of the tooltip appearing in the wrong place first is the actual engineering challenge, usually solved by measuring in a layout effect before the first paint.

Interview Answer:

"The real distinction between the two is that a Tooltip is pure text with no interactive content, while a Popover can contain buttons or a form, and that difference is exactly why ARIA disallows focusable elements inside a tooltip. Both need positioning logic that measures the trigger's actual bounding box and recalculates on scroll and resize rather than trusting a static offset. For the overflow problem, I'd measure the floating element after it renders and flip to the opposite side if the preferred placement would clip the viewport, then shift along the cross-axis if even that would clip, which is the same approach a library like Floating UI takes. And critically, a tooltip has to show on keyboard focus, not just mouse hover, or keyboard-only users never see it."

Q15. [Mid] Design a Carousel/image slider with autoplay. What accessibility requirements does autoplay specifically introduce that a static image gallery wouldn't have?

Answer:

A Carousel is a deceptively common source of accessibility complaints because autoplay actively works against several groups of users at once: screen reader users who are still reading a slide when it changes, and users with vestibular disorders for whom continuous motion can be a real physical problem, not just an annoyance. **The core design point is that any auto-advancing carousel needs a way to pause, and WCAG (Web Content Accessibility Guidelines) explicitly requires that pause control for content that moves automatically for more than five seconds.**

Props / API

- `slides`, `autoPlay`, `interval`, `loop`, and `onSlideChange`, plus a way to render a visible pause/play control, which should not be optional if `autoPlay` is enabled.

Internal State

- `currentIndex`, a `paused` flag (set true on hover, focus-within, or explicit pause button press), and, for touch support, the in-progress drag delta while a swipe gesture is underway.

Accessibility Considerations

- Give the carousel container `aria-roledescription="carousel"` and announce slide changes via a live region (e.g., "Slide 2 of 5"), and respect the user's `prefers-reduced-motion` media query setting by disabling autoplay (or at least the sliding transition) for users who have that preference set at the OS level.
- Pause autoplay whenever any element inside the carousel receives keyboard focus or hover, not only via the explicit pause button, since a user tabbing through slide content shouldn't have the content change underneath them mid-read.

Tricky Edge Case

- If autoplay keeps advancing while a screen reader user is in the middle of reading the current slide's content (which can take much longer than the visual interval), the announcement of "slide 3 of 5" can interrupt them mid-sentence on slide 2, and by the time they finish listening the visible slide has already changed twice more. This is exactly the scenario WCAG's pause requirement is aimed at, and it's why a genuinely accessible carousel treats focus-within and hover as automatic pause triggers, not just a nice-to-have.

Interview Answer:

"Autoplay is the part that actually creates accessibility obligations here, not the sliding mechanism itself. WCAG requires a way to pause anything that auto-advances past five seconds, so I'd build a visible pause control and also pause automatically on hover or when focus lands anywhere inside the carousel, since a screen reader user reading slide content shouldn't have it change out from under them mid-sentence. I'd also respect `prefers-reduced-motion` and skip the sliding animation, or autoplay entirely, for users who've set that preference. State-wise I'd track the current index and a paused flag separately from the interval timer itself."

Q16. [Mid] Design a Progress bar component that supports both a determinate value and an indeterminate "still working, no known percentage" state. What's the ARIA difference between the two?

Answer:

A progress bar seems like the simplest component in this chapter, but the determinate versus indeterminate distinction actually changes what you're allowed to put in the ARIA attributes, and getting it wrong actively misinforms screen reader users about how much progress has actually been made. **The core design point is that an indeterminate progress bar must never report a fake numeric value just to satisfy `aria-valuenow`, since that would tell assistive technology a specific percentage is known when it isn't.**

Props / API

- `value`, `max` (defaulting to 100), `indeterminate` (boolean), and `label` (a human-readable description of what's progressing, like "Uploading file.pdf").

Internal State

- None for the determinate case, it's a fully controlled reflection of `value`; the indeterminate case is typically a pure CSS animation with no JavaScript-driven state at all.

Accessibility Considerations

- For a determinate bar, set `role="progressbar"` with `aria-valuenow`, `aria-valuemin`, and `aria-valuemax`, and optionally `aria-valuetext` when a plain number isn't descriptive enough (e.g., "Step 2 of 4" instead of "50").
- For an indeterminate bar, use `role="progressbar"` but omit `aria-valuenow` entirely, which is the ARIA-correct signal that progress is happening but a specific value isn't known, rather than making one up (like always showing 50%).

Tricky Edge Case

- If progress updates arrive from the server faster than your CSS transition can visually keep up, or if a step is retried and the value actually needs to move backward (say, an upload restarts partway through after a dropped connection), you have to decide whether to force the visual bar to always move forward monotonically for a reassuring feel, or to snap/animate backward and risk it looking broken to a user who doesn't expect a progress bar to ever regress.

Interview Answer:

"The distinction I'd bring up unprompted is the ARIA difference: a determinate bar sets `aria-valuenow`, `aria-valuemin`, and `aria-valuemax` so assistive technology can announce a real percentage, but an indeterminate bar should omit `aria-valuenow` entirely rather than faking a number like 50%, since that would misinform the user that a specific progress amount is known. Functionally the determinate version is just a controlled reflection of a value prop, no internal state needed. The edge case worth mentioning is what happens if progress needs to move backward, like a retried upload, where you have to decide between

keeping the bar visually monotonic for reassurance or accurately snapping it back, which can look like a bug to an unprepared user."

Q17. [Mid] Design an infinite-scroll list that loads more items as the user scrolls down. How do you avoid firing duplicate fetches, and would you offer a keyboard-friendly alternative to scrolling?

Answer:

Infinite scroll is popular for feeds (Twitter, Instagram) because it keeps users engaged without an explicit "next page" click, but it introduces a request-timing problem and a genuine accessibility gap that a simple "Load more" button doesn't have. **The core design point is guarding against duplicate fetches with an explicit in-flight flag, since the scroll-triggered fetch mechanism (an IntersectionObserver watching a sentinel element near the bottom of the list) can fire multiple times before the first request even resolves.**

Props / API

- `fetchPage(cursor)`, `pageSize`, `hasMore`, and `renderItem`, plus an optional `loadMoreLabel` for an explicit button that supplements the scroll trigger.

Internal State

- The accumulated `items` array, `loading`, `error`, the current `cursor`/page token, and a ref to a sentinel `<div>` near the bottom of the list observed by an `IntersectionObserver` (a browser API that reports when an element enters or leaves the viewport, more efficient than manually calculating scroll position on every scroll event).

Accessibility Considerations

- Announce "Loading more items" and then the updated count via a live region when a new page loads, so screen reader users get feedback equivalent to the visual spinner.
- Provide a real, focusable "Load more" button as an alternative trigger alongside the scroll-based one, since pure scroll-triggered loading is awkward or effectively broken for keyboard-only users (there's no natural "scroll" keyboard gesture that reliably crosses the sentinel the same way a mouse wheel does) and because it can make reaching the page footer or other content below the list practically impossible.

Tricky Edge Case

- If the user scrolls quickly, the `IntersectionObserver` callback can fire again while the first `fetchPage` request is still in flight, triggering a second, duplicate request for the same next page before `hasMore` or the cursor has updated. The fix is an explicit `loading` guard checked before issuing any new fetch, not just relying on the observer firing once, since observers can fire more than once for the same intersection event depending on threshold configuration and layout shifts.

Interview Answer:
"I'd use an IntersectionObserver watching a sentinel element near the bottom of the list rather than manually tracking scroll position, since it's more efficient and doesn't run on every scroll event. The bug I'd specifically guard against is duplicate fetches: if the user scrolls fast, that observer callback can fire again before the first request resolves, so I'd gate every fetch behind an explicit loading flag rather than trusting the observer to only fire once. And I'd always pair the scroll trigger with a real 'Load more' button, because pure scroll-based loading is a rough experience for keyboard-only users and can make it effectively impossible to reach content below the list."

Q18. [Mid] Design a star Rating component that supports half-star increments. How would you handle the fact that touch devices have no hover state?

Answer:

A star rating widget looks trivial until you have to support half-stars and realize your hover-preview interaction model, moving the mouse across the stars to preview a rating before clicking, has no equivalent on a touchscreen. **The core design point is separating the "preview" value from the "committed" value in state, since hover preview and final selection are genuinely different pieces of state that touch devices can't populate the same way.**

Props / API

- `value`, `onChange`, `max` (default 5), `allowHalf`, and `readOnly` (for a display-only rating shown on, say, a product card rather than an input the user can change).

Internal State

- `hoverValue` (the value being previewed as the pointer moves across the stars, reset to null on pointer leave) kept separate from `value` (the actually committed rating), so moving the mouse around never accidentally commits a rating before a click.

Accessibility Considerations

- Use `role="radiogroup"` (or a slider pattern, `role="slider"` with `aria-valuenow`) with arrow keys incrementing or decrementing the rating, and `aria-valuetext` announcing something like "3.5 out of 5 stars" rather than a bare number that a screen reader user would have to guess the scale of.

Tricky Edge Case

- Half-star hover preview relies entirely on mouse position within a star (left half versus right half) to decide the preview value, and touch devices have no `mouseenter`/`mousemove` equivalent before a tap commits, so a design that only works via continuous hover position simply has no way to preview a half-star on mobile. The practical fix is either dropping to a tap-based two-step interaction (tap once to preview at the nearest whole or half value, tap again to confirm) or accepting that touch users get a slightly coarser, tap-to-commit interaction rather than trying to force hover-based logic onto a device that doesn't support it.

Interview Answer:
"I'd keep a hover-preview value separate from the actual committed value in state, so moving the mouse across the stars previews a rating without committing it until a click. The interesting design constraint is that this whole preview interaction depends on continuous hover position to detect left-half versus right-half of a star for the half-star case, and touch devices simply don't have that signal before a tap. So for touch, I'd fall back to a coarser tap-based interaction rather than pretending hover-based logic will translate directly, and I'd make sure keyboard users get the same granularity through arrow keys and an `aria-valuetext` that spells out something like '3.5 out of 5 stars' instead of a bare number."

Q19. [Senior] Design a list component that can render 100,000 rows without the browser choking. Explain windowing conceptually, and sketch how you'd track which rows to actually render.

Answer:

Rendering 100,000 DOM nodes at once will make any browser slow to paint and scroll, no matter how simple each row is, so the standard fix is windowing (also called virtualization), rendering only the small slice of rows currently visible in the viewport, plus a small buffer, and swapping their content as the user scrolls, rather than ever mounting all 100,000 at once. **The core design point is that the list needs a fixed-height (or accurately measured) scroll container where only `visibleRange` rows are real DOM nodes, while the rest of the list's height is faked with empty space so the scrollbar still behaves correctly.**

Props / API

- `items`, `itemHeight` (a fixed pixel height, or an estimator function for variable-height rows), `overscan` (extra rows rendered just outside the visible area to reduce blank flashes during fast scrolling), `height` (the viewport's visible height), and `renderRow`.

Internal State

- `scrollTop`, updated on the container's scroll event, from which you derive `startIndex` and `endIndex` (the currently visible row range) using simple arithmetic for fixed-height rows: `startIndex = Math.floor(scrollTop / itemHeight)`.
- A "total height" value (`items.length * itemHeight`) used to size a spacer element so the scrollbar thumb and scroll range look correct even though only a handful of real rows exist in the DOM at any moment.

Accessibility Considerations

- Screen readers can only perceive the rows that are actually mounted in the DOM at any given moment, which is a real, inherent limitation of virtualization, not a bug you can fully engineer away; you mitigate it by adding `aria-setsize` (total item count) and `aria-posinset` (this row's position) to each rendered row, so a screen reader can at least announce "item 4,213 of 100,000" even though items 1 through 4,212 aren't in the DOM to be navigated directly.

Tricky Edge Case

- Fixed-height windowing is straightforward arithmetic, but variable-height rows (like a chat log with messages of different lengths) break the simple `scrollTop / itemHeight` formula, since you don't know a row's real height until it renders at least once. The common fix is maintaining a cache of measured heights per row (measuring the first time a row mounts) plus an estimated height for unmeasured rows, and using a running cumulative-height array (or a binary search over it) to figure out which index corresponds to a given scroll offset, which is exactly what libraries like `react-window` and `react-virtualized` do internally.

```
function VirtualList({ items, itemHeight, height, renderRow, overscan = 5 }) {
  const [scrollTop, setScrollTop] = useState(0);
  const startIndex = Math.max(0, Math.floor(scrollTop / itemHeight) - overscan);
  const endIndex = Math.min(
    items.length,
    Math.ceil((scrollTop + height) / itemHeight) + overscan
  );
  const visibleItems = items.slice(startIndex, endIndex);

  return (
    <div style={{ height, overflowY: "auto" }} onScroll={(e) =>
setScrollTop(e.target.scrollTop)}>
      <div style={{ height: items.length * itemHeight, position: "relative" }}>
        {visibleItems.map((item, i) => (
          <div
            key={startIndex + i}
            style={{ position: "absolute", top: (startIndex + i) * itemHeight, height:
itemHeight }}
          >
            {renderRow(item, startIndex + i)}
```

```
          </div>
        ))}
      </div>
    </div>
  );
}
```

Interview Answer:

"The concept is windowing: instead of mounting 100,000 rows, I only mount the handful that are actually visible in the viewport, plus a small overscan buffer, and I fake the total scrollable height with a spacer so the scrollbar behaves correctly. I'd track `scrollTop` in state, derive `startIndex` and `endIndex` from it, and absolutely position each rendered row at `index * itemHeight` inside a container sized to the full list's total height. The part that actually gets hard is variable row heights, where you don't know a row's real height until it's rendered once, so you need a measured-height cache and a cumulative height lookup, ideally with a binary search, rather than simple division. And I'd flag the accessibility tradeoff directly: a screen reader genuinely can't perceive rows that aren't mounted, so I'd add `aria-posinset` and `aria-setsize` to at least communicate position and total count."

Q20. [Senior] Design a drag-and-drop list reorder component. The native HTML5 drag-and-drop API isn't keyboard accessible at all, so how would you make this usable without a mouse?

Answer:

Drag-and-drop reordering (think a Trello board or a to-do list you can reorder by dragging) is one of the few UI patterns where the "obvious" native browser API, the HTML5 Drag and Drop API, is fundamentally inaccessible, since it has no built-in keyboard equivalent at all. **The core design point is that you need a second, entirely separate keyboard interaction model running in parallel with the pointer-based drag, not an afterthought bolted on, which is exactly why libraries like dnd-kit ship dedicated keyboard sensors alongside their pointer sensors.**

Props / API

- `items`, `onReorder(newOrder)`, `renderItem`, and a `keyboardReorderEnabled` flag (though in a well-designed system this should really just always be on, not optional).

Internal State

- `draggedIndex` and `dropTargetIndex` while a pointer drag is active, plus, for touch, the raw pointer position delta used to compute a visual drag offset.
- For the keyboard path, a separate `pickedUpIndex` representing an item that's currently "lifted" via keyboard and awaiting arrow-key movement, distinct from normal focus.

Accessibility Considerations

- The keyboard interaction typically works as: focus an item, press Space to "pick it up" (announced via a live region, e.g., "Picked up item 3 of 8"), use Up/Down arrows to move it within the list (each move announced, e.g., "Moved to position 2 of 8"), and press Space again to "drop" it in place, with Escape canceling and returning it to its original position.
- Every move, pickup, and drop needs a live region announcement, since none of this state change is otherwise visible to a screen reader the way a mouse-based visual reorder is to a sighted user.

Tricky Edge Case

- If the list is also receiving live updates from elsewhere while a user is mid-drag, for example a websocket pushing a newly inserted item from another collaborator while the user is reordering the same list, the indices the drag operation is tracking can become stale the instant the underlying array changes, causing the drop to land in the wrong position relative to what the user actually sees. A safe design either locks reordering (queues incoming updates) while a drag is actively in progress, or reconciles by item identity (stable ids) rather than raw array index once the drag completes, so a background insert doesn't silently corrupt an in-progress reorder.

Interview Answer:

"The thing I'd raise immediately, because it's easy to miss, is that the native HTML5 drag-and-drop API has no keyboard equivalent at all, so a component built only on `draggable` and `ondragstart` is inaccessible by construction. I'd build a parallel keyboard interaction: focus an item, Space to pick it up, arrow keys to move it, Space again to drop, Escape to cancel, with every step announced through a live region since none of that state change is otherwise visible to a screen reader. On the state side I'd track drag and drop-target indices for the pointer path and a separate 'picked up' index for the keyboard path. The edge case I'd bring up is background data changes during an active drag, like a websocket inserting a new item mid-reorder, which is exactly the kind of thing that corrupts index-based reordering unless you reconcile by stable item id rather than raw position."

Q21. [Senior] Design a multi-step Form wizard with per-step validation. If a user goes back to step 1 and changes an answer that invalidates something they already filled in on step 3, what happens?

Answer:

A form wizard's hardest problem isn't rendering steps one at a time, it's owning form state consistently across steps that the user can freely move backward and forward between. **The core design point is that form values need to live in one shared source of truth across the whole wizard, not scattered as local state per step, otherwise going back and forth silently loses data, and that same shared store is what makes cross-step revalidation even possible.**

Props / API

- `steps` (each with its own field schema and a `validate` function), `onComplete(values)`, `initialValues`, and often an `allowSkipAhead` flag controlling whether users can jump to a step they haven't reached yet via a step indicator.

Internal State

- `currentStepIndex`, a single `values` object holding every field from every step (not per-step local state), a `errors` map keyed by field, and a `visitedSteps` set used to drive a step indicator that shows which steps have already been completed versus which are still ahead.

Accessibility Considerations

- On every step change, announce it via a live region ("Step 2 of 4: Shipping address") and move focus to either the new step's heading or its first form field, since a purely visual step transition leaves a screen reader user still positioned wherever they last were, with no cue that the content around them just changed entirely.
- The step indicator itself (often a horizontal progress track of numbered circles) needs `aria-current="step"` on the active step, and completed versus upcoming steps should be distinguishable through more than just color.

Tricky Edge Case

- If a user selects "Canada" as their country on step 1, then picks "Ontario" as their state/province on step 3, then navigates back to step 1 and changes the country to "United States," the previously selected "Ontario" value on step 3 is now invalid but the wizard has no built-in reason to notice, since step 3 hasn't been visited again and its own local validation isn't re-run. A correct design revalidates (or at least flags as stale) any already-completed step whose fields depend on a value that just changed, which requires the validation logic to express cross-step dependencies explicitly rather than assuming each step's validity, once passed, stays true forever.

Interview Answer:

"The design decision that matters most is keeping all form values in one shared object across every step, not local state per step, because otherwise navigating back and forth loses data, and you also need that shared state to make cross-step validation possible at all. The scenario I'd walk through is someone changing their country on step 1 after already picking a state on step 3, if the wizard only validates whatever step you're currently on, that now-invalid state selection just sits there silently wrong until submission. So I'd design the validation layer to express dependencies between fields across steps, and revalidate or flag dependent steps as stale when an upstream value changes, rather than treating 'this step passed validation once' as permanent. And on every step transition I'd move focus to the new step's heading and announce it through a live region, since a screen reader user gets no other signal that the whole form context just changed."

Q22. [Senior] Design a DataTable component with sorting and filtering. Would you make the sort/filter state internal to the table or lifted to the parent, and how does that decision change once the data is server-paginated?

Answer:

A DataTable is where several features that seem independent, sorting, filtering, pagination, and row selection, actually interact constantly, and the state ownership decision you make early determines whether those interactions compose cleanly or produce inconsistent combinations. **The core design point is that sort, filter, and pagination state should be lifted to a single controlling owner (the parent, or a headless table library like TanStack Table) rather than each feature managing its own isolated internal state, because filtering has to reset pagination to page 1 and sorting has to apply to the filtered set, not the raw one.**

Props / API

- `columns` (each with an `accessor`, `sortable` flag, and optional `filterFn`), `data`, and either fully controlled `sortState`/`filterState`/`onSortChange`/`onFilterChange` props, or an uncontrolled mode with sensible internal defaults for a simpler use case.
- A `mode` distinction (client-side versus server-side) matters enough to be explicit in the API, since it changes who is responsible for actually doing the sorting and filtering work.

Internal State

- If uncontrolled: current sort column and direction, current filter values per column, and a selected-rows set.
- For genuinely large datasets, the table often also needs virtualization internally (the same windowing concept as a plain virtualized list), which adds visible-row-range state layered on top of the sort/filter state.

Accessibility Considerations

- Use real table semantics (`<table>`, `<thead>`, `<th scope="col">`), and make each sortable header a button with `aria-sort` (`ascending` | `descending` | `none`) reflecting current state, so a screen reader announces the sort state change rather than only a sighted user seeing an arrow icon flip.
- Filter inputs need to be clearly labelled and associated with the specific column they filter (e.g., `aria-label="Filter by status"`), since a row of unlabelled filter boxes above the header row is a common, confusing pattern for screen reader users.

Best Choice

Lift sort and filter state to a single controlling owner, ideally a headless table library like TanStack Table, rather than letting each feature (sort, filter, pagination, selection) manage its own local state independently.

Why?

These features aren't actually independent: filtering needs to reset the current page back to 1, sorting needs to operate on the already-filtered subset rather than the full raw dataset, and row selection needs to survive a re-sort without silently pointing at the wrong rows; a single source of truth is what keeps those combinations consistent, whereas four independently-managed pieces of local state will eventually produce a combination (like a "selected" row that's actually a different row after a sort) that's wrong and hard to trace.

Tricky Edge Case

- The moment your data is server-paginated (say, an API returning 50 rows at a time out of 2 million total), client-side sorting and filtering physically cannot work correctly, since you don't have the full dataset in memory to sort or filter against, sorting the 50 rows you happen to have loaded is not the same as sorting the real, full dataset. The component's API needs to explicitly distinguish this server-driven mode, delegating the actual sort/filter operation to the parent (which re-fetches from the server with new sort/filter parameters) rather than quietly doing an incorrect, partial client-side sort that looks plausible but is wrong.

Interview Answer:

"I'd lift sort, filter, and pagination state to one owner rather than letting each feature manage itself independently, because they interact: filtering has to reset the page back to 1, sorting has to apply to the already-filtered rows, and selection has to survive a re-sort without pointing at the wrong row. In practice I'd reach for a headless library like TanStack Table for exactly that reason. The edge case that actually changes the architecture is server-side pagination: once you're only holding 50 of 2 million rows in memory, client-side sorting is just wrong, it can only sort the page you happen to have, so the table's API needs an explicit server mode where sort and filter changes are delegated back to the parent to refetch, rather than quietly doing a partial sort that looks fine but isn't."

Q23. [Senior] Design a rich text editor conceptually, the kind behind a comment box with bold, italic, and lists. Would you build it on raw contentEditable, and what's the hardest part to get right?

Answer:

A rich text editor is one of the few UI components where "just use contentEditable" (a native browser attribute that lets any element become directly editable by the user) sounds simple but hides a genuinely hard problem: contentEditable's actual DOM output for the same user action, like pressing Enter inside a bulleted list, varies inconsistently across Chrome, Safari, and Firefox. **The core design point is that production rich text editors are almost always built on a structured document model (a JSON tree**

representing bold spans, lists, and paragraphs) that gets rendered into the DOM and kept in sync with the user's edits, rather than trusting the browser's raw contentEditable output directly as the source of truth.

Props / API

- `value` (often a JSON document structure rather than an HTML string, since HTML strings are lossy and hard to validate), `onChange`, and a `toolbar` configuration describing which formatting commands are available.

Internal State

- A selection/cursor range model, tracked independently from the DOM's native selection object so it survives programmatic content changes.
- An undo/redo history stack of document snapshots or operations, and the currently active formatting marks near the cursor (used to highlight, say, the "Bold" toolbar button when the cursor sits inside bolded text).

Accessibility Considerations

- contentEditable regions have historically inconsistent and sometimes poor screen reader support across browsers for anything beyond plain text, so formatting changes (like applying bold) need to be handled carefully to avoid confusing or silent announcements, and custom keyboard shortcuts for formatting commands must avoid colliding with browser or OS-level shortcuts (Cmd+B is safe territory, but plenty of single-letter shortcuts are not).

Best Choice

For any production use case, build on a maintained rich text framework, like Lexical, Slate, or TipTap (built on ProseMirror), rather than hand-rolling raw contentEditable handling.

Why?

Raw contentEditable produces inconsistent HTML for identical user actions across browsers, and correctly reimplementing undo/redo, paste sanitization (stripping a user's pasted Microsoft Word formatting down to something sane), and reliable cursor/selection restoration is a multi-year effort that these libraries have already solved; a hand-rolled contentEditable comment box is fine for a quick prototype, but breaks down fast under real-world content and real-world browsers.

Tricky Edge Case

- Restoring the exact cursor position after the document model changes programmatically (for example, the user's typed markdown-style `**bold**` gets auto-converted into actual bold formatting as they type) is genuinely hard, because a naive React re-render of the contentEditable's inner content resets the browser's native cursor position back to the start of the element. This is exactly why editor libraries maintain their own selection model, synced to the DOM via careful, imperative updates, rather than trusting a declarative re-render to preserve cursor position the way it would for a plain text input.

Interview Answer:

"I wouldn't hand-roll this on raw contentEditable for anything beyond a prototype, I'd build it on a library like Lexical or TipTap, because contentEditable produces genuinely inconsistent DOM output for the same action across browsers, and reimplementing undo/redo and paste sanitization correctly is a huge amount of hidden work those libraries have already done. Conceptually, the editor's real source of truth should be a structured document model, not raw HTML, with the DOM as a rendering target kept in sync with it. The hardest specific problem is cursor position: if the document model changes programmatically, like auto-converting typed markdown into real formatting, a naive re-render collapses the cursor back to the start of

the field, which is exactly why these libraries maintain their own selection model instead of trusting a declarative re-render to preserve it."

Q24. [Senior] Design a Date Picker component with a full keyboard-navigable calendar grid. What's the classic timezone bug that shows up in date pickers, and how do you avoid it?

Answer:

A date picker combines two separate hard problems: a keyboard-navigable grid widget, and the surprisingly common bug of a selected date silently shifting by a day once it crosses a timezone boundary. **The core design point on the timezone side is treating a calendar date the user picked ("August 21, 2026") as a timezone-naive value, not as a JavaScript `Date` object carrying an implicit local time, since converting that local-midnight `Date` to UTC for storage or transmission is exactly what causes the date to silently roll back a day.**

Props / API

- `value`, `onChange`, `minDate`, `maxDate`, `locale`, and `isDateDisabled(date)` for blocking specific dates (like already-booked days on a hotel booking calendar).

Internal State

- A `viewDate` (which month/year is currently displayed) kept separate from the actually selected `value`, since a user can navigate months to browse without changing their selection, and a `focusedDate` for roving tabindex across the day grid, distinct from both.

Accessibility Considerations

- Use `role="grid"` with `role="gridcell"` for each day, arrow keys move by one day, Up/Down move by a full week (7 days), Page Up/Page Down move by a month, and Home/End jump to the start/end of the current week.
- Each day cell needs a full, unambiguous accessible label, like "Friday, August 21, 2026," not just the bare number "21," since a screen reader user navigating cell to cell needs the full date, not just a digit stripped of month and context.

Tricky Edge Case

- If a user selects August 21 in a browser running in, say, Pacific time, and that selection is stored as a JavaScript `Date` object representing local midnight on August 21, converting that value to UTC for an API call (`toISOString()`, for instance) shifts it to August 21 at 7 AM UTC, which is still fine, but for a user in a timezone ahead of UTC, local midnight on August 21 can convert to August 20 late at night in UTC, silently turning the stored date into the wrong day once the server (running in UTC) reads it back. The fix is to never let a pure calendar date pass through a timezone-aware `Date` object at all: store and transmit it as a plain string like `"2026-08-21"` and only construct a timezone-aware `Date` at the specific point where you actually need one (like formatting it for display in the user's local timezone).

Interview Answer:

"For the widget itself, I'd build a `role='grid'` of day cells with roving tabindex, arrow keys moving by a day, week navigation moving by seven, and Page Up and Page Down jumping by a month, with each cell labelled with the full date, not just the bare number, so a screen reader user gets real context. The bug I'd specifically bring up is the timezone off-by-one, where a date picked as local midnight gets converted to UTC for storage and can roll back to the previous day for users ahead of UTC. The fix is to never treat a picked calendar date

as a timezone-aware JavaScript Date at all, I'd store and pass it around as a plain string like '2026-08-21' and only construct an actual timezone-aware Date object at the point where I need to format it for display."

Q25. [Senior] Design a command palette, the kind of Cmd+K search-everything overlay you'd see in Linear or VS Code. What goes wrong with a naive implementation of the global keyboard shortcut that opens it?

Answer:

A command palette is really an autocomplete search box wrapped in a modal, layered on top of a global keyboard shortcut that has to work no matter what the user is currently focused on. **The core design point most naive implementations get wrong is scoping and cleaning up that global keydown listener correctly, since an unscoped or leaked listener either fires while the user is typing in an unrelated text field or double-fires when a second instance of the palette gets mounted.**

Props / API

- `commands` (each with an `id`, `label`, `action`, and optional `keywords` for fuzzy matching beyond the literal label), `isOpen`, `onClose`, and an optional `recentCommandIds` list for surfacing recently used actions first.

Internal State

- The current query text, a filtered and ranked results list (usually fuzzy-matched, not just a substring check, so typing "cls" can still match "Close Tab"), `highlightedIndex`, and a small cache of recently executed commands to bias ranking.

Accessibility Considerations

- The palette is a `role="dialog"` containing a `role="combobox"` input wired to a `role="listbox"` of results, following the same combobox pattern as autocomplete, and on open, focus must move immediately into the search input, then return to whatever had focus before on close.

Tricky Edge Case

- The global `keydown` listener for Cmd+K (or Ctrl+K) has to be attached once, typically at the document level near the app root, and carefully removed on unmount; a common bug is attaching it inside a component that can mount more than once (say, inside a modal that itself gets opened and closed), which either leaks a duplicate listener that fires the palette open twice, or, if the shortcut isn't guarded against firing while the user is actively typing inside an unrelated text input or another modal, hijacks keystrokes the user actually intended for that other field. The fix is registering the listener exactly once at a stable, single-mount location (a root-level provider), checking `event.target` isn't itself an editable field before acting in ambiguous cases, and calling `preventDefault()` deliberately rather than accidentally suppressing the browser's own Ctrl+K/Cmd+K behavior in a way that surprises the user.

Interview Answer:
"The palette itself is basically an autocomplete combobox inside a modal, with fuzzy-matched, ranked results and a recent-commands cache biasing what shows up first. The part I'd actually probe on is the global keyboard shortcut: it needs to be registered exactly once, at a stable root-level location, not inside a component that could mount more than once, or you end up with duplicate listeners double-opening the palette. I'd also guard against the shortcut hijacking keystrokes while the user is legitimately typing in some other field or a second modal is open, and be deliberate about calling preventDefault so I'm not silently breaking a browser-level shortcut the user expects to still work elsewhere."

Q26. [Senior] You're building a design system used by a dozen product teams. How would you design a component's API so it's flexible enough for teams to customize, but consistent enough that it can't easily be built into something inaccessible?

Answer:

This is the meta-question behind every component in this chapter: a design system's components aren't just used once, they're reused hundreds of times by teams who didn't build them and may not read the accessibility notes closely, so the API itself has to make the accessible path the easy, default path. **The core design point is favoring composition, compound components sharing state through context, a polymorphic slot/`asChild` pattern, and consistent ref forwarding, over one giant component with dozens of boolean props trying to anticipate every use case.**

Props / API

- Compound components (like `<Select.Root><Select.Trigger /><Select.Content>…</Select.Content></Select.Root>`) let teams rearrange, restyle, or omit parts of a component without the library needing a new prop for every possible layout variation, which is the difference between one `<Select>` with 25 props and a handful of composable pieces.
- A slot pattern, often called `asChild`, lets a consumer render the component's behavior and accessibility wiring onto their own element (say, rendering a styled Button's behavior directly onto a Next.js `<Link>`) instead of forcing an extra wrapper `<button><a>…</a></button>` that produces invalid, nested-interactive-element markup.
- Every component that wraps a DOM element should use `forwardRef` (a React API letting a parent component get a ref to a child's actual underlying DOM node) so consumers can call `.focus()`, measure the element, or integrate with other libraries that need direct DOM access, without that being blocked just because the element is wrapped in your library's component.

Internal State

- Compound components share state (like a Select's open/closed status and the currently highlighted option) through a shared context created by the `Root` component, rather than requiring the consumer to manually wire props between `Trigger` and `Content`, which is what keeps the pieces feeling like one cohesive component despite being separately composable.

Accessibility Considerations

- Bake correct ARIA wiring directly into the primitives so a consumer physically can't easily assemble an inaccessible instance: auto-generate and connect ids between a label and its input via `useId` (a React hook that generates a stable unique id, avoiding hardcoded ids that collide if the same component renders twice on a page) rather than requiring the consumer to remember to pass matching ids themselves.

```
const SelectContext = createContext(null);

const Select = forwardRef(function Select({ children, value, onChange }, ref) {
  const [isOpen, setIsOpen] = useState(false);
  return (
    <SelectContext.Provider value={{ isOpen, setIsOpen, value, onChange }}>
      <div ref={ref}>{children}</div>
    </SelectContext.Provider>
  );
});

function SelectTrigger({ children }) {
  const { isOpen, setIsOpen } = useContext(SelectContext);
  return (
```

```
    <button aria-expanded={isOpen} onClick={() => setIsOpen((o) => !o)}>
      {children}
    </button>
  );
}
```

Tricky Edge Case

- A team inevitably needs to intercept an event the library component already handles internally, for example, they want their own `onClick` on a Select's trigger to fire (to log analytics) without breaking the library's internal open/close toggle logic. Naively overwriting the internal handler with the consumer's `onClick` silently breaks the component; the more robust pattern is composing both handlers together (calling the internal handler and then the consumer's, or vice versa), or exposing an internal handler that returns a boolean the consumer's handler can use to signal "prevent the default internal behavior," similar to how native DOM events support `preventDefault()`, rather than a plain prop overwrite that stomps on required internal logic.

Interview Answer:

"I'd lean on composition rather than one component with dozens of boolean props, compound components like `Select.Root`, `Select.Trigger`, and `Select.Content` sharing state through context, so teams can rearrange or restyle pieces without needing a new prop for every layout. I'd forward refs on everything that wraps a DOM element so consumers can call `.focus()` or measure it directly, and I'd support an `asChild` slot pattern so a team can render, say, a Button's behavior onto their own routing Link instead of nesting an anchor inside a button and producing invalid markup. On the accessibility side, I'd bake ARIA wiring into the primitives themselves, auto-generating and connecting ids with `useId` rather than trusting every consumer to remember to wire them by hand, since the goal is making the accessible path the default one, not an opt-in extra step. And I'd specifically design an escape hatch for event handler composition, so a team adding their own `onClick` for analytics doesn't accidentally stomp on the component's internal open/close logic."

If you remember nothing else from this chapter: *a component with fourteen boolean props isn't flexible, it's fourteen different components wearing one name.*

Chapter 10: Design Systems & Component Library Architecture

Almost every frontend organization above a certain size ends up building a shared component library, but the interesting engineering happens at the edges: how flexible the API surface is, how breaking changes roll out without freezing a dozen teams, and who gets to decide what belongs in the shared layer versus a single app. If the only way you can describe a design system is "a shared button library," that's a sign you haven't actually operated one at scale. Use these questions to practice thinking about API design, versioning and migration mechanics, and the organizational governance that make a design system survive contact with real, independently deployed teams.

War Story: *A "small" visual tweak to a shared button component's padding shipped in a minor version bump and silently broke layout across eleven different product teams' screens overnight.*

Avoid: treating a shared design-system component the same as an ordinary app component when it comes to versioning discipline

Aim For: instinctively treating any visible change to a shared component as a breaking change, not a minor one

Q1. [Junior] Your company has five product teams, each shipping its own app, and each has quietly built its own version of a modal, a dropdown, and a date picker. What problem does a shared design system actually solve here, beyond "we'd have fewer files"?

Answer:

A shared design system solves two distinct problems at once: visual and behavioral consistency for the end user, and development velocity for the engineering organization. **The real payoff isn't deduplicating code, it's that product teams stop re-solving accessibility, interaction, and visual-consistency problems that have already been solved once, correctly, elsewhere.**

Consistency problem

- Five independently built date pickers likely have five different keyboard navigation behaviors, five different screen reader announcements, and five slightly different visual treatments, all under the same company brand.
- Users notice this as "this app feels different from that one," which erodes trust even if no individual screen looks broken.

Velocity problem

- Every team currently pays the full cost of designing, building, testing, and accessibility-auditing a date picker from scratch. That is weeks of work, times five.
- A shared library amortizes that cost once, so a sixth team building a new app gets a production-grade, accessible date picker on day one instead of week three.

What it is not

- It is not primarily about saving a few kilobytes of duplicated code. Bundle size savings are a minor side benefit; the dominant costs being solved are engineering time and UX fragmentation.

Interview Answer:
"The five-teams-five-date-pickers scenario is really a story about paying the same tax five times. Each team independently burns weeks on interaction design, accessibility, and edge cases like timezone handling, and the end user experiences five subtly different products under one brand. A shared design system centralizes that cost once so future teams start from a solved, accessible, on-brand component instead of reinventing it, and users get a consistent experience across the whole product suite. It's fundamentally an investment in velocity and consistency, not a code-deduplication exercise."

Q2. [Junior] You're designing the API for a shared `Button` component that needs to support a primary style, a danger style, a loading state, and an icon-only variant. A teammate proposes `<Button primary danger loading iconOnly />` with a boolean prop for each variant. What's the problem with this approach, and what would you do instead?

Answer:

Independent booleans imply combinations are all valid when most aren't, like `primary` and `danger` set simultaneously, and every new visual variant requires another boolean forever. **Boolean props model a component's states as independently toggleable flags, when in reality most variant dimensions are mutually exclusive choices that should be modeled as a single enum-like prop.**

Why booleans break down

- `primary` and `danger` together is a nonsensical combination the type system doesn't prevent; the component has to defensively resolve conflicts at runtime.
- Adding a "success" variant means adding yet another boolean, and the prop list grows linearly with every new design decision forever.

Better shape

```
type ButtonVariant = "primary" | "danger" | "secondary" | "ghost";

interface ButtonProps {
  variant?: ButtonVariant; // mutually exclusive, one wins
  loading?: boolean;       // truly independent state
  icon?: React.ReactNode;  // presence implies icon-only if no children
  children?: React.ReactNode;
}
```

- `variant` collapses all mutually exclusive visual choices into one prop with a closed set of valid values, which TypeScript can enforce at compile time.
- `loading` stays a boolean because it genuinely is an independent, orthogonal state, a button can be `primary` and `loading` at the same time.
- Icon-only mode is derived from the presence or absence of `children` rather than a separate `iconOnly` flag, removing a way to misconfigure the component.

Interview Answer:
"The tell here is that primary and danger being on at the same time doesn't mean anything, so booleans are the wrong model. I'd collapse the mutually exclusive visual choices into a single variant prop typed as a union, which the type system can enforce, and keep only genuinely independent states like loading as separate booleans. That way adding a new variant later is a one-line change to the union type instead of a

new boolean threaded through every conditional in the component, and it's structurally impossible to request two conflicting variants at once."

Q3. [Junior] A designer asks for a `Card` component that can show an image, a title, a description, action buttons, and sometimes a badge, in different orders depending on the product. Would you build this as one big configurable component with props like `imageUrl`, `title`, `showBadge`, or would you design it differently?

Answer:

A single component with a prop for every possible slot turns into a wall of optional props that can't express arbitrary ordering or novel layouts, and every new use case means touching the shared component again. **Composition, exposing `Card`, `Card.Image`, `Card.Title`, and `Card.Actions` as separate pieces that consuming teams arrange themselves, scales to layouts the library author never anticipated without ever changing the shared component's code.**

Problem with the all-in-one prop approach

- `imageUrl`, `title`, `showBadge`, `actionLabel1`, `actionLabel2` cannot express "badge appears before the title" or "three actions instead of two" without more props or an escape-hatch render prop.
- Every new layout request becomes a pull request against the design system repo, which doesn't scale once there are dozens of consuming teams.

Compositional shape

```
<Card>
  <Card.Image src={product.image} alt={product.name} />
  <Card.Badge>New</Card.Badge>
  <Card.Title>{product.name}</Card.Title>
  <Card.Description>{product.summary}</Card.Description>
  <Card.Actions>
    <Button variant="primary">Buy</Button>
  </Card.Actions>
</Card>
```

- Ordering, omission, and repetition are all free: a team that needs the badge before the title, or no image at all, just arranges the children.
- The shared component only owns shared concerns, spacing, alignment, responsive behavior, not the specific content mix.

Interview Answer:

"I'd push back on the giant-props-object version because it can only express the layouts its author thought of in advance, and every new arrangement becomes a change request against the design system team. I'd expose Card as a set of composable subcomponents, Card.Image, Card.Title, Card.Actions, and let consuming teams arrange, omit, or repeat them freely, while the parent Card still owns shared layout concerns like spacing and responsive behavior. That's the difference between a component that scales to unanticipated use cases and one that requires a design system PR for every new page layout."

Q4. [Junior] A junior engineer on your team wants to add a `theme` prop to every component so pages can look different, by passing color values directly, like

`<Button theme={{ background: '#FF0000' }} />`. What's the architectural concern with that approach, and what would you suggest instead?

Answer:

Letting every component accept arbitrary inline color overrides means the design system stops being the source of truth for what colors are allowed, and it becomes impossible to audit or globally change the palette later. **Theming should flow through a small set of named design tokens, semantic values like `color.action.primary`, provided once at the app root, not as ad hoc color literals passed into individual component props.**

Why per-component raw colors are risky

- `#FF0000` passed directly bypasses every accessibility contrast check and brand guideline the design system was supposed to enforce.
- If the brand's primary red changes next year, someone has to grep every consuming app for hardcoded hex values instead of changing one token definition.

Token-based approach

```
// design-system/theme.ts
const lightTheme = {
  colorActionPrimary: "#0B5FFF",
  colorDangerBackground: "#D92D20",
};

// Button.tsx internally references the token, never a raw value
// consumer just does:
<ThemeProvider theme={lightTheme}>
  <Button variant="primary">Save</Button>
</ThemeProvider>
```

- Components reference semantic tokens (`colorActionPrimary`) internally, never raw hex values passed in from the outside.
- A rebrand or dark mode becomes a change to the token map at the root, not a search-and-replace across every app that uses the library.

Interview Answer:

"Passing raw hex values into individual components defeats the purpose of having a design system, because now the color is defined at the call site instead of in one governed place. I'd have components reference semantic design tokens internally, like a primary action color, and provide the actual palette once at the app root through a theme provider. That way a rebrand or a new dark theme is a change to the token definitions in one file, not a hunt through every app for hardcoded colors, and it keeps the design system as the actual source of truth for what's on-brand."

Q5. [Junior] Your design system's `Modal` component currently only supports a title and a message, as separate string props. Product now wants a modal with a custom form inside it. Do you add a `formFields` prop, or handle this differently?

Answer:

A `formFields` configuration prop would force the design system to reinvent form-building as a config schema, duplicating whatever form library the app already uses. **Once content needs to be arbitrary and interactive, the component should accept `children` and let the consumer render whatever they need inside a structural shell the design system still controls.**

Why a config prop is the wrong direction

- Forms have validation, field types, conditional visibility, and submission states; encoding all of that as a generic `formFields` prop means rebuilding a forms API inside the modal.
- The design system team becomes a bottleneck for every new field type a product team wants to add.

Children-based shape

```
<Modal isOpen={isOpen} onClose={handleClose} title="Invite teammate">
  <InviteForm onSubmit={handleInvite} />
</Modal>
```

- `Modal` still owns the things that must be consistent everywhere: focus trapping, escape-to-close, overlay click behavior, animation.
- The content is fully open-ended because it's just `children`, so the same `Modal` shell works for a form, a confirmation message, or a media preview.

Interview Answer:

"I wouldn't build a formFields prop, because that's the design system trying to become a forms library, and it'll always be one step behind whatever validation or field type product actually needs. I'd keep Modal focused on the things that genuinely need to be consistent everywhere, like focus trapping and escape-to-close, and accept arbitrary children for the content. That way the same Modal shell works whether the content is a form, a confirmation dialog, or an image preview, without the design system ever needing to know what's inside."

Q6. [Mid] Explain the difference between a "styled" component library and a "headless" component library (like building on top of Radix or downshift-style primitives), and describe a real scenario where you'd choose headless over pre-styled.

Answer:

A styled library ships both the behavior (open/close logic, keyboard handling, focus management) and the visual output as one package, while a headless library ships only the behavior and interaction logic, leaving all markup and styling to the consumer. **Headless components separate "how does this behave" from "how does this look," which matters enormously the moment one org has to support multiple, visually distinct brands on the same underlying interaction logic.**

When pre-styled is fine

- A single-brand internal tool or admin dashboard where every team should look identical: a fully styled library like a Material-based kit gets you there fastest, with no styling work needed.

When headless is the right call

- Example: a company running three consumer brands, each with a different visual identity, that all need an accessible, keyboard-navigable combobox (searchable dropdown). Building three separate combobox implementations means three separate a11y (accessibility) bugs to fix independently.
- A headless combobox primitive provides the state machine, keyboard handling, and ARIA attributes once; each brand's design system wraps it with its own markup and CSS.
- This also matters when the org's own design system is styled on top of a headless base internally, so the design system team fixes interaction bugs in one place while each brand controls its own visual layer.

Trade-off

- Headless costs more upfront integration work per component, since someone still has to write the styled wrapper, but it decouples "is this accessible and behaviorally correct" from "does this match brand X," which pays off the moment there's more than one visual identity to support.

Interview Answer:

"The distinction is behavior versus presentation. A styled library bundles both together, which is great for a single brand but breaks down the moment you have multiple visual identities sharing the same interaction patterns. I'd reach for a headless base, something like Radix primitives, in a scenario like three consumer brands all needing an accessible combobox, because the keyboard handling and ARIA wiring gets solved once and each brand layers its own CSS and markup on top. It costs more integration work upfront since you're writing the styled wrapper yourself, but it means an accessibility fix happens in one place instead of three."

Q7. [Mid] Your design system's component library is published as an npm package and consumed by twelve independently deployed apps, each on its own release schedule. You need to ship a breaking change to the `Tooltip` component's API. Walk me through how you'd roll this out without forcing all twelve teams to update in lockstep.

Answer:

Forcing synchronized updates across twelve independently deployed apps is an organizational non-starter, since it requires coordinating twelve teams' release calendars around your change. **The rollout should support both old and new APIs simultaneously for a deprecation window, communicated through semantic versioning and a codemod, so each team migrates on its own schedule instead of everyone migrating at once.**

Step-by-step rollout

1. Ship the new API alongside the old one in a minor version release, with the old prop shape marked `@deprecated` in the type definitions and a console warning in development builds.
2. Publish a codemod (an automated script that rewrites consumer code to match the new API) so teams can run one command to migrate their usage instead of hand-editing every call site.
3. Announce a deprecation window, for example two release quarters, during which both APIs work, and communicate the removal date through changelog, Slack, and a banner in the component's Storybook docs.
4. Release the actual breaking removal as a major version bump per semver (semantic versioning), so any team still on the old major version is simply unaffected until they choose to upgrade.
5. Track adoption: query which apps have upgraded past the major version, and follow up directly with the teams still on the old version as the deprecation window closes.

Why this works over a big-bang change

- Nobody is forced to stop their own roadmap work to accommodate the design system's release; the cost of migrating is decoupled from the cost of staying current on other things.

Interview Answer:

"With twelve independently deployed consumers, I can't assume everyone will drop what they're doing to migrate on my timeline. So I'd ship the new Tooltip API alongside the old one in a minor release, mark the old prop shape deprecated with a dev-time warning, and provide a codemod so teams can migrate

mechanically instead of by hand. I'd give a defined deprecation window, maybe two quarters, before actually removing the old API in a major version bump, which per semver means nobody's build breaks until they intentionally upgrade. The whole point is decoupling my release from their release calendar."

Q8. [Mid] Two teams both need a "Select" component that behaves almost like your design system's existing `Select`, but one needs async-loaded options and the other needs multi-select with tag chips. Would you extend the existing component with more props, wrap it, or fork it? Walk me through your decision process.

Answer:

The decision hinges on whether the new requirement is a variation of the same underlying interaction model or a genuinely different one. **Extend the existing component when the new need is a configuration of the same behavior, compose a wrapper around it when the need is additive but separable, and only fork when the interaction model itself fundamentally diverges.**

Decision framework

1. Is this "the same thing with a different data source"? Async-loaded options are still a single-select dropdown; the only change is how options arrive. This should be an added prop, like `loadOptions`, on the existing component, because the underlying keyboard navigation and rendering are unchanged.
2. Is this "the existing thing plus an additional concern"? Multi-select with tag chips changes the value shape (an array instead of a single value) and adds chip rendering and removal. This can often be composed: a `MultiSelect` wrapper component that uses the base `Select` internally for the dropdown mechanics, adding chip UI around it, rather than cramming multi-select branching logic into the single-select component.
3. Is this "a different interaction model entirely"? If a team needs a combobox with free-text entry and async search combined with a completely different keyboard model, forking (or building a new, separate component) may be justified, but this should be rare and should trigger a conversation with design system governance about whether it becomes a first-class new component.

Why not just pile props onto one component

- Cramming both async loading and multi-select tag logic into one `Select` implementation means every consumer pays the bundle-size and complexity cost of features they don't use, and testing the matrix of combinations balloons.

Interview Answer:
"I'd start by asking whether the new requirement is the same interaction model with different data, or a genuinely different pattern. Async options for the existing Select is the same single-select behavior with a different data source, so that's just a new prop like loadOptions. Multi-select with tag chips changes the value shape and adds chip rendering, so I'd lean toward composing a MultiSelect wrapper that uses the base Select internally rather than bolting multi-select branching onto the single-select component. Forking is the last resort, reserved for when the interaction model itself is fundamentally different, and even then I'd loop in whoever governs the design system before creating a parallel component."

Q9. [Mid] Your organization is deciding what belongs in the shared design system versus what should just live as a one-off in a single app's codebase. A team wants to contribute a highly specific "shipment tracking timeline" component. How do you decide?

Answer:

The test isn't "is this well-built" or "could other teams theoretically use it," it's whether there's actual, current demand from more than one team and whether the component's logic is generic enough to be decoupled from one product's specific domain model. **Something belongs in the shared design system when at least two teams need it now and its API can be expressed in domain-agnostic terms, not when it's merely well-engineered or hypothetically reusable.**

Signals it should stay app-local

- Only one team currently needs it, and speculative reuse ("someone might want this someday") isn't evidence of real demand.
- Its props or data model are tightly coupled to one product's domain, for example a shipment timeline whose steps are hardcoded to a logistics company's specific fulfillment stages.

Signals it should move into the design system

- A second team independently builds or requests something with the same shape, for example an order-status timeline and an onboarding-progress timeline turn out to be the same generic "step-based timeline" pattern with different labels.
- The component can be redesigned around a generic, data-driven API, like accepting an array of `{ label, status, timestamp }` steps, rather than domain-specific props like `shipmentId`.

Practical middle ground

- The team can build it locally first, proven in production, and only propose promoting it to the shared library once a second real consumer appears, avoiding premature abstraction based on a single use case.

Interview Answer:
"My first question is always whether a second team actually needs this today, not whether it's theoretically reusable, because premature generalization based on one use case usually produces the wrong abstraction. For the shipment tracking timeline, I'd look at whether its core shape, a sequence of steps with statuses and timestamps, is actually generic once you strip out the shipping-specific labels. If another team's onboarding flow needs the same step-based timeline pattern, that's real evidence it belongs in the design system, generalized around a data-driven API. Until then, it should stay in that one app, built well, and get promoted later if real cross-team demand shows up."

Q10. [Mid] Explain how you'd architect theming in a component library that needs to support both a light/dark mode toggle at runtime and multiple distinct corporate brands, without duplicating every component's styles per brand.

Answer:

Light/dark and multi-brand are two independent axes, and conflating them into one flat list of theme variants creates a combinatorial explosion. **Theming should be layered as a small set of semantic design tokens that components consume, with brand and color-mode each resolving independently into concrete values for those same token names.**

Layered token architecture

- Define semantic tokens components actually reference, like `colorSurfacePrimary`, `colorTextOnSurface`, `colorBorderSubtle`, never raw hex values inside component code.

- Each brand provides a token map; each color mode (light/dark) provides a modifier or a second token map per brand.
- Components are written once against the semantic token names and never know which brand or mode is active.

Example

```
const tokens = {
  brandA: {
    light: { colorSurfacePrimary: "#FFFFFF", colorTextOnSurface: "#111111" },
    dark:  { colorSurfacePrimary: "#0B0B0C", colorTextOnSurface: "#F5F5F5" },
  },
  brandB: {
    light: { colorSurfacePrimary: "#FFF8F0", colorTextOnSurface: "#20140A" },
    dark:  { colorSurfacePrimary: "#101010", colorTextOnSurface: "#FDE8D0" },
  },
};

// Resolved once at the app root:
const activeTokens = tokens[currentBrand][currentColorMode];
```

- CSS custom properties (variables that can be swapped at runtime without a JS re-render) are a good implementation vehicle here, since flipping `data-theme="dark"` on the root element can swap every token value instantly without React re-rendering the whole tree.

Why not duplicate styles per brand per mode

- Four brands times two modes as eight hardcoded style sets per component means eight places to fix any visual bug, and it doesn't scale when brand nine is added.

Interview Answer:

"I'd treat brand and light/dark mode as two orthogonal axes, not as one flat list of theme variants, because otherwise four brands times two modes becomes eight hardcoded style sets to maintain per component. Components should only ever reference semantic tokens, like a surface color or an on-surface text color, and never a raw hex value. Then brand and mode each resolve independently into concrete values for those same token names, and I'd usually implement that with CSS custom properties so switching mode or brand at runtime is just swapping a data attribute at the root, no component re-render or per-brand style duplication required."

Q11. [Mid] Your Storybook (a tool for browsing and testing UI components in isolation) instance for the design system has grown to over 200 component stories over two years. Engineers increasingly complain they can't find the right component, or don't trust the docs because several stories are stale and don't match production behavior. How do you fix this?

Answer:

The core problem is that documentation quality decayed because nothing kept it in sync with the actual component code, and search/discovery wasn't designed for scale. **Treat the documentation as a build artifact validated in CI, not a one-time writing task, and organize discovery around use-case and pattern, not just an alphabetical component list.**

Fixing trust (staleness)

- Wire stories into CI so a component's props changing without a corresponding story update fails the build, for example using TypeScript types to auto-generate the props table so it can never silently drift from the real API.

- Add visual regression testing (automated screenshot diffing) against the stories, so a story that no longer reflects real rendered output gets caught before merge, not discovered by an engineer months later.
- Assign ownership: each component has a named owning team responsible for keeping its story and docs accurate, reviewed on a cadence, not "whoever wrote it two years ago."

Fixing discovery (findability)

- Reorganize navigation around task ("I need a way to let users pick a date") and pattern (forms, feedback, navigation, data display), not just a flat alphabetical list of 200 names.
- Add a search that indexes on tags and use-case descriptions, not just component names, since engineers often know what they want to do before they know what it's called.
- Surface a small "recommended/stable" set distinctly from experimental or soon-to-be-deprecated components, so engineers don't have to guess which of three similar-looking options is the current standard.

Interview Answer:

"Two hundred stories failing to be trusted or found are really two separate problems. For trust, I'd make the documentation part of the build, auto-generating the props tables from the actual TypeScript types so they can't drift, and adding visual regression tests against the stories so a stale story gets caught in CI instead of by a confused engineer. For discovery, I'd reorganize navigation around what someone's trying to accomplish, forms, feedback, data display, rather than an alphabetical component dump, and make search understand use-case terms, not just component names. And I'd flag which components are the current recommended standard versus deprecated or experimental, since with 200 components engineers need a signal for what to actually reach for."

Q12. [Mid] A team requests a brand-new "Notification Banner" component be added to the design system. Who should decide whether it gets built, what its API looks like, and how does that request actually flow through your organization? Design the governance process.

Answer:

Without a defined process, design system contributions either bottleneck entirely on one overloaded team or fragment into inconsistent ad hoc additions with no review. **Governance should combine a lightweight intake process anyone can use, a small cross-functional review (design system engineers plus a designer) that owns API consistency, and clear criteria for approval so decisions aren't arbitrary.**

Intake

- A standard request template capturing: the concrete use case, whether other teams have a similar need, and a rough sketch of the desired API or visual design, submitted as an issue in the design system's repo or tracker.

Review

- A small standing group, for example two design system engineers and one product designer, reviews new component requests on a regular cadence (say, biweekly), evaluating against the "does this belong in the design system" criteria (cross-team demand, generic API) from earlier.
- The review focuses specifically on API shape at this stage, catching a boolean-prop-pile-up before it's built, not after.

Build and ownership

- Either the requesting team builds it with design system engineers reviewing for API consistency and accessibility, or the design system team builds it directly if the requesting team lacks bandwidth or design system-specific expertise (accessibility, theming integration).
- Once merged, the component has a named owning team responsible for maintenance, not an orphaned contribution nobody revisits.

Why this beats the two failure modes

- Without any process: either the design system team becomes a bottleneck reviewing everything ad hoc, or teams bypass governance and each build their own banner, recreating the original consistency problem.

Interview Answer:
"I'd set up a lightweight intake, a request template that captures the use case and whether other teams need something similar, filed against the design system's own repo. A small standing review group, maybe two design system engineers and a designer, looks at these on a regular cadence and specifically reviews the proposed API shape before anything gets built, since that's where boolean-prop pileups get caught early. The requesting team can build it with design system engineers reviewing for accessibility and consistency, or the design system team builds it if that expertise is needed, but either way it gets a named owner afterward so it doesn't become an orphaned component nobody maintains."

Q13. [Senior] Your design system's `Table` component is used in 40 places across 15 apps. You need to deprecate it in favor of a new `DataGrid` with a different API, better performance, and virtualization (rendering only visible rows) for large datasets. Design the migration strategy for moving all 40 usages without a big-bang rewrite.

Answer:

A big-bang rewrite of 40 usages across 15 independently deployed apps is both a huge coordination cost and a huge risk, since any subtle behavior difference gets discovered in production 40 times over. **The migration should run both components side by side behind usage analytics and a codemod, prioritized by actual impact, so teams migrate incrementally on their own schedule while the design system tracks real progress instead of assuming it happened.**

Step-by-step strategy

1. Ship `DataGrid` as a new, independent component; do not touch `Table` yet. This lets early-adopter teams (ones with the most pain from missing virtualization) opt in immediately without anyone else being affected.
2. Instrument both components with lightweight usage telemetry, or at minimum grep-based auditing in CI across the monorepo or dependency graph, to get a real, current count of the 40 usages rather than trusting a stale spreadsheet.
3. Write a codemod that handles the mechanical parts of the API translation, for example renaming `columns` prop shape or `onRowClick` to `onRowSelect`, and dry-run it against all 15 apps to see what fraction of usages it can auto-migrate versus what needs a human.
4. Segment the 40 usages by migration difficulty and business impact: tables with 10,000+ rows suffering real performance problems get migrated first since they have the clearest win; tables with 20 static

rows and heavy custom cell rendering get scheduled last since they benefit least and cost more to hand-migrate.

5. Mark `Table` `@deprecated` with a console warning once `DataGrid` has proven itself in production for a few cycles, giving a concrete removal timeline communicated well in advance, not an open-ended "eventually."
6. Track migration completion centrally (a dashboard of remaining `Table` usages by app) and follow up directly with the owning teams of the stragglers as the deadline approaches, rather than assuming silence means completion.

Why not force it all at once

- Forcing 15 teams to migrate simultaneously means the design system team owns risk it doesn't control, since it can't verify 40 call sites all behave identically under a different API in the time available.

Interview Answer:

"I would never try to migrate all 40 usages in one push, since that puts the design system team on the hook for verifying correctness across 15 codebases it doesn't control. I'd ship DataGrid alongside Table, let it prove itself with the teams who have the most pain, like ones with large unvirtualized tables, and build a codemod to handle the mechanical API translation for the rest. Then I'd segment the remaining usages by actual impact and difficulty, migrate the highest-value ones first, and only mark Table deprecated with a real removal date once DataGrid has track record in production. Throughout, I'd track progress centrally instead of assuming teams migrated just because we announced it, and follow up directly with whoever's left as the deadline gets close."

Q14. [Senior] Your component library ships as a single npm package containing every component. One team complains their production bundle includes 400KB of design system code even though they only use six components. Diagnose the likely cause and describe how you'd restructure the library's packaging to fix it.

Answer:

A single monolithic package usually means either the bundler isn't tree-shaking (eliminating unused code) effectively, or the package has hidden cross-component dependencies that pull in far more than what's imported. **Fix it by ensuring the package is structured and built for proper tree-shaking, ES modules with no side-effecting imports, and by decoupling components from each other internally, so importing one component's file doesn't transitively drag in unrelated ones.**

Likely root causes

- The package is published as a single CommonJS bundle or one large `index.js` that re-exports everything, which many bundlers cannot tree-shake effectively even if only six named imports are used.
- Components internally import from a shared "kitchen sink" utils file that itself imports heavy, unrelated dependencies (like a full date library used only by the date picker), so importing `Button` accidentally pulls in date-handling code.
- The `package.json` is missing a `sideEffects: false` declaration, so bundlers conservatively assume every module has side effects and can't safely drop unused ones.

Fix

1. Publish as ES modules (the standard JavaScript module format that static analysis tools can tree-shake) with per-component entry points, so `import { Button } from 'design-system'` resolves cleanly to just Button's code, not the whole library.
2. Set `"sideEffects": false` in `package.json` (or list the specific files that do have side effects, like a global CSS reset), giving bundlers explicit permission to eliminate unused exports.
3. Audit internal imports so components depend only on genuinely shared primitives, and heavy dependencies used by only one component, like a date library, stay scoped to that component's own module rather than a shared utils barrel file.
4. Add a bundle-size budget check in CI, using a tool that fails a pull request if importing a single component's size grows past a threshold, catching regressions before they ship to all 15 consuming apps.

Interview Answer:

"400KB for six components strongly suggests tree-shaking isn't happening, which usually comes down to two things: the package isn't structured as clean ES modules with a sideEffects false declaration, so bundlers can't safely drop unused code, or components have hidden internal dependencies on each other or on heavy shared utilities, so importing Button accidentally pulls in a date library used only by DatePicker. I'd restructure the package with per-component entry points, mark it side-effect free, and audit the internal import graph so components only depend on genuinely shared primitives. Then I'd add a bundle-size budget in CI so a future regression, like someone accidentally importing something heavy into a shared utils file, gets caught before it ships to every consuming app."

Q15. [Senior] Two teams building on your design system have both independently discovered a real gap: neither `Button` nor `IconButton` supports a "split button" pattern (a primary action button attached to a secondary dropdown of related actions). Walk me through how you'd decide the API design, and what specifically you'd do to avoid this becoming another pile of booleans two years from now.

Answer:

The risk with any new composite component is designing its API around today's two known use cases instead of the underlying pattern, which guarantees a third team's slightly different need turns into more boolean props bolted on. **Design the API around composition of already-existing primitives, `Button` plus a dropdown/menu primitive, rather than inventing a new monolithic `SplitButton` with props for every anticipated variation.**

Wrong instinct

```
// Brittle: every new requirement becomes another prop
<SplitButton
  primaryLabel="Save"
  secondaryActions={[{ label: "Save as draft" }, { label: "Save and close" }]}
  dropdownAlign="right"
  disableSecondary={false}
/>
```

- This looks reasonable for two use cases but breaks the moment a third team needs, say, icons per secondary action, or a secondary action that's itself disabled conditionally; each of those becomes another prop.

Compositional API

```
<SplitButton>
  <Button variant="primary" onClick={handleSave}>Save</Button>
```

```
  <Menu>
    <Menu.Item onClick={handleSaveDraft}>Save as draft</Menu.Item>
    <Menu.Item onClick={handleSaveAndClose} disabled={!canClose}>
      Save and close
    </Menu.Item>
  </Menu>
</SplitButton>
```

- `SplitButton` becomes a thin structural wrapper handling only the shared visual seam and keyboard behavior (arrow key navigation between the button and the menu trigger, for example) between an existing `Button` and an existing `Menu`.
- Every future requirement, icons on menu items, conditionally disabled items, nested submenus, is already solved because it's just `Menu`'s existing API, not a new prop on `SplitButton`.

Process to avoid future prop pileup

- Before building, explicitly write down "what does SplitButton actually need to own" (the visual seam, focus management, ARIA relationship between button and menu) versus "what should it delegate" (everything about what's inside the menu), and hold that boundary in review.

Interview Answer:

"The instinct to avoid is building a SplitButton with a primaryLabel prop and a secondaryActions array, because that looks fine for two teams' current needs but turns into more props the moment a third team wants icons or conditionally disabled items in the secondary list. I'd instead design SplitButton as a thin wrapper that composes the existing Button and Menu primitives, owning only the shared concerns like the visual seam between them and the keyboard relationship, like arrow-key navigation from the button into the menu. Everything about what's inside the dropdown, icons, disabled state, submenus, is then just Menu's existing API for free, not a new prop surface on SplitButton. The discipline is writing down what the new component actually needs to own versus what it should delegate, before writing any code."

Q16. [Senior] Your company is doing a full visual rebrand: new color palette, new typography, new spacing scale, rolling out across 30 apps that all consume your central design system. Design the rollout strategy. Assume you cannot get all 30 teams to redeploy in a coordinated window.

Answer:

A synchronized rebrand across 30 independently deployed apps is operationally impossible to coordinate and creates a visible seam the moment even one team is late. **Decouple the visual change from app deploys entirely by driving the rebrand through the token layer and a runtime-switchable flag, so the new brand ships inside the design system itself and each app "picks it up" on its own schedule without a code change.**

Foundational move: push the rebrand into tokens, not components

- Since theming is already token-based (see Q10), the rebrand is primarily a change to token *values* (new colors, new type scale, new spacing scale), not a change to component code or APIs.
- This means the design system package itself can ship both the old and new token sets simultaneously, gated by a runtime flag, without any consuming app needing a code change to keep functioning.

Rollout sequence

1. Implement the new brand as a second complete token set inside the design system, verified against every existing component in Storybook and via visual regression tests, before any app sees it.

2. Ship a runtime theme flag (for example, resolved from a feature flagging service or a config value) that lets the design system pick old-brand or new-brand tokens without an app redeploy, since the app is already on a design system version that contains both.
3. Roll out by cohort rather than by coordinated date: start with one low-risk internal or low-traffic app, verify visually and via analytics (no regression in engagement, no accessibility contrast regressions), then expand cohort by cohort.
4. For apps that are behind on the design system version entirely and can't get the new tokens without an upgrade, treat that as a separate, tracked upgrade backlog, and set an outer deadline after which the old brand token set is removed.
5. Maintain a small honest exception list, since some apps will have hardcoded values outside the token system (see Q4), and those need direct code fixes tracked individually rather than blocking the rest of the rollout.
6. Communicate a hard cutover date for when the old token set is removed from the design system entirely, giving stragglers a deadline instead of an indefinite dual-brand period.

Why this avoids the big-bang trap

- Because the switch is a flag flip on top of already-deployed code, most of the 30 apps need zero redeploy to get the new brand, only a config change, which sidesteps needing 30 coordinated releases.

Interview Answer:

"The key move is realizing a rebrand is fundamentally a token value change, not a component API change, so if theming is already token-based, I can ship the new brand as a second complete token set inside the design system itself, verified against every component before any app sees it. Then I'd gate it behind a runtime flag so apps that are already on a recent design system version pick up the new brand through a config change, no redeploy needed, and roll it out cohort by cohort starting with low-risk apps rather than trying to coordinate 30 simultaneous releases. Apps still behind on the design system version become a tracked upgrade backlog with their own deadline, and I'd keep an honest list of places with hardcoded values outside the token system that need individual fixes. The whole strategy is about decoupling the visual change from the deploy cycle, so 30 independent teams don't have to move in lockstep."

Q17. [Senior] A senior engineer proposes that every component in the design system accept a `className` or `style` prop so any team can override any style they need for their specific use case. What's your position on this, and how would you actually handle the legitimate need for one-off visual customization?

Answer:

Unrestricted style escape hatches quietly transfer control of the design system's visual consistency to whoever calls the component, which defeats the reason the design system exists. **Allow narrow, intentional customization points, like specific style props for spacing or a documented set of CSS variables a component exposes, rather than an unrestricted `className`/`style` override that lets any visual property be changed from outside.**

Why unrestricted overrides are dangerous

- Once `className` can override anything, the design system can no longer guarantee what a `Button` looks like in production; a visual audit or an automated contrast check becomes meaningless since any consumer might have silently overridden the color.

- It becomes a support and versioning nightmare: bumping the component's internal CSS structure can silently break someone's override that depended on an internal class name, even though that was never a documented, stable API.

What to do instead

- Identify what customization is actually legitimate and expose it as a real, typed prop: if teams need custom spacing around a component, expose a `spacing` prop with defined valid values, not an open className escape hatch.
- Expose a small set of documented CSS custom properties (`--button-radius`, for instance) that are explicitly part of the public API and covered by the same versioning guarantees as props, so overriding them is supported, not a hack.
- For truly one-off visual needs that don't generalize, that's a signal the use case belongs in the consuming app's own code, possibly composing around the design system component, rather than being solved by cracking open the shared component's styles.

Interview Answer:

"My position is no, not as an unrestricted escape hatch, because the moment className or style can override anything, the design system can't actually guarantee what its own components look like in production, which undermines the entire reason it exists. Instead I'd figure out what customization is legitimately needed and expose it as an explicit, typed prop, or as a small documented set of CSS custom properties that are part of the versioned public API, not an internal implementation detail someone happened to be able to reach into. If a use case is so one-off that it doesn't fit even that, that's usually a sign it belongs in the consuming app's own code, composed around the design system component, rather than being solved by letting anyone override any internal style."

Q18. [Senior] Your design system has been running for three years. You discover that four different teams have each built their own, slightly different wrapper around the shared `Input` component to add validation error display, because the base `Input` never had opinions about error state. Diagnose what went wrong and how you'd prevent this class of problem going forward.

Answer:

This is a sign the design system shipped a component with a gap in its core, extremely common use case, error state, so every consuming team was forced to solve the same problem independently, exactly the fragmentation the design system was supposed to prevent. **The fix is both tactical, retrofit `Input` with a first-class, well-designed error/validation API and migrate the four wrappers onto it, and process-level, add a feedback loop so gaps like this get caught from real usage instead of being rediscovered four separate times.**

Diagnosing the gap

- Error display around form inputs is not an edge case, it is one of the most common needs in any form-heavy product, so its absence in the base `Input` API was a real design system gap, not an unreasonable ask.
- Four independent teams solving it independently means the original "does this belong in the design system" signal (cross-team need) was actually present all along; it just wasn't visible to the design system team because there was no channel surfacing it.

Fix

```
interface InputProps {
  value: string;
  onChange: (value: string) => void;
  error?: string;          // presence of a message implies error styling + ARIA wiring
  "aria-describedby"?: string; // still overridable for advanced cases
}
```

- Add `error` as a first-class prop that handles both the visual treatment (border color, icon) and the accessibility wiring (`aria-invalid`, associating the error text via `aria-describedby`) in one place, so every consumer gets a correct, accessible error state without reinventing it.
- Provide a codemod and migration guide to move the four existing wrapper components onto the new native `error` prop, then deprecate the wrappers per the earlier migration pattern (Q13).

Preventing recurrence

- Instrument or periodically audit consuming app codebases for repeated wrapper patterns around design system components, since a wrapper appearing in multiple apps is a strong, objective signal of a real gap, stronger than a single feature request.
- Add a lightweight "gap report" channel in the contribution process (Q12) explicitly for "I had to build a workaround" reports, not just "please add a new component" requests, since workaround-driven gaps often hide inside existing components rather than looking like brand-new component ideas.

Interview Answer:

"This tells me the base Input shipped with a real gap, no first-class error state, and because that's such a common need in any form, four teams independently rediscovered and patched around it instead of one team, which is exactly the fragmentation the design system exists to prevent. The immediate fix is adding a proper error prop to Input that handles both the visual state and the accessibility wiring, like aria-invalid and describedby, in one place, then migrating the four wrappers onto it with a codemod. The longer-term fix is process: I'd want some way of noticing when multiple apps have built structurally similar wrappers around the same design system component, since that pattern is a much stronger signal of a real gap than any single feature request, and I'd rather catch it after one team builds a wrapper than after four do."

Q19. [Senior] Compare building your design system's component library fully in-house from scratch versus building it as a themed layer on top of an existing open-source library like Material UI or Chakra. What would actually drive that decision for a mid-size company building its first real design system?

Answer:

This decision trades control and long-term flexibility against speed and reduced maintenance burden, and the right call depends heavily on how differentiated the company's product design actually needs to be and how much engineering capacity exists to own infrastructure work indefinitely. **Building on top of an existing library is usually right when speed and low maintenance matter more than deep visual differentiation, while a from-scratch (or headless-primitive-based) build is justified when the product's visual identity or interaction requirements are strategically important enough to be worth owning indefinitely.**

Best Choice

For most mid-size companies building their first real design system, start on top of an existing library or headless primitives, and only invest in fully custom, from-scratch components for the small number of components that are genuinely core to product differentiation.

Why?

- An existing library like Material UI already solved accessibility, keyboard interaction, and cross-browser rendering for dozens of common components; rebuilding all of that from scratch is months of work that produces no new user-facing value on its own.
- Theming an existing library, especially one designed for it, can get a company 80% of the way to a distinct visual identity, fast, at a fraction of the engineering cost of custom-building everything.
- The real cost of building from scratch isn't the initial build, it's the ongoing maintenance burden of owning every accessibility fix, every browser quirk, and every new component forever, a cost that doesn't show up until year two or three.
- The exception is when a specific component is core to the product experience and needs behavior no off-the-shelf library provides well, for example a highly custom data visualization interaction; that specific component may justify custom build even while the rest of the system rides on an existing base.
- A middle path many teams land on is building on headless primitives (see Q6) rather than a fully styled library, getting the accessibility and interaction correctness without inheriting an opinionated visual layer that then has to be fought against during theming.

Interview Answer:

"For a mid-size company doing this for the first time, I'd usually recommend starting on top of an existing library, or on headless primitives, rather than building everything from scratch. Something like Material UI has already solved accessibility and cross-browser behavior for dozens of common components, and theming it can get you most of the way to a distinct visual identity for a fraction of the cost of building and maintaining all of that yourselves indefinitely. I'd reserve fully custom builds for the small number of components that are genuinely core to the product's differentiation, where an off-the-shelf option doesn't fit the actual interaction needed. The real trap with building everything from scratch isn't the initial cost, it's that you're signing up to own every accessibility bug and browser quirk forever, and that cost doesn't show up until a year or two in."

Q20. [Senior] Your design system currently has no automated way to know if a breaking change to a shared component silently breaks a consuming app until that team files a bug report weeks after your release. Design a system to catch this before it ships.

Answer:

Relying on downstream bug reports means the feedback loop between a breaking change and discovering its impact is measured in weeks, by which point the change is already deployed and the fix is reactive. **Close the loop with cross-repository integration testing that runs consuming apps' actual test suites against the design system's proposed change before it's published, turning "we found out weeks later" into "CI told us before merge."**

Architecture

1. Maintain a registry of consuming apps, or at least their critical UI test suites, that the design system's CI can trigger against a candidate release, not just its own internal unit tests.
2. On a pull request that changes a shared component, run an integration step that builds the candidate version of the package and runs it against a curated set of consuming apps' test suites (unit and end-to-end), similar to how a monorepo's affected-tests detection works, even if the apps live in separate repositories.

3. Where full integration runs are too slow or expensive to run on every PR, use a canary channel: publish a prerelease version (an npm dist-tag like `next`) automatically on every merge to main, and have a small number of volunteer or high-signal consuming apps run their CI against `next` continuously, surfacing breakage within a day instead of weeks.
4. Pair this with the visual regression testing from Q11, since some breakage is visual rather than functional, a layout shift that doesn't throw an error but looks broken, which functional tests alone won't catch.
5. Track a lightweight compatibility matrix, showing which consuming apps have verified against which design system versions, so a release manager can see at a glance whether it's safe to promote a version from `next` to `latest`.

Why this beats "wait for the bug report"

- Weeks-later bug reports mean the fix competes with the app team's actual roadmap for priority, whereas catching it in CI before publish means it's fixed once, by the people with full context, before it ever reaches a consumer.

Interview Answer:
"Right now the feedback loop is measured in weeks, because we only find out a component change broke something when a consuming team happens to notice and files a bug. I'd want to shorten that by running actual consumer test suites against candidate design system builds before publishing, either as a direct integration step in CI for critical apps, or more practically, by publishing a next prerelease tag on every merge and having a set of high-signal consuming apps run their own CI continuously against it. I'd pair that with visual regression testing, since some breakage is a layout shift that doesn't throw any error but still looks wrong. And I'd keep a simple compatibility matrix of which apps have verified against which version, so promoting a release from next to the stable tag is a decision backed by real evidence, not just the absence of complaints yet."

If you remember nothing else from this chapter: *a design system's real customers are the teams consuming it silently through a lockfile, not the team maintaining it.*

Chapter 11: Micro-Frontends & Scaling Frontend Architecture

Micro-frontend questions separate people who have only read a blog post from people who have actually lived through the pain of shipping a shared component across six teams on six different release schedules. Use this chapter to practice going beyond the reflexive "best practice" answer and articulating the real organizational problem micro-frontends solve and the real cost they add, since recommending them without weighing that cost can steer an org into years of avoidable complexity.

War Story: *A micro-frontend split that gave each team independence also gave every page five separate copies of the same framework runtime, until someone finally measured the bundle and found more code loading the app than running it.*

Avoid: treating splitting into micro-frontends as free, with no mention of runtime duplication cost

Aim For: raising shared dependency duplication as a cost to actively manage, not an acceptable side effect

Q1. [Junior] In plain terms, what is a micro-frontend architecture, and what problem is it actually trying to solve?

Answer:

A micro-frontend architecture splits a single web application into smaller, independently built and independently deployed pieces, each owned by a different team, that get composed together into one product the end user experiences as a single site. **It exists to solve an organizational problem, not a technical one: it lets multiple teams ship features to production on their own schedules without waiting on a shared release train or stepping on each other's code in one giant repository.**

The Core Idea

- Think of a large retail site where one team owns the product catalog, another owns checkout, and a third owns account settings. In a monolith, all three teams commit to the same codebase and typically ship on the same deploy cadence.
- In a micro-frontend setup, each team builds and deploys their piece independently, and the pieces are stitched together at runtime or build time into what looks like one seamless site to the shopper.

Why It Matters

- It is often described as "the frontend equivalent of microservices" because the motivation is identical: as an engineering org grows past a handful of teams, a single shared codebase and release process becomes the bottleneck, not the technology.
- At a junior level, you should at least connect the pattern to team scaling and independent deployment, not just describe it as "splitting up the code."

Interview Answer:

"A micro-frontend architecture breaks a large web app into smaller pieces that different teams build and deploy on their own, and then those pieces get combined into one product for the user. It's really the frontend version of microservices. The problem it solves isn't a technical limitation, it's an organizational

one: once you have several teams working in one big frontend codebase, they start blocking each other on releases and stepping on each other's code, so splitting ownership lets each team move at their own pace."

Q2. [Junior] Say a company has a search team, a checkout team, and a recommendations team all contributing to the same product page. What's the argument for letting each of them deploy independently instead of merging into one shared release?

Answer:

The argument is deployment independence: each team can ship a bug fix or a new feature the moment it's ready, without waiting for the other two teams to finish their work and without a shared code freeze blocking everyone. **The core benefit is that a team's release velocity is no longer capped by the slowest or most cautious team sharing their codebase.**

Concrete Scenario

- If the checkout team needs to ship a payment provider fix at 2pm on a Tuesday, they should not have to wait for the recommendations team to finish testing an unrelated carousel change first.
- With micro-frontends, checkout, search, and recommendations each live in separate repositories and pipelines, so that Tuesday fix goes out in isolation.

The Catch a Junior Often Misses

- Independence at deploy time does not mean independence at runtime. All three pieces still render on the same page, share the same browser tab, and need to agree on things like page layout and design system versions.
- A junior answer that stops at "teams can deploy separately" is fine as a starting point, but a strong answer notes that this independence has to be managed carefully or the page becomes visually and behaviorally inconsistent.

Interview Answer:
"The main win is that each team's release schedule stops being tied to everyone else's. If checkout needs a hotfix, they can ship it without waiting on search or recommendations to finish their own work. That's a big deal once you have several teams touching the same page, because in a shared codebase everyone is stuck waiting on the same CI pipeline and the same code freeze windows. The trade-off is that even though they deploy separately, they still render together on one page, so you need some coordination to keep things consistent."

Q3. [Junior] What is the iframe approach to building a micro-frontend, and why is it usually the first thing teams try?

Answer:

An iframe (a browser element that embeds another full HTML document inside the current page, like a window within a window) lets a host page load a completely separate app, built and deployed by another team, without either app knowing anything about the other's code, build tooling, or framework. **It's the first thing teams try because it requires almost zero coordination: any app that already runs standalone in a browser can be dropped into an iframe with no code changes.**

Why Teams Reach for It

- If the account settings page is a legacy Angular app and the new dashboard is React, an iframe lets you show both on one URL without rewriting either one or worrying about their build tools conflicting.
- It gives complete style and script isolation for free: CSS from the host page cannot leak into the iframe's content and vice versa.

Interview Answer:
"An iframe just embeds another team's fully separate web page inside your own page, the same way you'd embed a YouTube video. Teams reach for it first because it needs almost no integration work: whatever app the other team already has running standalone, you can drop straight into an iframe without touching either codebase. It's the fastest way to get two apps on one page, which is exactly why it's tempting, but that isolation comes at a real cost once you need the two apps to actually talk to each other."

Q4. [Junior] What breaks down once you try to build a real product experience out of iframes instead of just embedding one isolated widget?

Answer:

Iframes give strong isolation, but that same isolation becomes the problem: the pieces cannot easily share state, cannot share the browser's URL and back button behavior, and cannot inherit the host page's fonts, colors, or spacing without extra work. **The strict isolation that makes iframes easy to adopt is the exact same property that makes them a poor fit for a cohesive, deeply integrated product experience.**

Concrete Drawbacks

- **Styling isolation cost**: the design system's fonts, spacing, and colors defined on the host page do not automatically apply inside the iframe, so the embedded app can visually look like a different product unless someone manually keeps both style sheets in sync.
- **Hard cross-app communication**: passing data between the host page and the iframe requires the `postMessage` API, a manual, string-based messaging channel, instead of just calling a function or sharing a variable, which is slower to build and easy to get wrong.
- **No shared routing or history**: the iframe has its own internal URL and back-button behavior, separate from the parent page's, so a user clicking "back" in the browser often does not do what they expect inside an iframe-based flow.

Interview Answer:
"The isolation that makes iframes so easy to bolt on is the same thing that hurts you later. Styling doesn't flow across the boundary, so keeping two apps looking like one consistent product means manually syncing CSS on both sides. Communicating between the host page and the iframe means using postMessage, which is a clunky, string-based channel compared to just calling a function directly. And routing is probably the worst part: the iframe has its own internal navigation and history, so the browser's back button often doesn't behave the way users expect. That's why most teams outgrow iframes pretty fast once the product needs deeper integration."

Q5. [Junior] What are the real trade-offs between a single monolithic frontend app and splitting into micro-frontends? Walk me through both sides.

Answer:

A monolith keeps one codebase, one build, and one deploy pipeline, which makes consistency easy but forces every team's changes through the same release process. **Micro-frontends trade that shared**

simplicity for team autonomy: you gain independent deploys and ownership boundaries, but you pay for it with duplicated dependencies, a real risk of inconsistent UX across the pieces, and genuine runtime integration complexity that a monolith never has to deal with.

In Favor of the Monolith

- One version of React, one design system implementation, one bundler configuration. There's no risk of two teams shipping two different versions of the same library that both end up in the user's browser.
- Refactoring a shared component across the whole app is a single pull request, not a multi-team coordination effort.

In Favor of Micro-Frontends

- Independent deploys mean a 50-person org isn't bottlenecked on one release train, and teams can pick their own upgrade cadence for frameworks and tooling.
- Ownership boundaries are enforced by the architecture itself, not just by convention, which matters once an org is too large for everyone to informally agree on norms.

The Cost Side, Concretely

- Duplicated dependencies: if the checkout micro-frontend and the search micro-frontend each ship their own copy of a date-formatting library, the user's browser downloads that library twice.
- Runtime integration complexity: getting two independently built apps to share a logged-in user's session, or to navigate smoothly between each other, requires deliberate engineering that a monolith gets for free.

Interview Answer:

"A monolith buys you consistency for free: one dependency tree, one build, one place to fix a shared bug. Micro-frontends trade that away for team autonomy: teams can deploy independently and pick their own pace, but you now have to actively manage things a monolith never worried about, like two teams each shipping their own copy of the same library, or the UI looking slightly different between two team's sections because they didn't update the design system at the same time. There's also real runtime complexity in getting independently built pieces to share session state or navigate between each other cleanly. Neither option is free, it's a trade of shared simplicity for independent velocity."

Q6. [Junior] What role does a design system play once you have several teams, whether they're in a monolith or split into micro-frontends, all shipping UI?

Answer:

A design system (a shared library of reusable UI components, design tokens like colors and spacing, and usage guidelines) is what keeps a product feeling like one coherent thing even when dozens of engineers across different teams are building different parts of it. **Without a shared design system, independent teams will each reinvent buttons, modals, and form inputs slightly differently, and the product will visibly fragment even if the underlying code works fine.**

What It Actually Provides

- A single source of truth for how a button, input, or modal looks and behaves, so a checkout team and a search team don't each build their own slightly-different version of the same component.
- Accessibility and interaction behavior built in once, correctly, instead of every team re-solving the same problem, like focus trapping in a modal, with varying quality.

Interview Answer:

"A design system is the shared library of components and design rules, buttons, inputs, spacing, colors, that every team pulls from instead of building their own. It's what keeps the product visually and behaviorally consistent once you have more than one or two teams shipping UI. Without it, you end up with five different button styles across the product because five teams each built their own version, and users notice that inconsistency even if they can't articulate why the product feels sloppy."

Q7. [Mid] Walk me through how Webpack 5's Module Federation actually lets one team's app load another team's app at runtime. What's happening under the hood?

Answer:

Module Federation lets a webpack build expose specific modules from one application, called a remote, that a separate application, called the host or shell, can import and load over the network at runtime, as if they were local modules, without either app being compiled together at build time. **The key shift is that the integration point moves from build time, where you'd install a package, to runtime, where the host fetches the remote's JavaScript bundle live from a URL and mounts it into the page.**

How It Works Mechanically

1. The remote app (say, a "checkout" micro-frontend) declares an entry file, typically `remoteEntry.js`, that exposes specific components or modules, like a `CheckoutWidget` component, under a public name.
2. The host app's webpack config declares that remote as a dependency by URL, not by npm package name, and can then `import` from it as though it were a normal module.
3. At runtime, the host's browser fetches `remoteEntry.js` from the checkout team's deployed URL, and webpack's federation runtime wires up the exposed module into the host's running app.
4. Shared libraries, like React itself, can be marked as singletons so both the host and the remote use the same instance loaded once, instead of each shipping their own copy.

```
// host app's webpack.config.js
new ModuleFederationPlugin({
  name: "shell",
  remotes: {
    checkout: "checkout@https://checkout.example.com/remoteEntry.js",
  },
  shared: { react: { singleton: true }, "react-dom": { singleton: true } },
});
```

Why Teams Reach for It

- The checkout team can redeploy their remote independently, and the next time a user loads the shell app, it fetches the updated `remoteEntry.js` automatically, with no rebuild or redeploy needed on the shell's side.
- Compare this to publishing an npm package: with Module Federation there's no version bump, no reinstall, no host redeploy required for the remote's changes to reach production.

Interview Answer:

"Module Federation lets an app expose parts of itself, like a component, through a file called remoteEntry.js, and lets a different app import that component live over the network instead of installing it as a package. So the host app's webpack config points at a URL for the remote, and at runtime the browser actually fetches that remote's JavaScript and wires it into the running page. You can also mark shared libraries like React as singletons so both apps use one copy instead of shipping two. The big advantage over an npm package is

that the remote team can redeploy independently and the host picks up the change automatically on the next page load, no rebuild needed on the host's side."

Q8. [Mid] For a given micro-frontend, would you integrate it at build time, as an installed npm package, or at runtime, as a loaded remote bundle? Compare the two.

Answer:

Build-time integration means the host app installs the other team's UI as an npm package and compiles it into its own bundle, so it behaves like any other dependency. **Runtime integration, like Module Federation, means the pieces stay in separate deployed bundles and get combined live in the browser, which is slower to wire up but is what actually delivers independent deploys.**

Build-Time (npm Package)

- The consuming app gets full type checking, a normal `npm install` upgrade flow, and no risk of runtime version mismatches, since everything is compiled together into one bundle.
- The cost: publishing a new version of the package doesn't reach production until every consuming team bumps their dependency and redeploys, which reintroduces the exact coordination problem micro-frontends were meant to avoid.

Runtime (Loaded Remote Bundle)

- The producing team can push a change and have it live everywhere that loads their remote on the next request, no downstream redeploys required.
- The cost: you're now dealing with live version compatibility in the browser, harder debugging since the pulled-in code wasn't part of your build, and a new class of runtime failure if the remote is down or has shipped a breaking change.

Best Choice

Runtime integration (Module Federation or similar) for the actual page-level micro-frontends that need independent deploy cadence, like the checkout widget or the account dashboard.

Why?

The entire point of adopting micro-frontends is independent deploys. If every UI change still requires the consuming team to bump an npm version and redeploy, you've recreated the monolith's release coupling with extra infrastructure on top. Build-time npm packages are still the right call for genuinely shared, slow-changing pieces, like the design system's component library itself, where stability and type safety matter more than day-to-day independent shipping.

Interview Answer:

"Build-time integration means you npm install the other team's code and it gets compiled into your bundle, which gives you type safety and a normal upgrade flow, but a change doesn't reach production until every consumer bumps the version and redeploys. Runtime integration, like Module Federation, keeps the bundles separate and stitches them together live in the browser, so the producing team can ship a change and it's live everywhere immediately, at the cost of harder debugging and new failure modes if the remote is unavailable. For the actual independently-owned page sections, I'd use runtime integration, because independent deploys are the whole reason you're doing this. But for something like the shared design system library itself, I'd still ship that as a versioned npm package, since you want stability and compile-time safety there, not every team pulling a live, possibly-changing bundle."

Q9. [Mid] Several independently deployed micro-frontends all need to use the same design system component library. How do you version and distribute it so that one team upgrading doesn't force every other team to upgrade in lockstep?

Answer:

You publish the design system as its own versioned package with a clear semver (semantic versioning: major.minor.patch, where major means breaking changes) contract, and you let each consuming team choose when to bump their dependency, while the design system team commits to supporting at least the current and previous major version for some deprecation window. **The key discipline is treating the design system as a product with its own release contract, so consumers upgrade on their own schedule instead of being forced to move the moment the design system team ships something new.**

Mechanics That Make This Work

- Semantic versioning lets consumers safely accept patch and minor updates automatically (bug fixes and new non-breaking additions) while treating major version bumps as a deliberate, planned upgrade.
- A changelog and migration guide for every major version, so a team upgrading from v3 to v4 has a concrete list of what changed and how to adapt, rather than discovering breakage in production.
- Running two major versions in parallel for a deprecation window, say six months, so the checkout team on v3 isn't broken while the search team has already moved to v4.

Where This Gets Harder in a Micro-Frontend World

- If two micro-frontends on the same page load two different major versions of the design system, you can end up with two versions of the same CSS or JS shipped to the browser, which is exactly the "duplicated dependencies" cost that comes with this architecture. Marking the design system as a shared singleton in Module Federation reduces this, but only if consumers agree on compatible version ranges.

Interview Answer:
"I'd publish the design system as its own semver-versioned package and let each team upgrade on their own timeline, rather than forcing everyone to move the moment we ship something new. Patch and minor bumps can usually be accepted automatically since they shouldn't break anything, but a major version bump comes with a migration guide and we'd support the previous major for a defined deprecation window, something like six months, so no team gets stranded. The tricky part in a micro-frontend setup is that if two teams on the same page are running two different major versions, you can end up shipping duplicate copies of the same library to the browser, so we'd also try to keep version drift small by communicating upcoming majors well ahead of time."

Q10. [Mid] Two independently deployed micro-frontends on the same page need to talk to each other, say, a cart icon in the header needs to update when the product page adds an item. How would you wire that up without recreating a tightly coupled monolith?

Answer:

You give them a narrow, well-defined communication surface instead of letting them import each other's internals: browser custom events, a small shared event bus, or state passed through URL and query parameters are the three common patterns. **The goal is to keep the contract between micro-frontends as**

thin and explicit as a public API, so either side can change its internal implementation freely as long as it keeps honoring that contract.

Custom Events

- One micro-frontend dispatches a `CustomEvent` on the shared `window` or `document`, and any other micro-frontend that cares can listen for it, without either side needing a direct reference to the other's code.

```
// product page micro-frontend, after adding to cart
window.dispatchEvent(new CustomEvent("cart:item-added", { detail: { sku: "SKU123", qty: 1
} }));

// header micro-frontend, listening
window.addEventListener("cart:item-added", (e) => updateCartBadge(e.detail));
```

A Shared Minimal Event Bus

- For a larger number of micro-frontends, a tiny shared pub/sub module (a simple "subscribe to a topic, publish to a topic" utility) loaded by every piece gives the same decoupling as custom events but with a cleaner API and easier testing than raw DOM events.

URL and Query Parameters

- For state that should survive a page reload or be shareable, like a selected filter or an open modal, encoding it in the URL lets independent micro-frontends read shared state without any direct messaging channel at all, just by reading `window.location`.

Why Not Just Share a Global Store Directly

- Wiring every micro-frontend directly into one shared Redux store, or having them call each other's internal functions, recreates the tight coupling of a monolith: now nobody can change their internal implementation without checking every other team's code first, which defeats the purpose of splitting them up in the first place.

Interview Answer:

"I'd keep the communication surface small and explicit rather than letting the pieces reach into each other's internals. Custom events dispatched on the window are the simplest option, the product page fires a 'cart:item-added' event with the SKU in the detail, and the header listens for it and updates the badge, with neither side needing a reference to the other's code. For more micro-frontends talking to each other regularly, a tiny shared event bus gives the same decoupling with a cleaner API. And for state that should survive a reload, like a selected filter, I'd just put it in the URL. What I'd avoid is wiring everyone into one shared global store or letting them call each other's internal functions directly, because that recreates the tight coupling you were trying to get away from."

Q11. [Mid] For an org running many independently deployed micro-frontends, would you keep them in one monorepo with a tool like Nx or Turborepo, or split them into separate polyrepos, one repo per team? Compare the two.

Answer:

A monorepo (one repository holding multiple projects) with a build tool like Nx or Turborepo gives you shared tooling, coordinated dependency versions, and the ability to run cross-project checks in one place, while a polyrepo (each team owns a fully separate repository) gives teams maximum autonomy over their own tooling and release process at the cost of harder cross-cutting changes. **The trade-off is coordination versus autonomy: a monorepo makes shared upgrades and cross-project visibility easy but couples teams'**

tooling choices together, while a polyrepo isolates teams completely but makes anything spanning multiple micro-frontends, like a design system upgrade, much harder to land atomically.

Monorepo (Nx / Turborepo)

- A single pull request can update the design system and every consuming micro-frontend in the same commit, with the build tool's dependency graph only rebuilding and retesting the projects actually affected by the change.
- Shared linting rules, TypeScript configs, and CI pipelines are defined once, so new micro-frontends don't each reinvent their own tooling setup.

Polyrepo

- Each team fully owns their release cadence, their own CI pipeline, and even their own framework choice, with zero risk of one team's build tooling change breaking another team's repo.
- The cost shows up the moment something needs to change across teams: a design system major version bump means opening and coordinating a separate pull request in every single repo, and there's no single dependency graph telling you what's actually affected.

Best Choice

A monorepo with Nx or Turborepo for most organizations running micro-frontends, unless a specific team genuinely needs a completely different toolchain or release process from the rest.

Why?

Micro-frontends already add real integration complexity between teams. A monorepo's incremental build caching and shared dependency graph claw back a lot of that complexity by making cross-cutting changes, like a shared component library bump, a single coordinated change instead of a fan-out across a dozen separate repositories. The polyrepo's autonomy is real, but at the scale most orgs are actually facing, six teams and fifty engineers, the coordination pain of a polyrepo tends to outweigh the tooling independence it buys.

Interview Answer:

"I'd lean toward a monorepo using something like Nx or Turborepo for most orgs doing micro-frontends. You still get independent deploys, since each project can have its own pipeline, but a shared design system change or a dependency bump can land as one pull request that the build tool's dependency graph knows exactly how to test and rebuild, instead of fanning out into a dozen separate PRs across separate repos. Polyrepos give teams more autonomy over their own tooling, and there are cases where a team genuinely needs a different stack, but for most orgs the coordination pain you save with a monorepo outweighs the tooling independence a polyrepo buys you."

Q12. [Mid] You wire up Module Federation between three micro-frontends, but don't mark React as a shared singleton. What actually goes wrong, and how would you catch it?

Answer:

Without marking a library like React as a shared singleton, each remote ships its own copy of React inside its own bundle, so the browser can end up loading two or three separate instances of React on one page. **That duplication isn't just wasted bytes, it actively breaks things like React context and hooks that assume there is exactly one React instance managing the component tree.**

What Breaks Concretely

- A context provider set up by the host app's React instance won't be seen by a component rendered from a remote running a different React instance, so `useContext` silently returns the default value instead of the value you expected.
- Bundle size grows noticeably. React plus React DOM is a meaningful chunk of JavaScript, and shipping it two or three times to every user is a direct, measurable performance regression.

How You'd Catch It

- Check the network tab for duplicate React bundles being downloaded, or add `shared: { react: { singleton: true, requiredVersion: false } }` to the Module Federation config and let webpack warn on version conflicts at build time.
- A code review checklist item for any new remote or host config change: confirm shared dependencies are actually configured as singletons, not just assumed to be.

Interview Answer:

"If React isn't marked as a shared singleton, each remote bundles its own copy, so you can end up with two or three separate React instances running on the same page. That's not just wasted download size, it actually breaks things, because a context provider from one React instance won't be visible to a component running under a different instance, so useContext quietly returns the wrong value instead of throwing an obvious error. I'd catch this either by checking the network tab for duplicate React downloads, or by making sure the Module Federation config explicitly marks React and React DOM as singleton shared dependencies, and treating that as a required item in review for any new remote."

Q13. [Senior] Your team owns a shared `Modal` component in the design system, used by all six product teams. You need to change its API in a breaking way. Walk me through how you'd roll that out without breaking every team's app at once.

Answer:

You never ship the breaking change as a silent replacement; you ship the new behavior alongside the old one, communicate a deprecation timeline, and let each team migrate on their own schedule before you remove the old API. **The core technique is running both the old and new APIs side by side for a defined window, so no team is forced to drop everything and migrate the moment you publish, while still giving you a hard, communicated end date for the old behavior.**

A Concrete Rollout Sequence

1. Ship a new major version of the design system with the new `Modal` API added, while keeping the old prop shape working, using a runtime deprecation warning (a `console.warn` when the old prop is used) rather than removing it outright.
2. Publish a migration guide with a code diff example, and if the change is mechanical enough, provide a codemod (an automated script that rewrites call sites) so teams can run one command instead of hand-editing every usage.
3. Announce a concrete removal date for the old API, something like eight weeks out, and track which teams have migrated versus which haven't, following up directly with teams that are lagging close to the deadline.
4. Only remove the old prop shape in a subsequent major version release, after the deprecation window has passed and usage has actually dropped to zero.

```
// Old and new both work during the deprecation window
type ModalProps =
  | { open: boolean; onClose: () => void } // new API
  | { isVisible: boolean; onDismiss: () => void }; // deprecated, warns at runtime
```

Why Not Just Ship It and Tell Teams to Fix Their Usage

- With six teams on independent deploy schedules, a hard breaking change with no transition period means whichever team hasn't redeployed yet breaks in production the next time they touch anything nearby, and you've turned a design system update into a fire drill across the org.

Interview Answer:

"I wouldn't just swap the API and tell teams to fix it. I'd ship the new Modal API alongside the old one in the same major version, with the old prop shape still working but throwing a console warning so teams know it's deprecated. Then I'd publish a migration guide, and if the change is mechanical enough, write a codemod so teams can run one script instead of hand-editing every call site. I'd announce a concrete removal date, something like eight weeks out, track which teams have actually migrated, and only pull the old API in a later major version once usage has dropped to zero. The whole point is that with six teams on independent release schedules, a hard breaking change with no transition window turns a routine design system update into a production incident for whoever hasn't redeployed yet."

Q14. [Senior] Beyond deployment, what's the hardest real runtime integration problem you've seen in a micro-frontend setup, and how do you actually solve it?

Answer:

Shared session and authentication state is usually the sharpest edge: several independently built apps on one page all need to know the user is logged in, know who they are, and agree on when that session expires, without each reimplementing auth from scratch or drifting out of sync with each other. **The reliable solution is to centralize authentication in the shell (host) application and pass identity down to each micro-frontend as one clearly-owned piece of shared state, rather than letting every micro-frontend independently manage its own copy of the session.**

Why This Is Genuinely Hard

- If the checkout micro-frontend and the account micro-frontend each independently check token expiry and each independently redirect to login, you can get race conditions where one redirects the user mid-flow while the other still thinks they're logged in.
- Token refresh logic duplicated across micro-frontends means it's easy for one team's copy to have a bug that silently logs users out early, or fails to refresh, while another team's copy works fine, producing inconsistent, hard-to-reproduce bug reports.

A Concrete Solution Pattern

- The shell app owns the single source of truth for the auth token and session state, handles refresh and expiry centrally, and exposes it to every mounted micro-frontend through a shared, well-typed interface, whether that's a shared singleton module through Module Federation or a small runtime context passed down at mount time.
- Each micro-frontend treats that identity as read-only input, the same way any app treats a prop, rather than independently fetching or refreshing its own session.

Interview Answer:

"The hardest one I've run into is shared session and auth state across independently built pieces. If every micro-frontend independently checks token expiry and independently redirects to login, you get race conditions, one part of the page thinks the user's logged out and redirects while another part still thinks they're fine, and token refresh bugs become inconsistent and hard to reproduce because each team has their own copy of that logic. The fix that actually works is centralizing auth in the shell app, the one hosting

all the micro-frontends, so there's a single source of truth for the session, and every micro-frontend just receives that identity as read-only input rather than managing its own copy of login state."

Q15. [Senior] How do you actually debug a production issue when the bug only shows up in the interaction between two independently deployed micro-frontends, and neither team's own error monitoring caught it?

Answer:

You need observability that spans the composed page, not just each micro-frontend's own isolated logs, because the bug lives in the seam between two systems that neither team can see end to end on their own. **The practical fix is to instrument the shell (host) application itself to capture cross-boundary events, like which micro-frontend versions were mounted, what custom events fired, and where an error boundary caught a failure, so there's one place to reconstruct what actually happened on that page load.**

Why Neither Team's Own Monitoring Catches It

- The checkout team's error tracking only sees errors thrown inside checkout's own bundle, and the header team's only sees errors inside theirs. If the failure is, say, the header dispatching a malformed custom event that checkout mishandles, neither team's dashboard shows a clean, attributable error.
- Independent deploys make it worse: the bug might only appear when checkout v14 is live alongside header v9, a specific version combination that isn't obviously visible from either team's own release notes.

What Actually Helps

- Wrap each mounted micro-frontend in its own error boundary at the shell level, and log which micro-frontend and which version threw, so a crash is attributable even if the root cause is really a contract mismatch with another piece.
- Log a version manifest on every page load, which remote versions were fetched and mounted, so when a bug report comes in you can correlate it against "this only happens with header v9 and checkout v14 together."
- Centralize logging for the shared communication layer itself, custom events or the event bus, so you can see the actual payload that one micro-frontend sent and the other received, instead of guessing.

Interview Answer:
"The core problem is that each team's own monitoring only sees inside their own bundle, so a bug that lives in the interaction between two micro-frontends, like a malformed custom event, doesn't show up cleanly in either team's dashboard. What's worked for me is instrumenting the shell app itself: wrap each mounted micro-frontend in its own error boundary and log which one failed and at what version, and log a version manifest on every page load so you can correlate a bug report against a specific combination of versions that were live together. I'd also centralize logging around the shared communication layer, the custom events or event bus, so when something breaks at the seam, you can actually see the payload that was sent versus what was received instead of guessing at both teams independently."

Q16. [Senior] Your organization has grown to 50 frontend engineers across 6 teams, all shipping into one product, and someone on leadership is asking whether you should adopt micro-frontends. How do you actually make that call?

Answer:

You don't start from the architecture, you start from where the org is actually feeling pain: is it release coordination and deploy bottlenecks, or is it something else, like an unclear ownership model or a slow CI pipeline, that micro-frontends wouldn't actually fix. **Fifty engineers across six teams is genuinely in the range where micro-frontends can be justified, but the deciding factor should be a specific, named bottleneck, like teams routinely blocked on each other's release windows, not the team size number by itself.**

Questions I'd Ask First

- Are teams actually blocked on each other today? If six teams commit to one repo and one release train and are shipping fine with a good CI setup and clear code ownership, the pain that micro-frontends solve may not exist yet.
- Is the real problem something a monorepo with good tooling, like Nx or Turborepo, could already fix, since a well-organized monolith with clear module boundaries and fast, incremental CI checks can support surprisingly large teams without a runtime architecture change.
- Do teams need genuinely different release cadences, for example, one team shipping daily and another needing a slower, more regulated release process for compliance reasons? That's a much stronger signal for micro-frontends than team size alone.

What I'd Actually Recommend

- If the concrete complaint is "we're blocked waiting on other teams' release windows" or "our CI takes 40 minutes and keeps getting slower as more teams commit," I'd first try tightening the monolith: better code ownership boundaries, an incremental build/test tool, and CI caching, since that's a much smaller investment than standing up runtime integration infrastructure.
- If, after that, teams still fundamentally need independent deploy pipelines, most commonly because one team's release process has to move at a different speed or through a different approval process than another's, then a micro-frontend architecture, likely starting with one boundary, like carving out the checkout flow as a Module Federation remote, is the right next step, not a full rewrite of all six teams at once.

Interview Answer:

"I wouldn't jump straight to micro-frontends just because the org hit fifty engineers. I'd first ask where the actual pain is: are teams genuinely blocked on each other's release windows, or is the real problem something a well-tuned monorepo with a tool like Nx could already fix, better ownership boundaries and faster incremental CI? Fifty engineers across six teams is in the range where micro-frontends can be justified, but the deciding signal should be a specific bottleneck, like one team needing a completely different release cadence than another, not the headcount by itself. If it turns out to be a real deploy-coupling problem, I'd start small, carve out one boundary, like the checkout flow, as a Module Federation remote, prove it out, and expand from there, rather than re-architecting all six teams into micro-frontends on day one."

Q17. [Senior] Walk me through how you'd migrate an existing large monolithic frontend to micro-frontends incrementally, without a multi-month rewrite that freezes feature work.

Answer:

You migrate one vertical slice at a time, starting with the piece that has the clearest ownership boundary and the least entanglement with the rest of the app, and you keep the monolith fully functional and shipping features the entire time. **The core principle is the strangler pattern: carve out one boundary at a time**

behind the existing shell, prove the integration works end to end, and only then move to the next slice, instead of attempting a big-bang rewrite.

A Realistic Migration Sequence

1. Pick the first candidate slice based on ownership clarity and low coupling, for example, an account settings page that one team owns outright and that doesn't share much state with the rest of the app, rather than starting with checkout, which usually touches everything.
2. Stand up the shell (host) application that will eventually host all micro-frontends, and have it initially render the monolith itself as its first "remote," so the shell exists and works before any real extraction happens.
3. Extract the chosen slice into its own deployable app, wire it into the shell via Module Federation or the chosen integration approach, and route real production traffic to it behind a feature flag so it can be rolled back instantly if something breaks.
4. Once that slice is stable in production, establish the shared contracts it needed, communication events, design system version, auth handoff, as the template for the next slice, and repeat, migrating one boundary at a time while the rest of the app keeps shipping normally.

Why Not a Full Rewrite

- A multi-month freeze on feature work to do a full rewrite is a massive opportunity cost, and it also means the first time you discover a real integration problem, like the auth handoff not working cleanly, you discover it after months of investment instead of after the first small slice.

Interview Answer:

"I'd treat it like a strangler migration: carve out one slice at a time behind the existing app, rather than freezing feature work for a big rewrite. I'd pick the first slice based on clear ownership and low coupling to the rest of the app, something like account settings, not checkout, which usually touches everything. I'd stand up the shell app first, initially just hosting the existing monolith as its one remote, then extract that first slice into its own deployable app, wire it in behind a feature flag so it's instantly reversible, and get it stable in production. Once that's proven, the contracts we figured out, communication, auth handoff, design system versioning, become the template for the next slice, and we keep going one boundary at a time while the rest of the app keeps shipping features the whole time."

Q18. [Senior] Two of your six teams are complaining that a shared micro-frontend they don't own broke their part of the page during an unrelated deploy. As the architect responsible for the platform, what do you actually change?

Answer:

This is usually a contract and ownership problem more than a pure technical one: the breaking team didn't realize their change had downstream consequences because there was no enforced contract making that dependency visible. **The fix is to make cross-micro-frontend contracts explicit and testable, shared TypeScript types for any events or props crossing the boundary, and integration tests that run in CI before a deploy, rather than relying on teams to informally remember what other teams depend on.**

What Actually Needs to Change

- Define and version the contract for anything crossing a micro-frontend boundary, custom event payloads, shared component props, URL parameter formats, as shared, versioned TypeScript types that both sides import, so a breaking change to the shape is a type error at build time, not a runtime surprise.

- Add integration or contract tests that run against a deployed staging version of the dependent micro-frontends before a production deploy goes out, catching the break in CI instead of in front of real users.
- Set up a lightweight registry or dashboard of "who depends on what," so before a team changes a shared event's payload shape, they can see which other micro-frontends are actually listening for it.

Interview Answer:

"I'd treat this as a missing contract problem, not just a bug. The team that broke things almost certainly didn't know anyone downstream depended on that shape, because nothing made the dependency visible or enforced. So I'd push for shared, versioned TypeScript types for anything crossing a micro-frontend boundary, custom event payloads, prop shapes, so a breaking change becomes a compile-time type error instead of a runtime surprise in production. I'd also add contract or integration tests that run against a staging version of the dependent micro-frontends as part of CI, so this kind of break gets caught before a deploy goes out, not after. And longer term, a simple dependency registry, so teams can see who's actually listening to a shared event before they change its shape, would have caught this specific case early."

If you remember nothing else from this chapter: *micro-frontend independence is bought with duplicated runtime weight, and the bill comes due on every single page load.*

Chapter 12: Accessibility Architecture

Most engineers can recite what alt text or an ARIA (Accessible Rich Internet Applications, a spec for exposing custom widget behavior to screen readers) role is, but accessibility failures in production almost never come from not knowing the definition. They come from nobody deciding whose job it was to enforce it, a component that technically has the right role but the wrong contract, or a shipped feature nobody tested with a keyboard. Use the questions in this chapter to practice thinking about accessibility as a system and organizational design problem, not a checklist of tags. A weak answer describes individual fixes; a strong one talks about where a decision gets made once so five hundred engineers don't have to make it correctly five hundred times.

War Story: *A checkout flow that passed every visual QA check turned out to be completely unusable with a screen reader, because focus order silently broke the moment a promotional banner was added above the form.*

Avoid: treating accessibility as a final QA checklist item rather than a property of how components are built

Aim For: considering keyboard and screen-reader behavior while designing a component, not after

Q1. [Junior] Should accessibility be baked into your shared design system's components, or should each feature team be responsible for making their own UI accessible? How would you decide?

Answer:

This is fundamentally a question about where you place a control point in the system, and the honest answer is that the design system should own as much of it as possible by default. **If every button, dropdown, and form field a team builds already has correct keyboard behavior and screen reader semantics baked in, teams get accessibility for free just by using the library, instead of having to re-learn and re-implement it feature by feature.**

Why centralizing is the leverage point

- A 40-person product org shipping features every sprint cannot rely on every engineer independently knowing how focus order, ARIA roles, and contrast requirements interact. Centralizing means you only need a handful of accessibility specialists to get it right once, in one component.
- When a bug is found (say, a checkbox component doesn't announce its checked state to VoiceOver), fixing it in the design system fixes it everywhere it's used, instantly, versus filing 30 separate tickets against 30 feature teams.
- It changes the default. A team building a settings page doesn't have to decide to be accessible; they just import `<Toggle>` and it already is.

Where per-team responsibility still applies

- The design system can't own everything: page-level structure (heading order, landmark regions), the meaning and clarity of your own copy, and any bespoke visualization (a custom chart, a canvas-based editor) still need the feature team to apply the underlying primitives correctly.

- The design system should ship guidance and lint rules for these cases, not just for its own components, so "not owned by the design system" doesn't become "not owned by anyone."

Interview Answer:
"I'd push as much of this into the design system as I can, because that's the highest-leverage place to fix it. If the design system's `<Modal>` or `<Combobox>` already traps focus correctly and exposes the right ARIA roles, every team that uses it gets that for free, and a fix in one place propagates everywhere instead of needing thirty separate tickets. What the design system can't own is page-level structure and custom one-off widgets, so I'd pair centralized components with linting and review for the stuff that's inherently feature-specific."

Q2. [Junior] You're designing a custom combobox (an autocomplete input with a dropdown list) for a shared component library. What API or contract does it need to expose so a consuming team can't accidentally build an inaccessible version of it?

Answer:

The risk with a combobox isn't that the component itself is inaccessible; it's that its API leaves so many "assembly" decisions to the consumer that they can wire it up wrong. **The component's props and internal behavior need to make the accessible path the only path, not an opt-in path a busy feature team can skip.**

Design the contract, not just the visuals

- The component should own its own ARIA wiring internally (`role="combobox"`, `aria-expanded`, `aria-activedescendant` pointing at the highlighted option) rather than asking the consumer to pass those attributes in. If the consumer has to remember to add `aria-expanded`, some team will forget.
- Keyboard behavior (arrow keys to move through options, Escape to close, Enter to select, Home/End to jump) should be built into the component, not left as a "you can add your own key handler" escape hatch.
- The only things the consumer should be able to control are the data (the list of options), the rendering of each option, and callbacks for selection, not the underlying semantics.

```
// The consumer only supplies data and render logic.
// Focus, keyboard nav, and ARIA wiring are NOT exposed as props to override.
<Combobox
  label="Assign owner"
  options={users}
  getOptionLabel={(u) => u.name}
  onSelect={(u) => setOwner(u)}
  renderOption={(u) => <UserRow user={u} />}
/>
```

Guard rails against misuse

- If the component supports a "headless" mode (logic without styling, common in design systems like Radix or Downshift), the accessible behaviors (focus management, ARIA attributes) should still ship in the logic layer, not be left to the styling layer the consumer writes.
- Ship the component with automated tests that assert on ARIA state transitions (does `aria-expanded` flip to `true` when the list opens), so a future refactor that breaks accessibility fails CI (continuous integration, the automated pipeline that runs on every code change) instead of shipping silently.

Interview Answer:

"The main risk with a combobox isn't the visual design, it's that if I expose too many raw props, some consuming team will wire up the ARIA attributes wrong or forget a keyboard handler. So I'd have the component own its own ARIA state and keyboard behavior internally, arrow keys, Escape, Enter, aria-activedescendant, all baked in, and only expose props for data, rendering, and selection callbacks. That way the only way to use the component is the accessible way, and I'd back it with tests that assert on the ARIA state transitions so a future change can't quietly break it."

Q3. [Junior] When would you build a fully custom interactive widget instead of using a native HTML element, even though the native element is less visually flexible? What's the actual cost/benefit here?

Answer:

Native elements come with a huge amount of accessibility behavior for free (keyboard handling, screen reader semantics, focus management, form integration), and reimplementing all of that in a custom widget is expensive and easy to get subtly wrong. **The real trade-off isn't "always use semantic HTML," it's whether the visual or interaction requirement is significant enough to justify rebuilding everything the browser already gives you for free.**

Best Choice

Default to the native element unless there's a specific, named requirement the native element genuinely cannot satisfy, and when you do go custom, budget real engineering time to rebuild the behaviors you just gave up.

Why?

- A native `<select>` gets keyboard support, screen reader announcements, mobile OS picker UI, and form submission behavior for free, across every browser and platform, without a single line of your code.
- A custom dropdown that visually needs multi-column layout, rich option previews (avatars, icons, secondary text), or a search filter inside the list is a real requirement a native `<select>` can't meet, that's a legitimate reason to go custom.
- The hidden cost of custom is not the initial build, it's that you now own keyboard navigation, focus trapping, ARIA roles, and screen reader testing forever, and every future contributor has to know not to regress it.
- A common bad pattern: a team builds a custom checkbox purely for a slightly different visual style, when CSS alone (styling the native `<input type="checkbox">` with `accent-color` or a styled label wrapping a visually-hidden native input) would have met the visual bar with zero accessibility cost.

Interview Answer:

"I default to the native element unless there's a concrete requirement it can't satisfy, because native controls give you keyboard support, screen reader semantics, and mobile behavior for free. Something like a rich autocomplete with avatars and grouped results is a legitimate reason to go custom, a checkbox that just needs a different visual style usually isn't, that's a CSS problem. When I do go custom, I treat it as taking on a real ongoing cost, because now my team owns keyboard handling, ARIA roles, and screen reader testing forever instead of getting it from the browser."

Q4. [Junior] Your team wants to add automated accessibility checks to CI using a tool in the axe-core family. What kinds of issues will that actually catch, what will it miss, and how would you decide what should fail the build versus just warn?

Answer:

Automated tools are excellent at catching structural and rule-based violations but they cannot judge whether an experience actually works for a real screen reader user. **Automated checks should be treated as a fast, cheap first filter that catches maybe a third of real accessibility issues, not a substitute for human testing.**

What axe-core style tools catch well

- Missing or empty `alt` attributes on images, missing form labels, insufficient color contrast ratios (the luminance difference between text and background used to judge readability), invalid or duplicate ARIA attributes, and missing document landmarks (like no `<main>` element).
- These are all things that can be verified by inspecting the DOM (Document Object Model, the browser's in-memory representation of the page) against a rule, with no understanding of intent required.

What they miss

- Whether the reading order actually makes sense when a screen reader moves through the page.
- Whether a custom widget's keyboard interaction matches what users expect (can you actually operate this component with only a keyboard, does Escape close it, does focus land somewhere sensible after an action).
- Whether alt text, when present, is actually meaningful (`alt="image1.png"` passes the automated check and is still useless).
- Anything involving cognitive load, motion sensitivity, or whether the overall flow makes sense.

Deciding what fails the build

- Fail the build on things that are unambiguous violations with no false-positive risk: missing labels, contrast failures below the legal/organizational bar, missing alt attributes.
- Warn (but don't block) on rules that require human judgment to confirm, like ambiguous heading structure, so the noise doesn't cause teams to stop trusting the pipeline.
- Pair the automated gate with a scheduled or milestone-based manual screen reader pass, since that's the only way to catch what the automated layer structurally cannot see.

Interview Answer:

"Axe-core style tools are great at catching things you can verify purely from the DOM, missing labels, bad contrast ratios, invalid ARIA, and I'd fail the build on those because they're unambiguous. What they can't catch is whether the experience actually works, whether a custom dropdown is keyboard operable, whether the reading order makes sense, whether alt text is actually meaningful rather than just present. So I treat automated checks as a fast first filter that catches maybe a third of real issues, and I still budget for manual and screen reader testing, especially before big feature launches, because that's the only way to catch the rest."

Q5. [Junior] In a single-page app, the router swaps the main content when the user navigates, but the browser doesn't do a full page reload. How do you architect

focus management for these route transitions so keyboard and screen reader users aren't left behind?

Answer:

On a traditional multi-page site, a full navigation resets focus to the top of the document and the browser announces the new page title automatically. A single-page app's router breaks both of those behaviors unless you rebuild them deliberately. **Every route change needs to explicitly move focus to a sensible anchor, usually the new page's heading, and announce the change, or keyboard and screen reader users will keep interacting with content that's no longer there.**

The core mechanism

1. On every successful route change, programmatically move focus to a stable, predictable target, typically the page's `<h1>` (made programmatically focusable with `tabindex="-1"` since headings aren't focusable by default).
2. Update the document `<title>` on every route change, since many screen readers announce title changes and some users rely on that as their primary "did the page change" signal.
3. For screen reader users specifically, pair this with an ARIA live region (a region marked so screen readers announce content changes without needing focus to move there) that announces something like "Navigated to Order History," since focus alone doesn't always get verbally announced consistently across screen readers.

```
function useRouteFocus(pageTitle: string) {
  const headingRef = useRef<HTMLHeadingElement>(null);

  useEffect(() => {
    document.title = pageTitle;
    headingRef.current?.focus();
  }, [pageTitle]);

  return headingRef; // consumer attaches this to <h1 ref={headingRef} tabIndex={-1}>
}
```

Common failure modes to avoid

- Focusing the very top of the `<body>`, which forces screen reader users to tab through the entire nav bar again on every navigation, a real and common complaint.
- Only handling this for "full page" route changes and forgetting nested/tab-style navigation inside a single route, which has the identical problem at a smaller scale.

Interview Answer:

"A single-page app's router breaks two things a real navigation normally gives you for free: focus resetting to the top and the page title being announced. So on every route change I move focus programmatically to the new page's heading, using tabindex negative one since headings aren't natively focusable, and I update the document title. For screen reader users I'll also pair that with a live region announcement like 'Navigated to Order History,' because focus movement alone isn't consistently announced across every screen reader. The failure mode I watch for is teams focusing the body tag, which forces users to tab through the whole nav bar again on every single navigation."

Q6. [Junior] You have dozens of teams independently choosing colors for buttons, badges, alerts, and charts, and contrast violations keep slipping into production.

How would you design the way your design system communicates and enforces color contrast rules across all of these teams?

Answer:

The failure here is almost never that engineers don't know contrast matters, it's that there's no system stopping a designer from picking an aesthetically nice color that fails the contrast ratio (the required luminance difference between text and its background, commonly a 4.5:1 minimum for normal text under WCAG, the Web Content Accessibility Guidelines) until it's already in production. **You fix this by removing the ability to pick an arbitrary color at all, restricting choice to a pre-vetted token palette, and backing that with an automated check as the final safety net.**

Restrict the palette, don't just document it

- Publish a fixed set of color tokens (named, reusable design values like `color-text-critical` rather than raw hex codes) where every token that's approved for text-on-background use has already been contrast-checked against its intended background by the design system team.
- Make the design tooling (Figma libraries, a code-level theme object) only expose these tokens, so a designer or engineer physically cannot select an arbitrary hex value inside the sanctioned workflow.

Back it with automated enforcement

- Add a lint rule or CI check that flags any raw hex color value introduced outside the token file, since that's usually the moment someone is about to bypass the system.
- Run contrast-checking automated tests (axe-core or similar) against rendered components in CI so a token combination that was fine on a white background but fails on a new dark surface color gets caught before merge.

Handle the "we want a brand color that fails" conversation

- When a marketing or brand color genuinely fails contrast for text, the answer is usually: allowed for large decorative elements or backgrounds, not allowed for text or interactive states, and the design system should encode that restriction directly in the token names (`color-brand-decorative` vs `color-brand-text`) so the constraint is visible at the point of use.

Interview Answer:
"The real fix isn't a style guide PDF telling people the contrast rule, it's removing the ability to pick an arbitrary color in the first place. I'd have the design system publish a fixed token palette where every text-safe token has already been contrast-checked against its intended background, and restrict the design and code tooling to only expose those tokens. Then I'd back that with a CI lint rule that flags raw hex values outside the token file, and automated contrast tests on rendered components, so a token that's fine on one surface but fails on a new one gets caught before it merges instead of after a designer notices in production."

Q7. [Mid] Design an accessible modal dialog component for a shared design system that dozens of teams will use for everything from confirmation prompts to multi-step forms. Walk me through the architecture.

Answer:

A modal is deceptively hard because it has to coordinate four things at once: what's visually on screen, what's reachable by keyboard, what a screen reader announces, and what happens to focus when it closes. **The component needs to own focus trapping, initial focus placement, and focus restoration internally so**

that no consuming team can render a modal that's visually correct but functionally a dead end for keyboard users.

Core architectural responsibilities

1. **Focus trap**: while open, Tab and Shift+Tab should cycle only through elements inside the modal, never escaping to the page behind it. This has to be implemented generically (walking focusable descendants), not hardcoded to a specific modal's contents, since every consumer's modal content is different.
2. **Initial focus**: on open, move focus to a sensible default, typically the first interactive element or a heading, and let the consumer override it via a prop for cases like a destructive-confirmation dialog where you want focus on "Cancel," not "Delete."
3. **Focus restoration**: on close, return focus to the element that triggered the modal (the button that opened it), not to the top of the page, otherwise a keyboard user loses their place entirely.
4. **Semantics**: `role="dialog"` (or `"alertdialog"` for a confirmation), `aria-modal="true"`, and `aria-labelledby` pointing at the modal's own heading, all wired internally, not left as optional props.
5. **Dismissal**: Escape key and a backdrop click both close it by default, with an option to disable backdrop dismissal for destructive flows where an accidental close could lose unsaved work.

```
<Modal
  isOpen={isOpen}
  onClose={handleClose}
  labelledBy="delete-confirm-heading"
  initialFocusRef={cancelButtonRef} // override default first-element focus
>
  <h2 id="delete-confirm-heading">Delete this project?</h2>
  <button ref={cancelButtonRef}>Cancel</button>
  <button>Delete</button>
</Modal>
```

Why this can't be left to consumers

- If focus trapping, restoration, and ARIA wiring are optional props instead of built-in behavior, some team under deadline pressure will ship the version without them, and the bug won't be visible to anyone testing with a mouse.

Interview Answer:

"I'd treat the modal as owning four things internally: trapping focus inside itself while open, placing initial focus somewhere sensible on open, restoring focus to the trigger element on close, and wiring up role dialog, aria-modal, and aria-labelledby itself rather than asking the consumer to pass those in. I'd still expose an override for initial focus, because a destructive confirmation dialog should default focus to Cancel, not Delete. The reason all of this needs to be built into the component rather than documented as 'please also do this' is that under deadline pressure, some team will ship the version without it, and it'll look completely fine to anyone testing with a mouse."

Q8. [Mid] Design a complex data table component, sortable columns, selectable rows, maybe 40+ columns, that will be used across an internal admin product. What does the accessibility architecture look like?

Answer:

A data table's biggest accessibility risk isn't the table markup itself, it's that as you add sorting, selection, and inline actions, you're layering interactive behavior on top of a structure that a screen reader is trying to read as a grid, and those two things can easily fight each other. **The architecture needs a single source of truth**

for which cell currently "has" keyboard focus, using a roving tabindex pattern, so that 40 columns of interactive controls don't turn into an unusable maze of hundreds of individually tabbable elements.

Structural foundation

- Use real `<table>`, `<thead>`, `<tbody>`, `<th>` semantics as the base, since that's what gives screen readers row/column context for free; only reach for `role="grid"` and cell-level ARIA when you need grid-style single-cell keyboard navigation (arrow keys moving between cells), which a plain table doesn't support.
- Sortable column headers should be real buttons inside the `<th>` with `aria-sort` (values `ascending`, `descending`, or `none`) reflecting current state, not a clickable `<div>` with a sort icon.

Keyboard navigation architecture

- Implement roving tabindex (a pattern where only one cell/control in the whole grid has `tabindex="0"` at a time, and arrow keys move that focus internally) rather than making every button in every cell independently tabbable, which would force a keyboard user to press Tab 40+ times to cross one row.

```
// Simplified shape: the table tracks one active [row, col] position,
// arrow keys move it, only that cell is tab-reachable.
function useRovingGridFocus(rowCount: number, colCount: number) {
  const [active, setActive] = useState({ row: 0, col: 0 });
  const isTabbable = (row: number, col: number) =>
    row === active.row && col === active.col;
  const moveBy = (dRow: number, dCol: number) =>
    setActive((pos) => clampToGrid(pos, dRow, dCol, rowCount, colCount));
  return { active, isTabbable, moveBy };
}
```

Selection and dynamic content

- Row selection checkboxes need an accessible label tied to row identity ("Select row: Acme Corp, Invoice #4021"), not a bare unlabeled checkbox, since a screen reader user tabbing through 40 identical checkboxes has no way to know which row they're on otherwise.
- When rows are added, removed, or re-sorted, use a live region (an ARIA region that announces content changes) to announce summary state changes ("Sorted by Amount, descending, 214 results") rather than expecting the user to notice a silent re-render.

Interview Answer:

"I'd start from real table markup, thead, tbody, th, since that gives you row and column context for free, and only escalate to a grid role if I actually need arrow-key navigation between individual cells. The part that takes real design work is keyboard navigation at scale: with 40 columns you can't make every interactive element independently tabbable, so I'd use a roving tabindex pattern where only one cell has focus at a time and arrow keys move it. I'd also make sure row checkboxes have labels tied to row identity, not bare checkboxes, and that sorting or filtering changes get announced through a live region, since a silent re-render is invisible to a screen reader user."

Q9. [Mid] You want to build accessibility checks into CI beyond just running an axe-core scan once. How would you architect the full pipeline, and where does manual or screen-reader testing fit in relative to the automated part?

Answer:

A single axe-core run on a single page state catches surface-level issues, but real products have dozens of dynamic states (open menus, error states, loading states) that a one-shot scan never sees. **The pipeline**

needs to run automated checks against every meaningful component state, gate merges on the unambiguous failures, and route anything requiring judgment to a scheduled manual and screen-reader review instead of trying to force it into an automated gate.

Layered pipeline design

1. **Component-level, per-state automated tests**: run accessibility assertions against a component's Storybook-style states (default, open, error, disabled), not just the default render, since a dropdown's violations often only exist in the "open" state.
2. **Page-level automated smoke tests**: run a broader scan against key user flows in a staging environment on every deploy, catching integration-level regressions that unit-level component tests miss.
3. **PR-level gating**: fail the build on unambiguous rule violations (contrast, missing labels, invalid ARIA); surface but don't block on rules that need human judgment.
4. **Scheduled manual/screen-reader pass**: a recurring cadence (say, before every major release, or monthly for high-traffic flows) where a person actually navigates key flows with a keyboard only and with a screen reader (VoiceOver, NVDA, or JAWS), because this is the only layer that catches "technically passes the automated rules but doesn't actually work."

```
// Example CI config shape: distinguishing blocking vs advisory rules
export const a11yGateConfig = {
  blocking: ["color-contrast", "label", "aria-valid-attr", "image-alt"],
  advisory: ["heading-order", "landmark-unique", "region"],
};
```

Why the split matters

- If everything blocks, teams either can't ship or start disabling the whole check out of frustration. If nothing blocks, the check becomes ignorable noise. The split keeps the gate credible.

Interview Answer:

"I wouldn't rely on a single axe scan of the default page state, because most real violations show up in dynamic states, an open menu, an error banner, a loading skeleton. So I'd run automated checks against each component's meaningful states, plus broader page-level scans on staging deploys, and split the rules into blocking versus advisory so the gate stays credible instead of either blocking everything or being ignored. On top of that, I'd keep a scheduled manual pass with an actual keyboard and screen reader before major releases, because that's the only layer that catches things that pass every automated rule but still don't work for a real user."

Q10. [Mid] Manually testing every pull request with a screen reader isn't realistic at scale. How do you design a manual/screen-reader testing process that actually scales across a large engineering org?

Answer:

Trying to make manual testing happen on every single PR either doesn't scale or gets skipped under deadline pressure, so the process has to be tiered based on risk, not applied uniformly. **You concentrate expensive manual and screen-reader testing on new or changed interactive components and high-traffic flows, and let cheaper automated checks cover the long tail of pages that reuse those already-tested components.**

Tiering by risk

- **New shared component in the design system**: always gets a full manual pass, keyboard-only navigation plus at least one screen reader (say VoiceOver on macOS and one of NVDA/JAWS on

Windows, since behavior genuinely differs between them), before it's published for other teams to consume.

- **Feature built entirely from existing, already-tested components**: relies mainly on automated checks, since the underlying primitives were already manually verified; a spot-check is still worthwhile but a full pass isn't necessary.
- **High-traffic or legally sensitive flows** (checkout, account creation, anything under an accessibility compliance deadline): scheduled recurring manual audits regardless of whether the code changed recently, since regressions can creep in from unrelated changes (a new analytics script, a CSS change elsewhere).

Making it repeatable, not hero-dependent

- Write down a fixed manual test script (specific tasks: "complete checkout using only Tab, Enter, and arrow keys," "complete checkout with VoiceOver enabled and eyes closed") so the process doesn't depend on one specialist's personal knowledge and can be run by any engineer trained on it.
- Rotate the responsibility across the team rather than routing everything through a single accessibility expert, both to build organizational knowledge and to avoid a single point of failure when that person is out or leaves.

Interview Answer:

"I wouldn't try to manually test every PR, that doesn't scale and it'll get skipped under deadline pressure. I'd tier it by risk: any new shared component always gets a full keyboard and screen reader pass before it's published, since everything downstream inherits its behavior. Features built entirely out of already-tested components lean on automated checks instead. And high-traffic or legally sensitive flows, like checkout, get a scheduled recurring manual audit regardless of whether the code changed, because regressions can sneak in from unrelated changes. I'd also write the manual tests down as a fixed script so it doesn't depend on one specialist and can be rotated across the team."

Q11. [Mid] You're asked to audit an existing large product, say 200 pages, for accessibility issues. How do you design the audit process, and how do you prioritize the resulting list of issues instead of just handing engineering a flat bug list?

Answer:

A flat list of a few hundred violations is useless to an engineering team; it doesn't tell them what to fix first, and "fix everything" isn't a real plan against limited sprint time. **The audit needs to score every issue on severity, reach, and fix cost, and prioritize by that combination rather than by raw violation count or the order pages happen to be listed in.**

Structuring the audit

1. Don't audit all 200 pages individually; identify the underlying shared components and templates those pages are built from, since fixing one shared component often resolves the same issue across dozens of pages at once.
2. Combine automated scans (fast, wide coverage of rule-based issues) with a manual keyboard and screen reader pass on a representative sample of page types (a form page, a data-heavy dashboard, a marketing page), since automated tools alone will miss the issues that actually block task completion.

The prioritization framework

- **Severity**: does this completely block a task (a form that can't be submitted via keyboard) versus degrade the experience (a slightly ambiguous heading level)?

- **Reach**: how many users and how many pages does this affect? A bug in a shared header component that renders on all 200 pages outranks a one-off bug on a rarely visited settings sub-page.
- **Fix cost**: is this a one-line contrast token change, or does it require rearchitecting a custom widget's keyboard handling? Two issues of equal severity and reach should still be sequenced by cost, since the cheap, high-impact fixes should ship first to show momentum and unblock the most users fastest.
- Combine these into a simple score (e.g., severity times reach, divided by cost, or a simpler high/medium/low matrix) and sort the backlog by that score, not by page order or component name.

Interview Answer:
"First, I'd map the 200 pages back to the shared components and templates they're actually built from, because fixing one shared header or form component might resolve the same issue across fifty pages at once, which is a much better use of engineering time than auditing every page as if it were unique. Then instead of handing over a flat list, I'd score every issue on severity, does it block a task or just degrade it, reach, how many pages and users does it touch, and fix cost, is this a one-line token change or a rearchitecture. I'd prioritize high severity, high reach, low cost fixes first, since those give you the most impact per engineering hour, and use that score to sequence the backlog instead of just going in the order issues were found."

Q12. [Mid] Design an internal accessibility review or gate process that every new feature has to pass before it ships. What does that process actually look like end to end?

Answer:

A gate that only exists as a check right before launch either gets skipped when the deadline is tight or catches issues so late that fixing them means redesigning the feature. **The gate needs to exist at multiple points across the feature's lifecycle, design, build, and pre-launch, so accessibility is checked while it's still cheap to fix, not just at the end when it's expensive.**

The three checkpoints

1. **Design review**: before engineering starts, a design reviewer (or checklist) verifies the mockups specify focus order, states (hover, focus, error, disabled) for interactive elements, and color choices that meet contrast requirements. Catching a contrast problem here costs a Figma color swap; catching it after launch costs an engineering ticket and a re-deploy.
2. **Build-time automated + peer review**: during implementation, CI-level automated checks run on every PR (see the CI-pipeline question), and PR review includes at least one explicit accessibility-focused pass, ideally by someone who wasn't the original author, since authors reliably miss their own blind spots.
3. **Pre-launch manual sign-off**: before the feature ships, a manual keyboard-only and screen-reader pass on the actual feature (not just the design mockups) is required, with a named owner who signs off, so the accountability for "was this actually tested" is explicit rather than assumed.

Making the gate stick, not just exist on paper

- Tie the pre-launch sign-off to the same release checklist teams already use for other launch requirements (legal review, performance budget), so it's not a separate, easily-skipped extra step.
- Give teams a fast escalation path for edge cases (a launch blocked by a genuinely ambiguous issue) to an accessibility specialist, rather than a rule that blocks indefinitely with no way to get an exception reviewed.

Interview Answer:
"I wouldn't put the gate only right before launch, because by then fixing an issue often means redesigning something instead of tweaking a color. I'd put a lightweight check at design review, focus order and contrast on the mockups, another at build time through CI plus an explicit accessibility pass in code review, and a final manual keyboard and screen reader sign-off before launch with a named owner. The key is folding that final sign-off into the same release checklist teams already use for things like legal or performance review, so it's a normal part of shipping rather than an extra hoop that gets skipped when the deadline is tight."

Q13. [Senior] Your product adds toast notifications, a live-updating dashboard, and content that gets injected dynamically without a page reload. How do you architect focus and announcement handling so keyboard and screen reader users don't lose track of what's happening?

Answer:

Dynamic content is where most production accessibility bugs live, because visually a sighted user just sees something appear, but a screen reader user has no equivalent peripheral vision; if nothing explicitly tells them something changed, they simply don't know. **You need a single, centralized announcement mechanism (a shared live region) that every feature routes through, rather than letting each feature decide independently whether and how to announce its own changes.**

Centralize the announcement mechanism

- Build one shared `LiveAnnouncer` service/hook that any part of the app can call to push a message into a persistent ARIA live region (a region marked `aria-live="polite"` or `"assertive"` so screen readers announce content changes without requiring focus to move there).
- `polite` for non-urgent updates (a toast confirming "Saved," a dashboard number ticking up); `assertive` only for things that need immediate interruption (a session-expiry warning), since overusing assertive is disorienting.

```
const { announce } = useLiveAnnouncer();

function handleSave() {
  saveDraft();
  announce("Draft saved", { priority: "polite" });
}
```

Deciding when to move focus vs just announce

- Toast notifications: announce, don't move focus, since focus should stay wherever the user was working; forcibly moving focus to a toast that then disappears is disorienting.
- A newly opened panel or a validation error block that appears after a failed form submission: move focus to it, since the user needs to act on it directly, an announcement alone isn't enough for something requiring a response.
- A live-updating dashboard number: announce summary changes at a controlled interval (not on every single tick, which would spam the user with constant noise) rather than on every data refresh.

Interview Answer:
"The failure mode with dynamic content is that a sighted user notices something appeared on screen, and a screen reader user has no equivalent unless you explicitly tell them. So I'd build one shared live announcer that every feature routes through, rather than letting each team decide independently, using polite for things like a save confirmation and assertive only for genuinely urgent interrupts like a session timeout warning. For things like a validation error after a failed submit, I'd actually move focus to it since the user

needs to act on it, whereas for a toast I'd announce it but leave focus where the user was working, since yanking focus to something that's about to disappear is worse than leaving it alone."

Q14. [Senior] A feature team wants to build their own custom dropdown instead of using the design system's shared one, because the shared component doesn't support a visual variant they need on a deadline. How do you handle this, both as an architecture decision and as an organizational one?

Answer:

This is the moment where a design system's accessibility investment actually gets tested, because the pressure to route around it is highest exactly when a deadline is tight. **The right response is almost never a flat "no," it's to fix the actual gap, extend the shared component to support the needed variant, or fast-track a reviewed exception, because every unofficial one-off dropdown becomes a permanent, unowned accessibility liability.**

Best Choice

Extend the shared component (or fast-track a supervised exception with a plan to fold it back in), rather than letting the team fork their own.

Why?

- A one-off custom dropdown built under deadline pressure will almost never get the same keyboard handling, focus management, and ARIA wiring the shared component has, because that team isn't accessibility specialists, and they're not incentivized to spend the extra days getting it right when the deadline is tomorrow.
- Once it ships, it becomes an orphan: no one owns updating it when accessibility standards evolve, and it will quietly diverge further every time someone touches it for an unrelated reason.
- The actual underlying problem is usually that the design system's component genuinely lacks a capability (say, grouped options with sticky group headers), which means other teams will hit the same wall later; fixing it once in the shared component pays off repeatedly.
- If there's truly no time to extend the shared component before this specific deadline, the better move is a time-boxed, reviewed exception (the design system team pairs on the one-off implementation to make sure it's still accessible) with an explicit follow-up ticket to fold the capability back into the shared component, rather than an unreviewed fork that never gets revisited.

Interview Answer:

"My default answer is not 'no,' it's 'let's fix the actual gap.' If the shared dropdown is missing a variant, that's useful signal, other teams will hit the same wall eventually, so extending the shared component pays off more than once. What I want to avoid is a team quietly forking their own version under deadline pressure, because it almost never gets the same keyboard and screen reader treatment, and it becomes an orphaned component nobody owns going forward. If there's genuinely no time to extend the shared component before this specific deadline, I'd rather fast-track a reviewed one-off, with the design system team involved to keep it accessible, and a concrete follow-up ticket to fold it back in, than let an unreviewed fork ship and disappear from anyone's radar."

Q15. [Senior] You're asked to lead an org-wide accessibility remediation effort on a large legacy product, ten-plus years of code, limited engineering time, and there's a compliance deadline. Design the rollout plan.

Answer:

The instinct to "fix everything before the deadline" is both unrealistic and the wrong plan, because a legacy product's issues are not evenly distributed or evenly risky, and a plan that tries to treat all of them equally will run out of time on low-value fixes. **The rollout has to be sequenced by risk and leverage: fix the shared components and highest-traffic flows first, since those changes compound across the whole product, and treat legal/compliance risk as a separate, explicit filter layered on top of the severity-reach-cost framework, not a reason to panic-fix randomly.**

Phase 1: Assessment and componentization (weeks, not months)

- Run the audit-and-prioritization process (severity times reach, divided by cost) across the product, but bias the "reach" scoring toward shared components and templates, since a fix there multiplies across every page that uses it.
- Identify which pages and flows are explicitly covered by the compliance deadline (often a specific regulation covers specific transaction flows, not literally every page), and flag those as a non-negotiable floor regardless of their cost-to-fix score.

Phase 2: Fix the multipliers first

- Put the design system/shared component fixes at the front of the queue, since a fixed shared `<Dropdown>` or `<Modal>` propagates to every consuming page automatically, this is the highest return on limited engineering hours.
- Stand up the automated CI gate now (even a partial one) so that as teams touch legacy code for unrelated reasons, they don't introduce new violations while the remediation is in progress; otherwise you're bailing water into a boat with a hole in it.

Phase 3: Targeted remediation sprints

- Pull a small rotating pool of engineers (not the whole org, which stalls unrelated roadmap work) dedicated to remediation, working down the prioritized backlog, starting with compliance-mandated flows, then high-reach/low-cost items, then the long tail.
- Track and report progress against the compliance deadline explicitly (percentage of mandated flows passing both automated and manual review), so leadership has real visibility rather than a vague "we're working on it."

Phase 4: Make it not regress

- Once the mandated flows and shared components are fixed, shift from a project to a standing process: the CI gate, the design/build/launch review gate from earlier, and periodic manual audits, so the remediation effort doesn't quietly decay back to its starting state a year later.

Interview Answer:

"I wouldn't try to fix everything before the deadline, that's not realistic and it treats unevenly risky issues as if they were equal. I'd run the severity, reach, and cost audit across the product, but weight reach toward shared components, since fixing a shared dropdown or modal once propagates everywhere it's used, that's the highest return on limited engineering time. I'd separately flag whatever flows the actual compliance deadline covers as a non-negotiable floor. Then I'd pull a small dedicated rotation of engineers to work the prioritized backlog, starting with compliance-mandated flows and high-reach low-cost fixes, while turning on at least a partial CI gate immediately so the legacy codebase stops accumulating new violations while we're

fixing the old ones. And once the deadline-critical work is done, I'd fold it into a standing review process instead of letting it be a one-time project that quietly decays."

Q16. [Senior] How do you design the governance model that decides which accessibility decisions live centrally in the design system team versus get delegated to individual feature teams? And how do you keep that boundary from eroding over time?

Answer:

Every design system eventually faces pressure to let feature teams "just do their own thing" for speed, and if the boundary isn't explicit, it erodes one exception at a time until the design system stops being a meaningful accessibility control point. **The governance model needs an explicit, written rule for what's centralized versus delegated, plus a real mechanism (not just goodwill) for catching and reversing drift, because an unenforced boundary always erodes toward whichever team is under the most deadline pressure that week.**

Draw the line explicitly

- Centralize: anything that's a reusable interactive pattern (modals, dropdowns, form controls, data tables), color and contrast tokens, and focus-management primitives. These are exactly the things where a single fix has to propagate everywhere.
- Delegate: page-level content structure, copywriting clarity, and any genuinely one-off visualization that has no reuse potential, since forcing every one-off into the design system just slows the design system down without adding leverage.
- Write this down as an actual document, not a shared understanding, since "shared understanding" is exactly what breaks down under deadline pressure six months later when half the original team has turned over.

Mechanisms that prevent erosion

- Automated detection: a lint rule or dependency scan that flags a feature team importing a non-design-system implementation of something that has a design-system equivalent (a raw `<div role="dialog">` instead of `<Modal>`), so drift gets caught by tooling, not by someone happening to notice in a code review months later.
- A lightweight exception process (see the earlier custom-dropdown question) so "no" isn't the only available answer when a team has a real gap, that's what actually keeps people from routing around the system quietly instead of asking.
- Periodic reconciliation: a recurring (quarterly, say) review of what's been built outside the design system, treating anything found as a signal to either fold the pattern in or explicitly re-affirm it as delegated, rather than letting it sit unaddressed.

Interview Answer:
"I'd centralize anything reusable, interactive components, color tokens, focus management primitives, since that's where a single fix propagates everywhere, and delegate page-level content and genuinely one-off visualizations. But the important part is writing that boundary down explicitly, because an unwritten understanding erodes the moment a team is under deadline pressure and quietly builds their own version. I'd back it with tooling, a lint rule that flags a team importing a raw dialog instead of the shared Modal, plus a real exception process so teams have a legitimate path when they hit a genuine gap instead of just routing around the system. And I'd do a periodic review of what's been built outside the system, because without that, drift just accumulates silently."

Q17. [Senior] How would you design metrics or dashboards to track an organization's accessibility progress over time, in a way that resists teams gaming the numbers?

Answer:

The obvious metric, "number of automated violations found," is exactly the one that's easiest to game, because a team can make that number drop by suppressing or ignoring rules instead of actually fixing anything, and it says nothing about whether real users can accomplish real tasks. **Good accessibility metrics need to combine an automated coverage signal with outcome-based signals (manual test pass rates on key flows, time-to-fix on new issues) that are much harder to fake than a raw violation count.**

Metrics that are easy to game (use with caution)

- Raw count of automated violations: can be reduced by disabling rules, narrowing scan scope, or fixing only the cheapest issues regardless of severity.
- "Percentage of components in the design system": says nothing about whether those components are actually used correctly by feature teams.

Metrics that are harder to game

- Manual/screen-reader test pass rate on a fixed set of critical user flows (checkout, sign-up, core task completion), re-run on a consistent cadence, since this reflects real usability, not just rule conformance.
- Time-to-fix for newly discovered issues, segmented by severity, since a team that lets high-severity issues sit for months is a clearer signal than a raw open-issue count that doesn't distinguish severity.
- Percentage of new PRs that pass the CI accessibility gate without needing an override/exception, since a rising override rate is a leading indicator that the gate is being routed around rather than satisfied.
- Trend of "escaped" issues (accessibility bugs found in production after shipping, versus caught pre-merge), since this measures whether the earlier-stage gates are actually working, not just whether a scan ran.

Reporting design

- Report trends over time per team/product area, not a single company-wide number, since a single aggregate number hides which specific teams need support or intervention.
- Pair every quantitative metric with a small number of qualitative spot-checks (an actual person occasionally re-verifying a "passing" flow) since automated self-reporting alone can quietly drift out of sync with reality.

Interview Answer:

"I'd avoid leaning on raw automated violation counts as the headline metric, because that number is trivially gameable, a team can make it drop by disabling rules or fixing only the cheapest issues. Instead I'd track things that are harder to fake: manual pass rates on a fixed set of critical flows like checkout, time-to-fix segmented by severity, and the rate of PRs needing an accessibility gate override, since a rising override rate tells you people are routing around the gate rather than meeting it. I'd also report this per team rather than as one company-wide number, since an aggregate hides exactly which teams need help, and I'd pair the dashboard with occasional human spot-checks, because a purely automated self-reported number can drift from reality without anyone noticing."

Q18. [Senior] You inherit a legacy custom dropdown/select component that's deeply embedded across 40 pages and is inaccessible (no keyboard support, no ARIA). Do you patch it in place or replace it, and how do you architect that migration?

Answer:

With something this embedded, the decision isn't really "patch vs. replace" as a binary, it's about how much of the existing component's public contract (its props, its DOM structure, the events it fires) you can preserve while swapping the internals, because a full rip-and-replace across 40 call sites simultaneously is itself a huge regression risk. **The lowest-risk path is usually to rebuild the component's internals to be accessible while preserving its existing external API, so the 40 consuming pages don't need to change at all, then only touch call sites individually for cases the new internals can't cleanly support.**

Best Choice

Rebuild internals behind the existing props/API where possible (an adapter approach), reserving full replacement for call sites that turn out to need incompatible new behavior.

Why?

- Rewriting the component's internals to add keyboard navigation, ARIA wiring, and focus management, while keeping its existing props (`options`, `value`, `onChange`) unchanged, means all 40 pages get the accessibility fix automatically on the next deploy, no coordinated migration needed.
- Trying to introduce a brand-new component and migrate 40 call sites to it in parallel multiplies the risk surface (40 separate PRs, 40 separate chances for a regression) for a problem that a single well-tested internal rewrite mostly solves at once.
- The exception is call sites doing something the old component's API can't express cleanly, say, one page needs multi-select and the legacy component was single-select only. Those become explicit, individually reviewed migrations to a newer/adjacent component, rather than being forced through the adapter.

```
// External contract stays identical; internals are fully rewritten
// to add keyboard nav, ARIA roles, and focus management.
<LegacyDropdown
  options={statusOptions}
  value={status}
  onChange={setStatus}
  // same props as before the rewrite; consuming pages are untouched
/>
```

Sequencing the rollout

1. Rebuild internals behind a feature flag, validate with automated tests plus a manual keyboard/screen-reader pass, on a handful of representative call sites first.
2. Roll the flag out incrementally (by page or by team) rather than flipping it globally, so a regression is caught on a handful of pages before it's everywhere.
3. Once stable, remove the flag and treat any call sites with incompatible needs as separate, scoped follow-up tickets, not blockers to shipping the accessibility fix for the other 39 pages.

Interview Answer:

"With something used on 40 pages, I try hard to avoid a full replace-and-migrate-everywhere, because that turns into 40 separate migration risks for a problem I can usually solve once. My default is to rebuild the component's internals, add real keyboard navigation, ARIA roles, focus management, while keeping its

existing props and API exactly the same, so all 40 pages get the fix automatically without any of them changing a line of code. I'd roll that internal rewrite out behind a flag to a handful of pages first, verify it with both automated and manual screen reader testing, then expand gradually. The only call sites I'd actually migrate to something different are the ones doing something the old API genuinely can't express, and I'd treat those as separate, scoped tickets rather than let them block shipping the fix for everyone else."

If you remember nothing else from this chapter: *accessibility bugs are nearly invisible in a visual review and completely disqualifying in real use, which is exactly why they need a direct check, not an assumption.*

Chapter 13: Frontend Security Architecture

Most frontend engineers can define XSS (cross-site scripting, getting attacker-controlled script to run in a victim's browser) or CSRF (cross-site request forgery, tricking a logged-in user's browser into firing a request the user didn't intend) in a sentence. That's not what gets you hired. What matters is whether you can take a specific product, a specific auth model, and a specific attacker, and design the concrete controls, storage choices, headers, and review processes that keep it safe without breaking the product. Use these questions to practice treating security as an architectural constraint you design around from day one, rather than a checklist you bolt on when a pen test comes back red. A weak answer recites "sanitize your inputs" and "use HTTPS." A strong answer states exactly where the token lives, what a specific attacker gets if they compromise it, and what the rollback plan is if the mitigation breaks a paying customer's integration.

War Story: *A single unescaped user-generated comment field, live for over a year, turned out to be a stored XSS vulnerability that any user could have used to steal every other logged-in user's session.*

Avoid: assuming any user input is safe to render because "it's just a comment" or "we validate on the backend"

Aim For: treating every point where user-generated content gets rendered as a place XSS could hide, by default

Q1. [Junior] You're building a simple internal admin tool, employees on the corporate VPN use it to look up shipment statuses. After login, where do you store the session token, and why does this being a low-stakes internal tool change your answer?

Answer:

The storage decision should always be driven by what an attacker gains and how they'd realistically get in, not by a blanket rule. **For a low-stakes internal tool, the risk of a token leaking is lower and the cost of engineering around XSS (cross-site scripting) is real, so a pragmatic team can accept more risk than they could on a banking app.**

Best Choice

- A short-lived session token in an httpOnly cookie (a cookie flag that blocks `document.cookie` and any injected script from reading the value) is still the safer default, even here, because it costs almost nothing to set up if the app already sits behind a session-based backend.
- If the tool is a pure SPA (single-page application) talking to an API with no server rendering, storing the token in memory (a JS variable, not `localStorage`) with a short expiry and silent refresh is an acceptable middle ground for an internal tool, since the blast radius of a leak is "someone sees shipment data," not "someone drains a bank account."

Why?

- `localStorage` is readable by any script running on the page, so any XSS bug, even a minor one introduced by a junior dev six months from now, becomes a full account takeover.
- An internal tool behind VPN and SSO (single sign-on) reduces the attacker population dramatically, which is exactly the kind of context that should shift effort, not eliminate it.

Interview Answer:
"I'd still default to an httpOnly cookie if the backend already does sessions, because it's basically free protection against XSS stealing the token. But I wouldn't block the roadmap over it for an internal VPN-only tool. If it's a pure SPA hitting a public API, I'd keep the token in memory rather than localStorage, accept the extra complexity of a refresh flow, and move on, because the actual damage from a leaked token here is limited to internal shipment data, not customer funds."

Q2. [Junior] Your team keeps debating how much client-side form validation to build, since everyone agrees a determined attacker can just hit the API directly with curl. What is client-side validation actually for, then, and what has to exist on the server no matter what?

Answer:

Client-side validation is a user experience feature, not a security boundary, and conflating the two is where teams get burned. **Every validation rule that matters for security or data integrity must be re-enforced on the server, because the client is fully under the attacker's control.**

What Client-Side Validation Is For

- Immediate feedback, like flagging an invalid email format or a password that's too short, before the user hits submit, so they don't wait on a round trip to discover a typo.
- Reducing pointless server load from obviously malformed requests, like an empty required field.
- Guiding the user through complex multi-step forms, like a checkout flow, where catching an error early avoids losing a session's worth of progress.

What Must Also Exist Server-Side

- Authorization checks, like confirming the logged-in user actually owns the order they're trying to cancel.
- Business rule enforcement, like a discount code only being valid once, since a client check is trivially bypassed with a direct API call.
- Data shape and size validation, to prevent a malicious payload from corrupting storage or causing a denial of service.

```
// Client: fast feedback only
function validateEmailFormat(email: string): boolean {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

// Server: the actual boundary, re-checks everything the client claimed
async function createAccount(req: Request) {
  if (!validateEmailFormat(req.body.email)) throw new BadRequest();
  if (await emailAlreadyExists(req.body.email)) throw new Conflict();
  // ...authorization, rate limiting, business rules all happen here
}
```

Interview Answer:
"I treat client-side validation purely as a UX layer, catching typos and giving instant feedback, since anyone can bypass it with a direct request. Anything that actually protects data integrity or enforces who's allowed to do what has to be duplicated on the server, no exceptions. In practice I write the validation once as a shared schema, like a Zod or Yup definition, and run it on both sides so I'm not maintaining two divergent rule sets, but I never trust the client-side pass as proof of anything."

Q3. [Junior] You're building a customer support dashboard that displays a user's saved credit card. Walk me through how you'd design the display of that card number, and what you'd want to check about the API response before you even get to rendering.

Answer:

The safest design decision here happens before any UI code gets written: deciding what data the API is allowed to send to the browser in the first place. **If the full card number never reaches the client, no frontend bug, XSS payload, or browser extension can leak it, so masking should happen server-side, not just in the display layer.**

Best Choice

- The API should return only the last four digits and the card network (e.g. "Visa ending in 4242"), never the full PAN (primary account number), and definitely never the CVV.
- The frontend then just renders what it's given, "•••• •••• •••• 4242", with no unmasking control exposed to a support agent unless there's a separate, audited, re-authenticated flow for the rare case a full number is genuinely needed.

Why?

- A field that exists in a JSON response is a field that will eventually be logged, cached, screenshotted, or read by a malicious browser extension, so "we'll just mask it in the UI" is not a real control.
- This is a case where the frontend engineer's job is partly to push back on the API contract, not just consume whatever shape the backend hands over.

Interview Answer:
"My first move isn't a component, it's a conversation with the backend team about the response shape. I want the API returning only the last four digits and card brand, so the full number literally never crosses the network to the browser. Once that's true, the frontend part is trivial, I just render the masked string. If there's a legitimate business need to reveal the full number, that should be a separate, logged, re-authenticated endpoint, not something baked into the default profile response."

Q4. [Junior] You're adding a comments section to a blog, users can write plain text with basic formatting like bold and links. What does your defense-in-depth plan against XSS look like here, beyond just saying "we'll sanitize it"?

Answer:

"Sanitize it" is the right instinct but a one-line answer, the strong move is breaking it into layers so that one mistake in one layer doesn't equal a breach. **Defense-in-depth for user-generated content means combining an allowlist-based sanitizer, safe rendering practices, and a Content Security Policy, so that no single failure point results in script execution.**

Layer 1: Sanitize on Write or Read (Allowlist, Not Blocklist)

- Run the comment HTML through a well-maintained sanitizer library, like DOMPurify, configured with an explicit allowlist of tags (`b`, `i`, `a`) and attributes (`href` on `a`, restricted to `http`/`https` schemes), rather than trying to blocklist dangerous patterns.
- Sanitize on the server before storage as the primary defense, since that protects every consumer of the data, not just the web client.

Layer 2: Safe Rendering on the Frontend

- Avoid `dangerouslySetInnerHTML` (React's escape hatch for injecting raw HTML) unless the content has already passed through the allowlist sanitizer, and re-sanitize client-side as a second check in case the stored data predates a sanitizer bug fix.
- Prefer rendering structured data (e.g., a small AST of "bold this range, link that range") over raw HTML wherever the feature allows it, since structured data has no execution surface at all.

Layer 3: Content Security Policy as a Backstop

- A CSP (Content Security Policy, a browser-enforced allowlist controlling what scripts/resources a page may load) that disallows inline scripts means that even if a malicious `<script>` tag slips through sanitization, the browser refuses to execute it.

```
import DOMPurify from "dompurify";

function renderComment(rawHtml: string) {
  const clean = DOMPurify.sanitize(rawHtml, {
    ALLOWED_TAGS: ["b", "i", "a"],
    ALLOWED_ATTR: ["href"],
  });
  return clean; // safe to pass to dangerouslySetInnerHTML after this
}
```

Interview Answer:

"I wouldn't rely on a single sanitizer call and call it done. I'd sanitize server-side with an allowlist library like DOMPurify before the comment is even stored, since that protects every client, not just the browser one. On the frontend, I avoid raw HTML injection unless it's gone through that same allowlist, and I'd re-sanitize client-side too as a belt-and-suspenders check. On top of that, a strict CSP that blocks inline script execution means that even if something gets past sanitization, the browser itself refuses to run it."

Q5. [Junior] A teammate wants to drop a new analytics vendor's script tag directly into the app's `index.html` to start tracking events by end of week. What do you want to know before approving that, and what would make you say no?

Answer:

Every third-party script that runs on your page runs with the same privileges as your own code, it can read cookies, intercept form input, and make requests, so approving it is a security decision, not a product decision. **A new third-party script should go through the same scrutiny as a new production dependency: what data it can access, what it's allowed to load, and how you'd contain it if it goes rogue.**

Questions to Ask Before Approving

- What data does this script touch? If it's on every page including checkout, does it see payment fields?
- Does the vendor support Subresource Integrity (SRI, a hash that ensures a fetched script matches an expected version) or, better, npm-installing and bundling it instead of loading from their CDN at runtime?
- Can it be scoped with CSP to only the domains it actually needs, rather than given a blanket allowance?

What Would Make You Say No

- The vendor requires loading the script directly from their CDN with no SRI hash and no changelog process, meaning they can push new code to your production site at any time without your review.

- The script needs access to pages with sensitive input, like a payment form, and the vendor has no clear data handling policy.

Interview Answer:
"I'd want to know what pages it runs on and what it can see, especially whether it's anywhere near the checkout or login flow. I'd push for installing it as an npm package we control and bundle ourselves, rather than a live script tag from their CDN, since a live tag means they can change what runs on our site without us reviewing it. If that's not possible, I'd at least want a Subresource Integrity hash pinned to a specific version, and I'd scope it out of any page that touches sensitive data via CSP."

Q6. [Junior] Your app uses a traditional session cookie for auth, and the backend doesn't currently do anything special for CSRF. Walk me through what you'd add and why the cookie's `SameSite` attribute matters here.

Answer:

Cookie-based session auth is exactly the setup CSRF (cross-site request forgery, a malicious site tricking a logged-in user's browser into sending an authenticated request) targets, since the browser auto-attaches cookies to requests regardless of which site triggered them. **The baseline fix is setting the session cookie's `SameSite` attribute to `Lax` or `Strict`, backed by a CSRF token for any state-changing request, since relying on `SameSite` alone leaves older browsers and edge cases uncovered.**

What to Add

- Set `SameSite=Lax` (or `Strict` if the app never needs cross-site GET navigation, like clicking an emailed link, to work while logged in) on the session cookie, which stops the browser from sending it on cross-site POST requests.
- Add a CSRF token: the server issues a random token tied to the session, the frontend reads it and includes it as a custom header on every state-changing request (POST/PUT/DELETE), and the server rejects requests where the header doesn't match.
- Reject state-changing requests that arrive without the custom header at all, since a cross-site form submission can't set custom headers.

Why Not Rely on SameSite Alone

- `SameSite` protection depends on browser support and correct configuration, and some legitimate flows (like a payment redirect back from a third-party processor) can require `SameSite=None`, which reopens the door.
- Defense-in-depth means the CSRF token check catches what the cookie attribute misses.

Interview Answer:
"I'd set the session cookie to SameSite=Lax as a first layer, since that alone stops the browser from attaching it to most cross-site POST requests. But I wouldn't stop there, because some flows need looser cookie settings and older browsers don't all honor SameSite consistently. So I'd also add a CSRF token that the server issues, the frontend sends back as a custom header on every mutating request, and the server validates against the session. That combination covers both the common case and the edge cases the cookie attribute alone doesn't."

Q7. [Mid] Now take that same design question to a banking app: after login, where does the auth token live, and specifically why is an httpOnly cookie the right call here in a way it might not be for a lower-stakes app?

Answer:

A banking app's risk profile flips the cost-benefit analysis from Q1: the value of a stolen token is now "an attacker can move money," so the extra engineering cost of the safest storage option is easily justified. **For a banking app, the token belongs in an httpOnly, Secure, SameSite cookie set by the server, because that is the only common storage option a client-side XSS bug cannot read directly.**

Best Choice

- httpOnly cookie, `Secure` flag (only sent over HTTPS), `SameSite=Lax` or `Strict`, short expiry, with a separate refresh mechanism handled entirely server-side.
- Pair it with a CSRF token strategy (per Q6) since httpOnly cookies solve the XSS-theft problem but reopen the CSRF surface that token-in-header approaches don't have.

Why?

- `localStorage` or an in-memory token exposes the token to any successful XSS injection, and a banking app is a high-value target where "we'll just avoid XSS bugs forever" is not a credible plan across years of feature development.
- The cost here, more backend coordination for httpOnly cookies plus CSRF tokens, is trivial compared to the cost of a single account-takeover incident at a bank.

What Changes vs the Internal Tool

- For the internal tool, in-memory storage was an acceptable trade because the blast radius was small and VPN access narrowed the attacker pool. For a banking app, the attacker pool is "anyone on the internet who can get one XSS payload to execute," so the same trade is unacceptable.

Interview Answer:

"For a bank app I go straight to an httpOnly, Secure cookie with SameSite set, issued and rotated entirely server-side, because that's the one storage option that a client-side XSS bug simply can't read. Yes, that means I now need CSRF protection too, since cookies auto-attach, but that's a well-understood, solvable problem, versus the alternative of a token sitting in JS-accessible storage where a single sanitizer bug turns into someone's life savings disappearing. The stakes justify the extra backend coordination."

Q8. [Mid] You've inherited a large e-commerce app that's been live for eight years, with a dozen third-party scripts already embedded: ad tags, a chat widget, analytics, a payment SDK. Leadership wants a Content Security Policy in place. How do you roll it out without breaking the site?

Answer:

Shipping a strict CSP (Content Security Policy) on day one on an eight-year-old app is close to guaranteed to break something, since nobody has a complete inventory of what's actually loading. **The safe rollout path is Content-Security-Policy-Report-Only mode first, collecting violation reports in production traffic for weeks, before ever switching to an enforcing policy.**

Rollout Steps

1. Inventory every script, style, iframe, and connection the app currently makes, using both a manual audit and a first pass CSP built loosely from that audit.
2. Deploy that policy as `Content-Security-Policy-Report-Only` with a `report-uri`/`report-to` endpoint, so violations are logged but nothing is blocked yet.
3. Let it run against real production traffic for at least a couple of weeks, since third-party scripts often load conditionally (a/b tests, geo-based ad networks) and a short window will miss cases.
4. Triage the violation reports: some are things you genuinely want to allow (the payment SDK's iframe origin), some are things you want to kill (an old marketing pixel nobody remembers approving).
5. Tighten the policy, move to enforcing mode for a narrow, low-risk subset of directives first, like `object-src 'none'` and `base-uri 'self'`, before enforcing the higher-risk `script-src`.
6. Keep the report-only listener running permanently on a secondary, slightly looser policy, so new third-party additions get flagged before they cause an incident, not after.

Why Report-Only First

- It turns "did we break something" from a production incident into a log line you review calmly, which is the entire point of the staged rollout.

Interview Answer:
"I would never flip straight to an enforcing CSP on an app this old, there's no way anyone has a complete mental model of every third-party origin it depends on. I'd start with Content-Security-Policy-Report-Only, built from a manual inventory, and let it collect violations against real traffic for a few weeks to catch the scripts that only load for some users or some campaigns. Then I'd triage: allow what's legitimate, kill what's dead weight, and only then start enforcing, directive by directive, starting with the lower-risk ones like object-src before touching script-src. I'd also keep a permanent report-only policy running afterward so new third-party scripts get flagged before they become an incident."

Q9. [Mid] Your product now needs a WYSIWYG (what-you-see-is-what-you-get) rich text editor, so users can paste content from Word or other websites, not just type plain comments. How does your XSS defense-in-depth strategy from a simple comment box need to change for this?

Answer:

A WYSIWYG editor is a much harder surface than plain-text comments because "paste from anywhere" means users routinely bring in HTML with embedded styles, tracked-change markup, and sometimes scripts they didn't even know were there. **The strategy still starts with an allowlist sanitizer, but now it has to run at multiple points: on paste, before storage, and again before render, because any one of those points can be bypassed by a clever payload.**

What Changes From the Comment Box Case

- Sanitize on paste in the editor itself (most rich text libraries like TipTap or Slate support a paste-transform hook), stripping unknown tags and attributes before they ever enter the document model, so the user sees clean content immediately rather than getting silently rejected later.
- Sanitize again server-side before storage, treating the paste-time sanitization as a UX nicety, not a security control, since a malicious client can send arbitrary payloads directly to the API regardless of what the editor UI does.

- Sanitize a third time on render for any consumer that isn't the original editor, like a public-facing preview page, since stored content that was safe under one sanitizer configuration might not be safe under a looser one used elsewhere.
- Explicitly ban dangerous constructs the plain comment box never had to think about: `style` attributes with `url()` referencing external resources, SVG (which can carry embedded scripts), and `iframe`/`object` embeds.

Additional Layer: Isolate Rendering

- For untrusted rich content shown to other users, consider rendering it inside a sandboxed iframe with a restrictive CSP of its own, so that even a sanitizer bypass is contained to a context with no access to the parent page's cookies or DOM.

```
const sanitizeConfig = {
  ALLOWED_TAGS: ["p", "b", "i", "u", "a", "ul", "li", "h1", "h2", "img"],
  ALLOWED_ATTR: ["href", "src", "alt"],
  ALLOWED_URI_REGEXP: /^(https?:|mailto:)/i, // block javascript: URIs
  FORBID_TAGS: ["style", "svg", "iframe", "object"],
};
```

Interview Answer:

"A WYSIWYG editor raises the stakes because users paste in HTML from anywhere, Word docs, other websites, and that content can carry things they never intended. I'd sanitize with an allowlist at three points: on paste for immediate UX, server-side before storage as the real security control, and again on render for any place that displays the content differently, like a public preview. I'd also explicitly strip things a plain comment box never worried about, like SVGs and inline styles with external URLs. If the content is shown to other users and the stakes are high enough, I'd consider rendering it in a sandboxed iframe with its own tight CSP, so a sanitizer bypass is contained rather than game over."

Q10. [Mid] Your SPA uses a token-based auth model, the frontend sends the access token as an `Authorization: Bearer` header rather than relying on a cookie. Does this app still need CSRF protection, and how does the architecture change from the cookie-based case?

Answer:

This is a common point of confusion, and it's worth reasoning through it rather than just stating the conclusion. **If the token is only ever sent via a manually-set Authorization header and never stored in a cookie, CSRF is largely neutralized, because a malicious site's cross-origin form or image tag can't force the browser to attach a header it doesn't control.**

Why Header-Based Auth Sidesteps CSRF

- CSRF works because the browser automatically attaches cookies to any request to the cookie's domain, regardless of which page triggered it. A forged cross-site form has no way to set a custom `Authorization` header, since that requires JavaScript running in the context of a page that already has the token, which is a same-origin restriction, or an XSS bug, which is a different problem entirely.

Where the Architecture Still Needs Care

- The access token has to come from somewhere and be stored somewhere. If it's kept in memory and refreshed via a refresh token sent in an httpOnly cookie (a common hybrid pattern), that refresh endpoint itself needs CSRF protection, since it is cookie-based.

- CORS (Cross-Origin Resource Sharing) configuration on the API becomes the actual gatekeeper: if the API's CORS policy is misconfigured to allow credentials from any origin, that reopens a similar class of cross-origin abuse.

Best Choice

- Bearer token in memory for API calls, refresh token in an httpOnly cookie scoped to a dedicated `/auth/refresh` endpoint, with CSRF protection specifically on that endpoint, and a strict CORS allowlist everywhere else.

Why?

- This gets the XSS-resistance benefit of httpOnly storage for the long-lived credential (the refresh token) while keeping the short-lived access token's blast radius small if it does leak, and it confines the CSRF-relevant surface to one well-guarded endpoint instead of the whole API.

Interview Answer:

"If the access token is only ever sent as a manually-attached Authorization header, classic CSRF mostly goes away, because a forged cross-site request can't make the browser add that header on its own. But I'd still look for where a cookie sneaks back in, usually the refresh token. A common pattern I'd design for is: access token in memory, refresh token in an httpOnly cookie hitting a dedicated refresh endpoint, and that specific endpoint gets CSRF protection since it's the one place cookies are doing the authenticating. I'd also lock down CORS tightly, since a loose CORS policy that allows credentials from arbitrary origins undermines this whole design."

Q11. [Mid] Your app needs to embed a third-party widget, say a chat support tool or a partner's booking calendar, inside an iframe, and it needs to exchange data with your page (like passing the logged-in user's name). How do you architect that communication securely?

Answer:

`postMessage` (the browser API for passing messages between windows/iframes of different origins) is the right tool, but it is dangerous by default because a naive implementation trusts any message from any origin. **The core design rule is to always specify and verify the exact origin on both the sending and receiving side, never use a wildcard `*` target origin, and treat every incoming message as untrusted input until validated.**

Sending Side

- Always pass the exact expected origin as the second argument to `postMessage`, never `"*"`, so that even if the iframe's `src` gets hijacked or redirected, your page won't leak data to an unintended origin.

Receiving Side

- Check `event.origin` against an explicit allowlist before doing anything with `event.data`.
- Validate the shape and type of `event.data`, since even a legitimate partner's widget could have a bug, or the origin check could have a subtle flaw, and treating the payload like any other untrusted API response costs little and catches a lot.
- Never use `event.data` to directly set `innerHTML` or evaluate code, apply the same allowlist-sanitization thinking from the XSS questions if any part of it is ever rendered.

```
const ALLOWED_WIDGET_ORIGIN = "https://widget.partner.com";
```

```
window.addEventListener("message", (event: MessageEvent) => {
  if (event.origin !== ALLOWED_WIDGET_ORIGIN) return; // reject anything else
  if (typeof event.data !== "object" || event.data?.type !== "widget:ready") return;
  handleWidgetReady(event.data.payload);
});

// Sending into the iframe, always scoped to the known origin
iframeRef.current?.contentWindow?.postMessage(
  { type: "user:context", userName: currentUser.name },
  ALLOWED_WIDGET_ORIGIN
);
```

Additional Containment

- Sandbox the iframe with the `sandbox` attribute and only the specific permissions it needs (e.g. `allow-scripts` but not `allow-top-navigation`), so that even if the widget's own code is compromised, it can't navigate your top-level page or access things outside its frame.
- Only send the minimum data the widget needs, a display name, not a full user object with an email or internal ID, since you don't fully control what that vendor does with data once it's inside their iframe.

Interview Answer:

"I'd treat the iframe as a fully untrusted third party even though we chose to embed it. On the sending side, I always target the exact origin, never a wildcard, so a hijacked or redirected iframe can't intercept our data. On the receiving side, I check event.origin against an allowlist before touching event.data at all, and I validate the message shape like I would any external API response, since I don't want to trust that the vendor's code is bug-free just because we picked them. I'd also sandbox the iframe with only the permissions it actually needs, and only pass the minimum data required, like a display name instead of a full user profile."

Q12. [Mid] You're designing an account settings page that shows a masked SSN (social security number) to the user, and a support agent's internal tool that sometimes needs to see the full SSN to resolve a dispute. How do you architect these two different needs without creating a leak?

Answer:

This is really two separate systems wearing the same "display an SSN" label, and treating them identically is how leaks happen. **The customer-facing page should never receive anything but the masked value from the API, while the support agent's full-SSN access should be a separate, explicitly audited, re-authenticated flow, not a toggle on the same endpoint.**

Customer-Facing Page

- API returns only the last four digits, the full value never serializes into that response at all, so there's no unmask button to accidentally expose, no caching layer that might retain it, and no browser extension or XSS bug that could exfiltrate something that was never sent.

Support Agent Tool

- A separate, narrowly scoped endpoint that returns the full SSN only for a specific, logged reason (e.g. "resolving dispute #4821"), requiring the agent to re-enter their credentials or complete a step-up auth challenge (an extra verification step beyond normal login) before the value is revealed.

- Every access is written to an audit log with who, when, and why, and ideally the value is displayed with an auto-expiring reveal (shown for 30 seconds, then re-masked) rather than sitting rendered indefinitely on screen.
- Rate limit and alert on unusual access patterns, like one agent viewing dozens of full SSNs in an hour.

Why Separate Systems, Not One Endpoint With a Permission Flag

- A single endpoint gated by a role check is one misconfigured permission away from leaking full SSNs to the customer-facing client. Physically separate endpoints and response shapes mean the customer app's code path has no way to accidentally receive the sensitive value, even if a role check elsewhere has a bug.

Interview Answer:
"I'd design these as two different systems, not one endpoint with a flag. The customer-facing profile page only ever gets an API response with the last four digits baked in, full stop, so there's no sensitive value sitting in that client's memory or cache to leak. The support tool gets its own endpoint that requires a re-authentication step and logs exactly who viewed the full number and why, tied to a specific case. That separation means a bug in one role check can't accidentally expose the full SSN to the wrong audience, because the customer-facing code path never had access to it in the first place."

Q13. [Senior] A stored XSS vulnerability is discovered in production, a crafted username is rendering as executable script on the profile page of anyone who views it, and it's already been active for two weeks. Walk me through your response, from the first hour to the postmortem.

Answer:

This question tests whether you can think in terms of containment and blast radius under time pressure, not just "we'll patch the code." **The response has to run in parallel tracks: stop the bleeding immediately, assess who was actually exposed, fix the root cause, and only then look at process changes, in roughly that order of urgency.**

First Hour: Contain

1. Take the vulnerable rendering path down or behind a feature flag immediately, even if that means temporarily disabling profile page rendering of usernames, accepting a degraded feature over an active exploit.
2. If the payload is identifiable, purge or neutralize the malicious stored data directly in the database as an interim fix, independent of the code deploy.
3. Rotate anything the payload could plausibly have stolen: force session invalidation for any user who visited an affected page, if session tokens were readable client-side.

Same Day: Assess Blast Radius

- Pull logs to find which profile pages were viewed during the exposure window and by whom, since that scopes who needs a security notification.
- Determine what the payload actually did: was it exfiltrating cookies, redirecting users, or just defacing pages, since that changes the severity and disclosure obligations.

Within Days: Fix Root Cause

- Patch the actual rendering bug, likely a missing or bypassed sanitizer on the username field, and add a regression test that specifically re-injects the discovered payload.
- Audit every other place user-controlled strings get rendered as HTML, since if one field skipped sanitization, others likely did too.

After: Process

- Add the payload as a permanent test case in the security regression suite.
- Review whether a CSP would have contained this even if the sanitizer failed, and if not, tighten it.
- Run a blameless postmortem: how did this ship, why didn't code review catch it, does the team need a linting rule that flags raw HTML injection in review.

Interview Answer:

"First priority is containment, not root cause analysis. I'd get the vulnerable page behind a flag or take the rendering path down immediately, even if it degrades the feature, and if I can identify the malicious payload in the data, I'd neutralize it directly in the database as an interim step. Then I'd figure out blast radius: who viewed affected pages during the exposure window, and what the payload actually did, since that determines whether we owe users a security notification. Only after containment do I fix the actual sanitizer bug, add a regression test with the exact payload, and audit other places we render user content, since one missed sanitizer usually means there are others. I'd close it out with a blameless postmortem focused on why review didn't catch it."

Q14. [Senior] Your org ships several frontend apps sharing a monorepo, and adds new npm dependencies weekly. Design the review process for evaluating new third-party dependencies given supply-chain risk, things like a compromised maintainer account pushing a malicious patch version.

Answer:

Supply-chain risk means the danger isn't just "is this library well-written," it's "can this library or its transitive dependencies change under us without anyone noticing." **A real process needs automated gates that run on every install, not just a one-time manual review when the dependency is first added, since the attack usually lands in a later version, not the one you initially vetted.**

Automated Gates (Every Install/CI Run)

- Lockfile enforcement (`package-lock.json` or equivalent) committed and verified in CI, so builds are reproducible and a silent transitive version bump can't sneak in unreviewed.
- Automated SCA (software composition analysis) scanning, like `npm audit`, Snyk, or Socket.dev, on every PR that touches the lockfile, flagging known CVEs and, ideally, suspicious install-time behavior (packages that run postinstall scripts, request network access at install time, or were just transferred to a new maintainer).
- A CI check that blocks merges introducing a new transitive dependency with no corresponding lockfile diff review.

Manual Review Trigger Points

- Any new direct dependency needs a lightweight review: what does it do, is there a lighter alternative, how actively maintained is it, how many maintainers, how many weekly downloads (a proxy for how quickly the community would notice a compromise).

- Dependencies with install scripts (`postinstall` hooks) get extra scrutiny, since that's a common supply-chain attack vector, arbitrary code execution at install time before any application code even runs.

Ongoing Controls

- Pin exact versions rather than broad semver ranges (`^` or `~`) for anything with elevated risk, and use a scheduled, reviewed update process (like Renovate or Dependabot with required review) rather than automatic minor/patch upgrades.
- Consider a private registry proxy that mirrors and can pin/block specific package versions org-wide, so a single compromised release can be blocked centrally rather than relying on every team catching it independently.

Interview Answer:
"I'd split this into automated gates and human review, because relying purely on a one-time manual review misses the real attack pattern, which is usually a compromised package getting a malicious version bump months after it was approved. So every PR that touches the lockfile runs through SCA tooling like Socket or Snyk automatically, flagging things like new postinstall scripts or a sudden maintainer change. For new direct dependencies, I'd want a lightweight human review of maintenance health and whether we actually need it versus writing the twenty lines ourselves. And for anything sensitive, I'd pin exact versions and route upgrades through a reviewed bot like Renovate rather than letting semver ranges silently pull in new code."

Q15. [Senior] Six months into the CSP report-only rollout from Q8, you're ready to move to enforcing mode on `script-src`, but the marketing team routinely adds new third-party tags through a tag manager (like Google Tag Manager) without engineering review. How do you architect the policy and process so enforcing CSP doesn't turn into a recurring fire drill?

Answer:

The technical policy and the organizational process have to be designed together here, because a tag manager that bypasses engineering review is structurally incompatible with a strict `script-src` allowlist, whichever one wins, the other has to change. **The sustainable architecture uses nonce-based or hash-based CSP for first-party scripts, combined with making the tag manager itself the single allowed script-loading gateway, so marketing keeps its independence while every script it loads is still contained.**

Technical Design

- Use a per-request nonce (a random value generated server-side per page load and injected into both the CSP header and the script tags) for first-party scripts, rather than a static allowlist of hashes, since that scales better as the app changes.
- For the tag manager itself, allowlist its specific loader domain in `script-src`, but rely on the tag manager's own sandboxing/permission model (e.g. GTM's consent and permission scopes) plus CSP's `script-src` still blocking anything the tag manager tries to load from a non-allowlisted domain, so a rogue tag inside the manager still can't pull in an arbitrary script from an unapproved origin.
- Use `Content-Security-Policy-Report-Only` permanently on a slightly wider "canary" policy in parallel with the enforced one, so new marketing tags show up as violations in reports before they show up as a broken production incident.

Process Design

- Require any new tag manager container change to go through a lightweight approval step, even if it's just a Slack bot that posts "this GTM change would add script-src violations for domain X" before it goes live, generated automatically from the report-only canary policy.
- Give marketing a self-service allowlist request path with a fast SLA (service level agreement, a promised turnaround time), like a same-day PR that adds one domain to the policy, so the friction of the process doesn't push them to work around it entirely.

Why This Balance Matters

- If the process is too strict, marketing route around it and lose the safety net entirely; if it's too loose, the CSP is decorative. The goal is a fast, mostly-automated approval loop, not a manual gate that becomes the bottleneck everyone learns to bypass.

Interview Answer:

"The real problem isn't the CSP syntax, it's that marketing's workflow and engineering's review process are pulling in opposite directions. I'd use nonce-based CSP for our own scripts so that scales without hand-maintaining hashes, and I'd allowlist the tag manager's loader domain specifically, since blocking it outright just means marketing routes around us. Then I'd run a permanent report-only canary policy that's slightly wider than the enforced one, and wire it to automatically flag new violations, ideally posted straight into the channel where marketing makes tag manager changes, so they see the impact before it's a production break instead of after. And I'd give them a fast, mostly self-service path to request a new domain, because if the process is slower than just shipping the tag anyway, they'll bypass it."

Q16. [Senior] You're reviewing a PR that adds a "share to social" feature: it embeds a third-party widget via a `<script>` tag that then injects an iframe showing live content from that platform, directly into the checkout confirmation page. What's your review focus, and what guardrails would you require before approving?

Answer:

The placement on the checkout confirmation page is the first red flag before you even read the widget's code, since that's one of the highest-value pages on the entire site for an attacker to compromise. **The review should treat the third-party script as an unaudited dependency with page-wide execution rights, and the required guardrails are isolation (don't load it on sensitive pages, or sandbox it if you must), integrity (pin and verify the exact script version), and containment (CSP scoped tightly to only what this widget needs).**

Placement Review

- Question why this needs to be on the checkout confirmation page at all, rather than a dedicated post-purchase page with no payment-adjacent context, since the checkout flow is exactly where you want the smallest possible third-party surface area.
- If it must stay there, the widget's script should not run in the same execution context as the checkout page at all, load it inside a sandboxed iframe with a locked-down `sandbox` attribute, so a compromised or buggy widget can't reach the parent page's DOM, cookies, or form fields.

Integrity Guardrails

- Require Subresource Integrity (SRI) hash pinning on the script tag if it's loaded directly, or better, vendor the SDK through npm with lockfile pinning instead of a live CDN reference, so the vendor can't silently change what runs on this page.

- Add the widget's specific domain to CSP `script-src`/`frame-src` explicitly, rather than a broad wildcard, so it can't pull in additional, unreviewed origins at runtime.

Data Exposure Guardrails

- Confirm the widget has no access to page context it doesn't need, no reading of the DOM outside its own iframe, no access to order details, payment status, or user PII (personally identifiable information) beyond what's explicitly passed via a scoped `postMessage` call, per the pattern from Q11.

What Would Block Approval

- A same-context `<script>` tag with no SRI, no CSP scoping, and no sandboxing, sitting directly on the checkout confirmation page, would be a hard no until moved to an isolated, sandboxed context with a tightly scoped CSP and pinned integrity.

Interview Answer:
"My first question in review is why this is on checkout confirmation at all, since that's about the worst page on the site to add unaudited third-party execution to. If there's a real product reason to keep it there, I'd require it run inside a sandboxed iframe, not a same-context script tag, so it has zero access to the parent page's DOM or cookies. I'd require Subresource Integrity or, better, that we vendor it through npm rather than pull live from their CDN, and I'd want the CSP scoped to exactly that widget's domain, not a wildcard. And I'd want to see exactly what data, if any, passes into that iframe via postMessage, because 'it just needs the order ID to show a share card' is very different from it having a handle on the full order object."

Q17. [Senior] Design the end-to-end token architecture for a fintech app that has both a web app and a mobile web experience, needs to support silent session refresh, and has to survive a worst-case scenario where an XSS bug does get through despite your defenses. Walk me through the full flow.

Answer:

At senior level, the interesting part isn't picking cookie-vs-localStorage in isolation, it's designing the whole token lifecycle so that even a successful XSS exploit has a small, time-boxed blast radius. **The strongest common pattern is a BFF (backend-for-frontend, a thin server layer that sits between the browser and the real APIs) that holds the long-lived refresh token server-side entirely, issuing only short-lived access tokens to the browser, so nothing an XSS bug could steal survives more than a few minutes.**

Architecture

1. Login goes through the BFF, which authenticates against the identity provider and stores the resulting refresh token server-side (in a session store, never sent to the browser at all).
2. The BFF sets a session cookie, httpOnly, Secure, SameSite=Strict, that identifies the server-side session, not the actual credential.
3. The browser calls the BFF for an access token; the BFF exchanges its server-held refresh token for a short-lived (say, 2-5 minute) access token and hands that to the browser, kept in memory only, never in storage.
4. On expiry, the browser silently calls the BFF again; since the BFF still holds the real refresh token server-side, the browser never needs to handle refresh tokens directly at all.
5. For mobile web specifically, the same BFF session model applies, since it's still a browser context, avoiding the temptation to fall back to storing anything long-lived client-side just because it's "mobile."

Why This Survives an XSS Bug Better

- If an XSS bug executes, the worst it can steal is the short-lived access token sitting in memory, which is valid for minutes, not the long-lived refresh token, which never left the server.
- The httpOnly session cookie identifying the BFF session is unreadable by the injected script, so the attacker can't even use it to keep calling the BFF for fresh tokens beyond whatever window the XSS payload itself is actively running in.

Additional Containment

- Bind the BFF session to characteristics like IP range or device fingerprint where feasible, so a stolen access token used from a wildly different context gets flagged.
- Keep access token lifetime aggressively short specifically to bound the worst case, and support immediate server-side session revocation (kill the BFF session, not just let the token expire) for incident response.

Interview Answer:

"I'd put a backend-for-frontend in front of the identity provider, and design it so the actual refresh token never leaves the server at all, ever. The browser only gets a short-lived access token, a few minutes, kept in memory, and an httpOnly session cookie that identifies the BFF session rather than being a usable credential itself. Mobile web goes through the exact same flow, since it's still a browser. The reason I'd design it this way specifically is the worst-case scenario in the question: if an XSS bug does get through, the blast radius is capped at whatever a few-minute access token can do, and I can kill the BFF session server-side immediately for incident response, versus a long-lived refresh token sitting somewhere the browser can reach, which would be catastrophic at a fintech company."

Q18. [Senior] You're setting engineering standards for the whole frontend org. Design a security review gate that runs as part of the normal PR and CI process, covering the ground we've discussed: XSS-prone patterns, dependency risk, and CSP violations, without making every PR feel like a security audit.

Answer:

The goal is to make the safe path the default path, catching the categories of mistakes covered in this chapter automatically, so human security review is reserved for genuinely novel design decisions rather than repetitive pattern-matching. **A layered CI gate, static analysis for dangerous patterns, automated dependency scanning, and CSP report-only monitoring, catches the large majority of issues before a human reviewer even looks at the diff, and human review is escalated only for flagged or genuinely new risk categories.**

Layer 1: Static Analysis (Every PR, Fast, Automatic)

- A lint rule (like `eslint-plugin-security` or a custom rule) that flags `dangerouslySetInnerHTML`, `eval`, direct `innerHTML` assignment, and unscoped `postMessage` target origins (`"*"`), requiring an explicit inline suppression comment with a linked justification if genuinely needed, so these patterns are visible and deliberate, not accidental.
- Fails the build, not just a warning, since a warning that's routinely ignored provides no actual protection.

Layer 2: Dependency Scanning (Every PR Touching the Lockfile)

- Automated SCA scanning per Q14, blocking merges on new critical CVEs or newly-added install scripts without a manual override from a designated reviewer.

Layer 3: CSP Violation Monitoring (Continuous, Not Per-PR)

- The permanent report-only canary policy from Q15 running in a staging environment, with any new violation surfaced automatically in the PR that introduced the change, when traceable, rather than discovered weeks later in a shared dashboard nobody checks.

Layer 4: Human Review, Reserved for Real Judgment Calls

- A short checklist template attached to PRs touching auth, payment data display, or third-party embeds, per the earlier questions in this chapter, prompting the reviewer to actually ask "where does this token live" or "what does this iframe have access to," rather than relying on the reviewer to remember to ask.
- A designated security-minded reviewer (not necessarily a dedicated security team at a smaller org) required specifically on PRs matching those categories, identified via a CODEOWNERS-style path rule.

Why This Balance

- Automating the repetitive 80% (raw HTML injection, known-vulnerable dependencies, CSP regressions) means the humans on the review are spending their attention on the 20% that actually requires judgment, like the checkout-page widget scenario in Q16, which is exactly where automation can't help.

Interview Answer:

"I'd design this as layers, most of it automatic, so we're not asking every engineer to become a security expert on every PR. Static analysis catches the mechanical stuff, dangerouslySetInnerHTML, wildcard postMessage targets, eval, and fails the build rather than just warning. Dependency scanning runs on anything touching the lockfile and blocks on new criticals or suspicious install scripts. I'd keep a permanent report-only CSP canary running and surface new violations directly on the PR that caused them. And then for the categories that actually need human judgment, auth flows, payment data display, third-party embeds, I'd require a specific reviewer and a short checklist that prompts the right questions, like where does this token live or what can this iframe reach, because that's where real judgment matters and automation can't cover it."

If you remember nothing else from this chapter: *the most dangerous XSS vulnerabilities aren't in the code people review carefully, they're in the field everyone assumed was harmless.*

Chapter 14: Testing & Release Strategy

Almost every frontend candidate can name the testing tools on their resume, but very few have had to decide how much testing is enough, where a test suite's time is best spent, or how to ship a risky change to production without it turning into an incident. This chapter is where you find out whether you've actually owned a release process end to end, made a call about what not to test, or fought a flaky end-to-end (E2E, tests that drive a real browser through the full app) suite back under control. A weak answer defaults to "write more tests" or "use Cypress"; a strong answer talks about cost, signal, blast radius, and how a team's size and constraints change the right answer.

War Story: *A test suite that was "one hundred percent passing" for months turned out to have a silently broken CI step, meaning no test had actually run in weeks, discovered only when an obviously broken build shipped straight to production.*

Avoid: treating a green CI checkmark as proof of correctness without ever asking what would happen if the tests silently stopped running

Aim For: building in a check that the test suite itself is actually running and catching things, not just producing a green checkmark

Q1. [Junior] You're the second frontend engineer at a four-person startup shipping an MVP every week. How would you design the test pyramid (the ratio of unit, integration, and end-to-end tests) for this team, and how would that be different at a 200-person company with a dedicated QA team?

Answer:

The right test mix isn't a universal ratio, it's a function of how much it costs the team to be wrong and how much time they have to spend on tests before shipping. **At a small startup, the priority is a thin, fast layer of unit tests plus a handful of E2E tests on the two or three flows that make the business money, because engineering time spent on tests directly competes with time spent finding product-market fit.**

Small startup, four engineers

- Heavy on fast unit and component tests for logic that's easy to get subtly wrong (form validation, pricing calculations, date handling).
- A small number (5 to 15) of E2E tests covering the critical path only: sign-up, checkout, the core action the product exists to perform.
- No dedicated QA, so every engineer owns the tests for the code they write, and code review is the main quality gate.
- Fast feedback matters more than completeness. A 3-minute CI (continuous integration, the automated pipeline that runs on every commit) run beats a 30-minute one that nobody waits for.

200-person org with dedicated QA

- Still unit-test business logic close to the code, but integration and E2E coverage can be much broader because there's a QA function to help maintain it and a release process that can absorb slower feedback.

- QA typically owns and extends the E2E suite and exploratory testing, while engineers stay responsible for unit and component-level tests, since QA can't scale to review every line of frontend logic.
- More room for a formal staging environment, a regression suite that runs before every release train, and manual exploratory testing for high-risk releases.

Why the ratio changes

- Team size changes who maintains the tests: a solo engineer can't maintain 300 E2E tests, but a QA team of five can.
- Release cadence changes what feedback loop is tolerable: weekly ad hoc deploys need sub-5-minute CI, a biweekly release train can tolerate an hour of regression testing.
- Cost of being wrong changes coverage depth: a payments flow needs more belt-and-suspenders coverage than an internal admin tool, regardless of company size.

Interview Answer:
"I'd anchor the ratio on two things: how fast we need to ship, and how expensive a bug in production actually is. At a four-person startup I'd keep the pyramid thin, fast unit tests for anything with real logic, and E2E coverage only on the two or three flows that make us money, because I don't have a QA team to carry a big suite and every hour spent on tests is an hour not spent finding product-market fit. At a 200-person company with a QA function, I'd let the E2E layer grow a lot wider since there's a team that can own and triage it, and I'd lean on a release train and staging environment instead of shipping every commit straight to prod. The ratio isn't a fixed number, it's a reflection of who maintains the tests and how much a mistake costs."

Q2. [Junior] You inherit a dashboard app with 200 React components, most of them fairly simple presentational pieces that just render props. The previous team wrote a unit test for almost every single one. Would you keep that practice going, and how do you decide what's actually worth testing in a UI-heavy codebase?

Answer:

Test coverage that doesn't correlate with real bugs is a liability, not a safety net, because it still has to be maintained every time the component's markup changes for a purely cosmetic reason. **Worth testing is a function of complexity and risk, not surface area, so a component that only renders `{props.title}` inside a `<div>` is a bad target for a unit test while a component with branching logic, formatting, or state is a good one.**

Low-value tests to stop writing

- Snapshot or render tests on purely presentational components with no logic, since they mostly just re-assert "the JSX I wrote is the JSX I wrote" and break on any styling change.
- Tests that assert internal implementation details (a specific CSS class name, a specific internal function call) rather than observable behavior.

High-value tests to keep writing

- Components with conditional rendering, calculated or formatted output (currency, dates, pluralization), or accessibility-critical markup.
- Custom hooks that encapsulate real logic, like a `useDebouncedSearch` hook or a hook that merges paginated API results.

- Anything that has already caused a production bug once, since that's a strong empirical signal it's error-prone.

```
// Low signal: testing that a dumb component renders its prop.
// A markup refactor breaks this test for no real reason.
test("renders title", () => {
  render(<CardTitle title="Hello" />);
  expect(screen.getByText("Hello")).toBeInTheDocument();
});

// High signal: testing behavior that can actually be wrong.
test("formats a negative balance in red with a minus sign", () => {
  render(<Balance cents={-4200} />);
  expect(screen.getByText("-$42.00")).toHaveClass("balance--negative");
});
```

The maintenance cost argument

- Every test is a liability that has to be updated when the code changes, so a low-signal test is a recurring tax with no corresponding safety benefit.
- A good heuristic: if you can't describe a realistic way the test would fail before the bug reaches production, it's probably not worth writing.

Interview Answer:

"I wouldn't keep testing every component just because the previous team did. I'd triage by asking what could actually go wrong: a component that only renders a prop into a div basically can't have a meaningful bug, so a test for it is just future maintenance work with no payoff. I'd focus effort on components with real logic, conditional rendering, formatting, or calculations, and on custom hooks that hold state or business rules. If a component already caused a production incident, that's a strong signal to add a test there specifically. The goal isn't maximum coverage, it's coverage that correlates with real risk."

Q3. [Junior] A teammate opens a PR that adds a unit test which mocks out every single dependency of the component and asserts that specific internal functions were called with specific arguments. What's the architectural problem with this test, and what would you ask them to do instead?

Answer:

A test like this is coupled to the implementation rather than the behavior, so it will break every time the component is refactored even if the user-facing behavior hasn't changed at all. **Tests should assert on what a user or caller can observe, like rendered output or emitted events, not on how the code internally achieves that output.**

Why over-mocking is a design smell

- If every dependency is mocked and every internal call is asserted, the test essentially re-states the implementation, which means it can't catch a bug where the implementation itself is wrong.
- It actively punishes safe refactors: renaming a helper function or restructuring internal logic breaks the test even though the component still behaves correctly.

What to ask for instead

- Test through the same interface a real consumer would use: render the component, interact with it via the accessible role or label, and assert on what appears on screen or what callback fired with what payload.

- Reserve mocking for true external boundaries (network calls, timers, browser APIs), not for the component's own internal collaborators.

```
// Fragile: encodes the implementation.
expect(formatCurrencyMock).toHaveBeenCalledWith(4200, "USD");
expect(internalSortHelper).toHaveBeenCalledTimes(1);

// Resilient: encodes the behavior.
render(<PriceList items={items} />);
expect(screen.getByText("$42.00")).toBeInTheDocument();
```

Interview Answer:

"The problem is that the test is really testing the implementation instead of the behavior. If they mock every dependency and assert on internal calls, the test will fail the moment someone refactors the component's internals, even if nothing the user sees actually changed. I'd ask them to rewrite it to interact with the component the way a real consumer would, render it, click or type into it, and assert on what's visible or what callback fired, and to only mock true external boundaries like a network request. That way the test protects behavior and doesn't become a tax on safe refactors."

Q4. [Junior] Your team ships a new marketing landing page and a completely separate internal admin tool used by five people on your own team. Would you set up visual regression testing (automated screenshot comparison to catch unintended UI changes) for both, one, or neither, and why?

Answer:

Visual regression testing has a real ongoing cost, since every intentional UI change requires reviewing and approving a new baseline screenshot, so it should be reserved for surfaces where a silent pixel-level regression is expensive and hard to catch another way. **The marketing landing page is a strong candidate because it's high-traffic, rarely changes, and a broken layout there directly costs conversions, while the internal admin tool used by five people is a poor candidate because it changes often and a broken pixel there is cheap to notice and fix.**

Where it earns its cost

- High-traffic, low-change-frequency surfaces: marketing pages, checkout, pricing pages, where a CSS regression could sit unnoticed and cost real revenue.
- Component libraries or design systems, where a shared button or input style change could silently ripple across dozens of consuming pages.

Where it's not worth it

- Internal tools with a small, technical user base that will immediately notice and report a visual bug.
- Pages that change layout frequently as part of normal iteration, since the baseline review overhead becomes constant busywork.

Best Choice

Set up visual regression testing on the marketing landing page and skip it for the internal admin tool.

Why?

The landing page has high traffic and low change frequency, so the signal-to-noise ratio of a screenshot diff is high and a caught regression is worth real money. The admin tool has a tiny, forgiving audience and changes often, so the same tooling would mostly generate baseline-approval busywork instead of catching real bugs.

Interview Answer:
"I'd set it up for the marketing page and skip it for the admin tool. Visual regression testing isn't free, someone has to review and approve a new baseline every time the UI intentionally changes, so it only pays for itself where a silent visual bug is expensive and where the page doesn't change shape very often. The landing page fits that: high traffic, rare changes, real revenue impact if a layout breaks. The admin tool is used by five people on our own team who'd notice and flag a visual bug in minutes, and it probably changes every sprint, so the same tooling would mostly just create baseline review noise."

Q5. [Junior] You're setting up CI for a new frontend repo. Design the gating strategy: what should actually block a pull request from merging, and what should just be a warning? How strict is too strict?

Answer:

Every gate you add has a false-positive cost paid by every future PR, so a gate should only block merges if it protects against something expensive and reliably detectable. **Blocking gates should be reserved for things that are objectively broken (failing tests, a build that doesn't compile, a type error), while subjective or noisy checks like code style nitpicks or coverage percentage deltas should be advisory, not blocking.**

What should block a merge

1. The build fails to compile or bundle.
2. Any existing automated test fails (unit, integration, or the small critical-path E2E suite).
3. A type-checking error (if using TypeScript).
4. A security or dependency vulnerability scan flags a critical issue.

What should be a warning, not a block

- Linting style issues that don't affect correctness (most of these should be auto-fixed by a formatter instead of surfaced as a gate at all).
- Bundle size increases, which are worth flagging to a human but shouldn't block a legitimate feature that needs a few more kilobytes.
- Code coverage percentage drops, since coverage percentage is a weak proxy for test quality and blocking on it encourages low-value tests written just to move the number.

How strict is too strict

- If engineers start describing the pipeline as something to "get around" rather than trust, it's too strict, or too flaky, or both.
- A pipeline that blocks on flaky or slow checks trains people to re-run CI instead of reading its output, which defeats the entire purpose of the gate.

Interview Answer:
"I'd only block on things that are unambiguously broken: the build failing, a test failing, or a type error. Those are cheap to verify and expensive to let through. Style nitpicks, bundle size changes, and coverage percentage moves I'd surface as visible warnings on the PR but not hard blocks, because they're judgment calls that a human should make, and blocking on something like coverage percentage just encourages people to write junk tests to satisfy the number. The strictness test I use is behavioral: if engineers start reflexively re-running a failed pipeline instead of reading why it failed, that's a sign the gate is either too strict or too flaky to trust, and both are worth fixing immediately."

Q6. [Junior] A checkout form has twelve different input validation rules (required fields, card number format, expiry date in the future, and so on). How would you decide which of those rules get a dedicated unit test versus relying on a broader integration test of the whole form?

Answer:

Individual validation rules are exactly the kind of pure, isolated logic that unit tests are best at, since each rule can be tested with a table of inputs and expected outputs without rendering anything. **Push each validation rule's logic into a small testable function and unit test it directly with a table of cases, then use one or two integration tests to confirm the form actually wires those rules to the right fields and error messages.**

Unit test the rules themselves

- Extract validation into pure functions (`isValidExpiryDate`, `isValidCardNumber`) so each can be tested with a matrix of valid, invalid, and edge-case inputs (expiry exactly this month, card number with a valid Luhn checksum but wrong length).
- This is fast, requires no rendering, and pinpoints exactly which rule broke if a test fails.

```
function isValidExpiryDate(month: number, year: number, now: Date): boolean {
  const expiry = new Date(year, month, 0);
  return expiry >= now;
}

test.each([
  [12, 2030, true],
  [1, 2020, false],
])("expiry %i/%i is valid: %s", (month, year, expected) => {
  expect(isValidExpiryDate(month, year, new Date("2026-01-01"))).toBe(expected);
});
```

Integration test the form's wiring

- One or two tests that fill out the whole form and confirm the right error message shows next to the right field, and that a fully valid submission actually calls the submit handler.
- This catches wiring bugs (rule attached to the wrong field, error message never rendered) that unit tests on the rules alone can't catch.

Interview Answer:
"I'd split it by what each type of test is actually good at. The twelve validation rules are pure logic, so I'd extract each one into its own small function and unit test it with a table of valid, invalid, and edge-case inputs, that's fast and tells me exactly which rule is broken. Then I'd add just one or two integration tests on the actual form to confirm the rules are wired to the right fields, the right error messages show up, and a fully valid submission actually fires. I wouldn't try to cover all twelve rules through the rendered form, that would be slow and any failure would be harder to diagnose."

Q7. [Junior] Your team's small startup has no dedicated QA function, five engineers, and ships to production multiple times a day. Who should own writing and maintaining tests, and how does that ownership model change once the company grows a real QA team?

Answer:

Ownership of testing should map to who can act on a failing test fastest, since a test nobody feels responsible for quickly becomes a test nobody trusts. **With no QA function, the engineer who writes a piece of code should also write its tests as part of the same unit of work, because they have the context and the fastest feedback loop; once a QA team exists, engineers keep owning unit and component tests close to the code while QA takes ownership of end-to-end and exploratory testing across features.**

Five-engineer startup, no QA

- Tests are part of "done," not a separate task, the engineer who builds the feature writes its tests in the same PR.
- Code review is the primary quality gate, since there's no second team downstream checking behavior.
- Ownership is diffuse by necessity, everyone can touch and fix any test, since the team is too small for specialization.

After a QA team exists

- Engineers still own unit and component tests, since QA can't realistically review frontend implementation details at that granularity.
- QA owns the E2E and exploratory testing layer, including cross-feature flows that no single engineer has full context on (does a change to the pricing page break the checkout flow's price display, for example).
- A clear escalation path matters here: if QA finds a bug, it should route back to the owning engineer or team, not sit in a shared "someone will get to it" backlog.

Interview Answer:

"At five engineers with no QA, ownership is simple: whoever writes the feature writes its tests, in the same PR, because they have the most context and the fastest feedback loop. Code review is really the only quality gate we have at that stage. Once there's a real QA team, I'd keep engineers owning unit and component tests close to their own code, since QA can't realistically dig into implementation details for every PR, but I'd hand E2E and exploratory testing across features to QA, because they're the ones with the cross-feature context to catch things like a pricing change silently breaking checkout. The key is making sure a bug QA finds routes back to a specific owner instead of landing in a shared backlog nobody feels responsible for."

Q8. [Mid] Your E2E suite has grown from 20 tests to 400 over the last year. It now takes 45 minutes to run and roughly 15% of runs fail due to flakiness rather than real bugs. Engineers have started ignoring red CI. How would you redesign the strategy to fix this?

Answer:

A 45-minute, 15%-flaky suite has stopped functioning as a signal, and once engineers learn to ignore red CI the suite is providing negative value, since it burns time and erodes trust in every other gate too. **The fix has three parts that need to happen together: parallelize the suite to cut wall-clock time, actively quarantine and fix flaky tests instead of tolerating them, and prune or demote tests that duplicate coverage other layers already provide.**

Cut wall-clock time

- Shard the suite across multiple parallel workers or machines, since most E2E runners (Playwright, Cypress) support this natively, splitting 400 tests across 10 workers can turn 45 minutes into under 10.

- Reserve full E2E runs for pre-merge on a subset of critical paths, and run the full 400-test suite on a schedule (nightly) or against a staging environment rather than on every single commit.

Fix flakiness instead of tolerating it

- Introduce a formal quarantine: a flaky test gets tagged and moved out of the blocking suite within 24 hours of being flagged, with an owner and a deadline to fix or delete it.
- Quarantining isn't free, a quarantined test provides zero coverage while parked, so it needs a hard SLA (service level agreement, an agreed time limit) or it just becomes a graveyard that grows forever.
- Common root causes to hunt for: unhandled async timing (asserting before a network call resolves), shared test data causing cross-test interference, and reliance on wall-clock sleeps instead of waiting for a specific condition.

```
// Flaky: race condition, assumes the request finishes in 500ms.
await page.click("#submit");
await page.waitForTimeout(500);
expect(await page.textContent("#status")).toBe("Success");

// Resilient: waits for the actual condition instead of a fixed delay.
await page.click("#submit");
await expect(page.locator("#status")).toHaveText("Success");
```

Prune redundant coverage

- Audit for E2E tests that are really re-testing logic a unit or integration test already covers faster and more reliably, and push that coverage down the pyramid.
- E2E should be reserved for things that can only be verified by exercising the real stack: navigation, real API integration, cross-page state.

Interview Answer:

"A 45-minute suite that's 15% flaky has already failed at its job, because engineers have learned to ignore it, which means real failures are getting ignored too. I'd attack it on three fronts at once. First, parallelize it, sharding 400 tests across ten workers should get us well under 10 minutes, and I'd move the full suite to a scheduled or staging run rather than blocking every commit on all 400 tests. Second, I'd set up a real quarantine process: a flaky test gets tagged out of the blocking path within a day, but with an owner and a deadline, so quarantine doesn't become a place tests go to die and silently stop providing coverage. Third, I'd audit for tests that are really just re-testing logic a unit test could cover in milliseconds, and push that coverage down the pyramid so E2E is reserved for things only a real browser and a real backend can verify."

Q9. [Mid] Design a policy for quarantining flaky tests: who can quarantine a test, what happens to it once it's quarantined, and what's the real risk of having this process at all?

Answer:

Quarantining exists to stop one unreliable test from blocking unrelated work, but it's a pressure release valve, not a fix, so the policy needs to actively fight the temptation to quarantine and forget. **A safe quarantine policy requires three things: a low bar to quarantine (any engineer can do it immediately when a flaky failure blocks their PR), a mandatory ticket with an owner and deadline, and a hard cap on how long or how many tests can sit in quarantine before it's treated as an incident.**

Who can quarantine, and how

- Any engineer should be able to quarantine a test the moment it blocks their unrelated PR, gated by a lightweight mechanism, like a `test.fixme` tag or a dedicated quarantine test suite, not a manual approval process, since requiring approval just means people merge past red CI instead.
- Quarantining must automatically file a ticket with an owner (usually whoever last touched that test or feature) and a target fix date, not just silently skip the test.

What happens to a quarantined test

- It stops blocking merges immediately but keeps running in a non-blocking, visible report, so the team can see it's still failing and hasn't been forgotten.
- It gets triaged within a set window (a week is a reasonable default) to determine if it's a real product bug, a test bug, or a genuinely flaky interaction that needs a different testing approach entirely.

The real risk

- Quarantine lists tend to only grow, since fixing a flaky test is unglamorous work that's easy to deprioritize against feature work, so without a hard cap and regular review, "temporary" quarantine becomes permanent and coverage quietly erodes.
- A useful forcing function: track quarantine list size as a visible team metric, and treat it crossing a threshold (say, more than 5% of the suite) as its own incident that pauses new feature E2E tests until it's worked down.

Interview Answer:
"I'd make quarantining easy and fast for the reporter, but expensive to leave unresolved. Any engineer should be able to tag a flaky test out of the blocking suite the moment it's in their way, no approval gate, because requiring approval just pushes people to merge past red CI instead. But that action has to automatically create a ticket with an owner and a deadline, and the test keeps running in a visible, non-blocking report so it's not forgotten. The real risk with any quarantine process is that it only grows, since fixing flaky tests isn't exciting work, so I'd track the quarantine list size as a team-level metric and treat it crossing a threshold, like 5% of the suite, as its own incident that has to get worked down before we add more."

Q10. [Mid] Your team is about to ship a full visual redesign of the main product dashboard, a change touching layout, colors, and component structure across the whole page. Design a canary and gradual rollout strategy using feature flags to ship this safely.

Answer:

A full redesign is exactly the kind of change where a binary "ship it" carries real risk, since visual and interaction regressions are often subjective and only surface once real users with real data and real habits touch the page. **Wrap the redesign behind a feature flag, roll it out to a small internal and beta cohort first, and progress the rollout percentage gated on both hard metrics (error rate, task completion, page load time) and qualitative feedback, with an instant kill switch at every stage.**

Staged rollout plan

1. Ship the redesign fully built behind a feature flag, defaulting to off, so the code lands in production without being visible to real users.
2. Enable it for internal employees first (dogfooding), catching obvious breakage with zero customer exposure.

3. Enable it for an opt-in beta cohort or 1 percent of users, chosen randomly to avoid selection bias, and watch for a fixed observation window (say, 48 hours) before increasing.
4. Ramp gradually, 1 percent to 5 percent to 25 percent to 100 percent, with an explicit go/no-go check at each step.

```
function useDashboardVariant(userId: string): "legacy" | "redesign" {
  const flag = featureFlags.get("dashboard-redesign", { userId });
  return flag.enabled ? "redesign" : "legacy";
}
```

What gates progression to the next stage

- Hard metrics: JavaScript error rate, Core Web Vitals (page load and interactivity metrics), and task completion rate (did users still complete the actions the dashboard supports) must stay within an agreed tolerance of the legacy baseline.
- Soft signals: support ticket volume and direct user feedback, since a redesign can be "technically working" and still be a usability regression that only shows up as complaints.
- A named decision-maker and a pre-agreed rollback threshold, decided before the rollout starts, not improvised under pressure when numbers look bad.

Why a flag instead of a big-bang release

- A flag makes rollback instantaneous (flip a value, no redeploy) versus a code revert and redeploy, which matters when the blast radius is the entire dashboard.
- It also lets you run the old and new versions side by side to compare real metrics directly, which a full cutover doesn't allow.

Interview Answer:

"I'd ship the whole redesign behind a feature flag so the code is in production but invisible until we choose to expose it. I'd start with internal dogfooding to catch obvious breakage, then move to a small random beta cohort, maybe 1 percent, and watch for 48 hours before increasing. At each step I'd gate progression on both hard metrics, error rate, page load, task completion versus the legacy baseline, and softer signals like support ticket volume, since a redesign can be technically fine and still be a usability regression that only shows up in complaints. The whole point of doing this behind a flag instead of a big-bang deploy is that rollback becomes an instant toggle instead of a revert and redeploy, which matters a lot when the blast radius is the entire dashboard."

Q11. [Mid] Your team is migrating a large app from Redux to a different state management library. This touches shared state used across dozens of components. Design the testing and rollout strategy so this migration doesn't introduce silent regressions.

Answer:

A state management migration is uniquely dangerous because the bugs it introduces are often not "the new code is broken" but "a component now reads slightly stale or slightly different data than before," which is easy to miss in manual testing and easy to catch with the right automated strategy. **Migrate incrementally behind a flag, feature-slice by feature-slice rather than all at once, and lean on integration and E2E tests that assert observable outcomes so they keep passing regardless of which state library is underneath.**

Before migrating any code

- Audit existing integration and E2E tests to make sure they assert on observable behavior (what's rendered, what an API call was made with) rather than on Redux-specific implementation details

(dispatched action types, store shape), since tests coupled to Redux internals will need a rewrite anyway and won't protect the migration.

- Add characterization tests, tests that pin down current behavior exactly as it is today, for any state-derived behavior that's under-tested, before touching the implementation.

Migrate incrementally

1. Pick the state library migration order by blast radius: start with an isolated, low-traffic slice of state (a settings panel, not global user auth state).
2. Run old and new state implementations side by side behind a flag where feasible, so a subset of users exercises the new store while most still use the old one.
3. Migrate slice by slice, verifying each slice's dependent components still pass their existing behavior-level tests before moving to the next.
4. Save the highest-risk, most-shared state (auth, cart, current user) for last, once the migration pattern has been proven on lower-risk slices.

Catching regressions

- Snapshot the output of key derived selectors under both implementations with the same input state and diff them, since subtle differences in memoization or update timing are exactly the kind of bug that's invisible in a quick manual check but easy to assert in a test.
- Keep the E2E suite's critical-path tests running against every migrated slice in CI, since those are what will actually catch a component silently receiving wrong data.

Interview Answer:

"The dangerous failure mode in a state migration isn't a crash, it's a component quietly reading slightly wrong or stale data, which is hard to catch by eye. So before touching any code, I'd make sure our integration and E2E tests assert on observable behavior, what's rendered, what's sent to the API, not on Redux-specific internals like dispatched actions, since those tests are what will actually catch a regression regardless of which library is underneath. Then I'd migrate incrementally, slice by slice, starting with low-traffic, isolated state like a settings panel and saving shared state like auth or cart for last, behind a flag so I can run old and new side by side. I'd also directly diff the output of key derived selectors under both implementations with the same inputs, because that catches subtle memoization or timing differences that are basically invisible in manual testing."

Q12. [Mid] Your frontend team consumes a checkout API owned by a separate backend team. Twice in the last quarter, a backend deploy silently changed a response shape and broke checkout in production before anyone noticed. Design a strategy using contract tests to prevent this.

Answer:

The core problem is that the frontend and backend teams are only discovering incompatibility in production, which means the actual contract between them exists only implicitly, in whatever each side currently happens to send and expect. **A contract test formalizes that shared expectation into an automated, versioned artifact that runs in both teams' CI pipelines, so a backend change that would break the frontend fails the backend's own build before it ever reaches production.**

How contract testing works here

- The frontend team defines, in code, the exact shape of requests it sends and responses it expects for each checkout endpoint (a "consumer contract").

- That contract is published (a tool like Pact does this, though the same idea works with a shared OpenAPI/JSON schema) and the backend team's CI verifies their actual API against it before every deploy.
- A break is caught in the backend team's own pipeline, before their code reaches any shared or production environment, rather than being caught by a frontend user seeing a broken checkout page.

```
// A consumer contract test: defines what the frontend expects from
// the checkout endpoint. If the real API stops matching this shape,
// the provider-side verification (run in the backend's CI) fails.
pact
  .given("a valid cart exists")
  .uponReceiving("a request for checkout summary")
  .withRequest({ method: "GET", path: "/checkout/summary" })
  .willRespondWith({
    status: 200,
    body: {
      totalCents: matchers.integer(4200),
      currency: matchers.string("USD"),
      items: matchers.eachLike({ sku: matchers.string("SKU-1"), qty:
matchers.integer(1) }),
    },
  });
```

Where contract tests fit relative to other tests

- They don't replace E2E tests against a real staging environment, they catch a narrower but earlier class of bug: shape and type mismatches, missing or renamed fields, changed status codes.
- They're cheap to run (no real network calls, no shared environment needed) so they can run on every commit in both teams' pipelines, unlike a full E2E suite against a live staging backend.

Interview Answer:

"The real issue is that the contract between our frontend and their checkout API only exists implicitly, in whatever each side currently sends and expects, so nobody finds out it's broken until a user does. I'd introduce contract tests, using something like Pact, where we define exactly what shape of request and response we expect for each checkout endpoint, and that contract gets published and verified against the real API in the backend team's own CI pipeline. That means a backend change that would break us fails their build before it ever reaches production, instead of us finding out from a broken checkout page. It's not a replacement for E2E tests against staging, it's a much earlier and cheaper tripwire for exactly the class of bug that bit us twice, a silently changed response shape."

Q13. [Mid] Your product needs to work across many combinations of browser, device, and network condition. You can't realistically test every combination. Design a strategy for deciding what actually gets tested.

Answer:

The combination space is infinite if you multiply browsers by devices by network speeds by OS versions, so the strategy has to be about prioritization by actual risk and real usage data, not attempted exhaustiveness. **Use real analytics on your actual user base to define a small tiered matrix of "must fully support," "must not crash on," and "best effort," and reserve automated cross-browser/device runs for the top tier while relying on responsive design principles and graceful degradation for the rest.**

Build the matrix from data, not guesses

- Pull real browser, device, and connection-speed analytics for your actual user base rather than assuming, a product with a large India or Southeast Asia user base has very different real-world network conditions than a US enterprise SaaS tool.
- Rank combinations into tiers: Tier 1 (must be fully tested and pixel/behavior verified, usually the top two browsers times the top few device classes), Tier 2 (must not crash or lose core functionality, spot-checked periodically), Tier 3 (best effort, no dedicated testing, relies on general responsive/progressive-enhancement practices).

What to automate versus what to skip

- Automate E2E runs for Tier 1 combinations using a cloud device/browser grid (BrowserStack, Sauce Labs, or Playwright's built-in multi-browser support), since this is the coverage worth paying to keep current on every release.
- For network conditions specifically, throttle a representative slow connection (simulate 3G-equivalent) in CI for the critical path rather than trying to test every bandwidth tier, since the failure modes (timeouts, spinners, retry logic) are usually similar across "slow" conditions.
- For Tier 2 and 3, invest in resilient patterns instead of test coverage: feature detection instead of browser sniffing, responsive layout instead of device-specific layouts, and graceful fallback for unsupported APIs.

Interview Answer:

"I wouldn't try to cover the full matrix, it's infinite once you multiply browsers, devices, and network speeds. I'd start from real analytics on our actual users, not assumptions, and build a tiered matrix: a small Tier 1 of browser and device combinations that we fully automate and verify on every release using a cloud device grid, a Tier 2 that we spot-check periodically and just need to not crash on, and a Tier 3 that's best effort, covered by general responsive design and feature detection rather than dedicated tests. For network conditions, instead of testing every bandwidth tier, I'd throttle to a representative slow connection in CI for the critical path, since the failure modes, timeouts and retry behavior, tend to be similar across anything in the 'slow' category."

Q14. [Mid] A production bug slips through and gets fixed. How do you decide whether that specific bug is worth writing an automated regression test for, versus just fixing it and moving on?

Answer:

Writing a regression test for every single bug sounds responsible but isn't free, since each one adds to the ongoing maintenance surface of the suite, so the decision should weigh how likely the bug is to recur against how expensive it would be if it did. **Write a regression test when the bug was in logic that's likely to be touched again, expensive if it recurs, or subtle enough that it wouldn't be caught by casual manual testing; skip it for one-off typos, environment-specific issues, or bugs in code that's about to be deleted or rewritten anyway.**

Worth a regression test

- The bug was in logic that gets modified often (pricing rules, date/timezone handling, permission checks), since it's likely to be perturbed again by a future unrelated change.
- The bug was expensive: it affected revenue, security, or a large share of users, so the cost of it recurring justifies a small ongoing maintenance cost.

- The bug was subtle, meaning a reasonable engineer reviewing the code or clicking through the feature manually wouldn't have caught it, since that's exactly the class of bug automated tests exist to catch.

Not worth a regression test

- A one-off typo or copy-paste mistake with no underlying logic error, fixing it is the whole fix.
- A bug in an environment-specific edge case unlikely to recur (a config issue in one deployment, not a code defect).
- A bug in code that's slated for deletion or a rewrite in the near term, where the test would be thrown away shortly after being written.

Interview Answer:
"I treat it as a cost-benefit call, not an automatic rule. If the bug was in logic that gets touched often, like pricing or date handling, or if it was expensive, revenue-impacting or affecting a lot of users, or if it was subtle enough that a normal manual check wouldn't have caught it, I'll add a regression test, because the odds of a repeat and the cost of a repeat both justify it. If it was a one-off typo, or something environment-specific that's unlikely to recur, or in code we're about to rewrite anyway, I'll just fix it and move on, since the test would just be added maintenance weight with no real payoff."

Q15. [Senior] Design an accessibility testing strategy that's built into a team's normal test suite and CI pipeline, rather than a one-time manual audit done before a big launch.

Answer:

A one-time audit finds a snapshot of problems and then the codebase drifts away from that baseline the next day, so accessibility (a11y, shorthand for making software usable by people with disabilities) needs the same continuous, automated backstop as functional correctness rather than a periodic manual sweep. **Combine automated static and rendered-DOM accessibility checks running in CI on every PR, targeted keyboard-navigation assertions in the existing component and E2E tests, and a recurring manual audit with assistive technology (screen readers) for the highest-traffic flows, since automated tools alone catch maybe a third of real accessibility issues.**

Automated layer, in CI on every PR

- Run an automated accessibility linter (like `eslint-plugin-jsx-a11y`) at the code-authoring stage to catch obvious issues (missing alt text, a click handler on a non-interactive `div`) before a PR is even opened.
- Run an automated rendered-DOM audit (axe-core, which checks the actual rendered accessibility tree) as part of component or E2E tests, so issues introduced by dynamic behavior, not just static markup, get caught too.

```
import { axe } from "jest-axe";

test("checkout summary has no automated a11y violations", async () => {
  const { container } = render(<CheckoutSummary items={items} />);
  const results = await axe(container);
  expect(results).toHaveNoViolations();
});
```

Behavioral layer, folded into existing tests

- Add specific keyboard-navigation and focus-management assertions into existing component/E2E tests for interactive elements (does a modal trap focus and return it on close, can a dropdown be

operated with only a keyboard), since this is the class of bug automated scanners are worst at catching.

- Treat this as an extension of existing test suites rather than a separate a11y-only suite, so it's maintained with the same rigor as functional tests instead of being the first thing cut under deadline pressure.

Manual layer, recurring, not one-time

- Automated tools (axe, Lighthouse) reliably catch roughly a third of WCAG (Web Content Accessibility Guidelines) violations; the rest, like whether a screen reader announces something in a genuinely usable order, require an actual human with assistive technology.
- Schedule recurring manual audits, quarterly for the whole app, and mandatory before launch for any redesigned critical flow, with real screen reader testing (VoiceOver, NVDA), not just automated tooling.

Why not gate merges on 100 percent automated a11y pass

- Automated tools do produce false positives, so a hard block on zero violations trains engineers to suppress warnings rather than fix them; a better gate is blocking on new violations introduced by that PR specifically, with existing debt tracked and worked down separately.

Interview Answer:

"I wouldn't treat accessibility as a pre-launch audit, because the moment that audit finishes, the codebase starts drifting away from it. I'd build it into the normal pipeline in layers. At the linting layer, something like eslint-plugin-jsx-a11y catches obvious static issues before a PR is even opened. In component and E2E tests, I'd run an automated rendered-DOM check like axe-core, and more importantly add real keyboard-navigation and focus-management assertions for interactive elements like modals and dropdowns, since that's exactly what automated scanners tend to miss. And because automated tools only catch roughly a third of real accessibility issues, I'd keep a recurring manual audit with actual screen readers, quarterly for the whole app and mandatory for any newly redesigned critical flow. I'd gate CI on new violations introduced by a given PR, not on a blanket zero, since a hard zero-tolerance gate mostly just trains people to suppress warnings instead of fixing them."

Q16. [Senior] You're designing the CI pipeline gating strategy for a frontend monorepo shared by 30 teams. Different teams have different risk tolerances and release cadences. Design a tiered gating strategy that avoids both under-protecting critical code and turning CI into an organization-wide bottleneck.

Answer:

A single flat gating policy across 30 teams either under-protects the highest-risk code paths or over-blocks low-risk ones, since "one size fits all" ends up being wrong for almost everyone. **Tier the gating strategy by the blast radius of the code being touched, ownership-critical shared packages get the strictest, most comprehensive gates, while a team's own isolated feature code gets a lighter, team-owned gate, with a shared, fast baseline (build, type check, lint-as-warning) applying to everyone.**

The tiering model

1. **Shared foundation, every PR, fast and strict**: build succeeds, type checks pass, unit tests for changed packages pass. This should complete in a few minutes and applies uniformly, since a broken build blocks everyone downstream regardless of team.

2. **Shared/critical packages (design system, auth, checkout, payments)**: additional mandatory gates, wider test coverage requirements, mandatory review from the owning team (via a CODEOWNERS-style rule), and a broader E2E run against consumers of that package, since a change here has organization-wide blast radius.
3. **Team-owned feature code**: the shared baseline plus whatever additional gates that specific team chooses to configure for their own package, since they best understand their own risk tolerance and shouldn't be forced into someone else's strictness level or slowed by someone else's slow tests.

Avoiding the bottleneck

- Run gates only against the parts of the monorepo actually affected by a given PR (affected-package detection via a tool like Nx or Turborepo), not the entire monorepo on every commit, or CI time scales with monorepo size instead of change size.
- Parallelize aggressively and cache aggressively (dependency installs, build outputs) so the fast baseline genuinely stays fast as the repo grows.
- Give each team ownership of their own gate configuration above the shared baseline, so a slow or strict gate one team wants doesn't become mandatory for all 29 other teams.

Avoiding under-protection

- Make the "critical package" designation an explicit, reviewed list, not an assumption, and require the platform or architecture team to sign off when a package graduates into that tier.
- Track and alert on production incidents by which gate tier the responsible code was in; if critical-tier code keeps causing incidents, the gate for that tier needs to get stricter, and if team-tier code never causes incidents despite lighter gating, that's a sign the tiering is calibrated correctly.

Interview Answer:

"With 30 teams, a flat policy is wrong for almost everyone, either too strict for low-risk feature work or too loose for genuinely shared, critical code. I'd tier it by blast radius. Everyone gets a fast, strict shared baseline on every PR, build, type check, unit tests for what changed, using affected-package detection so CI scales with the size of the change, not the size of the monorepo. Shared critical packages, things like the design system, auth, or checkout, get an additional tier: mandatory owning-team review, broader test requirements, and a wider E2E run against known consumers, because a bug there has organization-wide blast radius. Everything else is a team's own feature code, where they get the shared baseline plus whatever extra gates they choose for themselves, since they understand their own risk tolerance and shouldn't be slowed by another team's strictness. I'd also explicitly track which gate tier production incidents come from, so the tiering stays calibrated by actual evidence instead of guesswork."

Q17. [Senior] Your team is migrating a critical checkout flow, the single highest-revenue path in the product, to a new frontend architecture, and leadership has told you there is zero tolerance for regressions. Design the complete testing and release strategy from before the migration starts to full rollout.

Answer:

"Zero tolerance for regressions" isn't achieved by writing more tests at the end, it's achieved by making regressions structurally hard to ship and trivially easy to detect and reverse if one slips through anyway. **The strategy needs four layers working together: a locked-down characterization of current behavior before touching any code, an incremental migration behind a flag with real production traffic mirrored for comparison, staged rollout gated on hard business metrics with an instant kill switch, and a rollback plan rehearsed before it's ever needed.**

Phase 1: Before writing any migration code

- Write characterization tests (tests that pin down exactly what the current system does today, bugs and all) across every state the checkout flow can be in: empty cart, out-of-stock item, expired promo code, saved versus new payment method, every supported payment type, every currency and tax jurisdiction the product supports.
- Instrument the current checkout flow with detailed business metrics if not already present: conversion rate by step, error rate by step, average time to complete, so there's a real baseline to compare the new implementation against, not just a gut feeling of "it seems fine."
- Freeze the current checkout's feature scope for the duration of the migration where possible, so the team isn't trying to hit a moving target.

Phase 2: Build the new architecture behind a flag

- Build the new checkout flow fully behind a feature flag, unreachable by default, and run the same characterization test suite against it before it ever sees traffic.
- Where feasible, use shadow traffic: real production checkout requests are processed by both the old and new implementations, but only the old implementation's result is shown to the user, with any divergence between the two logged and triaged. This surfaces real-world edge cases no test suite anticipated, without any user-facing risk.

Phase 3: Staged, metric-gated rollout

1. Internal employees only, verifying no crashes across the team's own real payment methods.
2. A tiny, randomly selected user percentage (0.1 to 1 percent), watched closely for a fixed window against the Phase 1 baseline metrics (conversion rate, error rate, completion time), not just "did it crash."
3. Gradual ramp with an explicit go/no-go decision and named decision-maker at each step, and a pre-committed automatic rollback trigger (for example, conversion rate drops more than 2 percent relative to baseline, checkout error rate exceeds a set threshold) so the decision to roll back isn't made under pressure in the moment.
4. Full rollout only after sustained parity with baseline metrics across a full business cycle (including weekend and peak traffic patterns, which often behave differently than weekday traffic).

Phase 4: Rollback and safety net

- The feature flag itself is the rollback mechanism, flipping it should take seconds, not a deploy, and this path should be tested and rehearsed before it's ever needed under real pressure.
- Keep the old implementation deployable and functional for a defined window after full rollout (weeks, not days) rather than deleting it immediately, since a low-frequency edge case might only surface after the new code has been at 100 percent for a while.
- Real-time alerting on the same business metrics used to gate rollout, so a regression that slips past the staged rollout is caught within minutes rather than discovered in a weekly business review.

Interview Answer:

"Zero tolerance for regressions means the strategy has to make regressions hard to ship in the first place and fast to catch and reverse if one gets through anyway, not just 'write a lot of tests.' I'd start before writing any new code by characterizing exactly what the current checkout does today across every state, every payment method, every edge case, and instrumenting real business metrics as a baseline. I'd build the new architecture fully behind a flag and, where possible, run it as shadow traffic against real production requests, comparing outputs to the old system without ever showing a user the new result, which surfaces edge cases no test suite would think of. Rollout would be staged, internal first, then a tiny random

percentage, ramping gradually with a pre-committed automatic rollback trigger tied to the same business metrics, conversion rate, error rate, so the decision to pull back isn't made under pressure in the moment. And I'd keep the old implementation live and deployable for weeks after full rollout, with real-time alerting on those same metrics, because at this level of criticality, catching a regression in minutes instead of a week is the difference that actually matters."

Q18. [Senior] Your organization's E2E suite has grown organically over several years. It's owned by nobody in particular, and PR merges are routinely blocked by flaky failures in tests unrelated to the change being made. Redesign the ownership and gating model to fix this at a structural level.

Answer:

An E2E suite with no owner degrades in exactly this predictable way, since anyone can add a test but nobody feels responsible for fixing one that breaks, so the suite becomes a shared tragedy of the commons. **Fix it structurally by assigning every E2E test explicit ownership tied to the feature area it covers (usually the team that owns that part of the product), decoupling a PR's gating from unrelated tests via change-based test selection, and instituting a standing budget and review cadence for suite health so it can't silently rot again.**

Fix the ownership gap

- Every E2E test file gets a mandatory owner annotation tied to a specific team, enforced by a CI check that rejects new E2E tests without one, so the "nobody owns this" state can't be reintroduced going forward.
- Retroactively assign existing tests to owning teams based on the feature area they cover; tests that can't be confidently assigned to any team are a strong signal they're testing something too broad or stale to justify existing, and are candidates for deletion.
- When a test in a team's owned area goes flaky, it's that team's responsibility to fix or quarantine it within an agreed SLA, not a shared "someone" responsibility.

Fix the gating coupling

- Use change-based test selection: a given PR's merge gate runs only the E2E tests relevant to the code paths it actually touches (via dependency graph analysis or tagged test-to-feature mappings), not the entire suite.
- Run the full suite on a schedule (nightly, or on merge to main) rather than blocking every individual PR on every test, so an unrelated flaky test in another feature area can't hold up unrelated work; failures from the scheduled full run route to the owning team, not to whichever engineer happened to be merging at the time.

Prevent it from rotting again

- Give the suite an explicit health owner (a platform or DX team, or a rotating on-call role) responsible for suite-wide metrics: total runtime, flake rate, quarantine list size, reported on a recurring cadence to engineering leadership.
- Treat suite health metrics crossing an agreed threshold as an incident with the same seriousness as a production incident, since an untrustworthy test suite has the same organizational cost as an outage, just spread out and less visible.

Interview Answer:

"The root cause here isn't the flaky tests themselves, it's that nobody owns them, so nobody's incentivized to fix them, and the suite degrades in exactly the way you'd expect from a shared resource with no owner. I'd fix it structurally in two moves. First, every E2E test gets a mandatory owning team, enforced by CI for new tests and retroactively assigned for existing ones, and if a test can't be confidently owned by anyone it's a signal it's testing something too broad or stale and should probably be deleted. Second, I'd decouple gating from the whole suite: a given PR should only run the E2E tests relevant to what it actually touched, using change-based test selection, with the full suite running on a schedule instead of blocking every PR, and failures from that scheduled run routing to the owning team, not whoever happened to be merging. On top of that I'd put a standing health owner and visible metrics, runtime, flake rate, quarantine size, so this can't quietly rot back into the same state a year from now."

Q19. [Senior] Your product has grown to thousands of components across dozens of teams, and visual regression testing, once useful, now generates hundreds of screenshot diffs per week that mostly get rubber-stamped approved without real review. Design a strategy to fix this before it silently stops catching real regressions.

Answer:

A visual regression suite that generates more diffs than a team can meaningfully review has already stopped working, since rubber-stamped approvals mean a real regression buried among the noise will get approved right along with everything else. **The fix is to shrink the surface area to what actually matters, snapshot at the component-library level rather than every page composition, and route diffs to the specific team that owns the affected component instead of a shared review queue that nobody feels ownership over.**

Diagnose why the noise happened

- At thousands of components, snapshotting every page composition means any shared component change (a design system button, a spacing token) generates diffs across every page that uses it, most of which are the intended, correct outcome of that change, not a regression.
- A shared review queue for all diffs, owned by nobody specifically, is exactly the setup that produces rubber-stamping, since the reviewer often lacks context on whether a given page's diff is expected.

Restructure what gets snapshotted

- Move the bulk of visual regression coverage to the component-library level (each component in isolation, in a tool like Storybook), where a diff has one unambiguous cause, and drastically reduce full-page snapshot coverage to only the small set of pages identified earlier as genuinely high-value (Q4's logic: high traffic, low change frequency, high cost of a silent break).
- This means a design system token change produces one diff at the component level instead of two hundred diffs across every consuming page, restoring signal.

Fix the review routing

- Route each diff to the team that owns the component or page it affects, not a shared queue, so the reviewer actually has context to judge "is this the intended outcome of the change I just made."
- Track and report the approval rate and review latency per team; a team with a near-100 percent instant-approve rate on their diffs is a signal the tool isn't providing them real signal and either needs recalibration (tighter pixel-diff thresholds) or should be turned off for that surface entirely.

Interview Answer:

"A visual regression suite generating hundreds of diffs a week that get rubber-stamped has already stopped catching real regressions, because a real one is now buried in noise that nobody's actually looking at. The core problem is usually that we're snapshotting at the wrong layer, full page compositions instead of individual components, so one shared design system change fans out into hundreds of diffs across every page that uses it, almost all of which are correct and expected. I'd move the bulk of coverage down to the component level in isolation, where a diff has one clear cause, and keep full-page snapshots only for the small set of pages that are genuinely high-value, high traffic, low change frequency, expensive if broken. I'd also route diffs to the specific team that owns the affected component instead of a shared queue, since a reviewer with real context on the change is far less likely to rubber-stamp something they shouldn't. And I'd actively track approval rates per team, because a team that's approving 100 percent of their diffs instantly is telling us the tool isn't giving them real signal anymore."

Q20. [Senior] Midway through a gradual feature-flag rollout of a risky change, the feature flag service itself has an outage and starts returning errors or stale values for every flag check in your app. Design the release architecture so this failure mode doesn't turn into a full production incident, and explain how you'd have tested for it beforehand.

Answer:

A feature flag system that's a single point of failure for every flag check has quietly become as critical as the app's own backend, which means its own outage needs to be planned for with the same seriousness as a database outage. **The architecture needs a safe, explicit default baked into the client for every flag (fail to the known-safe, already-proven variant, not to an undefined or random state), a short client-side cache so a transient flag-service blip doesn't immediately affect every request, and this exact failure mode needs to be part of the standard pre-launch test plan, not discovered live.**

Architectural safeguards, decided before any rollout

- Every flag check must have an explicit, hardcoded fallback value baked into the client bundle, and that fallback should always be the safe, already-battle-tested variant (the old checkout, the old dashboard), never the new risky one, so a flag-service outage fails toward safety rather than toward risk.
- Cache the last known-good flag evaluation on the client for a short window (seconds to low minutes depending on the product), so a brief flag-service blip doesn't cause every single request to fall back simultaneously, which would look like the rollout percentage suddenly changing for everyone at once.
- Treat the flag service itself as a dependency with its own uptime and latency SLOs (service level objectives), monitored the same way any other critical backend dependency would be.

```
function useDashboardVariant(userId: string): "legacy" | "redesign" {
  try {
    const flag = featureFlags.get("dashboard-redesign", { userId, timeoutMs: 200 });
    return flag.enabled ? "redesign" : "legacy";
  } catch {
    // Explicit, safe fallback: never fail open into the risky variant.
    return "legacy";
  }
}
```

How this gets tested before it ever happens live

- As part of the pre-launch test plan for any flagged rollout, explicitly simulate the flag service being unreachable, slow, and returning malformed data, in a staging environment, and verify the app falls

back to the safe variant in each case, not just the happy path where the flag service responds correctly.

- Include this scenario in load and chaos testing (deliberately injecting failures to observe system behavior) for any migration flagged as high-risk, treating "what happens when the flag service itself fails" as a required pre-launch question with a documented answer, not an assumption.

Interview Answer:

"Once a feature flag service is gating a risky rollout, it's become a critical dependency in its own right, so its own outage needs a designed answer, not an improvised one. Architecturally, every flag check in the client needs a hardcoded, explicit fallback, and that fallback should always be the safe, already-proven variant, never the new risky one, so an outage fails toward safety. I'd also cache the last known-good evaluation client-side for a short window so a brief blip doesn't cause every user's experience to flip at once. And critically, this isn't something to discover during a live incident, it should be part of the standard pre-launch test plan for any flagged rollout: explicitly simulate the flag service being down or returning garbage in staging, and verify the app falls back correctly, the same way you'd chaos-test a database outage for a normal backend dependency."

If you remember nothing else from this chapter: *a test suite you trust blindly is one failure away from becoming a green checkmark that means nothing.*

Chapter 15: Frontend Production, Monitoring & Deployment

This chapter is a quick, deliberately short check on whether you've actually operated a frontend in production, not just built one in a sandbox and handed it off. If you've only ever worked in a dev environment, you'll tend to talk about "shipping code" as the finish line; if you've been on-call for a frontend, you'll immediately talk about what happens to that code an hour, a day, or a week after it goes live. Use these questions to build that instinct.

> **War Story:** *A gradual rollout that looked healthy on every aggregate dashboard was actually failing completely for one specific, unpopular browser version, invisible until someone finally sliced the error rate by browser instead of just looking at the average.*

> **Avoid:** only looking at aggregate metrics and never slicing error rates by browser, device, or region
> **Aim For:** instinctively asking to slice a metric by segment before declaring a rollout healthy

Q1. [Junior] What's the difference between real user monitoring and synthetic monitoring, and why would you want both running on the same site?

Answer:

Real user monitoring (RUM) collects performance data straight from actual visitors' browsers as they use the site, while synthetic monitoring runs a scripted page load on a schedule from a controlled machine, whether or not any real visitor shows up. **RUM tells you what real users on real devices and networks actually experienced, synthetic monitoring gives you a repeatable baseline you can alert on even during a quiet traffic period.**

Real User Monitoring (RUM)

- Captures Core Web Vitals (Google's named metrics for perceived speed, like LCP for how long the main content takes to render, and CLS for how much the page visually jumps around) from a script running in production, then beacons them to an analytics backend.
- Surfaces variance you'd never see from a single test machine, for example a page that's fast on a fiber connection in an office but takes eight seconds to load on a three-year-old Android phone on a spotty 3G connection in a different country.
- Needs enough traffic to be statistically meaningful, a low-traffic internal admin tool won't generate useful RUM data.

Synthetic Monitoring

- Runs the same scripted check (often via a tool like Lighthouse CI or a scheduled Pingdom/Datadog check) every few minutes regardless of real traffic, so it catches a regression on a rarely visited page before a real user ever hits it.
- Removes real-world noise, which makes it good for pass/fail alerting, but it can't tell you what's happening on that old Android phone because it's testing from a controlled environment.

Best Choice

Run both, RUM for understanding real-world impact and prioritizing fixes, synthetic checks for fast, noise-free alerting right after a deploy.

Why?

Relying only on synthetic monitoring means you can be green on your dashboard while a meaningful slice of real users are having a bad experience, and relying only on RUM means you might not notice a regression for hours if traffic to the affected page is low.

Interview Answer:

"RUM is data collected from actual users in the wild, it tells me what real people on real devices and networks experienced, while synthetic monitoring is a scripted check that runs on a timer from a controlled location regardless of whether anyone's visiting. I'd want both: synthetic checks give me a fast, low-noise signal right after a deploy, and RUM tells me the real-world story, like discovering a page is fine on office wifi but painfully slow on a mid-range Android phone. Core Web Vitals like LCP and CLS are the metrics I'd track in both."

Q2. [Junior] Why would you add a tool like Sentry to a production frontend app instead of just relying on `console.error`?

Answer:

A JavaScript error thrown in a user's browser three time zones away doesn't show up anywhere unless something is actively watching for it. **Without client-side error tracking, you typically find out about production bugs from a support ticket hours or days later, instead of from a dashboard within seconds.**

What It Actually Does

- Hooks into global error events (`window.onerror` and `unhandledrejection` for uncaught promise rejections) and reports the stack trace, browser, OS, and a trail of recent actions (breadcrumbs) to a central dashboard.
- Groups identical errors together so you can see "this exact null reference happened 4,000 times in the last hour" instead of drowning in individual reports.
- Lets you tag errors with the release version and user context, so you can tell whether a spike is new or has been happening quietly for weeks.

Interview Answer:

"`console.error` only helps if someone's got dev tools open watching that exact user's browser, which never happens in production. A tool like Sentry or Bugsnag hooks the global error and unhandled promise rejection handlers, ships the stack trace and some context like browser and recent user actions to a dashboard, and groups identical errors together. That means I find out about a bug from a spike on a dashboard within minutes, tagged to the exact release that caused it, instead of from an angry support ticket two days later."

Q3. [Junior] Your production JS is minified and bundled, so an error report just shows `bundle.js:1:48213`. How do you get back to a readable stack trace, and why can't you just serve that mapping file publicly?

Answer:

Minification (shrinking code by renaming variables and stripping whitespace) and bundling (concatenating many source files into one) make production code fast to load but unreadable in a stack trace. A source map

is a file that records exactly how each minified line and column maps back to the original source file, function, and line number. **Source maps get uploaded privately to your error-tracking provider as part of the build so it can de-minify stack traces after the fact, they generally should not be served publicly from your web server.**

Why Not Serve Them Publicly

- A public source map hands out your original, unminified source, including any comments or internal logic you didn't intend to expose.
- Most teams wire source map upload into the CI/CD pipeline (for example, the Sentry webpack plugin or CLI) so the map lands directly on the error-tracking provider's servers and is deleted or excluded from the public build output.

Interview Answer:
"Minified code turns every stack trace into meaningless line and column numbers in one giant bundle file, so I'd generate source maps at build time and upload them straight to the error tracking provider, usually as a CI step using something like the Sentry CLI or webpack plugin, rather than deploying the map files to my public CDN. That way the tool can de-minify a stack trace into the real file and line when an error comes in, but a random visitor can't just download my unminified source by requesting `bundle.js.map`."

Q4. [Junior] What's a feature flag, and how does it let you roll a new feature out gradually instead of turning it on for everyone at once?

Answer:

A feature flag is a runtime toggle, backed by a config service rather than a code branch, that decides whether a given user sees a feature. **Feature flags decouple deploying code from releasing a feature, so the code can already be live in production while almost nobody is actually seeing it yet.**

Gradual Rollout In Practice

- Ship the new feature behind a flag that defaults to off, so the code is deployed but dormant.
- Turn it on for a small percentage of users first, for example 1%, and watch error rates and key metrics before increasing that percentage.
- Ramp up over hours or days, 1% to 10% to 50% to 100%, with the option to pause or reverse at any step.

Interview Answer:
"A feature flag is basically an `if` check whose condition lives in a remote config service instead of the codebase, tools like LaunchDarkly or Unleash, or even a simple homegrown JSON config, let you flip it without a deploy. That means I can merge and deploy a risky feature while it's still off for everyone, then gradually ramp it up, say 1% of users, then 10%, then 100%, watching error rates at each step. It separates 'this code is deployed' from 'users can actually see this', which is a really important distinction operationally."

Q5. [Junior] How would you handle configuration, like API URLs and analytics keys, differently across dev, staging, and production for a frontend app?

Answer:

Each environment needs different values, a dev API running on localhost, a staging API pointed at a test database, a production API with real customer data, plus different analytics or third-party keys per

environment. **Environment-specific values should be injected at build or runtime through environment variables or a config endpoint, never hardcoded into the source.**

Common Approaches

- Build-time injection: tools like Vite or Webpack read `.env.development` / `.env.production` files and bake the right values into the bundle for that build.
- Runtime config: the app fetches a small `/config` JSON on startup so the exact same build artifact can run in staging or production, just pointed at a different backend, which avoids rebuilding per environment.
- Secrets (real API keys, not public client IDs) still shouldn't live in frontend env files at all, since anything bundled into client JS is visible to anyone who opens dev tools.

Interview Answer:

"I'd keep environment-specific values, like API base URLs and analytics keys, out of the source code entirely, either injected at build time through something like Vite's `.env.production` files, or fetched at runtime from a small config endpoint so the same build artifact works in staging and production. The runtime approach is nicer because you're not rebuilding per environment, you're just pointing the same artifact at a different config. Either way, anything that's a real secret shouldn't be in frontend config at all, since it ends up visible in the shipped JS."

Q6. [Mid] You just shipped a new checkout widget behind a feature flag and errors are spiking. Walk me through turning it off without doing a redeploy.

Answer:

This is exactly the scenario feature flags exist for. You flip the flag in the flag service's dashboard or API, and connected clients pick up the change, typically within seconds via a streaming connection or within the next polling interval, without touching the build pipeline. **A kill switch only works if the flag check wraps the whole broken feature at a coarse enough boundary, flipping it needs to fully remove the bad code path, not just hide its visible output.**

Making The Kill Switch Actually Work

- Wrap the entire new checkout widget, including whatever network calls it fires, behind the flag, not just its rendering, otherwise a hidden component can still throw or fire bad requests in the background.
- Design the flag check to fail safe: if the flag service is unreachable, default to the old, known-good behavior rather than the new feature.
- Confirm the rollback actually reached clients by watching the error rate drop in the monitoring dashboard, don't just assume the flip worked.

Interview Answer:

"Since it's behind a feature flag, I'd flip it off through the flag service's dashboard or API rather than touching code, and most flag SDKs push that change to connected clients within seconds through a streaming or fast-polling connection. The important design detail is that the flag needs to wrap the entire feature, including any network calls it makes, not just the visible UI, otherwise you can flip it off and still have background errors firing. I'd also make sure the flag defaults to the safe, old behavior if the flag service itself is unreachable, and then watch the error dashboard to confirm the rate actually drops before considering it resolved."

Q7. [Mid] You're running an A/B test on a redesigned pricing page. How do you make sure a user who's bucketed into Variant B keeps seeing Variant B every time they come back?

Answer:

Bucketing has to be deterministic, not re-rolled on every page load. **A common approach is to hash a stable identifier, like a user ID or a persistent device ID, together with the experiment's key, then map that hash into a bucket range, so the same input always produces the same bucket.**

How It Works

- Compute something like `hash(userId + experimentKey) % 100`, then map the result into ranges, for example 0-49 is Variant A and 50-99 is Variant B.
- Persist the assignment, either server-side against the user's account or in a long-lived cookie/localStorage entry for anonymous visitors, so a page refresh or a new session doesn't re-bucket them.
- Use a different hash salt per experiment so the same user isn't always in the "B" bucket across every unrelated experiment you run.

```
function getBucket(userId: string, experimentKey: string): "A" | "B" {
  const hash = simpleHash(userId + experimentKey);
  return hash % 100 < 50 ? "A" : "B";
}
```

Interview Answer:

"I'd want the bucketing to be deterministic rather than random on every request, typically by hashing a stable ID, like the user's account ID or a persistent device ID, combined with the experiment's key, then mapping the result into a percentage range. That guarantees the same user always lands in the same bucket. For anonymous users I'd persist that assignment in a long-lived cookie so a refresh doesn't reshuffle them, and I'd use a different salt per experiment so being in variant B on one test doesn't correlate with being in variant B on every other test."

Q8. [Mid] Marketing says the homepage visibly jumps, the hero banner swaps out right after the page loads because of an A/B test. What's causing that and how would you fix it?

Answer:

This is the classic "flash of original content" problem, the page paints the default version first, then client-side experiment code runs, decides the user's bucket, and swaps the content, causing a visible flicker or a layout shift (content unexpectedly moving after the user has already started looking at it, which also hurts your CLS score from Q1). **Deciding the experiment variant as early as possible, ideally before the page ever paints, is the real fix, everything after first paint costs you either a flicker or a layout shift.**

Fixes, From Best To Workaround

- Decide the variant server-side or at the edge (a CDN function reading the user's bucket cookie) before the HTML is even sent, so the browser paints the correct variant the first time, no swap needed.
- If the experiment truly has to run client-side, use a small synchronous script in the `<head>` that decides the bucket and hides the content until it's ready, with a short timeout so a slow experiment script doesn't block the whole page indefinitely.

- Reserve the layout space for whichever variant is loading (a skeleton placeholder sized like the real content) so even if content swaps in, the page doesn't jump around.

Best Choice

Move the decision server-side or to the edge whenever the experiment and infrastructure allow it.

Why?

Client-side-only experimentation is simpler to wire up, but it structurally guarantees a moment where the wrong content is visible, either as a flash before the swap or as a layout shift when it swaps, server-side decision removes that moment entirely.

Interview Answer:

"That's the classic flash-of-original-content problem, the browser paints a default version, then a client-side script figures out the bucket and swaps content in, and that swap is what you're seeing as a jump. The real fix is deciding the variant before the HTML ships, at the edge or server-side using a bucket cookie, so the correct version paints the first time. If it has to stay client-side, the fallback is a synchronous anti-flicker snippet in the head that hides content until the bucket's decided, with a timeout so it can't hang the page, plus reserving layout space so nothing shifts even if content does swap."

Q9. [Mid] A user has your app open in a tab from this morning. You deploy at 2 PM and change the shape of an API response. At 3 PM they click a button in that stale tab. What breaks, and separately, why might a plain refresh not even give them your new code?

Answer:

There are two related but distinct problems here. First, the stale tab is still running the JS you deployed this morning, so when it calls the now-changed API, it can throw trying to read a field that moved or was renamed, or worse, silently send malformed data. Second, even a refresh might not fetch new code if the browser aggressively cached the old JS bundle. **The fix for the caching half of this problem isn't clearing the cache, it's naming assets after a hash of their content so a changed file automatically gets a new name and old cached files simply stop being referenced.**

Handling The Stale API Call

- Version the API or support the old response shape for a deprecation window, so an old client calling it doesn't immediately break.
- Detect the mismatch client-side and show a "new version available, please refresh" banner, driven by polling a small `/version.json` or a lightweight websocket push.

Handling The Stale Cached Bundle

- Give every built asset a content hash in its filename, for example `bundle.a3f9c1.js`, so the file's name only changes when its contents change.
- Set a very long, immutable cache lifetime on those hashed files (`Cache-Control: max-age=31536000, immutable`), since they'll never be reused under the same name once content changes.
- Set a short or no-cache lifetime on the entry HTML file that references those hashed filenames, so a fresh page load always discovers the current set of hashed assets.

Interview Answer:

"Two separate things are happening. One is that the open tab is still running this morning's JS, so when it calls the API after the shape changed, it either throws on a missing field or sends the old shape back, which is why I'd want the backend to tolerate the old shape for a deprecation window, or have the frontend detect the mismatch and prompt the user to refresh. The other is caching, if the browser cached the old bundle, a refresh might not even get new code. The standard fix is content-hashed filenames, so `bundle.a3f9c1.js` becomes `bundle.7d2e90.js` the moment its content changes, which lets you cache those files forever while keeping the small entry HTML file that references them on a short cache lifetime, so a fresh load always picks up the latest hashes."

Q10. [Mid] Your static assets are served through a CDN. What actually needs cache purging on deploy, and what doesn't?

Answer:

Pushing build output (JS, CSS, images) to a CDN like Cloudflare, Fastly, or CloudFront puts copies of those files on edge servers close to users. **If your assets already use content-hashed filenames, the hashed files almost never need purging, since a new deploy produces new filenames, the only thing you typically need to actively purge is the small set of un-hashed entry files, like `index.html`.**

Why The Split Works

- Hashed, immutable assets (`app.9f2c1a.js`) can sit on the CDN edge with a cache lifetime of a year, because they'll simply never be requested again under that name once the content changes.
- The entry HTML (and something like an un-hashed `service-worker.js`) has to be purged or set to a very short cache lifetime, since it's the one file responsible for pointing browsers at the current set of hashed filenames.
- A deploy pipeline typically triggers a purge call automatically, for example CloudFront's invalidation API for `/index.html`, or Cloudflare's "purge by URL", right after uploading the new build.

Best Choice

Rely on content hashing to make most files self-invalidating, and only purge the handful of un-hashed entry files as part of the deploy pipeline.

Why?

Mass cache purges are slow, cost money on some CDNs, and can cause a "thundering herd" (a sudden wave of requests hitting your origin server at once because the cache was wiped everywhere simultaneously), purging a single small entry file avoids all of that.

Interview Answer:

"With content-hashed filenames, most of what's on the CDN doesn't need purging at all, a new deploy just produces new filenames like `app.7d2e90.js`, so the old cached file is simply never requested again. The one thing that does need to be purged, or just kept on a very short cache lifetime, is the small entry HTML file that references those hashed names, since that's the file responsible for telling the browser which build to load. I'd wire an automatic purge of just that file into the deploy pipeline, for example a CloudFront invalidation call, rather than purging everything, since a full purge risks a thundering herd of requests hitting the origin all at once."

Q11. [Mid] Your error tracking and analytics tools capture a lot by default, session replays, breadcrumbs, request data. How do you make sure you're not accidentally

shipping a user's PII (personally identifiable information, like email or name) to a third-party vendor?

Answer:

Monitoring tools capture whatever's present in the DOM, the URL, and request/response data by default, which means it's easy to accidentally leak a user's email from a URL query parameter or an auth token from a request header straight into a third-party dashboard. **PII has to be actively scrubbed before an event leaves the browser, most error tracking SDKs provide a hook for exactly this, and it should never be treated as an afterthought.**

Concrete Mitigations

- Use a pre-send hook (Sentry calls it `beforeSend`) to strip or mask sensitive fields, like auth headers and email addresses, before the event is transmitted.
- Avoid putting PII in URLs in the first place, a password reset link with a token is fine, a URL with `?email=jane@company.com` in it is not, since URLs get logged in a lot of places.
- Tell session replay tools which input fields are sensitive (most support a "mask" attribute or class) so a recording of the page never shows a credit card field or a password box in plaintext.

```
Sentry.init({
  dsn: "...",
  beforeSend(event) {
    if (event.request?.headers) delete event.request.headers["Authorization"];
    if (event.user?.email) event.user.email = maskEmail(event.user.email);
    return event;
  },
});
```

Interview Answer:

"By default, error trackers and session replay tools capture pretty much everything visible, the DOM, request headers, URL parameters, which means PII can leak to a third-party vendor without anyone intending it. I'd use a pre-send hook, like Sentry's `beforeSend`, to strip things like auth headers and mask fields like email before the event ever leaves the browser, and I'd make sure session replay tools are told which input fields are sensitive so a password or card number field never shows up in a recording. I'd also just avoid putting PII in URLs in the first place, since URLs get logged in more places than people expect."

Q12. [Senior] Ten minutes after a deploy, error rates triple. Walk me through your rollback strategy.

Answer:

The fast path isn't "revert the commit and redeploy," a fresh deploy still has to go through the same build and release pipeline, which costs time you don't have while users are actively affected. **The fastest rollback is repointing traffic at the previous, already-built artifact, which is why every deployed build should stay addressable rather than overwritten.**

Making Rollback Fast

- Store each build under its own identifier, for example `/builds/<git-sha>/`, on the CDN or object storage, instead of overwriting a single "latest" folder.
- Roll back by swapping which build the entry point or CDN configuration points to, an atomic pointer flip, rather than rebuilding from source.

- If the bad change was behind a feature flag, flip the flag off first, that's even faster than a full build rollback and might resolve the issue without touching the base deploy at all.

Interview Answer:
"My first move is not to revert the commit and push a new deploy, because that still has to go through the full build and release pipeline while users are actively hitting errors. Instead, I want every build to be stored as its own addressable artifact, so rolling back is just repointing the entry point or CDN config at the previous build's identifier, an atomic swap rather than a rebuild. If the change happened to be behind a feature flag, flipping that off is even faster and might be all that's needed. Either way, the goal is to stop the bleeding in seconds or low minutes, and only dig into root cause once the immediate impact is contained."

Q13. [Senior] A security review flags that anyone can download `bundle.js.map` from your production site and read your unminified source. Is that a real problem, and how should source maps actually be handled?

Answer:

Yes, it's a real problem, a publicly downloadable source map hands out your original, human-readable source, including internal comments, variable names, and logic you never intended to expose to the public. **Source maps should be uploaded directly to your error-tracking provider as a build step and excluded from the public deploy, not served from the same CDN as the app itself.**

How To Do It Right

- Wire source map upload into CI/CD, for example via the Sentry CLI or its webpack/Vite plugin, so maps land on the provider's servers as part of the build, and are then deleted or excluded before the public bundle is deployed.
- If a team genuinely needs maps reachable over HTTP for some internal tooling reason, they should sit behind authentication or an IP allowlist, not on the same open path as the public bundle.
- This is a good moment to also check whether other build artifacts, like `.env` files or unminified debug builds, are accidentally exposed the same way.

Interview Answer:
"That's a legitimate finding, a public source map is basically your original source code with a different file extension, comments and all. The right setup is to upload source maps directly to the error tracking provider as part of the CI build, using something like the Sentry CLI or its build plugin, and then exclude them from the actual public deploy entirely. If there's a real need to have them reachable over HTTP, they should sit behind auth or an IP allowlist rather than living at the same public path as the bundle. I'd also use that finding as a prompt to check whether anything else, like env files or debug builds, is leaking the same way."

Q14. [Senior] How do you avoid finding out about a bad deploy from a customer support ticket instead of catching it yourself within minutes?

Answer:

The goal is to bound both the blast radius (how many users are affected) and the time-to-detect, not to catch every possible bug before release, since some bugs only show up under real production traffic. **A staged rollout combined with deploy-tagged alerting and an immediate post-deploy smoke test catches most bad deploys before they reach most users, and catches the rest within minutes instead of hours.**

The Pieces

- Canary or staged rollout: release to a small slice of traffic or users first, and hold there for a fixed "bake time" before promoting to everyone, so a bad deploy only ever hits a small fraction of users.
- Release-tagged error tracking: tag every error event with the release version, so a dashboard can alert "error rate for release v482 is 5x baseline within 10 minutes of rollout" automatically, rather than someone eyeballing a graph.
- Synthetic smoke tests immediately after deploy: an automated script hits critical flows, like login and checkout, right after rollout and gates further promotion if any of them fail.

Interview Answer:

"I'd want a staged rollout, releasing to a small slice of users first and holding there for a bake time before going to 100%, so a bad deploy is naturally contained to a small blast radius. Every error event should be tagged with the release version, so I can set up an alert like 'error rate for this release is five times baseline within ten minutes,' rather than relying on a human to notice a graph moving. On top of that, I'd run an automated synthetic smoke test against critical flows like login and checkout right after each deploy, so a broken checkout button gets caught by a script within seconds, not by a customer support ticket hours later."

Q15. [Senior] Client-side errors spike right after a deploy. Walk me through how you'd actually diagnose it, in order.

Answer:

This is a triage problem before it's a debugging problem, users are actively affected while you're still reading stack traces, so the order of operations matters. **The triage order is always confirm it's real, find the dominant error, and mitigate the blast radius, before spending time on full root cause analysis.**

Diagnosis Steps

1. Confirm the correlation: filter the error dashboard by release version and confirm the spike actually lines up with the new deploy, and isn't a coincidental third-party outage, a traffic spike, or a bot crawl.
2. Find the dominant error: group errors by message and stack trace, since a single bug, like a null reference from a renamed API field, can generate thousands of near-identical events that are drowning out everything else, and fixing that one thing may resolve most of the spike.
3. De-minify a few real samples: pull the actual file and line from the source-mapped stack trace to rule out a source map upload issue masking what's really going on.
4. Segment by breadcrumbs: check the browser, OS, and locale breakdown, an error isolated to an old Safari version or a specific browser extension is a very different fix than something happening globally.
5. Rule out the stale-client-and-changed-API scenario: check whether the errors are coming from clients still running the previous bundle now hitting a changed API contract, which looks like "a new bug" but is really old code against a new backend.
6. Mitigate first, understand fully second: flip the feature flag off if the change was flag-gated, or roll back to the previous build, before finishing a deep root cause investigation, since containment shouldn't wait on full understanding.

Interview Answer:

"First I'd confirm the spike genuinely correlates with the new release by filtering the error dashboard by version, rather than assuming, since it could be an unrelated third-party outage. Then I'd group errors by message and stack trace to find the one dominant error, because usually a single bug, like a renamed API field causing a null reference, accounts for most of the volume. I'd de-minify a couple of real samples to make sure I'm looking at the actual failure and not a source map issue, and check the breakdown by browser

and locale to see if it's global or isolated. I'd also specifically check whether this is stale clients on the old bundle hitting a changed API, since that looks like a new bug but isn't. And critically, I'd mitigate immediately, flag off or roll back, before I've fully finished root causing it, because users are affected the entire time I'm investigating."

Q16. [Senior] You're about to replace the entire checkout flow, a genuinely risky frontend change. Design the end-to-end safety net you'd want in place before it ships to 100% of users.

Answer:

This question is really asking whether you can combine everything in this chapter into one coherent release strategy, rather than treating flags, monitoring, and rollback as separate unrelated tools. **The feature flag should be the primary safety valve because flipping it is instant, and the deploy-level rollback exists as a fallback for the case where the flag's boundary wasn't wide enough to contain the bug.**

The Layered Plan

1. Ship the new checkout behind a flag that defaults off, so the code is live in production but dormant, deploy risk is separated from release risk.
2. Dogfood internally first, employees only, watching RUM and error tracking tagged by both the flag and the release version.
3. Canary to a small percentage of real users, holding for a fixed bake time, alerting on error rate and on conversion rate specifically, since a silently broken checkout that doesn't throw a visible error but tanks completions is arguably worse than a loud crash.
4. Ramp gradually, for example 5% to 25% to 100%, with a deliberate pause between steps rather than a straight line to full rollout.
5. If any threshold is breached at any step, flip the flag off immediately, which resolves the incident without needing to touch the base deploy at all.
6. Keep the previous production build addressable regardless, in case the base application itself, not just the flagged feature, ever needs a full rollback for an unrelated reason.

Interview Answer:
"I'd treat the feature flag as the main safety valve here, since it's the fastest thing to flip. I'd ship the new checkout dark behind a flag defaulting off, dogfood it internally first while watching error tracking and RUM tagged by both the flag and the release, then canary to a small percentage of real users with a fixed bake time. Critically, I'd alert on conversion rate as well as error rate, since a checkout that fails silently without throwing is arguably a worse outcome than one that visibly crashes. I'd ramp gradually with pauses between steps, and if anything breaches a threshold, flip the flag off instantly rather than reaching for a full deploy rollback. I'd still keep the previous build addressable underneath all of that, in case the flag's boundary turns out not to be wide enough and the base app itself needs to roll back too."

If you remember nothing else from this chapter: *an average error rate can look perfectly healthy while one entire segment of users has a completely broken experience.*

Chapter 16: Putting It Together: Frontend System Design Case Studies

Every earlier chapter tests whether you understand one concept at a time; this chapter tests whether you can assemble a dozen of those concepts into a coherent system while a prompt is still ambiguous. Treat these six case studies as 30-45 minute open-ended prompts, and practice volunteering the tricky parts (a race condition, a layout shift, a conflict-resolution rule) before anyone has to drag them out of you, since that unprompted instinct is what actually separates someone who has shipped this kind of feature from someone who has only read about it.

Design a typeahead/autocomplete search box

Interviewer Pacing: budget roughly forty-five minutes end to end, five to ten on requirements and scoping, ten to fifteen on the core architecture, fifteen to twenty on the hardest trade-off, and keep five in reserve for a curveball edge case. A junior pass gets the boxes right; a senior pass defends the one decision that actually mattered once you push on it.

Q1. [Junior-Mid] Design the frontend for a search box that shows live suggestions as the user types, like the search bar on Amazon or GitHub's repo search. Walk me through the whole thing.

Answer:

A typeahead is a small system with a surprisingly large number of failure modes: too many network requests, suggestions arriving out of order, and a dropdown that's unusable from the keyboard. **The hardest part of a typeahead is not debouncing the input, it's guaranteeing that a slow response to an old keystroke can never overwrite a faster response to a newer one.** A strong answer treats the request layer and the UI layer as separate concerns and calls out that race condition unprompted.

Clarifying questions to ask

- What's the backend for suggestions: a REST endpoint, a search index, or a small local dataset that could just be filtered in the browser?
- What's an acceptable latency budget, and roughly how many requests per second should we expect at peak, since that affects whether we need client-side caching at all?
- Do we need to support IME composition (the multi-keystroke input method used for languages like Chinese or Japanese, where intermediate keystrokes shouldn't trigger a search)?
- Should recent queries persist across page reloads, or is an in-memory cache for the current session enough?
- Do we need to highlight the matched substring inside each suggestion?

Component/architecture breakdown

- **Input component**: a controlled text input that owns the raw keystroke value and forwards it upward on every change.
- **Debounce layer**: a small utility (often a custom hook) that waits for a pause in typing, typically 150 to 300 milliseconds, before treating the value as a "committed" query.
- **Fetcher/request manager**: issues the network call for the committed query and is responsible for cancelling or ignoring stale in-flight requests.

- **Cache**: a simple key-value store, often just a `Map` keyed by query string, so backspacing to a previously typed prefix is instant.
- **Dropdown/listbox**: renders the suggestion list, owns the currently highlighted index, and exposes keyboard and pointer interaction.

State and data flow

- Raw input value lives in the input component as local controlled state.
- A debounced version of that value is the actual "query" that drives fetching, kept separate so every keystroke does not trigger a request.
- Results, loading, and error states live alongside the query, typically in a single hook like `useAutocomplete(query)`.
- The active/highlighted suggestion index is separate state, reset to `-1` whenever new results arrive.
- The cache lives outside React state entirely (a `useRef` or a module-level `Map`), since updating it should never itself trigger a re-render.
- Flow: keystroke updates raw value, debounce timer fires, cache is checked first, on a cache miss a request goes out with an `AbortController`, and on success both the cache and the results state update together.

Key implementation details / tricky parts

- Cancel or ignore stale responses. Either abort the previous request with `AbortController` when a new one starts, or tag each request with an incrementing id and drop any response whose id isn't the latest, since both fixes the flicker where an old, slower response overwrites new results.

```
function useAutocomplete(query: string, delay = 250) {
  const [results, setResults] = useState<string[]>([]);
  const latestRequestId = useRef(0);
  const cache = useRef(new Map<string, string[]>());

  useEffect(() => {
    if (!query) {
      setResults([]);
      return;
    }
    if (cache.current.has(query)) {
      setResults(cache.current.get(query)!);
      return;
    }
    const requestId = ++latestRequestId.current;
    const controller = new AbortController();
    const timer = setTimeout(async () => {
      const res = await fetch(`/api/search?q=${query}`, { signal: controller.signal });
      const data: string[] = await res.json();
      if (requestId === latestRequestId.current) {
        cache.current.set(query, data);
        setResults(data);
      }
    }, delay);
    return () => {
      clearTimeout(timer);
      controller.abort();
    };
  }, [query, delay]);

  return results;
}
```

- Keyboard navigation needs arrow up/down to move the highlighted index, Enter to select, and Escape to close, all implemented as `role="combobox"` with `aria-activedescendant` pointing at the highlighted option's id, so a screen reader announces the current selection.

Performance considerations

- Debounce delay is a real trade-off: too short and you spam the backend, too long and the UI feels laggy; 200 to 300 milliseconds is a reasonable default to state out loud.
- The client-side cache eliminates duplicate requests for the extremely common pattern of typing, backspacing, and retyping the same prefix.
- Consider prefetching the top suggestion's detail page on hover or on highlight, since the user is statistically likely to pick it.
- Result lists are usually short (five to ten items), so windowing (rendering only visible items) is unnecessary here, but worth mentioning it would matter if the dropdown ever showed hundreds of rows.

Edge cases

- The user types quickly, then deletes several characters back to a prefix already in the cache. The cache should serve that instantly with no network round-trip.
- A network error occurs mid-search. The dropdown should show an inline error state rather than an empty list, since an empty list is indistinguishable from "no results."
- The user is typing in a CJK input method. The component should ignore `compositionstart` through `compositionend` and only treat the final composed text as a query, or you'll fire garbage searches on every partial keystroke.

Interview Answer:

"I'd split this into an input that tracks raw keystrokes, a debounce step that turns rapid typing into a single committed query, and a fetcher that's responsible for both caching and cancelling stale requests. The part I'd flag early is the race condition: if someone types 'cat' then quickly 'car', the response for 'cat' can come back after the response for 'car' if the network is uneven, so I'd tag each request with an id or use an AbortController and only accept the response that matches the latest query. I'd cache results by query string so backspacing feels instant, and I'd build the dropdown as a proper combobox with arrow-key navigation and aria-activedescendant so it's usable without a mouse. Edge cases I'd handle explicitly are empty results, network failures, and IME composition for non-Latin input methods."

Design an infinite-scrolling image gallery or social feed

Q1. [Mid] Design the frontend for an infinite-scrolling feed, like Instagram's grid or a Twitter-style timeline, that loads more content as the user scrolls and needs to restore scroll position when they navigate back to it.

Answer:

An infinite feed looks simple until you account for three things happening at once: the DOM growing without bound, images arriving at different speeds and shoving content around, and the user expecting the feed to look exactly the way they left it after clicking into a post and hitting back. **Every layout-shift bug in an infinite feed traces back to not reserving space for content before it actually arrives, whether that's an image, a card, or a whole page of items.**

Clarifying questions to ask

- Is this a single-column feed (Twitter-style) or a multi-column grid/masonry layout (Pinterest-style), since that changes how hard virtualization is?
- Roughly how many items will a typical session load, tens or thousands, since that determines whether virtualization is a nice-to-have or a hard requirement?
- Does the initial page need to be server-rendered for SEO, or is this entirely behind a logged-in, client-rendered app shell?
- Do new items get pushed in at the top in real time (like a live timeline), and if so, should that disturb the user's current scroll position?
- Does "back" navigation need to restore both the loaded items and the exact scroll offset, or just drop the user back at the top?

Component/architecture breakdown

- **Virtualized list/grid**: renders only the items currently in or near the viewport, using an estimated or measured item height to compute which indices to render, the technique usually called windowing.
- **Sentinel + IntersectionObserver**: an invisible element near the bottom of the rendered list whose visibility triggers the next page fetch, replacing manual scroll-position math.
- **Item component**: renders an image or card with a fixed aspect-ratio placeholder, so it occupies its final size in the layout before the image itself has loaded.
- **Pagination/data layer**: manages cursor-based fetching, appends new pages to an accumulated items array, and tracks loading and end-of-feed state.
- **Scroll/state cache**: a small store, keyed by feed or route, that remembers loaded items and scroll offset when the user navigates away and restores both on return.

State and data flow

- The accumulated `items` array and the pagination cursor live in a data layer above the list, not inside the virtualization logic itself.
- The virtualized list keeps its own derived state: current scroll offset, container height, and the computed visible index range.
- On navigating away (say, opening a photo detail view), the current items array, cursor, and scroll offset get written into a cache, keyed by the feed identity, before unmount.
- On returning, the list rehydrates from that cache instead of refetching from page one, so the user sees the same items at the same scroll position.
- Real-time inserts at the top go into a separate "pending new items" bucket with a visible "n new posts" affordance rather than being spliced directly into the array, so the user's scroll position never jumps out from under them.

Key implementation details / tricky parts

- Use `IntersectionObserver` on the sentinel instead of a scroll event listener, since scroll listeners fire far more often and require manual throttling.

```
function useInfiniteScroll(onLoadMore) {
  const sentinelRef = useRef(null);

  useEffect(() => {
    const observer = new IntersectionObserver(
      (entries) => {
        if (entries[0].isIntersecting) onLoadMore();
      },
      { rootMargin: "400px" }
    );
    if (sentinelRef.current) observer.observe(sentinelRef.current);
```

```
    return () => observer.disconnect();
  }, [onLoadMore]);

  return sentinelRef;
}
```

- For variable-height content (masonry-style grids), maintain a cache of measured heights per item so the virtualizer doesn't need to re-measure on every render, and use that cache to size the spacer elements above and below the visible window.
- For image loading specifically, set an explicit `width`/`height` or CSS `aspect-ratio` on the image container up front, so the browser reserves the box before the image byte stream even arrives, which prevents cumulative layout shift (CLS, a Core Web Vitals metric measuring how much visible content jumps around during load).

Performance considerations

- Virtualization keeps the number of live DOM nodes roughly constant no matter how many pages the user has scrolled through, which is what keeps memory and paint cost bounded in a feed with thousands of items.
- Use `loading="lazy"` or a per-image IntersectionObserver so off-screen images don't compete for bandwidth with the ones currently visible.
- Debounce or dedupe rapid-fire page requests when a user scrolls fast enough to cross the sentinel multiple times before the first request resolves.
- Prefer responsive images (`srcset`/`sizes`) so mobile clients aren't downloading desktop-resolution assets.

Edge cases

- The user scrolls very fast and crosses the load-trigger threshold several times before the first page returns, which without deduping would fire three or four redundant requests for the same next page; guard with an in-flight flag.
- Images load at wildly different speeds because of size or CDN latency, and without reserved space, faster-loading images below a slower one cause everything to jump as the slow one finally resizes into place.
- The user opens a post, then hits back, expecting the exact scroll position and the exact set of already-loaded items, not a fresh fetch from page one, which is why the scroll/state cache needs to be keyed by route and restored before first paint of the returning view.

Interview Answer:

"I'd build this as a virtualized list so the DOM only ever holds what's near the viewport, with an IntersectionObserver-based sentinel at the bottom triggering the next page fetch instead of a scroll listener. The data layer keeps an accumulated items array and cursor above the virtualization logic, and I'd cache that array plus the current scroll offset keyed by route so that going back from a detail view restores the feed exactly as the user left it instead of refetching from scratch. For images, the thing I'd insist on is reserving the aspect ratio of every card before the image loads, because otherwise you get constant layout shift as images at different network speeds pop in and shove content around. I'd also guard against firing duplicate page requests when someone scrolls fast enough to cross the trigger multiple times before the first request resolves."

Design a client-side multi-step form builder with dynamic validation

Q1. [Mid-Senior] Design the frontend for a multi-step, schema-driven form, like a job application wizard or an onboarding flow, where the fields themselves come from a JSON schema and can change based on earlier answers.

Answer:

This prompt is really two systems in one: a generic renderer that turns a schema into real form fields, and a wizard controller that manages steps, validation timing, and state that must survive moving back and forth between steps. **Form state has to live above and outside the individual step components, in a single central store, or navigating between steps will silently lose whatever the user typed.**

Clarifying questions to ask

- Is the schema fixed at build time, or fetched from the server and potentially different per user (for example, different steps for a business account versus a personal account)?
- What field types do we need to support: text, select, date, file upload, and do any fields conditionally show or hide based on earlier answers?
- What validation timing does product want: as-you-type, on blur, or only on step submit? Each has a different UX cost.
- Should partial progress be saved so a user can close the tab and resume later?
- What accessibility bar are we targeting, and are there specific assistive technologies QA will test with?

Component/architecture breakdown

- **Schema**: a JSON description of every field (name, type, validation rules, and optional `dependsOn` conditions for conditional visibility), plus a grouping of fields into steps.
- **Field renderer**: a component that maps a schema field's `type` to a concrete input component, so adding a new field type to the schema doesn't require touching the wizard itself.
- **Wizard controller**: owns the current step index, the ordered list of steps, and next/back navigation, and asks the validation engine whether the current step is allowed to advance.
- **Validation engine**: runs per-field rules (often via a schema validation library) and step-level rules (all required fields on this step are valid).
- **Central form store**: a single object of field name to value, plus per-field touched and error state, that outlives any individual step's mount/unmount.
- **Persistence layer**: periodically or on-blur writes the central store to local storage or a draft API so a refresh doesn't lose progress.

State and data flow

- All field values live in one flat object in the central store, not scattered across per-step local state, which is exactly why switching steps doesn't lose data.
- Each field also carries a `touched` flag, since a common validation-timing choice is "don't show an error until the user has interacted with the field."
- Visibility of conditional fields is derived state, recomputed from current values whenever they change, rather than stored independently, since two sources of truth for "is this field visible" inevitably drift apart.
- The current step index is separate from field data, ideally synced into the URL as a query parameter so back/forward browser navigation moves between steps correctly.

- Data flow on a keystroke: field change updates the central store, which recomputes derived visibility, runs validation according to the configured timing, and the field renderer re-reads only its own slice of state.

Key implementation details / tricky parts

- Schema-driven rendering means the renderer is a big switch (or lookup table) from field type to component, kept intentionally dumb so all the real logic (validation, conditional visibility) stays in the engine, not scattered across field components.

```
type Field = {
  name: string;
  type: "text" | "select";
  options?: { value: string; label: string }[];
};

function FieldRenderer({
  field,
  value,
  error,
  onChange,
}: {
  field: Field;
  value: string;
  error?: string;
  onChange: (name: string, value: string) => void;
}) {
  if (field.type === "select") {
    return (
      <select
        id={field.name}
        value={value ?? ""}
        aria-invalid={!!error}
        onChange={(e) => onChange(field.name, e.target.value)}
      >
        {field.options?.map((opt) => (
          <option key={opt.value} value={opt.value}>
            {opt.label}
          </option>
        ))}
      </select>
    );
  }
  return (
    <input
      id={field.name}
      value={value ?? ""}
      aria-invalid={!!error}
      aria-describedby={error ? `${field.name}-error` : undefined}
      onChange={(e) => onChange(field.name, e.target.value)}
    />
  );
}
```

- Validation timing trade-off: validating on every keystroke feels reactive but is noisy (an email field shows "invalid" while the user is still typing it); validating on blur, then re-validating on every subsequent change once a field is `touched`, is the pattern most production forms converge on.
- Accessibility specifics worth calling out unprompted: every input needs a `label` associated by `htmlFor`/`id`, invalid fields need `aria-invalid` and `aria-describedby` pointing at the error message's id, radio/checkbox groups need `fieldset`/`legend`, and on a failed step submission, focus should move to the first invalid field while an `aria-live` region announces the step change to screen reader users.

Performance considerations

- A schema with many conditional fields can trigger a full form re-render on every keystroke if visibility is recomputed naively; memoize the visibility calculation and scope re-renders to the fields that actually changed.
- For large forms, prefer field-level state subscriptions (the pattern used by libraries like react-hook-form) over one `useState` for the whole form object, so a single keystroke doesn't re-render every field on the page.
- Debounce any async validation, like checking whether a username is already taken, so it doesn't fire a network request on every keystroke.

Edge cases

- The user hits the browser's back button mid-wizard. If the step index is synced to a URL query parameter, back/forward naturally moves between steps instead of leaving the wizard or losing data.
- A field has async validation still pending (a username check) when the user tries to submit the step; the submit action should wait for or block on that pending validation rather than accepting a stale "valid" state.
- A conditional field is filled in, then hidden again because an earlier answer changed. Decide explicitly whether to keep the value in the store (in case the user re-reveals it) or clear it (to avoid submitting an answer to a question the user no longer sees), and be able to justify whichever you pick.

Interview Answer:
"I'd treat this as a schema-driven renderer plus a wizard controller sitting on top of one central form state object, and the thing I'd flag immediately is that the form values have to live above the individual step components, not inside them, otherwise going back a step wipes out what someone typed. The schema describes fields and their type, validation rules, and any conditional dependsOn logic, and a generic FieldRenderer maps field type to a real input component so adding a new field type doesn't mean touching the wizard logic. For validation timing I'd default to on-blur plus continuous re-validation once a field's been touched, since validating on every keystroke is noisy. I'd also be explicit about accessibility: labels tied to inputs, aria-invalid and aria-describedby on errors, and moving focus to the first invalid field on a failed submit. And I'd sync the current step into the URL so back and forward work the way users expect."

Design the frontend for a real-time collaborative app like a shared document or whiteboard

---

Q1. [Senior] Design the frontend architecture for a real-time collaborative editor, like Google Docs or a Figma-style whiteboard, where multiple users can edit the same content at the same time.

Answer:

This is the case study where getting the data model wrong makes every other decision harder, so it's worth spending real time up front on how conflicting edits get merged before touching UI components. **Optimistic local updates only feel safe to the user if the underlying data structure guarantees that merging in a remote edit can never silently discard the user's own unsent change.** That's exactly what CRDTs (conflict-free replicated data types, data structures designed so concurrent edits merge deterministically without a central coordinator) are built for, which is why most modern collaborative editors are built on one (Yjs and Automerge are the two most commonly used in production).

Clarifying questions to ask

- Is this a linear document (text) or a spatial canvas (whiteboard with x/y/z ordering and overlapping shapes), since that changes the data model significantly?
- Is the conflict-resolution strategy, operational transform (OT) versus CRDT, already decided at the backend, or is that part of what I should weigh in on?
- What's the expected concurrency: two or three editors, or a hundred people on the same whiteboard at once?
- Does the client need to support offline editing with changes replayed on reconnect, or is a live connection assumed at all times?
- Do we need live cursors and a presence list showing who's currently viewing or editing?

Component/architecture breakdown

- **WebSocket connection manager**: opens the socket, sends periodic heartbeats, and reconnects with exponential backoff on drop.
- **Local CRDT instance**: the actual source of truth for document content, living entirely on the client and syncing with peers over the socket.
- **Optimistic update layer**: applies the local user's edits to the CRDT immediately, before any server acknowledgment, so typing feels instant.
- **Reconciliation layer**: merges incoming remote operations into the local CRDT as they arrive; because CRDT merges are commutative, this requires no special-casing for arrival order.
- **Presence channel**: a separate, lightweight broadcast of cursor position, selection, and online user list, since presence data is ephemeral and shouldn't go through the same durability guarantees as document content.
- **Editor/canvas rendering layer**: subscribes to CRDT state and re-renders only the parts of the document or canvas that actually changed.

State and data flow

- The CRDT document is the single source of truth for content; there is no separate "app state" that duplicates it, since duplicated state is exactly what drifts out of sync during conflicts.
- A queue of locally applied but not-yet-acknowledged operations backs both the optimistic UI and offline replay after a reconnect.
- Connection state (connecting, open, reconnecting, offline) is tracked separately and drives UI like a "reconnecting" banner.
- Presence state is a simple map of userId to cursor position, selection range, and a display color, held outside the CRDT since it's transient and doesn't need to survive a reload.
- Flow: a keystroke applies an operation to the local CRDT immediately (optimistic), the UI re-renders from the CRDT's new state, the operation is sent over the socket, peers receive and merge it into their own CRDT, and their UI re-renders from that merge, all without any client waiting on a round trip to see its own edit.

Key implementation details / tricky parts

- Reconnection needs to resync state, not just reopen the socket: on reconnect, either request a diff since the client's last known version vector (a CRDT concept tracking which operations each replica has already seen) or, for a very stale client, fetch a fresh snapshot.

```
class RealtimeConnection {
  private attempt = 0;

  connect(url: string, onMessage: (op: unknown) => void, onOpen: () => void) {
    const socket = new WebSocket(url);
```

```
    socket.onmessage = (event) => onMessage(JSON.parse(event.data));

    socket.onopen = () => {
      this.attempt = 0;
      onOpen();
    };

    socket.onclose = () => {
      const delay = Math.min(1000 * 2 ** this.attempt, 30000);
      this.attempt += 1;
      setTimeout(() => this.connect(url, onMessage, onOpen), delay);
    };

    return socket;
  }
}
```

- Batch or coalesce rapid local operations (a burst of keystrokes within a single animation frame) into one outgoing message rather than sending one WebSocket frame per keystroke, which both reduces network chatter and gives the CRDT fewer, larger operations to merge.
- Throttle presence broadcasts (cursor position on every `mousemove` is far too frequent); sending updates every 30 to 50 milliseconds is usually indistinguishable to onlookers from sending on every pixel of movement.

Performance considerations

- For a whiteboard with thousands of shapes, cull or virtualize off-screen elements at the rendering layer, same idea as list virtualization but applied to a 2D canvas.
- Use fine-grained subscriptions so a remote edit to one paragraph or one shape re-renders only that piece of the UI, not the entire document tree.
- Avoid shipping the full operation history to a newly joined client; serve a compacted snapshot plus only the operations since that snapshot.

Edge cases

- The network drops mid-edit. Local operations queue up, and on reconnect they replay against the CRDT's merge logic against whatever the server's state has become in the meantime, so no edit is lost and no edit is silently dropped in favor of someone else's.
- Two users edit the same word or move the same shape at the same instant. The CRDT resolves this deterministically (for example, by a tie-break rule based on client id and timestamp), but the UI should visually reflect the merged outcome clearly rather than making it look like one person's edit vanished.
- A user closes their laptop lid without a clean disconnect. The server needs a heartbeat timeout so their cursor and "online" status expire after a few seconds instead of showing them as present indefinitely.

Interview Answer:

"I'd build the client around a local CRDT instance as the actual source of truth for the document, not a separate app-state object, because CRDTs let concurrent edits from different clients merge deterministically without needing a central lock. Local edits apply to that CRDT immediately for optimistic UI, get queued, and get sent over a WebSocket that reconnects with exponential backoff; on reconnect I'd resync from a version vector rather than trusting the socket reopening alone. Presence, meaning cursors and who's online, I'd keep as a separate, throttled, ephemeral channel since it doesn't need the same durability as actual content. The trickiest edge case I'd call out is two people editing the same shape or word simultaneously: the CRDT merge resolves it deterministically, but I'd make sure the UI actually shows the merged result clearly instead of making it look like someone's edit just disappeared."

Design a toast notification system plus a real-time notification center

---

Q1. [Mid-Senior] Design the frontend for two related pieces: transient toast popups (like "message sent") and a persistent bell-icon notification center with an unread count, fed by WebSocket or polling, that has to handle the user going offline and reconnecting.

Answer:

These look like the same feature but they're not: toasts are ephemeral UI that nobody expects to survive a refresh, while the notification center is a persisted list that has to stay consistent with the server even through dropped connections. **Unread count must be a value derived from the notification list's read flags, never a separately maintained counter, or it will drift out of sync the very first time an update gets missed.**

Clarifying questions to ask

- Do toasts and the notification center share a single underlying event, or can something toast without ever appearing in the center (and vice versa)?
- Is delivery via WebSocket, server-sent events (SSE, a one-way push protocol over HTTP), or polling, and is there a fallback if the primary transport fails?
- Does history need to survive a page reload, meaning we fetch a backlog from a REST endpoint on mount?
- Do notifications have priority levels that affect stacking order or how long a toast stays visible?
- If the same user has two tabs open, does marking something read in one tab need to update the other?

Component/architecture breakdown

- **Toast subsystem**: a provider/context holding a small ephemeral queue, a `Toast` component with an auto-dismiss timer that pauses on hover, and a cap on how many toasts show at once.
- **Notification center subsystem**: the bell icon with a badge showing unread count, and a dropdown panel listing notifications with mark-as-read actions.
- **Transport layer**: a WebSocket (or SSE) client that normalizes incoming push events into a common `Notification` shape, with a polling fallback if the socket repeatedly fails to connect.
- **Notification store**: the persisted list of notifications (id, payload, timestamp, read boolean), the single source of truth both the bell badge and the dropdown read from.
- **Sync/persistence layer**: fetches history on mount, sends mark-as-read calls to the server, and reconciles any gap after a reconnect.

State and data flow

- Toasts are a short-lived array, added on incoming events and removed on timeout or explicit dismissal; nothing about them is persisted.
- Notifications are a longer-lived list, ideally managed by a reducer or a data-fetching cache (something like React Query), each entry carrying its own `read` boolean.
- Unread count is computed as `notifications.filter(n => !n.read).length` every render, never stored as its own piece of state that could fall out of sync with the list.
- Flow: a push event arrives at the transport layer, gets normalized into a `Notification`, gets appended to the store (deduped by server-issued id), and, if configured to also toast, gets pushed

into the separate toast queue at the same time, so both surfaces read from the same underlying event rather than racing each other.

Key implementation details / tricky parts

- Dedupe by a server-issued id whenever a notification is received, since the same notification can legitimately arrive twice: once live over the socket, and once again from a backlog fetch after a reconnect.

```
type Notification = { id: string; message: string; read: boolean; createdAt: number };
type Action =
  | { type: "RECEIVE"; payload: Notification }
  | { type: "MARK_READ"; id: string };

function notificationsReducer(state: Notification[], action: Action): Notification[] {
  switch (action.type) {
    case "RECEIVE":
      if (state.some((n) => n.id === action.payload.id)) return state;
      return [action.payload, ...state];
    case "MARK_READ":
      return state.map((n) => (n.id === action.id ? { ...n, read: true } : n));
    default:
      return state;
  }
}
```

- On reconnect after any gap, call a "notifications since timestamp X" endpoint to backfill anything missed while disconnected, rather than assuming the socket alone caught everything.
- For cross-tab consistency, broadcast mark-as-read actions via `BroadcastChannel` (or a `storage` event as a fallback) so the badge count updates in every open tab, not just the one the user clicked in.

Performance considerations

- Cap in-memory notification history and paginate older entries from the server rather than keeping an ever-growing array in the client.
- Batch bursts of incoming events (fifty likes arriving in one second) into a single UI update and a single summarized toast ("50 people liked your post") instead of fifty separate re-renders and fifty stacked toasts.
- Use an `aria-live="polite"` region for toast announcements so screen reader users are notified without the interruption level of `aria-live="assertive"` for anything non-critical.

Edge cases

- The user is offline for ten minutes and reconnects. The reconnect handler must fetch missed notifications and merge them without creating duplicates of ones already shown live, which is exactly what id-based deduping in the reducer handles.
- The WebSocket keeps failing to establish (a restrictive corporate proxy, for example). The transport layer should detect repeated failures and fall back to polling on an interval rather than retrying the socket forever.
- A notification references content the user can no longer see, such as a comment that was deleted after the notification was generated. The center should either filter that entry out or render a graceful fallback rather than erroring or linking to a broken page.

Interview Answer:

"I'd separate toasts, which are ephemeral and never persisted, from the notification center, which is a real list that has to survive reconnects and reloads. Both read from the same underlying event stream, but toasts go into their own short-lived queue while the notification center appends to a persisted, reducer-managed

list keyed by a server-issued id so duplicates from a backlog re-fetch after reconnecting don't show up twice. The one thing I'd insist on is that the unread count is always computed from that list's read flags, not tracked as its own counter, because a separately maintained counter will eventually drift, usually the first time an event gets missed during a dropped connection. For reconnects specifically, I'd have the client call a 'notifications since last timestamp' endpoint to backfill any gap, and I'd fall back to polling if the WebSocket keeps failing to connect."

Design the frontend for a large e-commerce product listing page

---

Q1. [Senior] Design the frontend for a large e-commerce category page, like an Amazon search results page, with filters, sorting, and pagination, that needs to be fast, SEO-friendly, and handle filter facets with thousands of possible values.

Answer:

A product listing page (PLP) looks like "just a grid with a sidebar" until you factor in that it needs to be crawlable by search engines, shareable as a URL, fast on a slow mobile connection, and correct when the user hits the back button after paging through results. **Treating the URL, not component state, as the source of truth for filters is what makes a PLP simultaneously shareable, crawlable, and back-button-correct at the same time.**

Clarifying questions to ask

- Does the initial page load need to be server-rendered with real product data in the HTML for SEO, or is this an authenticated app where crawlers don't matter?
- Roughly how many facets and how many options per facet are we talking about, since a "brand" facet with five products behaves very differently from one with five thousand?
- Is pagination numbered pages, infinite scroll, or a "load more" button, and does that choice need to be consistent across mobile and desktop?
- Do filters need to be shareable and bookmarkable as a URL a user could paste into Slack?
- What's the acceptable time-to-first-product-visible on a mid-tier mobile device, since that constrains how much can happen client-side before first paint?

Component/architecture breakdown

- **Server-rendered shell**: the first page of results, with the applied filters already baked into the server response, so both crawlers and the user's first paint get real product data without waiting on a client-side fetch.
- **URL/query-param layer**: the canonical place filter, sort, and page state live (`?brand=nike&size=10&sort=price_asc&page=2`), read by both the server on direct hits and the client on in-app navigation.
- **Filter panel**: checkbox groups, range sliders, and for large facets, a search-within-facet input plus a virtualized option list rather than rendering every possible value.
- **Product grid**: fixed-size skeleton cards matching the real card's dimensions, so the layout doesn't shift once data arrives.
- **Pagination/data layer**: reads the current URL, calls the search API, and returns both products and updated facet counts (how many results each remaining filter option would produce).

State and data flow

- The URL query string is the single source of truth for filters, sort, and page; there is no parallel React state object holding "the real" filter selection, since two sources of truth for the same thing is exactly how PLPs end up out of sync with the URL a user bookmarks.
- Changing a filter pushes a new URL (via the router), which both updates the visible browser address and triggers a refetch keyed by that exact query string.
- On a direct hit to a filtered URL, the server reads the same query string and renders page one with those filters already applied, so SEO and shareability come for free from the same mechanism that drives client-side navigation.
- Ephemeral UI state that isn't part of the "result," like whether the mobile filter drawer is currently open, stays local and explicitly out of the URL.

Key implementation details / tricky parts

- Reading and writing filters through `URLSearchParams` (or a router's equivalent) rather than a separate state object keeps navigation, sharing, and refresh all working the same way by construction.

```
function useFilterState() {
  const [params, setParams] = useSearchParams();

  function setFilter(key: string, value: string | null) {
    const next = new URLSearchParams(params);
    if (value) {
      next.set(key, value);
    } else {
      next.delete(key);
    }
    next.set("page", "1");
    setParams(next);
  }

  return { filters: Object.fromEntries(params), setFilter };
}
```

- Debounce URL writes for continuous inputs like a price range slider, since pushing a new history entry on every pixel of drag would flood the browser's history stack; commit the URL update only when the user releases the slider or pauses briefly.
- For a facet with thousands of options (a brand list, for instance), virtualize the facet's own option list and add a type-ahead filter inside the panel, rather than rendering every checkbox up front.

Performance considerations

- Server-render the first page's HTML with real product data embedded, then hydrate, avoiding a client-only fetch waterfall where the user sees a spinner before anything meaningful appears.
- Reserve fixed height and aspect ratio for every product card and its image so incoming data never shifts already-rendered content, which is the same cumulative-layout-shift concern as the image gallery case study, just applied to a grid of cards instead of a feed.
- Cache result pages by normalized query string (many different users land on the exact same popular filter combination), so a CDN or an app-level cache like React Query can serve repeat requests without hitting the search backend again.
- Compute facet counts server-side as part of the search response rather than deriving them client-side from a full product list, since a full list large enough to compute counts from defeats the point of pagination.

Edge cases

- A filter combination yields zero results (a very specific brand plus a very specific size). Show an empty state that suggests removing a specific filter rather than a blank grid, which otherwise looks like the page is broken.
- The user pages forward several times, then hits back. The list should restore the exact prior page's products and scroll position from cache rather than refetching, the same URL-keyed caching pattern used for typeahead and infinite-scroll cases.
- The user rapid-fires several filter toggles before the first request resolves. Later requests can resolve before earlier ones due to network variance, so the same "ignore stale responses" fix from the typeahead case study applies here too, keyed by the query string that generated each request.

SEO considerations worth naming explicitly

- Server-render canonical HTML containing actual product data and structured data (schema.org `Product` markup) so crawlers see real content, not a shell waiting on JavaScript.
- Add `rel=canonical` tags pointing filtered or paginated URL variants back at a preferred canonical version where appropriate, to avoid duplicate-content penalties from near-identical filtered views of the same catalog.

Interview Answer:
"The decision I'd lead with is that the URL, via query params for filters, sort, and page, is the single source of truth, not a separate React state object, because that's what makes the page simultaneously shareable, back-button-correct, and server-renderable for SEO from the exact same mechanism. The server reads that same query string to render page one with real product data and facet counts already baked in, so crawlers and first paint both get real content instead of a loading shell. For the huge facets, like a brand filter with thousands of values, I'd virtualize the option list itself and add a type-ahead inside the panel rather than rendering every checkbox. I'd reserve fixed dimensions for every product card to avoid layout shift as data streams in, and I'd cache results by normalized query string so both a CDN and the back button can serve previously-seen filter combinations instantly. And I'd handle the same stale-response race condition here that shows up in a typeahead, since rapid filter toggling can easily return responses out of order."

If you remember nothing else from this chapter: *the case study that goes well isn't the one with zero mistakes, it's the one where you catch and fix your own mistake the moment it's pointed out.*

Chapter 17: When Your Tech Lead Says This... (Rapid-Fire Judgment Calls)

None of these are "design a system" questions. Each one is a single opinionated statement a tech lead, a senior engineer, or a code review comment might actually make about frontend architecture, some correct, some wrong, most somewhere in between depending on context. Use these to practice evaluating a claim on its merits instead of either agreeing with everything an authority figure says or reflexively contradicting it. The answer you want to give is rarely a flat yes or no, it's "here is when that is right, here is when it breaks, and here is what I would actually say back."

Q1. [Mid] Your tech lead says: "Let's just use CSR everywhere, SSR adds unnecessary complexity." What is your reply?

Answer:

CSR-everywhere is a reasonable default for an authenticated, logged-in app where nobody needs to index the page or see it render instantly on a slow connection, but "everywhere" ignores that different routes have very different needs.

Where it breaks

- Public, unauthenticated, or marketing pages lose SEO entirely, since a pure CSR shell is empty until JavaScript downloads, parses, and runs, and some crawlers still do not execute JavaScript reliably.
- First paint on a slow connection or low-end device is a blank screen for longer than users tolerate, exactly the pages meant to make a first impression.

A better default

- Pick the rendering strategy per route based on what that route actually needs: SSR or SSG for public and SEO-sensitive pages, CSR for authenticated, interaction-heavy app screens where an empty first paint barely matters.

The right rendering strategy is a per-route decision based on who needs to see the page and how fast, not a single blanket choice for the whole app.

Interview Answer:

"I'd push back on 'everywhere.' CSR is a fine choice for the authenticated parts of the app, but any public or SEO-sensitive page needs something that renders real content without waiting on JavaScript. I'd pick the rendering strategy per route based on what that specific page needs, not apply one strategy to the whole app."

Q2. [Junior] Your tech lead says: "We don't need a CDN, our server is fast enough." What is your reply?

Answer:

That conflates two different things: how fast the server can compute a response, and how long it takes that response to physically travel to the user. A CDN is mostly solving the second problem, not the first.

What a CDN actually buys you

- Serving static assets from a location physically closer to the user cuts network latency that no amount of server-side optimization can touch, the speed of light is the speed of light.

- It also absorbs traffic spikes and a meaningful share of malicious traffic before it ever reaches your origin server, and offloads TLS termination.

A fast server only controls compute time; a CDN controls network distance, and those are two independent bottlenecks.

Interview Answer:

"Server speed and network distance are two different problems. Even an instant server response still has to travel from wherever it's hosted to wherever the user is, and a CDN is what shrinks that distance, plus it absorbs traffic spikes before they hit our origin at all. I wouldn't skip it just because the origin itself is fast."

Q3. [Mid] Your tech lead says: "Let's lazy-load everything, including the hero image above the fold." What is your reply?

Answer:

Lazy-loading the one image that is visible the instant the page loads works against the exact metric lazy-loading is supposed to help.

Why this backfires

- The hero image is very likely the Largest Contentful Paint element on the page, and lazy-loading defers its request until the browser realizes it is needed, which directly delays LCP instead of improving it.
- Lazy-loading exists to defer requests for content the user has not scrolled to yet, applying it to something already on screen at load time removes the "lazy" part of the trade-off entirely.

Lazy-loading helps exactly the assets the user has not reached yet; applying it to the one thing visible immediately actively delays the page's most important paint metric.

Interview Answer:

"I'd only lazy-load what's actually below the fold. The hero image is probably our Largest Contentful Paint element, so lazy-loading it would delay the one metric we're usually trying hardest to protect. I'd eager-load or even preload that one image and lazy-load everything genuinely off-screen."

Q4. [Mid] Your tech lead says: "Let's just increase the cache TTL to fix the stale data complaints." What is your reply?

Answer:

That treats a symptom without asking whether the real bug is in the cache invalidation logic, and it moves in exactly the wrong direction: a longer TTL makes any given piece of stale data stay wrong for even longer.

The question to ask first

- Is data going stale because nothing invalidates the cache on write, or because the TTL genuinely needs tuning for a legitimately slow-changing resource? Those are two very different problems with two very different fixes.
- If writes are not properly invalidating or updating the cache, a longer TTL just widens the exact window where users see wrong data, it does not shrink the number of complaints, it delays them.

A longer TTL only helps if staleness was ever meant to be tolerated; if the real bug is a missing invalidation on write, a longer TTL makes the same bug visible for longer, not less often.

Interview Answer:

"Before touching the TTL, I'd want to know whether we're invalidating the cache correctly on writes. If invalidation is broken, a longer TTL just means users see stale data for even longer, it doesn't fix anything. I'd only extend the TTL if the actual problem is that we set it too aggressively short for data that's genuinely fine to serve a bit stale."

Q5. [Mid] Your tech lead says: "Let's put everything in global state, even a modal's open/close flag." What is your reply?

Answer:

Global state is for data that genuinely needs to be shared across distant, unrelated parts of the app. A modal's open/closed flag is almost always only relevant to that modal and its immediate parent, which is exactly what local component state is for.

The cost of over-using global state

- Every consumer of the global store can re-render on changes it does not actually care about, depending on the state library, which is wasted render work that local state would never trigger.
- It adds indirection, boilerplate, and a much larger surface of "what could have changed this value" for state that only ever needed to live in one component.

State should live at the smallest scope that actually needs it; global state is for data genuinely shared across the app, not a default home for every flag.

Interview Answer:

"I'd keep purely local UI state, like whether one modal is open, in that component, and reserve global state for data that's genuinely needed in multiple, unrelated parts of the app. Putting everything in global state adds re-render cost and indirection for state that never needed to leave its own component."

Q6. [Junior] Your tech lead says: "Let's just disable the lint rule that's failing the build." What is your reply?

Answer:

A failing lint rule is a signal, and disabling it removes the signal instead of addressing whatever it's pointing at. Sometimes the rule genuinely does not apply to a specific case, but that should be a deliberate, scoped exception, not a blanket disable.

What it is often actually catching

- A missing dependency in a hook, which causes a stale closure bug that will not show up until production.
- Dead code or an unused variable that is masking a mistake elsewhere in the same function.

Disabling the rule silences the alarm without checking what it was alarming about; the fix should target the actual code, or a narrowly scoped, deliberate exception, not the rule itself.

Interview Answer:

"I'd want to understand what the rule is actually flagging before disabling it, since a lot of these rules catch real bugs, like a stale closure from a missing hook dependency. If it's a genuine false positive for this one case, I'd add a scoped, commented exception right there, not turn the rule off globally."

Q7. [Senior] Your tech lead says: "Let's put the JWT in localStorage, it's simpler than cookies." What is your reply?

Answer:

It is simpler, but the simplicity comes at a real security cost that is worth naming explicitly rather than accepting quietly.

The trade-off

- Anything readable by JavaScript on the page, which is exactly what localStorage is, becomes instantly stealable the moment there is any XSS vulnerability anywhere on the page, no matter how minor. There is no mitigation once that happens.
- An httpOnly cookie is never readable by page JavaScript at all, so the same XSS bug cannot steal it directly, but cookies bring their own requirement, CSRF protection, since the browser attaches them automatically to requests.

This is a real security trade-off, not a simplicity-for-free choice: localStorage trades CSRF-safety for total exposure to any XSS bug anywhere on the page.

Interview Answer:

"I'd flag that this isn't just a simplicity choice, it's a security trade-off. A token in localStorage is fully exposed to any XSS bug anywhere on the page, with no mitigation once that happens. An httpOnly cookie protects against that specific risk but needs CSRF protection in return, since the browser sends it automatically. I'd rather take on CSRF protection, which is well understood, than leave the token readable by any injected script."

Q8. [Mid] Your tech lead says: "It's a GET request, so we don't need CSRF protection there." What is your reply?

Answer:

That is correct for a properly-built GET endpoint, and worth agreeing with, with one important condition attached.

Why it is usually true

- Classic CSRF exploits a browser automatically attaching cookies to a state-changing request the attacker's page triggers; a GET that only reads data and changes nothing has no state to corrupt even if triggered cross-site.

The condition that matters

- This only holds if the GET truly has no side effects. A GET route that secretly deletes something or changes state, a REST design mistake, would still be vulnerable, since a forged cross-site request could trigger it just as easily as a POST.

The safety of a GET from CSRF depends entirely on it having no side effects; the moment a GET changes state, the same protection a POST needs applies to it too.

Interview Answer:

"I'd agree, as long as the GET is genuinely read-only. CSRF exploits a request that changes state, so a side-effect-free GET has nothing for an attacker to corrupt. But if this GET secretly writes or deletes anything, that's a REST design problem on its own, and it would still need the same CSRF protection a POST would."

Q9. [Mid] Your tech lead says: "Let's inline all our CSS to avoid extra network requests." What is your reply?

Answer:

Inlining the critical, above-the-fold CSS is a well-known, legitimate performance technique. Inlining all of it is a different move with a real downside: it trades one kind of cost for another.

What is lost by inlining everything

- An external stylesheet is fetched once and cached by the browser across every page on the site; CSS inlined into the HTML ships again, in full, on every single page load, growing every response.

The actual best practice

- Inline only the critical CSS needed for the first paint, and load the rest as an external, cacheable stylesheet, getting the fast-first-paint benefit without losing caching for the bulk of the styles.

Critical CSS should be inlined for a fast first paint; the rest belongs in a cacheable external file, or every single page load re-downloads styles a normal stylesheet would have cached once.

Interview Answer:

"I'd only inline the critical, above-the-fold CSS, and keep the rest in an external, cacheable stylesheet. Inlining everything means every page load re-ships the full CSS with no caching benefit at all, which is a real cost for a real product with more than one page."

Q10. [Junior] Your tech lead says: "Let's show a loading spinner everywhere, users like feedback." What is your reply?

Answer:

Feedback matters, but a spinner is not the only, or always the best, form of it, and overusing it can make an app feel slower than it actually is.

Where spinners hurt more than help

- For near-instant operations, a spinner that flashes for a fraction of a second reads as jank, not feedback, and can feel worse than showing nothing at all.
- A spinner gives no sense of layout or progress; a skeleton screen shaped like the real content gives the user a sense of what is coming and avoids a jarring layout shift when the content pops in.

A skeleton screen that mirrors the eventual layout gives useful feedback without the jarring pop-in a spinner-then-content pattern creates; reserve spinners for genuinely long, indeterminate waits.

Interview Answer:

"I'd use skeleton screens shaped like the real content for most loading states, since they avoid the layout jump when content finally appears and give the user a sense of what's coming. I'd reserve an actual spinner for operations that are genuinely long and indeterminate, where there's no layout to preview yet."

Q11. [Mid] Your tech lead says: "Third-party scripts, like analytics or a chat widget, can just go in the head, order doesn't matter." What is your reply?

Answer:

Where and how a third-party script loads is one of the most well-documented performance levers on a page, and getting it wrong can block rendering on code you do not even control.

Why placement and loading strategy matter

- A blocking script tag in the head with no async or defer attribute pauses HTML parsing until it downloads and executes, and that delay is now dictated by a third party's server, not yours.
- A slow or hanging third-party script can visibly delay first paint and interactivity for the entire page, for functionality, like a chat widget, that most users never even open.

An unmanaged third-party script in the head gives an external vendor control over your page's render-blocking time; async or defer, and placement relative to critical content, is a deliberate decision, not an afterthought.

Interview Answer:

"I'd push back on that. An unmanaged script tag in the head can block rendering on a third party's server, for a widget most users never interact with. I'd load it with async or defer, or inject it after the main content is interactive, so a slow or failing third party can't hold our own page hostage."

Q12. [Mid] Your tech lead says: "Let's just increase the API timeout on the frontend to fix these errors." What is your reply?

Answer:

That treats the symptom the user sees, not the cause. If calls are timing out, something is actually slow on the backend, and a longer timeout just means the user waits longer before seeing the same underlying failure, or waits longer for a request that was always going to succeed anyway.

What is worth checking first

- Whether the slow calls share a pattern, one specific endpoint, one query, one dependency, that points at a fixable backend issue rather than a frontend configuration knob.

A longer timeout hides a backend performance problem behind a worse user experience instead of fixing it; it is a stopgap at best, not a fix.

Interview Answer:

"I'd want to understand why those calls are slow before touching the timeout, since a longer timeout doesn't fix anything, it just makes the user wait longer to hit the same problem, or wait longer for something that would have succeeded anyway. I'd treat this as a signal to dig into the backend, not a frontend configuration fix."

Q13. [Mid] Your tech lead says: "Feature flags can just be hard-coded booleans we flip and redeploy each release." What is your reply?

Answer:

That gets you version control on the flag, but it gives up the entire point of feature flagging, the ability to change behavior without a deploy.

What a hard-coded boolean cannot do

- Turn a feature off instantly if something breaks in production, without waiting on a build and deploy pipeline, which is the single most valuable property of a real flag system.

- Roll a feature out to a percentage of users, or a specific cohort, for gradual, monitored rollout rather than an all-or-nothing switch.

The entire value of a feature flag is changing behavior without a redeploy; a hard-coded boolean that needs a release to flip is just a compile-time constant wearing a feature flag's name.

Interview Answer:

"That's really just a compile-time constant, not a feature flag, since it still needs a full release to change. The value of a real flag system is being able to kill a broken feature instantly, or roll it out gradually to a subset of users, neither of which works if flipping it requires a deploy."

Q14. [Junior] Your tech lead says: "We don't need error boundaries, our code doesn't throw." What is your reply?

Answer:

Every codebase eventually throws somewhere, an unexpected API response shape, a null where one was not expected, a browser extension interfering with the page, and an error boundary is not there because current code is bad, it is there to contain the blast radius when something inevitably does throw.

What happens without one

- An uncaught error in one component can blank out the entire page for the user, rather than degrading just the one broken widget while everything else keeps working.

An error boundary is not a bet that code will fail; it is insurance so that when something eventually does fail, the damage is contained to one component instead of the whole page.

Interview Answer:

"I'd add them anyway, not because I expect this code to throw today, but because something eventually will, an unexpected response shape, a null we didn't account for, and without a boundary that takes down the whole page instead of just the one component that broke."

Q15. [Junior] Your tech lead says: "SEO doesn't matter, we're mostly an internal tool." What is your reply?

Answer:

For a genuinely internal, authenticated-only tool, that is a reasonable call, SEO exists to help strangers discover public content, and an internal tool has no strangers to discover it.

The one thing worth double-checking

- Any public-facing pages the same product might still have, a marketing site, public documentation, a status page, still need proper rendering for search and for link previews, even if the core app behind login does not.

Deprioritizing SEO for an authenticated internal tool is the right call; the risk is quietly extending that same decision to any public pages the product also happens to have.

Interview Answer:

"I'd agree for anything behind login, since there's no one to discover it via search. I'd just double-check we're not lumping in any public pages the product has, a marketing site or public docs, under the same decision, since those still benefit from being properly indexable."

Q16. [Mid] Your tech lead says: "Let's version our shared design-system components by just bumping the npm package's minor version freely." What is your reply?

Answer:

A shared design-system package is consumed by many teams at once, often silently, through a lockfile bump they did not review closely, so the usual rules about what counts as a "safe" minor change are stricter here than in a typical app.

Why this is riskier than normal

- A visual or behavioral change to a widely-shared Button or Modal component can break dozens of consuming apps at once, in ways that only show up visually, not as a compile error.

What it actually needs

- Strict semver discipline, anything visually or behaviorally different is a major version, plus visual regression testing across consumers before a release goes out, not just unit tests on the component itself.

A shared design-system package has many silent downstream consumers, so a "minor" version needs a stricter, more conservative definition than it would in a typical application package.

Interview Answer:

"I'd be more conservative here than a normal app package, since dozens of teams consume this without necessarily reviewing every bump closely. Any visual or behavioral change should be a major version, and I'd want visual regression testing across real consumers before shipping, since a broken component here breaks every app that uses it at once."

Q17. [Junior] Your tech lead says: "We'll do client-side-only validation, no need to also validate on the server." What is your reply?

Answer:

Client-side validation is purely a UX convenience, it gives instant feedback, but it provides zero actual protection, since anyone can bypass it entirely by calling the API directly, editing the request in devtools, or disabling JavaScript.

Why the server check is not optional

- The server has to independently validate and authorize every request as if the client-side check never happened, because from a security standpoint, it never did.

Client-side validation is a UX nicety the user sees; server-side validation is the actual security and data-integrity boundary, and skipping it means the boundary does not exist.

Interview Answer:

"I'd keep client-side validation for instant feedback, but the server has to validate everything independently regardless, since client-side checks are trivially bypassed by anyone calling the API directly. Skipping server validation isn't a shortcut, it just means we have no real validation at all."

Q18. [Senior] Your tech lead says: "Micro-frontends will let each team ship independently, so let's split by button-level granularity." What is your reply?

Answer:

The goal, independent shipping per team, is right, but button-level granularity is far finer than any real team boundary, and it multiplies the cost micro-frontends already carry without buying anything extra.

What over-fragmentation costs

- Every micro-frontend boundary can mean its own copy of a framework and shared dependencies loaded at runtime, so splitting too finely multiplies duplicate payloads across a single page instead of just at real team boundaries.
- Coordination overhead, contracts between fragments, shared design-system versioning, scales with the number of boundaries, not with team count, so tiny fragments add coordination cost without adding real team independence.

Micro-frontend boundaries should match actual team and domain boundaries; splitting more finely than that multiplies runtime and coordination cost without buying any additional independence.

Interview Answer:

"I'd align the split with actual team boundaries, a whole page or a clear product domain, rather than something as fine as a single button. Splitting that granularly multiplies duplicate runtime payloads and coordination overhead without giving us any more independence than a coarser, team-aligned split already would."

Q19. [Mid] Your tech lead says: "We can just poll the API every second for real-time updates." What is your reply?

Answer:

Polling is a legitimate, simple default for low-frequency updates, but "every second" for something meant to feel real-time is worth questioning on cost grounds, not correctness grounds.

Why the interval matters

- Load scales directly with concurrent open tabs, a thousand users polling every second is a thousand requests per second doing mostly nothing, since most polls return unchanged data.

When to reach for something else

- If updates are frequent and truly need to feel instantaneous, a WebSocket or Server-Sent Events connection pushes only when something actually changes, instead of paying for constant "anything new?" requests regardless of the answer.

Polling cost scales with concurrent users regardless of whether anything changed; a push-based connection only costs something when there is actually something to send.

Interview Answer:

"Polling is a fine simple default, but a one-second interval multiplied by every open tab adds up fast for mostly-unchanged data. If the update frequency and real-time feel genuinely matter, I'd move to WebSockets or Server-Sent Events, which only cost something when there's actually an update to push."

Q20. [Junior] Your tech lead says: "Since we use HTTPS, XSS isn't really a concern anymore." What is your reply?

Answer:

HTTPS and XSS protect against two completely unrelated threats, and one has no bearing on the other.

What each actually protects

- HTTPS protects data in transit between the browser and the server, confidentiality and integrity against anyone listening on the network in between.
- XSS is about an attacker getting their own script to execute inside your page, in the same trusted origin as your legitimate code, which HTTPS does nothing to prevent since the malicious script arrives over the same secure connection as everything else.

HTTPS secures the pipe the data travels through; XSS is about what runs inside the page once it arrives, and a perfectly encrypted connection can deliver an XSS payload just as easily as a legitimate one.

Interview Answer:

"Those are unrelated problems. HTTPS protects data in transit from network eavesdroppers, but it has nothing to say about a malicious script executing inside our own page, which is what XSS is. We still need output encoding, a Content Security Policy, and careful handling of user-generated content regardless of HTTPS."

Q21. [Senior] Your tech lead says: "Let's cache the entire page, including the user's personalized data, at the CDN edge." What is your reply?

Answer:

This is one of the more dangerous mistakes in this list: a shared edge cache serving personalized content risks leaking one user's private data to a completely different user.

How the leak happens

- If the cache key does not vary by identity, the first user's personalized response gets cached and then served as-is to the next user who hits the same URL, showing them someone else's data.

The safer pattern

- Cache the shared shell or non-personalized parts of the page at the edge, and fetch the personalized pieces separately, client-side or via a request that is explicitly excluded from the shared cache.

A shared edge cache must never serve personalized content under a cache key that does not vary by identity, since that is exactly how one user ends up seeing another user's data.

Interview Answer:

"I'd treat this as a real security risk, not a performance tweak. If the cache key doesn't vary per user, we risk serving one user's personalized data to the next person who requests the same URL. I'd cache the non-personalized shell at the edge and fetch personalized data through a path that's explicitly excluded from shared caching."

Q22. [Mid] Your tech lead says: "Let's just raise the bundle size limit in CI since the build keeps failing." What is your reply?

Answer:

A bundle size budget exists precisely to catch a slow, creeping regression before it reaches users; raising the ceiling every time it is hit turns the budget into a formality that never actually blocks anything.

What to check first

- What actually pushed the bundle over the limit, a genuinely necessary new dependency, an accidental duplicate copy of a library, or code that should have been code-split instead of bundled into the main chunk.

A budget that gets raised every time it fails is not a budget anymore; the size increase deserves investigation before the ceiling gets moved to accommodate it.

Interview Answer:

"I'd want to know what actually pushed us over the limit before raising it, since a budget that moves every time it fails stops being a real check. If it's a genuinely necessary dependency, raising the limit deliberately makes sense, but if it's a duplicate library or something that should have been code-split, that's the actual fix."

Q23. [Junior] Your tech lead says: "Everyone on the team is on fast office WiFi, so we don't need to worry much about performance." What is your reply?

Answer:

The team's own network and devices are close to the least representative sample of the real user base you could pick, and performance problems that are invisible on a fast office connection can be the entire experience for a real user on a mid-range phone or a slow connection.

What this leads to

- Performance regressions ship unnoticed because nobody on the team ever experiences them, and they only surface once real users, on real networks and devices, start complaining.

Testing performance only on the team's own fast, modern setup measures the best case a real user will basically never experience.

Interview Answer:

"I'd push back on using our own setup as the bar, since office WiFi and team devices are close to the best case, not a representative case. I'd want us testing under throttled network and mid-range device conditions that actually match our real users, since problems invisible to us can be the whole experience for them."

Q24. [Mid] Your tech lead says: "One global isLoading flag is enough for the whole app." What is your reply?

Answer:

A single shared flag collapses every independent async operation into one bit of information, which breaks down the moment more than one thing can be loading at once.

How it actually breaks

- One request finishing can flip the flag back to false while a completely unrelated, still-pending request is still in flight, so part of the UI shows "loaded" while it is actually still waiting on real data.
- Two different loading spinners on screen at once, for two unrelated pieces of data, end up controlled by the same single flag, so they cannot be shown or hidden independently.

A single shared loading flag cannot represent more than one thing happening at once; it needs to be tracked per request or per resource, not globally.

Interview Answer:

"That works fine as long as only one thing is ever loading at a time, which almost never stays true. I'd track loading state per request or per resource instead, otherwise one request finishing can incorrectly flip the flag while something else entirely is still in flight."

Q25. [Mid] Your tech lead says: "Let's just retry every failed API call automatically, forever, until it succeeds." What is your reply?

Answer:

Unbounded retries can turn a brief backend hiccup into a self-inflicted overload the moment the backend recovers, since every client that was silently retrying suddenly succeeds and piles on at once.

What is missing

- A retry limit, so a genuinely broken request eventually surfaces as a clear failure instead of hanging silently forever.
- Backoff between attempts, so retries spread out over time instead of hammering a recovering backend all at once, and ideally some jitter so many clients don't retry in lockstep.

Retries need a bound and a backoff; retrying forever with no limit just delays a visible failure and risks a thundering-herd pile-on the moment the backend recovers.

Interview Answer:

"I'd cap the number of retries with exponential backoff, rather than retrying forever. Unbounded retries with no backoff can turn a brief backend blip into a flood of requests the moment it recovers, and the user is left staring at something that never resolves either way."

Q26. [Mid] Your tech lead says: "Accessibility is something we'll add at the end, right before launch." What is your reply?

Answer:

Accessibility is far cheaper to build in from the start than to retrofit, since a lot of it is about the underlying structure of components, semantic markup, focus order, keyboard navigation, not a coat of paint applied afterward.

Why bolting it on later costs more

- A component built without keyboard and screen-reader support in mind often needs real structural changes, not just added ARIA attributes, to become genuinely accessible, which is a much bigger job after the fact than during initial design.

Accessibility built into component design from the start is a design decision; accessibility bolted on right before launch is usually a rebuild wearing a deadline.

Interview Answer:

"I'd push to build accessibility in as we design each component, not as a pass at the end, since a lot of it, keyboard navigation, focus order, semantic structure, is about how a component is built, not something you can sprinkle on afterward without real rework."

Q27. [Mid] Your tech lead says: "We can detect the user's device from the User-Agent string and serve completely different code based on it." What is your reply?

Answer:

User-Agent sniffing is a fragile foundation for this: the string is trivially spoofable, changes across browser versions in ways that are hard to keep up with, and does not reliably tell you what the device or browser can actually do.

A more reliable approach

- Responsive design and feature detection, checking whether a specific API or capability actually exists, describe what the environment can do directly, instead of guessing from a string that was never designed to be a reliable capability signal.

Where UA-based branching is still reasonable

- A narrow case like redirecting to a native app deep link is a defensible, limited use, but branching core rendering or behavior on UA parsing is a maintenance liability.

A capability should be detected by checking whether it exists, not inferred from a spoofable, constantly-changing string that was never designed to answer that question reliably.

Interview Answer:

"I'd be cautious about branching core behavior on User-Agent parsing, since it's spoofable and drifts constantly as browsers update. I'd reach for responsive design or actual feature detection, checking whether a capability exists, instead of guessing from a string. A narrow case like an app deep-link redirect is fine, but not our core rendering logic."

Q28. [Junior] Your tech lead says: "Let's just disable the back button so users can't leave the checkout flow." What is your reply?

Answer:

Fighting the browser's native navigation breaks a deeply ingrained user expectation, and it usually does not even work reliably, since a user can still close the tab, type a URL, or use forward/back in ways a page cannot fully intercept.

A better way to handle it

- Design the flow to be resumable and handle partial state gracefully, and if leaving mid-flow needs a warning, use an explicit, standard "are you sure you want to leave" prompt instead of trying to trap the user in place.

Users leaving a flow unexpectedly is a state-management problem to solve with resumability, not a navigation problem to solve by disabling a browser control that never fully disables anyway.

Interview Answer:

"I'd avoid trying to disable the back button, since that fights a control users expect to work, and it's not even reliable, there are other ways to leave a page a script can't block. I'd instead make the checkout flow resumable and use an explicit 'are you sure' prompt if leaving mid-flow would lose real progress."

Q29. [Junior] Your tech lead says: "We'll store the shopping cart only in React state, no persistence needed." What is your reply?

Answer:

That loses the entire cart the moment the user refreshes the page, closes the tab, or the app crashes, which is a common, ordinary action, not an edge case, and a frustrating way to lose a customer's intent to buy something.

What to persist it to

- localStorage or a backend-synced cart, depending on whether guest, logged-out carts need to survive too, so a reload restores the cart instead of silently emptying it.

In-memory-only state disappears on every refresh, which is one of the most ordinary things a user does, so anything worth keeping across a reload needs to live outside React state.

Interview Answer:

"I'd persist the cart to localStorage or sync it to the backend, since keeping it only in React state means a simple refresh, something users do constantly and without thinking, wipes out everything they've added. That's a real, frequent frustration, not an edge case worth ignoring."

Q30. [Senior] Your tech lead says: "Since our app is a SPA, we don't need to worry about the browser's back and forward buttons." What is your reply?

Answer:

That is backwards. A traditional multi-page site gets back and forward navigation for free from the browser, but a single-page app doing its own client-side routing has to explicitly manage browser history, or those buttons stop working correctly.

What breaks without explicit handling

- Without pushing state into browser history on each client-side navigation, pressing back can do nothing, or exit the app entirely, instead of moving to the previous in-app view the user actually expects.

A SPA needs more deliberate handling of browser history, not less, since it owns routing itself instead of getting it from the browser automatically.

Interview Answer:

"I'd say the opposite is true. A normal multi-page site gets back and forward for free from the browser; a SPA has taken over routing itself, so it has to explicitly push state into browser history on every navigation, or the back button either does nothing or exits the app instead of going to the previous screen."

If you remember nothing else from this chapter: *agreeing with everything an authority figure says, or disagreeing with everything they say, gives away less confidence than saying "it depends, and here's when."*

Chapter 18: Top 15 Most Asked Frontend System Design Questions

These fifteen prompts are the frontend equivalent of "design a URL shortener": they show up in almost every senior frontend loop, in some form, at almost every company that ships a consumer or B2B product at scale. Use this chapter as a fast cram sheet. Aim to structure the whole system in a few minutes (clarify scope, sketch components, describe data flow, name the one or two trade-offs that actually matter) without needing it dragged out of you. Jumping straight to component names and CSS before establishing what data moves where, or not mentioning a single edge case unprompted, are both weak habits worth catching in yourself now. Asking clarifying questions before designing is often the single best signal of seniority in this format, so make it a reflex.

Q1. [Mid-Senior] Design the frontend architecture for a Twitter/X-like social feed. Walk me through it.

Answer:

This is fundamentally a data-freshness and rendering-performance problem disguised as a UI problem. **The core design decision is how you cache and normalize posts so the same post rendered in the home feed, a profile, and a reply thread stays consistent when its like count or content changes.**

Clarifying questions to ask

- Is this an infinite-scroll feed or paginated? Is there a "ranked" feed (algorithmic) or strictly chronological?
- Do new posts appear live (a "10 new posts" banner) or only on manual refresh/pull-to-refresh?
- What actions are optimistic (like, retweet, follow) versus requiring a server round trip before UI updates?

Architecture / component breakdown

- **Feed container**: owns pagination state (a cursor, an opaque token from the server marking where the next page starts) and orchestrates fetch-more-on-scroll.
- **List virtualization layer** (a technique that only renders DOM nodes for rows currently in or near the viewport, e.g. via `react-window`): keeps a feed with thousands of posts from degrading scroll performance.
- **Post component**: presentational, memoized, receives a normalized post object plus dispatch functions for like/retweet/reply.
- **Composer**: separate from the feed, writes into the same normalized cache so a newly posted tweet appears without a full refetch.
- **New-content banner**: a lightweight poller or WebSocket listener that only fetches counts/ids, not full post bodies, until the user taps it.

Data flow and key design decisions

- Store posts in a normalized cache keyed by post id (like a small client-side database table) rather than as flat arrays per view, since the same post object needs to update everywhere it appears when its like count changes.
- Each view (home feed, profile feed, thread) holds only an ordered list of ids that points into that shared cache.

- Likes/retweets are applied optimistically (the UI updates instantly, before the server confirms) with a rollback path if the request fails, because waiting on a round trip for every tap feels broken at this scale.
- Use cursor-based pagination, not page numbers, since posts are inserted continuously and offset-based paging causes skipped or duplicated rows.

Edge cases

- New posts arrive while the user is mid-scroll through older pages: cursor-based pagination avoids re-showing or skipping posts, but the "new posts" banner must not silently reorder content under the user's thumb.
- An optimistic like fails server-side (e.g. rate limited): the UI must roll back the count and show a toast, not leave the client and server permanently disagreeing.

Interview Answer:

"I'd start by clarifying whether the feed is chronological or ranked, and whether new content pushes live or waits for refresh, because that changes the whole data layer. Architecturally I'd keep a normalized cache of posts keyed by id, since the same post shows up in the home feed, a profile, and a thread, and I want a like or edit to reflect everywhere at once. Each view just holds an ordered list of ids pointing into that cache. I'd virtualize the list so we're not rendering thousands of DOM nodes, use cursor-based pagination so inserts don't skip or duplicate rows, and make likes and retweets optimistic with a rollback path for failures. The trickiest edge case is new posts landing while someone's scrolled deep into history, so I'd surface those via a manual 'new posts' banner instead of silently reordering content under them."

Q2. [Mid-Senior] Design the frontend for a real-time chat application. What's the architecture?

Answer:

Chat looks like a simple list-and-input UI, but the real design problem is connection reliability and message ordering under a flaky network. **The system has to treat every outgoing message as "pending until acknowledged" and be able to resync cleanly after any connection drop, because you cannot assume the WebSocket stays open.**

Clarifying questions to ask

- 1:1 only, or group chats too? Read receipts and typing indicators?
- Does the app need to work offline and queue messages for later send?
- Single device or does state need to sync across multiple logged-in devices?

Architecture / component breakdown

- **Connection manager**: owns the WebSocket (a persistent two-way connection, unlike a normal HTTP request/response) lifecycle, including reconnect-with-backoff (waiting progressively longer between retry attempts) when the connection drops.
- **Per-conversation message store**: keyed by conversation id, holding messages in server-assigned order.
- **Outgoing queue**: messages the user sent that are not yet acknowledged, rendered with a "sending" state.
- **Presence/typing layer**: a low-priority side channel, separate from message delivery, that can drop updates without breaking correctness.

Data flow and key design decisions

- New message typed: added to the local store immediately in a "pending" state (optimistic UI), sent over the socket with a client-generated unique id, then reconciled (marked "sent") when the server acknowledges it by that id.
- Order messages by server-assigned sequence number or timestamp, not client send time, since two devices' clocks and network latency can disagree.
- Persist the outgoing queue to local storage (e.g. IndexedDB) so a message typed while offline survives an app reload and sends once connectivity returns.
- On reconnect, the client requests "everything since sequence N" rather than blindly trusting the socket resumed cleanly, to catch anything missed while disconnected.

Edge cases

- Network drops for 30 seconds mid-conversation: on reconnect, the client must detect the gap and backfill missed messages rather than just resuming from wherever the socket happens to reconnect.
- The same message gets delivered twice (a common failure mode of at-least-once delivery systems): the client-generated message id must be used to deduplicate, so retries don't produce duplicate bubbles.

Interview Answer:
"The UI is the easy part; the hard part is treating the network as unreliable by default. I'd have a connection manager that owns the WebSocket and reconnects with backoff, and every outgoing message gets a client-generated id and a 'pending' state until the server acks it, so sends feel instant but we can reconcile or roll back cleanly. Messages get ordered by server sequence number, not client timestamp, because clocks and latency can't be trusted. I'd persist the outgoing queue locally so a message typed offline survives a reload. The edge case I'd call out unprompted is reconnecting after a drop: the client needs to ask for everything since the last known sequence number, not just resume the socket and hope nothing was missed, and duplicate delivery needs to be deduplicated by that same client-generated id."

Q3. [Senior] Design the frontend for a collaborative document editor like Google Docs.

Answer:

The interesting problem is not the text editor UI, it is merging simultaneous edits from multiple people without losing anyone's changes or requiring a lock. **The whole architecture hinges on choosing a conflict resolution strategy, OT or CRDT, that lets every client apply local edits instantly and merge remote edits deterministically without a central "who wins" decision at the server.**

Clarifying questions to ask

- How many simultaneous editors realistically, two people or two hundred?
- Does it need to work offline and reconcile later, or is it always-online?
- Do we need live cursors/presence and comments, or just the document body?

Architecture / component breakdown

- **Editor core**: the text-editing surface (contentEditable-based or a custom canvas renderer) that produces a stream of local operations (insert, delete, format) rather than just a final string.
- **Sync engine**: translates local operations into a wire format, sends them to the server, and applies incoming remote operations to the local document model.

- **Presence layer**: a separate lightweight channel broadcasting cursor positions and selections, decoupled from the document content channel since losing a cursor update is harmless but losing a text edit is not.
- **Local document model**: the client's source of truth for rendering, kept in sync with, but not blocked by, the server.

Data flow and key design decisions

- **OT (Operational Transformation)** versus **CRDT (Conflict-free Replicated Data Type)**: OT transforms each operation against concurrent ones using a central server as arbiter of order; CRDTs are data structures designed so merges are automatically consistent regardless of order, which is friendlier to offline/peer-to-peer but has more client-side complexity and memory overhead.
- **Best Choice**: CRDT for a product that must support long offline editing sessions and later reconciliation; OT is simpler to reason about if the server is always in the loop and offline support is minimal.
- **Why?**: OT needs a central sequencing authority to transform operations correctly, so it degrades poorly when a client has been offline for a while and has a large backlog of local edits to reconcile.
- Local edits are applied to the document immediately (local-first), then queued and sent to the sync engine, so typing never waits on network round trips.
- Autosave/sync is continuous (streaming small ops), not a periodic "save" button, since the whole point is that everyone sees near-live changes.

Edge cases

- Two users edit the same paragraph at the same instant: the merge algorithm (OT or CRDT) must produce a deterministic result both clients converge to, without either user's keystrokes silently disappearing.
- A client reconnects after being offline for an hour with a large batch of local edits: the sync engine needs a resync strategy (replay ops in order, or a snapshot diff) rather than assuming a clean incremental catch-up.

Interview Answer:

"The UI is a rich text editor, but the actual system design question is conflict resolution. I'd design the editor to emit a stream of operations, inserts, deletes, formatting, rather than just tracking a final string, and route those through a sync engine that either does operational transformation against a central server or uses a CRDT so merges are automatically consistent. I'd lean CRDT if the product needs real offline support, since OT relies on a server to sequence operations and that breaks down when a client has a large backlog after being offline. Presence, like live cursors, I'd put on a totally separate channel from the document content, since dropping a cursor update is harmless but dropping a text edit is not. The edge case I'd flag is a client reconnecting after a long offline stretch with a big batch of local edits still needing a real resync strategy, not just an assumption that the socket reconnecting fixes everything."

Q4. [Mid-Senior] Design the frontend for a video streaming platform's watch page.

Answer:

The watch page has one latency-critical component (the video player) surrounded by several non-critical ones (comments, recommendations, related metadata), and the design decision is making sure the non-critical parts never block or degrade the critical one. **The player itself should almost never be built from scratch; the real frontend design work is around adaptive playback, buffering resilience, and making sure everything else on the page loads without competing with the video for bandwidth or main-thread time.**

Clarifying questions to ask

- Live stream, video-on-demand (VOD), or both? Does autoplay-next matter?
- Are there ads, and do they need to be woven into the same player instance?
- What telemetry does the business need (watch time, buffering events, quality switches)?

Architecture / component breakdown

- **Player wrapper**: a thin layer around a proven player library (e.g. hls.js or shaka-player) handling adaptive bitrate streaming (automatically switching video quality based on measured bandwidth) rather than reinventing that logic.
- **Buffering/bandwidth manager**: usually built into the player library, but the wrapper needs to surface buffering state to the UI (spinner, quality indicator).
- **Below-the-fold content**: recommendations rail, comments, description, all lazy-loaded and code-split so they don't compete with the player for the initial bandwidth and JS execution budget.
- **Analytics beacon**: batches playback events (play, pause, seek, quality change, buffering duration) and sends them on an interval or on page unload, not one network call per event.

Data flow and key design decisions

- On page load, fetch the video manifest (a file describing available quality levels/segments) first and prioritize it over everything else on the page; recommendations and comments fetch after or in parallel but render is deferred.
- Adaptive bitrate switching happens client-side based on a rolling bandwidth estimate, trading a brief quality drop for avoiding a full stall.
- Prefetch the next video's manifest (not the full video) when autoplay-next is likely, so the transition feels instant without wasting bandwidth on content that might not get watched.

Edge cases

- Network degrades mid-playback: the player must drop quality and keep playing rather than stalling, and the UI must clearly show "lower quality due to network" rather than silently confusing the user.
- User seeks to an unbuffered part of the video: the UI needs a distinct "seeking/loading" state, separate from initial load, so it isn't mistaken for a broken player.

Interview Answer:
"I would not build a video player from scratch, I'd wrap something like hls.js and focus the design on everything around it. The manifest fetch and adaptive bitrate switching get top priority on page load, and I'd defer or lazy-load the recommendations rail and comments so they're not competing with the player for bandwidth or main-thread time during that critical first few seconds. Analytics events get batched rather than fired individually, since play/pause/buffering events can be frequent. The edge case I'd bring up unprompted is mid-playback network degradation: the player should drop quality and keep playing instead of stalling, and the UI should clearly communicate that a quality drop happened rather than leaving the user thinking something's broken."

Q5. [Mid-Senior] Design the frontend architecture for an e-commerce checkout flow.

Answer:

Checkout is where correctness matters more than anywhere else in the app, since a bug here either loses a sale or double-charges a customer. **The two design decisions that matter most are treating the checkout as**

a state machine gated by validation at each step, and making order submission idempotent so a retry or double-click can never create two orders or two charges.

Clarifying questions to ask

- Guest checkout allowed, or account required? Multiple payment methods/providers?
- Single scrolling page or a multi-step wizard? Any inventory or price changes possible mid-checkout (flash sale, low stock)?
- Do we need to support saved addresses/cards from a previous session?

Architecture / component breakdown

- **Checkout state machine**: explicit steps (cart review, shipping, payment, confirmation), each with a defined "can I advance" validation gate.
- **Form state layer**: holds shipping/billing data; validated both client-side (for instant feedback) and server-side (as the authoritative check, since client validation can be bypassed).
- **Payment field isolation**: card fields are typically rendered inside an iframe or hosted component owned by the payment provider (e.g. Stripe Elements), so raw card numbers never touch the merchant's own JavaScript, for PCI compliance (the card industry's security standard).
- **Order submission service**: a single call that creates the order, guarded by an idempotency key (a unique token per checkout attempt so retries don't create duplicates).

Data flow and key design decisions

- Refetch cart/price/stock state at the start of the payment step, not just at cart-add time, since prices and stock can change between when someone adds an item and when they actually pay.
- Attach an idempotency key to the "place order" request generated once per checkout attempt; if the request is retried (double-click, network blip, back-button resubmit), the server recognizes the key and returns the original result instead of creating a second order.
- Single-page checkout reduces navigation friction but makes partial-state validation harder; a multi-step wizard is easier to validate incrementally and instrument for drop-off analytics.
- **Best Choice**: multi-step wizard for anything with meaningful shipping/payment complexity.
- **Why?**: it lets you validate and save progress at each gate, and gives product teams step-by-step funnel analytics to see exactly where users abandon.

Edge cases

- User submits payment, then hits the browser back button and hits "place order" again: the idempotency key must prevent a duplicate charge.
- An item goes out of stock or changes price between adding to cart and reaching the payment step: the UI must surface that clearly before charging, not silently charge the old price.

Interview Answer:

"I'd model checkout as an explicit state machine with steps like cart review, shipping, and payment, each gated by its own validation, rather than one big form. Payment fields I'd isolate inside the provider's hosted component, like Stripe Elements, so card data never touches our own code, for PCI compliance. The design decision I'd highlight is idempotency: the final 'place order' call needs a unique key per checkout attempt so a retry, a double-click, or a back-button resubmit can't create a duplicate order or double-charge the customer. I'd also refetch price and stock right before payment, not trust what was cached from when the item was added to cart, since prices and inventory can change in that window."

Q6. [Senior] Design the frontend for a ride-sharing app's live map/tracking screen.

Answer:

The map itself is a solved problem via existing SDKs; the design work is making choppy, infrequent location pings look like smooth, continuous motion without misleading the user about the driver's real position. **The key decision is interpolating (smoothly animating between two known points) the driver's marker on the client rather than snapping it directly to each raw location update, while still reconciling to server truth often enough that the interpolation never drifts far from reality.**

Clarifying questions to ask

- How often does the server push location updates, every second, every five seconds?
- Does the screen need to work when the app is backgrounded, or only foregrounded?
- Is ETA recalculated client-side from position, or always pushed from the server?

Architecture / component breakdown

- **Map layer**: a third-party SDK (Mapbox, Google Maps) handling tiles, zoom, and base rendering.
- **Location stream handler**: receives periodic driver location pushes (typically over WebSocket) and feeds them into an interpolation layer.
- **Marker interpolation/animation layer**: smoothly animates the marker's screen position between the last two known real coordinates over the expected interval, instead of teleporting it on each update.
- **ETA/status banner**: a separate, lower-frequency UI element that doesn't need to re-render on every animation frame.

Data flow and key design decisions

- Server pushes a raw lat/lng roughly every few seconds; the client animates the marker smoothly across that interval using the previous and current point, rather than jumping the marker instantly, which looks broken (cars don't teleport).
- Throttle re-renders of anything not directly tied to the animated marker (like the ETA text) to avoid re-rendering the whole screen on every location tick.
- Balance update frequency against battery and bandwidth on the driver's device; pushing location every second is smoother but costs more battery and data than every five seconds.

Edge cases

- GPS signal briefly drops or jumps (e.g. driving through a tunnel), causing a wildly inaccurate coordinate: the client should smooth over or discard clear outliers rather than snapping the marker to a nonsensical position.
- App is backgrounded for a few minutes then foregrounded: on resume, the client should fetch a fresh location snapshot immediately rather than resuming stale interpolation from before it was backgrounded.

Interview Answer:

"The map rendering itself I'd hand off entirely to an SDK like Mapbox; the actual design problem is making infrequent location pings feel like smooth motion. I'd animate the driver marker between the last two known points over the expected update interval, client-side, rather than snapping it directly to each raw update, since a teleporting car marker looks broken. I'd keep the ETA and status banner rendering separately and at a lower frequency so they're not re-rendering every animation frame. For edge cases, GPS glitches like a tunnel causing a wild coordinate jump need outlier smoothing so the marker doesn't snap

somewhere nonsensical, and resuming from a backgrounded app should trigger an immediate fresh location fetch rather than resuming a stale interpolation."

Q7. [Mid-Senior] Design the frontend for a large-scale admin dashboard with dozens of independent data widgets.

Answer:

The failure mode this design has to prevent is one slow or broken widget taking down the whole dashboard. **Each widget should be architected as an isolated unit that owns its own data fetching, loading state, and error boundary, so a failure or slowdown in one widget's API never blocks or crashes the rest of the page.**

Clarifying questions to ask

- Roughly how many widgets, and do they share underlying data sources or are they fully independent?
- Is the layout user-customizable (drag/resize/add widgets), or fixed?
- Do widgets need to auto-refresh, and if so, does staggering matter for backend load?

Architecture / component breakdown

- **Widget shell**: a common wrapper providing consistent loading, error, and empty states around whatever a specific widget renders, so individual widgets don't each reinvent that handling.
- **Grid/layout engine**: manages widget positions and sizes, persisting user customization if the layout is editable.
- **Per-widget data hook**: each widget's own fetch/poll logic, isolated from others, so one widget's refresh schedule or API shape has zero coupling to another's.
- **Shared cache layer**: for cases where multiple widgets query overlapping underlying data (e.g. two widgets both need "orders this week"), a shared cache avoids duplicate network requests without coupling the widgets' rendering logic together.

Data flow and key design decisions

- Each widget fetches and owns its own loading/error/success state; wrap each in an error boundary (a component that catches rendering errors in its subtree) so a widget crash renders a small inline error instead of blanking the whole dashboard.
- Code-split and lazy-load widgets below the fold, since a dashboard with thirty widgets should not ship and execute all thirty widgets' JS on initial load if only six are visible.
- Stagger polling intervals across widgets (e.g. slight random jitter added to each widget's refresh timer) rather than all widgets refreshing on the exact same tick, to avoid a "thundering herd" spike of simultaneous requests hitting the backend.

Edge cases

- One widget's API is slow or times out: it must show its own loading/error state indefinitely without blocking the rest of the dashboard from rendering or interacting.
- A user rearranges widgets and reloads the page: the custom layout needs to persist (server-side per-user preference, not just in-memory), or the customization is pointless.

Interview Answer:

"The main design principle here is isolation: each widget should own its own data fetching, loading state, and error boundary, so one broken or slow widget can't take down the rest of the dashboard. I'd wrap each widget in a common shell that standardizes loading and error UI, then code-split and lazy-load widgets that

are below the fold so we're not shipping and executing thirty widgets' worth of JS upfront. For refresh, I'd stagger polling intervals with a bit of jitter across widgets rather than having all of them hit the backend on the same tick, since that thundering-herd pattern is a real production issue at scale. The edge case worth calling out is a single slow API: it should degrade gracefully to its own inline error state, not hang or crash the whole page."

Q8. [Mid-Senior] Design the frontend for a photo-sharing app's image upload and gallery flow.

Answer:

The two hard parts are making uploads resilient on bad mobile networks and making the gallery feel instant despite uploads taking real time. **Uploads should be resumable and the gallery should show an optimistic local preview the moment a photo is selected, well before the server confirms it, so the app never feels like it's waiting on the network.**

Clarifying questions to ask

- Single or multi-file upload? Any client-side compression/resizing requirement?
- Does upload need to survive the app being closed or the network dropping mid-transfer?
- How large is the gallery expected to get, hundreds or tens of thousands of photos per user?

Architecture / component breakdown

- **Upload queue manager**: tracks each pending upload's state (queued, compressing, uploading, done, failed) independent of the gallery UI.
- **Client-side image processing pipeline**: resizes/compresses the image and strips EXIF metadata (like embedded GPS location) before upload, for both bandwidth and privacy reasons.
- **Resumable upload client**: uploads in chunks with the ability to resume from the last successful chunk after a network interruption, rather than restarting a large file from zero.
- **Gallery grid**: virtualized for large collections, using low-res placeholder images (e.g. a blurred thumbnail, sometimes called blurhash) while the full-resolution image loads.

Data flow and key design decisions

- On file selection, immediately insert a local preview into the gallery (using the local file object, before any network call) so the user sees their photo appear instantly; replace it with the server-hosted version once the upload completes.
- Compress/resize on the client before uploading, trading a bit of CPU time for a meaningfully smaller upload, which matters a lot on cellular connections.
- Use a resumable/chunked upload protocol for anything beyond a trivial file size, since restarting a 20MB upload from byte zero after a network blip is a bad experience.

Edge cases

- User backgrounds or closes the tab mid-upload: the queue needs to either continue in a background task/service worker or clearly mark the upload as incomplete and resumable on next open, rather than silently losing it.
- A batch upload of 50 photos has 3 fail: those 3 need individual retry affordances rather than forcing the user to re-select and re-upload all 50.

Interview Answer:

"I'd design this around two things: an upload queue that's resumable, and an optimistic gallery that shows the photo locally the instant it's selected, before the network is even involved. Client-side I'd resize and compress images and strip EXIF metadata before upload, both for bandwidth and privacy. For anything beyond a small file, I'd use chunked, resumable uploads so a network blip doesn't mean restarting a large file from scratch. The gallery itself needs virtualization at scale and low-res placeholders while full images load. The edge case I'd flag is a batch upload where a handful of files fail: those need individual retry, not an all-or-nothing re-upload of the whole batch."

Q9. [Mid-Senior] Design the frontend architecture for a multi-step onboarding/signup flow that supports save-and-resume.

Answer:

The feature that makes this harder than a normal form wizard is "resume," which means progress cannot live only in the browser tab that started it. **Progress has to be persisted server-side as a draft record tied to the user's account, not just cached in local storage, or resuming on a different device or after clearing browser data simply won't work.**

Clarifying questions to ask

- Can users resume on a different device, or only the same browser?
- Are all steps required, or can some be skipped and revisited later?
- Does the flow itself ever change (steps added/reordered) in a way that could affect users mid-flow?

Architecture / component breakdown

- **Step router / state machine**: decoupled from the visual step components, tracking which step the user is on and which are complete, independent of the URL or component tree.
- **Draft persistence layer**: saves each step's data to the server as it's completed (a "draft" record keyed by user, not by browser session).
- **Local cache**: mirrors the server draft locally for instant back/forward navigation without a network round trip on every step change.
- **Step validation gates**: each step defines what "complete" means before allowing advancement, and re-validates on resume in case previously entered data has gone stale.

Data flow and key design decisions

- Each completed step triggers a save to the server-side draft, not just a local state update, since local-only storage (e.g. localStorage) breaks resuming on a different device or browser.
- On resume, fetch the draft, restore to the furthest completed step, and re-validate each field, in case something (a promo code, an available username) is no longer valid.
- Version the flow definition itself (e.g. store which flow version a draft was created under) so that if the onboarding steps get reordered or changed by a later deploy, an in-progress user doesn't land in a broken half-migrated state.

Edge cases

- A user starts onboarding, abandons it for two weeks, and a promo code or reserved username they entered has since expired or been taken: resuming must re-validate rather than blindly trusting stale saved data.

- The product team ships a redesigned onboarding flow with different steps while some users still have in-progress drafts under the old flow: the draft needs a flow version so old in-progress users don't break.

Interview Answer:
"The key requirement that shapes this design is save-and-resume across devices, which means progress can't just live in localStorage, it has to be a server-side draft tied to the account. I'd decouple a step router or state machine from the actual step components, save progress to that draft after each completed step, and mirror it locally for snappy back-and-forth navigation. On resume, I'd re-validate the saved data rather than trust it blindly, since something like a promo code or reserved username could have gone stale in the meantime. One edge case I'd bring up unprompted is versioning the flow itself: if the onboarding steps get redesigned later, users with an in-progress draft under the old flow shouldn't land in a broken, half-migrated state."

Q10. [Mid-Senior] Design the frontend for a live sports score or stock ticker style real-time updating page.

Answer:

The failure mode unique to this kind of page is a burst of updates (a goal, a market swing) causing so many re-renders that the UI falls behind or freezes. **The design should multiplex all live data over a single shared connection and batch incoming updates into the UI on a fixed interval, rather than re-rendering on every single message, so a burst of activity degrades gracefully instead of overwhelming the render loop.**

Clarifying questions to ask

- How many symbols/games can realistically be on screen at once?
- What's the acceptable latency, sub-second, or is a couple seconds fine?
- Is there a fallback needed for environments where WebSocket isn't available (some corporate networks)?

Architecture / component breakdown

- **Single shared connection manager**: one WebSocket (or similar) for the whole page, not one per widget/ticker, since dozens of separate connections for dozens of symbols doesn't scale.
- **Subscription manager**: multiplexes which symbols/games the client currently cares about over that single connection, subscribing/unsubscribing as the user scrolls or navigates.
- **Keyed live-data store**: holds the latest value per symbol/game id, updated as messages arrive.
- **Batched render layer**: rather than re-rendering on every incoming message, applies a fixed-interval "tick" (e.g. every 250ms) that flushes accumulated updates to the UI at once.

Data flow and key design decisions

- Server pushes deltas (just the changed fields, e.g. new price and timestamp) rather than full objects, to keep message size small at high frequency.
- The UI subscribes only to the slice of the store it actually renders (e.g. a component only re-renders when its specific symbol's value changes), so an update to one stock doesn't re-render unrelated rows.
- Batch/throttle rendering on a fixed tick instead of on every message, which is the single most important decision for handling bursty update patterns like a goal or a market spike without the UI stuttering.

- Fall back to polling if a WebSocket connection can't be established (some networks block it), accepting higher latency as the trade-off for reliability.

Edge cases

- A burst of dozens of updates arrives within one second (a goal is scored, driving a flurry of stat updates): the batching layer must coalesce these into the next tick rather than queuing an unbounded backlog of individual re-renders.
- Connection drops and reconnects: on reconnect, request a fresh full snapshot of currently subscribed symbols rather than trying to replay every missed delta, since replaying an unknown number of missed messages is both harder and less reliable than just re-fetching current state.

Interview Answer:

"The core design decision here is treating this as a fan-out and batching problem rather than a per-widget data-fetching problem. I'd use one shared WebSocket connection for the whole page with a subscription manager multiplexing whichever symbols or games are currently visible, instead of one connection per ticker. Incoming updates go into a keyed store, and I'd render off a fixed tick, say every 250 milliseconds, batching accumulated updates instead of re-rendering on every single message, since that's what keeps the UI stable during a burst, like a goal or a big market move, rather than stuttering or falling behind. On reconnect after a drop, I'd fetch a fresh snapshot of current values rather than trying to replay every missed delta, since replay assumes you know exactly what you missed, and a snapshot is simpler and just as correct."

If you remember nothing else from this chapter: *these ten come up constantly for a reason, each one forces you to touch requirements, trade-offs, and a hard edge case in a single sitting.*

Q11. [Mid-Senior] Design the frontend for a music streaming app like Spotify. Walk me through the architecture.

Answer:

Global, router-independent player state

The current track, queue, playback position, and play/pause state must survive route navigation, since users keep browsing while music keeps playing. This argues for a top-level state store or context that lives above the router, not component-local state that would unmount the moment the user navigates away from a "now playing" screen.

A single playback engine

One page-level audio element, or the Web Audio API for advanced needs like crossfade or gapless playback, backs every mini-player UI shown across the app. You render many visual player widgets, but they all control the same one actual audio source.

Data layer and buffering

Track, album, and playlist metadata is fetched and cached like any content API, but playback progress needs a fast local ticking mechanism, an interval or the audio element's timeupdate event, rather than a network round trip for every second of progress. Pre-fetching and caching upcoming queue tracks (via the Cache API or IndexedDB for offline downloads) keeps playback from stalling on network hiccups.

Cross-tab and cross-device sync

Opening the app in two tabs should pause one when the other starts playing, typically coordinated via BroadcastChannel or a shared service worker, or a lightweight server-side "active session" signal for cross-device sync.

The defining architectural decision is that playback state has to be global and router-independent, since audio must keep playing across navigation; everything else, queue, mini-player, full-screen player, is just a different view onto that one shared piece of state.

Interview Answer:

"The core decision is treating playback as global state that lives above the router, not tied to any one screen, since audio has to keep playing while the user navigates. One real audio element backs every player UI shown across the app, and I'd handle cross-tab conflicts with BroadcastChannel so two open tabs don't play over each other."

Q12. [Senior] Design the frontend for a webmail client like Gmail or Outlook.

Answer:

A virtualized message list

A mailbox can have tens of thousands of messages, so the list view must use windowing, rendering only visible rows, rather than mounting a DOM node per email.

Optimistic actions with undo

Archiving, starring, marking read or unread, and deleting all need to feel instant: update the UI immediately, sync to the server in the background, and offer a clear "undo" affordance (a well-known real-world pattern is a toast after archive or delete) rather than blocking on server confirmation.

Search and drafts

Search needs to feel fast against a huge mailbox, typically backed by debounced queries to a dedicated search-indexed backend, with recent results cached client-side. Compose windows auto-save drafts locally, debounced writes to local storage or IndexedDB plus periodic server sync, so a crashed tab or lost connection doesn't lose an in-progress email.

Real-time new mail

New incoming mail needs to appear without a manual refresh, handled by a persistent connection (WebSocket or long-polling against a "new mail" endpoint) that triggers a targeted re-fetch of the inbox list rather than a full page reload.

The hardest part isn't any single screen, it's making a huge, frequently-mutated list feel instantly responsive through virtualization plus optimistic updates, while staying consistent with a server that might reject an action.

Interview Answer:

"The two things that make this hard are scale and mutation speed together: virtualize the message list since it can have tens of thousands of rows, and make every action, archive, star, delete, optimistic with a clear undo path, since users expect those to feel instant on a list this size."

Q13. [Senior] Design the frontend for a collaborative canvas or diagramming tool like Excalidraw.

Answer:

Canvas-based rendering, not the DOM

Shapes are drawn onto an HTML5 canvas (or SVG, with different trade-offs), not rendered as individual DOM elements, since a scene with thousands of independently draggable and resizable shapes would be far too slow and complex to manage as real DOM nodes.

A scene model decoupled from rendering

The actual source of truth is a plain data structure, an array of shape objects with type, position, size, and style. Every interaction, drag, resize, drawing a new shape, mutates this model, and a render loop redraws the canvas from that model, never the other way around.

Manual hit-testing

Since there's no DOM element per shape to click on, the app has to manually calculate which shape a given click's coordinates fall inside, nontrivial for rotated or overlapping shapes, usually solved by iterating shapes in reverse z-order and testing bounding boxes or paths.

Real-time collaboration

Multiple users editing the same canvas need a conflict-resolution strategy for concurrent edits. Excalidraw itself layers a CRDT-like approach over the shape list so two users moving different shapes, or even the same shape, converge to a consistent result without a central lock.

Performance at scale

As shape count grows, naively redrawing everything every frame breaks down. Real implementations use dirty-rectangle redrawing, only repainting the region that changed, and spatial indexing like a quadtree so hit-testing and redraw don't scale linearly with total shape count.

The core architectural shift from every other case study in this book is that the UI isn't primarily DOM-driven, it's a custom rendering engine over canvas, with the DOM only used for surrounding chrome like toolbars and menus.

Interview Answer:

"The biggest shift here is that the UI isn't DOM-driven at all, shapes live in a plain data model and get drawn onto a canvas by a render loop, with manual hit-testing to figure out what the user clicked. For collaboration, I'd use a CRDT-style merge over that shape list so concurrent edits from different users converge without a central lock."

Q14. [Senior] Design the frontend for a web-based spreadsheet like Google Sheets.

Answer:

Two-dimensional virtualization

A spreadsheet can have millions of cells, so the grid must be virtualized in both directions, rows and columns, rendering only the visible viewport window plus a small buffer, exactly like a virtualized list but two-dimensional.

A dependency-graph-driven formula engine

Cell formulas like =A1+B2 require building a dependency graph. When a cell's value changes, only cells that depend on it, directly or transitively, need to be recalculated, not the entire sheet. That means detecting cycles and computing an efficient recalculation order, a topological sort over the dependency graph, rather than a brute-force full recompute on every edit.

Editing and selection state

A single active cell, an editable formula bar, multi-cell range selection (click-drag, shift-click), and copy/paste that correctly handles relative versus absolute references shifting as you paste into a new location, an easy amount of state to underestimate.

Real-time collaboration and undo

Concurrent multi-user editing of the same cells needs operational transformation or CRDT-style conflict resolution, plus showing other users' live cursors and selections. Every edit, cell value, formatting, row insert/delete, needs to be captured as a reversible operation in a history stack, scoped carefully so undo affects only your own recent actions, not a collaborator's.

The two hardest problems are two-dimensional virtualization for scale and a dependency-graph-driven formula engine for correctness; everything else, selection, copy/paste, collaboration, builds on top of getting those two right.

Interview Answer:

"The two problems that actually make this hard are virtualizing a grid in two dimensions instead of one, and building a dependency graph for formulas so an edit only recalculates the cells that actually depend on it instead of the whole sheet. Everything else, selection, copy-paste, collaborative cursors, is secondary to getting those two right."

Q15. [Mid-Senior] Design the frontend for a Kanban board like Trello or Jira's board view.

Answer:

Representing card order cheaply

Columns (statuses) each contain an ordered list of cards. Ordering within a column and a card's column membership both change constantly through drag-and-drop, so the model needs a clean way to represent "position," commonly a sortable numeric or string rank per card (a "fractional indexing" scheme) that avoids having to re-index every other card whenever one moves.

Optimistic drag-and-drop

Dragging a card triggers an optimistic local reorder immediately so the UI feels instant, while the actual persisted move, new column, new position, is sent to the server in the background. If the server rejects it, a permission error or a stale board state, the UI rolls back to the last known-good arrangement.

Real-time multi-user sync

Other users moving cards on the same board need to be reflected live, typically via a WebSocket subscription to board-level events, and incoming remote moves have to be merged carefully with any in-progress local drag so a card doesn't visually "fight" between two conflicting positions.

Performance

A board with thousands of cards across many columns needs virtualization within each column's card list, plus care that drag-and-drop libraries, which often rely on measuring DOM node positions, still work correctly when off-screen cards aren't actually mounted.

The central design challenge is representing "where does this card sit" in a way that's cheap to update on every drag, avoiding a full re-index of every card, while staying consistent under concurrent edits from other users in real time.

Interview Answer:

"The trickiest part is the position model, I'd use a fractional ranking scheme per card so a drag only needs to update one card's rank, not re-index the whole column. Drag moves are optimistic locally with a rollback path, and other users' moves come in over a WebSocket and need to merge cleanly with whatever the local user is dragging at that moment."

Chapter 19: Top 10 Most Asked Frontend Component Design (LLD) Questions

These are the component-level "low-level design" (LLD) prompts that show up in almost every frontend interview loop, the ones where you're asked to design a single widget rather than a whole system. Use this list to practice turning a vague one-line prompt like "design a modal" into a real props contract, a state model, and a list of accessibility and edge-case concerns, since that gap between "I can build this" and "I can build this so a whole team can reuse it safely" is exactly what separates junior from senior in this format.

Q1. [Junior-Mid] Design a reusable star-rating input for a product review form. What are the props, the internal state, and how does it behave for keyboard and screen reader users?

Answer:

A star rating is a specialized numeric input, so the cleanest design treats it as a controlled form control (a component whose displayed value is driven entirely by props, not by its own internal memory) with custom visuals layered on top. **Design it as a controlled input first, and let the star icons be a rendering detail on top of that, so it composes cleanly into any form library.**

API / Props Surface

- `value: number`, the committed rating.
- `onChange(value: number)`, fires only when the user commits a rating, not on hover.
- `max: number`, total number of stars, default 5.
- `allowHalf: boolean`, whether half-star precision is supported.
- `readOnly` / `disabled: boolean`, for display-only contexts like an average rating badge.

Internal State

- `hoverValue`, a local, non-committed value used only to preview what the rating would become if the user clicked at the current pointer position.
- No other local state is needed since the committed value always comes from props, keeping the component predictable inside forms like Formik or React Hook Form.

Key Implementation Decisions

- Hover changes only update the visual preview, they never call `onChange`, so parent state does not thrash on every mouse move.
- Arrow keys (right/up to increase, left/down to decrease) move the committed value by one full star, Home and End jump to the min and max.
- Half-star support requires hit-testing which half of a star icon the pointer is over, which is more fragile than integer-only ratings, so many teams intentionally skip it unless the product explicitly needs it.

Accessibility

- Expose it with `role="radiogroup"` and an `aria-label` like "Rate this product", with each star as `role="radio"` and `aria-checked` reflecting the committed value, or alternatively use `role="slider"` with `aria-valuenow`/`aria-valuemin`/`aria-valuemax` if you want a single tab stop.

- Keyboard focus must have a visible outline distinct from the hover preview style, otherwise keyboard users cannot tell where focus is.

Edge Cases

- Touch devices have no hover state at all, so the first tap must directly commit a value rather than only previewing it.
- A `readOnly` average rating (like "4.3 stars") needs a way to render partial/fractional stars purely visually without exposing any interactive radio semantics to assistive technology.

Interview Answer:

"I'd build this as a controlled component, value and onChange, so it drops into any form library without extra glue code. Internally I only keep a hoverValue for the preview state, the real value always comes from props. For accessibility I'd expose it as a radiogroup with one radio per star and support arrow key navigation, because a rating widget that only works with a mouse fails an accessibility audit immediately. The trickiest edge case is half-star precision, since that requires hit-testing which half of the icon the pointer is over, so I'd only add it if the product spec actually calls for it."

Q2. [Junior-Mid] Design a toast/notification queue system that multiple parts of the app can push messages into. How do you manage the queue, timing, and stacking?

Answer:

A toast system is really two things: a global queue of notification objects, and a renderer that decides how many are visible and for how long. **Model toasts as a queue in a single global store, not as ad-hoc state in whichever component happens to trigger them, so any part of the app, including API error handlers, can push a toast without knowing about the UI tree.**

API / Props Surface

- A hook or function like `toast.show({ message, type, duration, action })` callable from anywhere, including outside React components (a raw API error handler, for example).
- `type: 'success' | 'error' | 'info' | 'warning'` drives icon and color.
- `duration: number | null`, `null` meaning "stays until dismissed" (used for errors that need acknowledgement).
- `action: { label, onClick }`, optional, for things like "Undo".

Internal State

- A single array of active toast objects, each with a generated id, held in a context provider or external store (Zustand/Redux) so any component can dispatch into it.
- A per-toast timer reference so a toast that's being hovered can pause its auto-dismiss countdown.

Key Implementation Decisions

- Render the toast list through a portal (a React feature that renders a component's output into a different part of the actual DOM tree, usually `document.body`) so toasts always stack above app content regardless of parent overflow or z-index (stacking order) rules.
- Cap the number of simultaneously visible toasts (commonly 3 to 5) and queue the rest, otherwise a burst of errors from a failed batch operation can flood the screen.
- Deduplicate identical messages within a short window (for example, don't show "Failed to save" five times if five requests fail at once), collapsing into a single toast with a count badge instead.

Accessibility

- Use `role="status"` with `aria-live="polite"` for informational toasts, and `role="alert"` for errors, so screen readers announce them without stealing keyboard focus from whatever the user was doing.
- Auto-dismiss timers should pause on hover and on keyboard focus, since a screen reader or low-vision user may need more than the default 4 seconds to read the message.

Edge Cases

- A toast triggered right as the user navigates to a new route: decide whether toasts persist across route changes (usually yes, since they represent the result of an action, not page content) or get cleared.
- Rapid-fire toasts from a retrying background sync job need throttling or deduplication, or the queue never drains.

Interview Answer:
"I'd centralize this as a queue in a global store with a `toast.show()` function that's callable from anywhere, not tied to any one component, because error toasts often need to fire from an API layer that has no idea what's currently rendered. I'd cap visible toasts at three to five and queue the rest, and dedupe identical messages so a burst of failures doesn't spam the screen. For accessibility I'd use aria-live regions, polite for info and alert for errors, and pause the auto-dismiss timer on hover or focus so people have time to actually read it."

Q3. [Junior-Mid] Design a reusable Modal/Dialog component for a design system. What's the props contract, and how do you handle focus and stacking?

Answer:

The interesting part of a modal isn't the box in the middle of the screen, it's everything around it: how it escapes the DOM hierarchy, traps focus, and restores the page to a sane state on close. **The core design decision is rendering the modal through a portal and implementing a focus trap, since without those two things the "modal" is just a styled div that a keyboard or screen reader user can still tab and read straight through.**

API / Props Surface

- `isOpen: boolean`, `onClose(): void`, controlled from the parent.
- `title`, `children` for content, `initialFocusRef` to specify which element gets focus on open (defaults to the first focusable element or the close button).
- `closeOnOverlayClick` and `closeOnEsc: boolean`, both default true but overridable for destructive confirmations where accidental dismissal is costly.
- `size` or `variant` for width presets (small confirmation vs large form modal).

Internal State

- No business state inside the modal itself, it's fully controlled via `isOpen`.
- Internally it does track the element that had focus before opening, so focus can be restored to it on close.

Key Implementation Decisions

- Render via `createPortal` into `document.body` so the modal is never visually clipped or z-index-fought by an ancestor with `overflow: hidden`.

- Implement a focus trap (keeping Tab/Shift+Tab cycling only among the modal's own focusable elements) so keyboard focus can't silently leak back to the page behind the overlay.
- Lock body scroll while open (`overflow: hidden` on `<body>`, carefully restoring the original value on close) so background content doesn't scroll under a fixed overlay.
- Support nested/stacked modals by tracking an open-modal stack, only the topmost modal should respond to Esc and own the focus trap.

Accessibility

- `role="dialog"` (or `"alertdialog"` for confirmations), `aria-modal="true"`, `aria-labelledby` pointing at the title element.
- On open, move focus into the modal, on close, return focus to the element that triggered it, both are required by WCAG (Web Content Accessibility Guidelines) for dialogs.

Edge Cases

- A modal opened from within another modal (a "delete confirmation" inside an "edit" modal) needs the stack-aware Esc/focus-trap handling above.
- Very tall content needs an internal scroll region, with the header/footer pinned, rather than letting the whole page scroll.

```
function Modal({
  isOpen,
  onClose,
  title,
  children,
  closeOnOverlayClick = true,
  closeOnEsc = true,
  initialFocusRef,
}) {
  // renders via createPortal(document.body), traps focus while open,
  // restores focus to the trigger element on close
}

<Modal
  isOpen={isDeleteOpen}
  onClose={() => setIsDeleteOpen(false)}
  title="Delete project?"
  closeOnOverlayClick={false}
>
  <p>This action cannot be undone.</p>
  <button onClick={handleDelete}>Delete</button>
</Modal>
```

Interview Answer:

"I'd make it a controlled component, isOpen and onClose, rendered through a portal into document.body so it's never clipped by an ancestor's overflow or z-index rules. The part I'd spend the most design time on is focus management: trap Tab and Shift+Tab inside the modal while it's open, move focus into it on open, honoring an initialFocusRef if given, and restore focus back to whatever triggered it on close. I'd also lock body scroll while it's open and think about a stacking case, like a confirmation dialog opened from inside a bigger form modal, where only the topmost one should own Esc and the focus trap. Getting the box to look right is easy, getting the focus and scroll behavior right is what actually makes it accessible."

Q4. [Mid] Design a multi-select dropdown component with full keyboard navigation. Walk me through the state model and the keyboard interaction contract.

Answer:

A multi-select dropdown has two separable concerns: which options are selected, and which option is currently highlighted for keyboard navigation, and conflating those two is the most common design mistake. **Keep "highlighted option" and "selected options" as two distinct pieces of state, since a keyboard user needs to move the highlight through the list without changing the selection until they explicitly commit.**

API / Props Surface

- `options: Array<{ label, value, disabled? }>`.
- `selectedValues: value[]`, `onChange(values: value[])`, controlled selection.
- `isOpen` / `onOpenChange`, if the parent needs to control open state (for example, to close it when a form resets).
- `placeholder`, `renderOption`, `maxSelected` (optional cap with a friendly message when hit).

Internal State

- `isOpen: boolean` (can be internal or lifted, per the prop above).
- `highlightedIndex: number`, which option the keyboard cursor is on, independent of selection.
- `searchQuery: string`, if the dropdown supports type-to-filter.

Key Implementation Decisions

- Arrow Up/Down move `highlightedIndex` only, Space or Enter toggles that option's membership in `selectedValues`, this separation is what makes keyboard use feel correct.
- Clicking a checkbox row must not close the dropdown (multi-select stays open across selections), whereas a single-select variant closes immediately on choice, so the two should share a base component but diverge on this one behavior.
- Typing while open should jump-scroll to (or filter to) matching options rather than doing nothing, since sighted mouse users get this "for free" by scanning the list visually.
- Selected items are usually echoed as removable "chips" in the closed trigger, each chip needs its own accessible remove button, not just a decorative "x".

Accessibility

- `role="listbox"` with `aria-multiselectable="true"` on the options container, each option `role="option"` with `aria-selected` reflecting selection, and the trigger button with `aria-expanded` and `aria-haspopup="listbox"`.
- Use `aria-activedescendant` on the listbox to announce the currently highlighted option to screen readers without moving actual DOM focus off the input, this is the standard pattern for combobox-style widgets.

Edge Cases

- Selecting "all" versus a very large option list (hundreds of items): pair this with virtualization (see Q7) if the list is long, since rendering every checkbox row eagerly will visibly lag.
- Async-loaded options (searched from a server) need a loading and an empty state inside the same listbox, and the highlighted index must reset or re-map sensibly when the option list changes underneath the user.

Interview Answer:

"The key modeling decision is splitting highlightedIndex from selectedValues, because a keyboard user arrowing through the list shouldn't be selecting things just by moving the cursor, only Space or Enter should commit. I'd expose it as a controlled selectedValues array with onChange, use role listbox with aria-

multiselectable, and drive the visual highlight with aria-activedescendant so focus technically stays on the input while screen readers still announce the right option. For multi-select specifically, choosing an option shouldn't close the dropdown, that's the main behavioral difference from a single-select combobox. And if the option list is large, I'd virtualize the listbox rather than rendering every row."

Q5. [Mid] Design a search box with autocomplete suggestions that debounces input and cancels stale in-flight requests. What happens on every keystroke?

Answer:

The naive version fires a request on every keystroke and just overwrites whatever renders last, which breaks the moment network latency is inconsistent, because a slow response to an earlier keystroke can arrive after a fast response to a later one and overwrite it with stale results. **Combine debouncing (waiting until typing pauses before firing a request) with request cancellation, keyed to the query that triggered it, so a slow, stale response can never overwrite a newer one.**

API / Props Surface

- `onSearch(query: string): Promise<Result[]>`, the caller-provided fetch function.
- `debounceMs: number`, default around 250 to 400ms.
- `minChars: number`, don't fire a request for a single character.
- `onSelect(result)`, when the user picks a suggestion.

Internal State

- `query: string`, the raw input value, updates synchronously on every keystroke so the input never feels laggy.
- `results: Result[]` and `status: 'idle' | 'loading' | 'success' | 'error'`.
- A ref holding the current `AbortController` (a browser API for canceling an in-flight fetch request) so the previous request can be aborted the instant a new one starts.

Key Implementation Decisions

- The visible input updates immediately and synchronously; only the network call is debounced, otherwise the box feels like it's lagging behind typing.
- On each new debounced fetch, abort the previous controller before creating a new one, and ignore `AbortError` in the catch block since that's an expected cancellation, not a real failure.
- Debounce vs throttle (firing on a regular interval regardless of pauses): debounce is the right choice here because the goal is "wait until the user probably stopped typing," not "sample periodically while they type," which is what throttle is for (like scroll or resize handlers).

Best Choice

Debounce with cancellation via AbortController, keyed per request.

Why?

Throttling would still fire requests mid-word and waste bandwidth on queries the user never intended to submit, while debouncing without cancellation leaves a race condition where an old, slow response can land after a newer one and visually overwrite correct results with stale ones.

Accessibility

- The suggestion list needs the combobox ARIA pattern: `role="combobox"` on the input, `aria-expanded`, `aria-controls` pointing at the listbox, `aria-activedescendant` for the highlighted suggestion, same pattern as Q4's dropdown.
- Announce result counts to screen readers via a visually-hidden live region ("5 results found") since the visual list update alone isn't announced.

Edge Cases

- Clearing the input entirely should cancel any pending request and clear results immediately, not wait for the debounce timer.
- Backspacing back to a previous query that was already cached should ideally serve from a local cache instead of re-fetching, which also sidesteps the race condition entirely for that case.

```
function useAutocomplete(query, fetchFn, delay = 300) {
  const [results, setResults] = useState([]);
  const [status, setStatus] = useState('idle');

  useEffect(() => {
    if (!query) {
      setResults([]);
      return;
    }
    const controller = new AbortController();
    const timeoutId = setTimeout(async () => {
      setStatus('loading');
      try {
        const data = await fetchFn(query, controller.signal);
        setResults(data);
        setStatus('success');
      } catch (err) {
        if (err.name !== 'AbortError') setStatus('error');
      }
    }, delay);

    return () => {
      clearTimeout(timeoutId);
      controller.abort();
    };
  }, [query, fetchFn, delay]);

  return { results, status };
}
```

Interview Answer:

"The input value itself has to update instantly on every keystroke so typing never feels laggy, but the actual network call gets debounced, so we only fetch once typing pauses for, say, 300 milliseconds. The part people forget is cancellation: every time a new debounced request fires, I abort the previous one with an AbortController and ignore AbortError in the catch, because otherwise a slow response to an old keystroke can arrive after a fast response to a newer one and stomp the results with stale data. I'd explicitly avoid throttling here since the goal is waiting for a pause, not sampling on an interval, that's a scroll-handler pattern, not a search-box pattern."

Q6. [Mid-Senior] Design a drag-and-drop reorderable list, like a task list a user can reorder by dragging rows. How do you track order during the drag, and what API do you choose?

Answer:

The two big decisions are which drag API to build on, and how to represent "order" so the list doesn't visually jump or flicker while an item is mid-drag. **Track order as an array of stable IDs that's optimistically reordered on every drag-over event, and commit it to the parent only on drop, so the UI shows the reorder in real time without waiting on a network round trip.**

API / Props Surface

- `items: Array<{ id, ...data }>`, `onReorder(newItems)`, called once on drop (not on every intermediate drag-over).
- `renderItem(item, dragHandleProps)`, so consumers control the row's look while the list owns the drag mechanics.
- `disabled` per item, for rows that shouldn't be draggable (a pinned/locked task, for example).

Internal State

- `draggedId`, which item is currently being dragged.
- A locally-mirrored `orderedIds` array that gets optimistically reordered live as the user drags over other rows, separate from the source-of-truth `items` prop, then reconciled/committed via `onReorder` on drop.
- `dropTargetIndex` or similar, to render an insertion indicator line between rows.

Key Implementation Decisions

- Native HTML5 Drag and Drop API vs pointer-events-based "sortable" implementation (like dnd-kit or react-beautiful-dnd's approach): native DnD has inconsistent styling hooks and poor mobile/touch support, while a pointer-events-based approach gives full control over drag previews, touch support, and animation, at the cost of implementing hit-testing yourself.

Best Choice

Pointer-events-based (or a library like dnd-kit) over native HTML5 drag-and-drop for anything that needs to work on touch devices or wants a polished animated reorder.

Why?

Native drag-and-drop's touch support is effectively nonexistent without extra polyfills, and its drag image/preview customization is clunky, whereas pointer events give full control and work identically on mouse and touch.

- Reorder the list optimistically in local state on every drag-over so the user gets live visual feedback, but only call `onReorder` once, on drop, to avoid spamming the parent (and any backend persistence) with dozens of intermediate states.
- Use stable item IDs as React keys, never array index, or the drag preview and animations will visually detach from the item actually being dragged.

Accessibility

- Pure drag-and-drop excludes keyboard and screen reader users entirely, so a production-grade version needs a keyboard fallback: focus a row, press a key (commonly Space to "pick up," then arrow keys to move, then Space again to "drop"), announced via an `aria-live` region ("Moved item to position 3 of 8").
- This keyboard path is not a nice-to-have, WCAG requires drag-and-drop functionality to have a non-drag alternative.

Edge Cases

- Dragging near the top or bottom edge of a scrollable container needs auto-scroll, otherwise users can't reorder into the currently-offscreen part of a long list.
- Reordering a filtered or paginated view (where `items` is a subset of the full dataset) requires mapping the visible reorder back to the correct positions in the full underlying dataset, not just reordering the visible slice.

Interview Answer:

"I'd track order as an array of stable item IDs, optimistically re-sorted on every drag-over so the user sees live feedback, and only fire onReorder once, on drop, so I'm not flooding the parent or a backend with every intermediate position. For the drag mechanics themselves I'd reach for a pointer-events-based approach, something like dnd-kit, rather than the native HTML5 drag-and-drop API, mainly because native drag-and-drop has weak touch support and clunky preview customization. The part that's easy to forget is accessibility: drag-and-drop by itself locks out keyboard and screen reader users completely, so I'd add a pick-up-and-move keyboard pattern with an aria-live announcement of the new position, since WCAG actually requires a non-drag alternative for this kind of interaction."

Q7. [Mid-Senior] Design a virtualized (windowed) list that can smoothly render a dataset of 100,000 rows. Walk me through how you decide what to render.

Answer:

Rendering 100,000 DOM nodes would crash the tab or freeze the page, so the core idea of virtualization (also called "windowing") is to compute which items are actually visible in the scroll viewport and render only those, plus a small buffer, while faking the total scrollable height. **Virtualization works by using scroll position and a known (or measured) item height to mathematically compute a small visible index range, rendering only those items absolutely-positioned inside a container sized to the full list's total height.**

API / Props Surface

- `items: T[]`, `itemHeight: number` (fixed) or `getItemHeight(index)` (variable/dynamic heights).
- `containerHeight: number` (or the component measures its own via a ref).
- `overscan: number`, extra rows rendered above/below the visible window as a buffer, default around 3 to 5.
- `renderItem(item, index)`.

Internal State

- `scrollTop: number`, updated on the container's scroll event, this single number drives everything else.
- For dynamic-height variants, a cache of measured heights per index (via `ResizeObserver`, a browser API that reports when an element's size changes), since you can't know an unmeasured item's height in advance.

Key Implementation Decisions

- Fixed-height items: `startIndex = floor(scrollTop / itemHeight)`, `endIndex = ceil((scrollTop + containerHeight) / itemHeight)`, both math operations, no measuring needed, this is the simple and fast case.
- Variable-height items are meaningfully harder: you need a running offset cache built from measured heights, and an estimate for not-yet-measured items below the fold, since you can't binary-search into offsets you haven't measured yet.

- Absolutely position each rendered row at its computed offset inside a spacer div whose height equals the full list's total height, this is what makes the scrollbar length and thumb position correct even though only a handful of rows actually exist in the DOM.
- Add overscan rows above and below the visible window so fast scrolling (or a screen reader's virtual cursor) doesn't show blank flashes before new rows paint.

Accessibility

- Screen readers rely on the DOM actually containing content, so a virtualized list can "disappear" content that isn't currently rendered, mitigate with `aria-setsize` and `aria-posinset` on each row (telling assistive tech the row's position within the full, non-rendered total) and generous overscan.
- Ensure the scroll container itself is reachable and operable via keyboard (Page Up/Down, Home/End) even though most rows aren't in the DOM at any given moment.

Edge Cases

- Jumping the scrollbar directly to the bottom (or a "scroll to item 87,000" deep link) must compute the target scroll offset mathematically rather than scrolling item-by-item.
- Items whose height changes after mount (an expandable row, or an image that loads and changes aspect ratio) invalidates the offset cache for every row after it, requiring an efficient recomputation strategy rather than remeasuring the entire list.

```
function VirtualList({ items, itemHeight, containerHeight, overscan = 5, renderItem }) {
  const [scrollTop, setScrollTop] = useState(0);

  const startIndex = Math.max(0, Math.floor(scrollTop / itemHeight) - overscan);
  const endIndex = Math.min(
    items.length,
    Math.ceil((scrollTop + containerHeight) / itemHeight) + overscan
  );
  const visibleItems = items.slice(startIndex, endIndex);

  return (
    <div style={{ height: containerHeight, overflowY: 'auto' }} onScroll={(e) =>
setScrollTop(e.target.scrollTop)}>
      <div style={{ height: items.length * itemHeight, position: 'relative' }}>
        {visibleItems.map((item, i) => (
          <div
            key={item.id}
            style={{ position: 'absolute', top: (startIndex + i) * itemHeight, height:
itemHeight, width: '100%' }}
          >
            {renderItem(item)}
          </div>
        ))}
      </div>
    </div>
  );
}
```

Interview Answer:

"The core trick is that scrollTop plus a known item height is just math, floor of scrollTop over itemHeight gives you the first visible index, and from there you compute how many rows fit in the viewport, so you only ever render a couple dozen actual DOM nodes no matter how big the dataset is. I'd absolutely-position each rendered row at its real offset inside a spacer div sized to the full list's total height, so the scrollbar behaves like the whole list is really there. Fixed-height rows make this straightforward math, variable-height rows are the harder version, since you need to measure and cache offsets as items render and estimate for the ones you haven't seen yet. I'd also add overscan, a few extra rows above and below the viewport, so fast scrolling

doesn't flash blank space, and I'd think about aria-posinset and aria-setsize so screen reader users get some sense of position in a list that's mostly not in the DOM at any given moment."

Q8. [Mid-Senior] Design a Tooltip/Popover component that positions itself smartly, flipping sides automatically so it never gets clipped off the edge of the viewport.

Answer:

A naive tooltip just renders "above the trigger" and gets clipped or clipped off-screen the moment the trigger is near a viewport edge. **Smart positioning means measuring the trigger's and content's actual rendered dimensions with `getBoundingClientRect`, checking that against the current viewport bounds, and flipping the preferred side (or nudging the offset) whenever the preferred placement would overflow.**

API / Props Surface

- `trigger: ReactNode`, `content: ReactNode`.
- `placement: 'top' | 'bottom' | 'left' | 'right' | 'auto'`, `auto` meaning "pick the side with the most available space."
- `offset: number`, gap between trigger and content, and `openOn: 'hover' | 'click' | 'focus'`.
- `closeDelay`, so moving the mouse from the trigger to the popover content itself (a common hover gap) doesn't instantly close it.

Internal State

- `isOpen: boolean`.
- `computedPlacement`, the actual resolved side after collision detection, which may differ from the requested `placement`.
- Cached trigger and content `DOMRect` measurements (from `getBoundingClientRect`), recomputed on open, on scroll, and on window resize.

Key Implementation Decisions

- Render the popover content through a portal into `document.body`, same reasoning as the modal in Q3: a popover positioned relative to a trigger buried inside an `overflow: hidden` card would otherwise get clipped by that ancestor.
- Position with `position: fixed` and coordinates computed in JavaScript (or delegate to a positioning library like Floating UI) rather than pure CSS, since CSS alone can't perform viewport collision detection.
- Recompute position on scroll and resize (throttled), and close the popover if the trigger scrolls fully out of view, otherwise it visually detaches from its trigger.
- Hover-triggered popovers need a small grace-period timeout on mouseleave, so a user moving their mouse from the trigger toward the popover content doesn't have it vanish mid-transit.

Accessibility

- Informational tooltips (text-only, on hover) use `role="tooltip"` with `aria-describedby` on the trigger pointing at the tooltip's id, and must also appear on keyboard focus, not only mouse hover, since a keyboard-only user needs the same information.
- Interactive popovers (with buttons/forms inside, like a filter panel) use `aria-haspopup` and `aria-expanded` on the trigger, and should trap or at least manage focus similar to a lightweight modal, plus close on Escape and on an outside click.

Edge Cases

- Trigger near a corner where both the preferred horizontal and vertical placements would overflow, needs a defined fallback order (for example: bottom, then top, then right, then left) rather than looping indefinitely.
- Content that's taller than the remaining viewport space even after flipping needs an internal max-height with its own scroll, rather than pushing off-screen regardless of which side it's placed on.

```
function Popover({ trigger, content, placement = 'auto', offset = 8 }) {
  // measures trigger + content DOMRects on open, on scroll, and on resize,
  // computedPlacement flips away from the requested placement if it would
  // overflow the viewport, and content renders through a portal at
  // position: fixed coordinates
}

<Popover trigger={<button>Filters</button>} content={<FilterPanel />}
placement="bottom" />
```

Interview Answer:

"The mechanical core is measuring both the trigger and the content with getBoundingClientRect and comparing that against the viewport, then flipping the placement when the preferred side would overflow, that's the whole idea behind 'smart' positioning, it's collision detection, not magic. I'd render the content through a portal at position fixed so it's never clipped by some ancestor's overflow hidden, and I'd recompute position on scroll and resize, throttled, and close it if the trigger scrolls fully offscreen so it doesn't visually detach. For hover-triggered popovers specifically, I'd add a small close delay so moving the mouse from the trigger toward the popover content doesn't slam it shut mid-transit. And for accessibility, a hover-only tooltip that never shows on keyboard focus is a real, common bug, so it has to trigger on focus too, not just mouseenter."

Q9. [Senior] Design a data table that supports sorting, filtering, and either pagination or infinite scroll, backed by a server API. What lives on the client vs the server, and how do you choose between pagination styles?

Answer:

The central design question is how much of sorting/filtering/paging happens client-side versus server-side, and that choice is driven entirely by dataset size, not by preference. **Once the dataset can't comfortably fit in memory on the client, sorting, filtering, and pagination all have to move server-side, and that in turn usually forces cursor-based pagination instead of offset-based, since offsets drift under concurrent writes.**

API / Props Surface

- `fetchPage({ cursor, pageSize, sortBy, sortDir, filters })`, returns `{ rows, nextCursor, totalCount? }`.
- `columns: Array<{ key, label, sortable, render(row), filterType }>`.
- `mode: 'pagination' | 'infinite-scroll'`.
- `onSortChange`, `onFilterChange`, both controlled if the parent needs to sync them to the URL (so a table's state survives a page refresh or a shared link).

Internal State

- `sortBy`, `sortDir`, `filters` (an object keyed by column), `cursor` or `pageIndex`.
- `rows`, `status: 'idle' | 'loading' | 'error' | 'success'`, and for infinite scroll, `isFetchingNextPage` kept separate from the initial `loading` state so the existing rows don't disappear while more load.

Key Implementation Decisions

- Offset-based pagination (`page=3&pageSize=20`) is simple and supports "jump to page 7," but rows can shift or duplicate across pages if rows are inserted/deleted between requests (a classic bug when a live feed is being paginated).
- Cursor-based pagination (`cursor=abc123`) is stable under concurrent writes since each cursor points at an actual row's position, but it can't jump to an arbitrary page number, only "next"/"previous."

Best Choice

Cursor-based pagination for any dataset with concurrent writes (a live orders table, a social feed), offset-based for small, mostly-static datasets where "jump to page N" is a real user need (an admin table of 200 fixed reference records).

Why?

Offset pagination silently shows duplicate or skipped rows the moment the underlying data changes between page loads, which is exactly the failure mode a paginated dashboard hits in production; cursor pagination avoids that at the cost of losing direct page-number navigation.

- Sorting and filtering must be sent as part of the same request that determines the cursor, since a cursor computed under one sort order is meaningless under a different one, changing sort or a filter should reset pagination to the first page/cursor.
- Debounce filter inputs (same technique as Q5) before firing a new server request, so typing in a filter box doesn't fire a request per keystroke.

Accessibility

- Use a real `<table>` with `<th scope="col">` for sortable headers, expose the current sort direction with `aria-sort="ascending" | "descending" | "none"` on the relevant header, and make the sort control a real button, not a bare clickable `<th>`.
- For infinite scroll, announce newly loaded rows via a polite live region ("20 more rows loaded"), since a screen reader user has no visual cue that more content just appeared below.

Edge Cases

- Infinite scroll combined with sort/filter changes: switching sort while several pages are already loaded must discard the loaded pages and restart from the first cursor under the new sort, not append onto old, now-invalid ordering.
- A user applying a filter that returns zero rows needs an explicit empty state, not a table that just silently renders no `<tr>`s and looks broken.

Interview Answer:

"The real decision here isn't sorting or filtering mechanics, it's where those operations happen, and that's dictated by data size: once you can't hold the whole dataset in memory client-side, sorting and filtering have to move to the server, which then usually means cursor-based pagination rather than offset-based, because offsets drift the moment rows are inserted or deleted between page loads, which is a real bug in something like a live orders table. Sort and filter changes both have to reset pagination, since a cursor computed under one sort order means nothing under a different one. For infinite scroll specifically, I'd keep isFetchingNextPage separate from the initial loading state so existing rows don't disappear while more load in the background, and I'd make sure changing a filter mid-scroll discards the stale pages instead of appending onto now-invalid data."

Q10. [Senior] Design a multi-step form wizard, like a checkout or onboarding flow, with per-step validation and the ability to go back a step without losing anything the user already typed. What's the state model?

Answer:

The failure mode this question is really probing for is a wizard that re-mounts each step as a fresh, uncontrolled form, so navigating back wipes out whatever the user entered on that step. **Hold all step data in a single object that lives above any individual step's lifecycle, and validate lazily per step on "Next" rather than eagerly across the whole wizard, so going back never touches, resets, or re-validates data the user hasn't tried to submit yet.**

API / Props Surface

- `steps: Array<{ id, component, validate(values): errors }>`, each step owns its own validation function but not its own data storage.
- `initialValues`, `onComplete(values)`, called once, on final submit.
- Optional `onStepChange(stepIndex)` for analytics or syncing the current step into the URL, so a page refresh mid-flow lands back on the right step.

Internal State

- `values`, one flat (or step-namespaced) object holding every field from every step, created once at the top of the wizard and never reset by step navigation.
- `stepIndex`, `errors` (scoped to the current step only), and `visitedSteps` (a set of step indices the user has reached), used to decide whether a not-yet-visited future step should show as "locked" in a step indicator.

Key Implementation Decisions

- Validate only the current step's fields on "Next," not the entire form, so an error on step 3 doesn't block a user from ever reaching step 3 to see what's wrong, and doesn't retroactively invalidate step 1 for no reason.
- "Back" never clears or re-validates the previous step's values, since the whole point of the feature is that it's still there exactly as entered, this is the detail that separates a wizard that feels trustworthy from one that quietly frustrates users into abandoning checkout.
- Persist `values` to `sessionStorage` (or the URL, for shareable state) on every change, so an accidental refresh or a back-button navigation away from the browser doesn't lose the user's progress, this matters a lot for anything long, like a multi-page insurance or loan application.
- Model the whole thing as a small state machine (steps as explicit states, "Next"/"Back"/"Submit" as the only transitions) rather than scattering `if (step === 2)` conditionals through the render tree, which becomes unreadable past four or five steps.

Accessibility

- Announce step changes to screen readers via a live region ("Step 2 of 4: Shipping address"), since the visual step indicator alone gives no cue to a non-visual user that the "page" changed.
- Focus should move to the new step's heading (not just silently rerender in place) on both Next and Back, and validation errors need `aria-describedby` linking the error message to its input, plus focus should jump to the first errored field on a failed "Next" attempt.

Edge Cases

- A step whose valid options depend on an earlier step (shipping method options depending on the address entered in step 1) needs to re-validate or reset just that dependent field, not the whole form, if the user goes back and changes the earlier value.
- Browser back/forward buttons interacting with wizard step state, if step is reflected in the URL, need to be handled explicitly (via `popstate`) or the browser's native back button and the wizard's own "Back" button will fight each other.

```
function useWizard(steps, initialValues) {
  const [stepIndex, setStepIndex] = useState(0);
  const [values, setValues] = useState(initialValues);
  const [errors, setErrors] = useState({});

  const goNext = async () => {
    const stepErrors = await steps[stepIndex].validate(values);
    if (Object.keys(stepErrors).length) {
      setErrors(stepErrors);
      return false;
    }
    setErrors({});
    setStepIndex((i) => Math.min(i + 1, steps.length - 1));
    return true;
  };

  const goBack = () => setStepIndex((i) => Math.max(i - 1, 0));

  return { stepIndex, values, setValues, errors, goNext, goBack };
}
```

Interview Answer:

"The state model has to separate 'where am I' from 'what has the user entered,' so I'd hold all field values for every step in a single object that lives above the wizard's step navigation, created once and never reset when the user moves between steps. Validation happens lazily, only on the current step when they hit Next, not eagerly across the whole form, and going Back never touches or re-validates the previous step's data, since that's the entire point of the feature. I'd persist that values object to sessionStorage on every change so a refresh doesn't wipe out a long form, and I'd model step transitions as an explicit small state machine rather than a pile of if-step-equals conditionals, since that's what stays readable once you're past four or five steps. For accessibility I'd move focus to the new step's heading on every transition and announce the step change in a live region, because the visual step indicator alone tells a screen reader user nothing."

If you remember nothing else from this chapter: *component design questions reward you when you can turn a vague prompt into a clean prop and state boundary without reaching for more complexity than the component has earned.*